

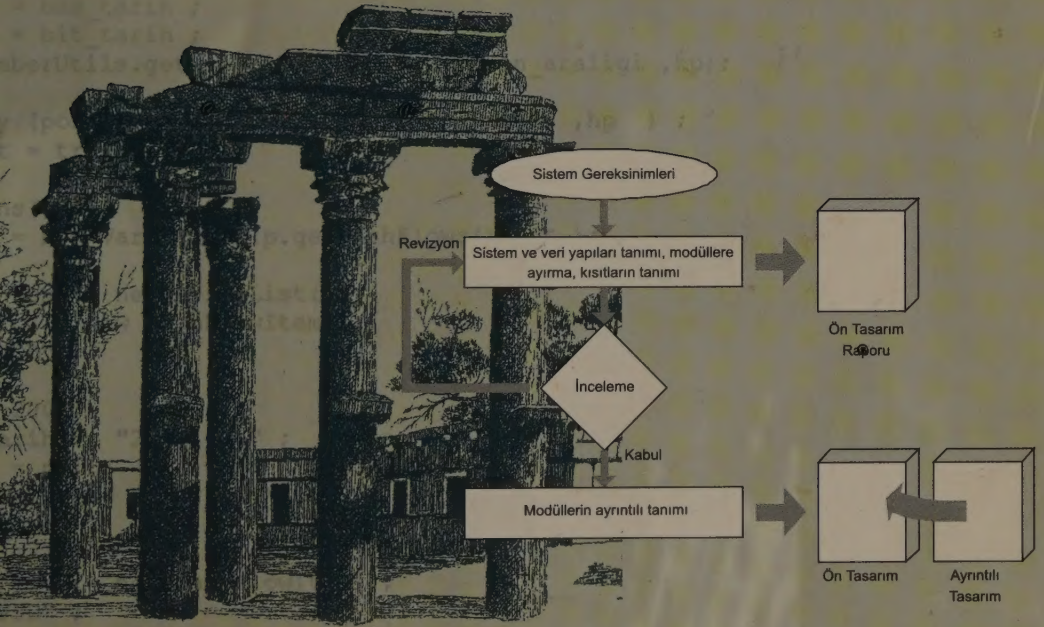
BİLGİSAYAR BİLİMLERİNDE SİSTEM ANALİZİ VE TASARIMI

NESNEYE YÖNELİK MODELLEME

Prof.Dr. Oya KALIPSIZ

Ayşe BUHARALI

Göksel BİRİCİK



PAPATYA

SİSTEM ANALİZİ VE TASARIMI

Prof. Dr. Oya KALIPSIZ
Ayşe BUHARALI
Göksel BİRİCİK

PAPATYA YAYINCILIK EĞİTİM

İstanbul, Ankara, İzmir ve Adana

© PAPATYA YAYINCILIK EĞİTİM - Ocak 2006
EĞİTİM BİLGİSAYAR SİS. SAN. VE TİC. A.Ş.

İnönü Cad. Hacıhanım Sok. 10/6, 80090, Gümüşsuyu/İstanbul

Tel : 212-245 37 40, Faks: 212-245 37 41
e-Posta : bilgi@papatya.gen.tr
Web : www.papatya.gen.tr
www.papatya.info

Sistem Analizi ve Tasarımı – Prof. Dr. Oya KALIPSIZ, Ayşe BUHARALI ve Göksel BİRİCİK

1. Basım Ocak 2006

Yayın Danışmanı : Dr. Rifat ÇÖLKESEN (Beykent Üniversitesi)
Üretim : Oğuz ÇÖLKESEN / Erdal GÜVEN
Sayfa Düzenleme : Papatya & Kelebek Tasarım
Kapak Tasarım : Papatya & Kelebek Tasarım
Basım ve Ciltleme : Altan Matbaacılık - İstanbul
Ana Dağıtım : Toros Dağıtım Kitap (212-520 42 25) - Ahmet KURTAY

© Bu kitabın her türlü yayın hakkı yayınevine aittir. Yayınevinden yazılı izin alınmaksızın alıntı yapılamaz, kısmen veya tamamen hiçbir şekil ve teknikle ÇOĞALTILAMAZ, BASILAMAZ, YAYIMLANAMAZ. Kitabın, tamamının veya bir kısmının fotokopi makinası, ofset vs. gibi tekniklerle çoğaltılması, hem çoğaltan hem de bulunduranlar için yasadışı bir davranıştır. Yayınevi, bu gibi yasadışı davranışlarda bulunan kurum ve kişilere karşı, her türlü haklarını korumakta kararlıdır.

Kalıpsız, Oya; Buharalı, Ayşe; Biricik, Göksel.

Sistem Analizi ve Tasarımı / Oya Kalıpsız, Ayşe Buharalı, Göksel Biricik - İstanbul: Papatya Yayıncılık, 2006.

XIV, 192 s. ; 24 cm.

Kaynakça ve izin var.

ISBN: 975-6797-68-1

1. Sistem Yaklaşımı 2. Bilgi Sistemleri 3. Nesneye Yönelik 4. Object Oriented I. Title.

*Bu kitabımızı, bilimsel araştırma konusunda yol gösterici olan
sevgili babam Prof. Dr. Abdülkadir KALIPSIZ'a
İthaf ediyorum.*

TEŞEKKÜR

Araştırma çalışmalarımızda yanımızda olan;

Yıldız Teknik Üniversitesi yönetimine,

YTÜ Bilgisayar Mühendisliği bölümündeki çalışma arkadaşlarımıza,

Metin düzeltmelerinde katkıları için *Ekin Su UĞURLU*'ya,

Kitabın basımında gerekli özeni gösteren *Papatya Yayıncılık* çalışanlarına ve

Kitabımızın hazırlanmasına ve tüm çalışmalarımıza özveri ile destek olan ailelerimiz;

Meliha, Ercan ve Olcay KALIPSIZ;

İsmail, Güler ve Ömer BUHARALI;

Nigar - Erdoğan BİRİCİK, Gökben - Orhan ve Gökhan KAYA' ya

Teşekkürlerimizi sunarız.

İÇİNDEKİLER

ÖNSÖZ	11
Şekil-Tablo Listesi	12
Kısım I SİSTEM VE BİLGİ SİSTEMLERİ	15
Bölüm 1. Sistem	17
1.1. Sistemin Özellikleri	17
1.2. Genel Sistem Teorisi	19
1.3. Sistem ve Ortam	20
1.4. Bir Sistem Örneği Olarak Hastane Yapısı	21
1.4.1. Hastane Yönetim Denetimi Sistemleri	23
1.5. Özet	24
1.6. Sorular	24
Bölüm 2. Bilgi Sistemleri	25
2.1. Bilgi Sistemi Tipleri	25
2.1.1. İşlemsel Bilgi Sistemleri (İBS)	26
2.1.2. Yönetim Bilgi Sistemi (YBS)	27
2.1.3. Ofis Otomasyon Sistemleri (OOS)	28
2.1.4. Karar Destek Sistemleri (KDS)	28
2.1.5. Üst Yönetim Bilgi Sistemleri (ÜBS)	29
2.1.6. Bilgi Tabanlı İş Sistemleri (BTİS)	30
2.2. Bilgi Sistemleri Arasındaki İlişkiler	30
2.3. Özet	32
2.4. Sorular	32
Bölüm 3. Bilgi Sistemi Tarafları	33
3.1. Kullanıcı	33
3.2. Yönetici	33
3.3. Programcı	34
3.4. Bilgi Sistem Destek Personeli (Operasyonel Personel)	34
3.5. Sistem Analisti	34
3.5.1. Sistem Analistinin Beceri ve Görevleri	34
3.5.1.1. Analitik Düşünme Bilgi ve Beceri	36
3.5.1.2. Teknik Bilgi ve Beceri	36
3.5.1.3. Yönetim ve İş Bilgi, Beceri	37
3.5.1.4. İnsan İlişkileri	37
3.6. Özet	38
3.7. Sorular	38

Kısım II	BİLGİ SİSTEMİ GELİŞTİRME SÜRECİ	39
Bölüm 4.	Sistem Geliştirme Süreci ve Modelleri	41
4.1.	Klasik Süreç	41
4.2.	Model Oluşturma	42
4.3.	RAD Model	43
4.4.	Evrimsel Süreçler	44
4.4.1.	Artımlı Model	44
4.4.2.	Spiral Model	44
4.5.	Aşırı Programlama (Extreme Programming - XP)	45
4.6.	Özet	47
4.7.	Sorular	48
Bölüm 5.	Ön İnceleme ve Fizibilite Çalışması	49
5.1.	Fizibilite Tipleri	49
5.1.1.	Teknik Fizibilite	49
5.1.2.	Ekonomik Fizibilite	49
5.1.3.	Zaman Fizibilitesi	51
5.1.4.	Sosyal Fizibilite	53
5.1.5.	Yönetim Fizibilitesi	53
5.1.6.	Yasal Fizibilite	53
5.2.	Sisteme Yönelik Farklı Çözüm Önerilerinin Değerlendirilmesi	53
5.3.	Özet	56
5.4.	Sorular	56
Bölüm 6.	Sistem Analizi -Gereksinim Analizi-	57
6.1.	Sistem Analizi Yöntemleri	58
6.2.	Araştırma Metotları	60
6.2.1.	Kişisel Görüşme Yöntemi	60
6.2.2.	Anket Yöntemi	62
6.2.3.	Araştırma Raporu	65
6.3.	Özet	65
6.4.	Sorular	65
Bölüm 7.	Sistem Analizi -Gereksinim Analizi- Modeli	67
7.1.	Veri Akış Diyagramı	68
7.1.1.	Veri Akış Diyagramı Kuralları	69
7.1.2.	Veri Akış Diyagramı Düzeyleri	70
7.2.	Varlık İlişki Diyagramı	73
7.3.	Karar Tabloları	74
7.4.	Veri Sözlüğü	76
7.4.1.	Veri Akış Sözlük Girişi	76

7.4.2. Veri Deposu Sözlük Girişi	77
7.4.3. Veri Yapısı Sözlük Girişi	77
7.4.4. Veri Elemanı Sözlük Girişi	78
7.4.5. İşlem Sözlük Girişi	78
7.5. Özet	79
7.6. Sorular	79
Bölüm 8. Sistem Tasarımı	81
8.1. Genel Tasarım	82
8.1.1. Sistem Altyapısını Belirleme	82
8.1.2. Sistem Mimarisi	87
8.2. Ayrıntılı Tasarım	91
8.2.1. Çıktı Tasarımı	91
8.2.2. Girdi Tasarımı	92
8.2.3. Veri (Yapısı) Tasarımı	95
8.2.4. Kullanıcı Arabirim Tasarımı	96
8.3. Özet	98
8.4. Sorular	98
Kısım III NESNEYE YÖNELİK YAKLAŞIMLA SİSTEM ANALİZİ VE TASARIMI	99
Bölüm 9. Nesneye Yönelik Yaklaşımın Genel Bakış	101
9.1. Özet	102
9.2. Sorular	102
Bölüm 10. Nesneye Yönelik Temel Kavramlar	103
10.1. Sınıf	103
10.1.1. Özellikler	104
10.1.2. İşlemler	105
10.2. Sınıflar Arası İlişkiler (Relations)	105
10.2.1. Bağlantı (Association)	105
10.2.2. Kümeleme (Aggregation) ve Oluşum (Composition)	106
10.2.3. Genelleştirme (Generalization)	107
10.2.4. Çok Şekillilik (Polymorphism) ve Soyut Sınıf (Abstract Class)	107
10.3. Özet	108
10.4. Sorular	109
Bölüm 11. Nesneye Yönelik Sistem Analizi	111
11.1. Kullanım Senaryosu Modellemesi (Use-Case Modeling)	112
11.1.1. Aktörler (Actors)	112
11.1.2. Kullanım Senaryoları	113

11.1.3. Kullanım Senaryolarında Sözleşmeler (Contracts)	115
11.2. Sınıf Modellemesi	115
11.2.1. Özelliklerin Belirlenmesi	116
11.2.2. Bağlantıların Belirlenmesi	117
11.2.3. Sınıf Diyagramı	117
11.3. Özet	119
11.4. Sorular	119
Bölüm 12. Nesneye Yönelik Sistem Tasarımı	121
12.1. Tasarım Kalıpları	121
12.1.1. Yaratıcı (Creator)	122
12.1.2. Bilginin Uzmanı (Expert)	122
12.1.3. Az Bağımlılık (Low Coupling)	122
12.1.4. Denetçi (Controller)	123
12.1.5. İyi Uyum (High Cohesion)	123
12.1.6. Çok Şekillilik (Polymorphism)	123
12.1.7. Yapay Sınıf (Pure Fabrication)	124
12.1.8. Dolaylılık-Arabirim (Indirection)	124
12.1.9. Değişimlerden Korunma (Protected Variation)	125
12.2. Etkileşim Diyagramları	125
12.2.1. Mesaj Sıra Numaraları	126
12.2.2. Kendine Mesaj	127
12.2.3. Koşullu Mesajlar	127
12.2.4. Döngüler	129
12.3. Kullanım Senaryolarının Gerçeklenmesi	131
12.3.1. Başlangıç İşlemleri	134
12.3.2. Görünürlük Tipleri	134
12.4. Tasarım Sınıf Diyagramı	135
12.5. Özet	136
12.6. Sorular	136
Kısım IV SİSTEM GERÇEKLEŞTİRME	139
Bölüm 13. Kodlama	141
13.1. Programlama Dilinin Seçimi	142
13.2. Yapısal Kodlama	143
13.3. Kodlama Biçimi	143
13.4. Özet	144
13.5. Sorular	144

Bölüm 14. Yazılımda Kalite ve Test Teknikleri	145
14.1. Yazılım Kalitesi Faktörleri	145
14.2. Kalitenin Güvence Altına Alınması	145
14.3. Yazılım Kalitesinin Sağlanması	146
14.4. Yazılımda Kalite Kontrolünün Planlanması	147
14.5. Yazılımın Sınanması	148
14.5.1. Yazılım Sınama Basamakları	148
14.5.1.1. Ünite Testi	149
14.5.1.2. Bütünleme Testi	150
14.5.1.3. Onaylama Testi	150
14.5.1.4. Sistem Testi	151
14.5.2. Hata ve Eksikliklerin Giderilmesi	151
14.5.3. Sınama Yöntemleri	152
14.5.3.1. Saydam Kutu Testi	152
14.5.3.2. Kara Kutu Testi	153
14.6. Özet	155
14.7. Sorular	156
Bölüm 15. Var Olan Sistemden Yeni Sisteme Geçiş	157
15.1. Kullanıma Hazırlık İşlemleri	157
15.1.1. Yeni Sistemin Eğitimi	157
15.1.2. Yeni Sisteme Mevcut Verilerin Aktarılması	157
15.2. Yeni Sisteme Geçiş Yöntemleri	158
15.3. Özet	160
15.4. Sorular	161
Kısım V İŞLETMELERDE BİLGİ SİSTEMİ UYGULAMALARI	163
Bölüm 16. İşletmelerde Bilgi Sistemlerinin Yeri	165
16.1. Organizasyonlar ve Bilgi Sistemleri Arasındaki İlişki	165
16.1.1. Bilgi Sistemlerinin Organizasyonlara Etkisi	166
16.1.2. Organizasyonlar Bilgi Sistemlerini Nasıl Etkiler	166
16.2. Bilgi Sistemleri ile Desteklenen Kurumsal İşlevler	167
16.2.1. Bilgi Sistemlerinin Stratejik Etkisi	167
16.3. İşletmelerde Uygulama Yazılımlarının Elde edilmesi ve Kullanımı	168
16.3.1. Uygulama Yazılımlarının Elde Edilmesi	168
16.3.2. Uygulama Yazılım Paketlerinin Kuruluşu ve Desteklenmesi	169
16.3.2.1. Yazılım Paketlerini Müşteriye Özelleştirme	170
16.3.2.2. Yazılım Paketlerini Bütünleştirme	170
16.3.2.3. Yazılım Paketlerini Güncelleştirme	171

16.4. Özet	171
16.5. Sorular	171
Bölüm 17. İşletmelerde Bilgi Sistemi Uygulamaları	173
17.1. Kurumsal Kaynak Planlaması	173
17.1.1. ERP ve İşletme Ortamı	173
17.1.1.1. ERP Sistemleri Gelişme Süreci	175
17.1.2. ERP Sistemlerinin Bilgi Sistemleri İçindeki Yeri	175
17.1.2.1. Kurumlarda ERP Sistemlerinin Sağlanması	176
17.1.3. ERP Özellikleri ve Fonksiyonları	176
17.1.4. ERP Sistemleri Uygulamalarında Kritik Başarı Faktörleri	177
17.1.5. ERP Sistemlerinin Organizasyonların Başarısına Katkısı	178
17.2. Tedarik Zinciri Yönetimi (SCM: Supply Chain Management)	179
17.3. Müşteri İlişkileri Yönetimi (CRM: Customer Relationship Management)	179
17.3.1. Pazarlama Bilgi Sistemlerinden Müşteri İlişkileri Sistemlerine	180
17.3.2. CRM Sistemi Yapısı	181
17.3.2.1. Yeni Ekonomi	182
17.3.3. Müşteri İlişkileri Yönetimi Sistemi Kuruluşu	182
17.3.4. CRM Uygulamalarında Başarı Faktörleri	183
17.3.5. Elektronik Müşteri İlişkileri Yönetimi (e-CRM)	183
17.4. Özet	184
17.5. Sorular	185
Kaynaklar	187
İnternet Kaynakları	188
Dizin	191

ÖNSÖZ

Ülkemizin gelişim sürecinde öncelikli alanlarımızdan olan *bilgi sistemi geliştirme projelerinin* başarılı olması, sistem analizi ve tasarımı kavram ve yöntemlerinin anlaşılmasını gerektirmektedir. Sistem Analizi ve Tasarımı derslerini alan öğrencilere ve bilgi sistemi geliştirme projesinde yer alan çalışanlara yol gösterici bir kaynak olması amacıyla hazırladığımız bu çalışmada sistem analiz-tasarımında yer alan aşamalar ve ilgili paydaşlar arasındaki ilişkiler açıklanmıştır.

Kitabımız, Yıldız Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü'nde 10 yıldır verdiğimiz Sistem Analizi ve Yönetim Bilgi Sistemleri derslerine ait notlarımızın ve bilgi sistemi geliştirme projelerinde kazandığımız deneyimlerin birleştirilmesini içeren 2 yıllık bir çalışma sonucunda hazırlanmıştır.

Çalışmamızda, sistem ve bilgi sistemleri ile ilgili temel tanımlar verildikten sonra bilgi sistemi geliştirme süreci, nesneye yönelik yaklaşımla sistem analizi ve tasarımı, sistem gerçekleştirme ve işletmelerde bilgi sistemi uygulamalarına yer verilmiştir.

“Sistem Analizi ve Tasarımı” kitabımız, bilgi sistemi modellemede geleneksel ve nesneye yönelik yeni yaklaşımları içermektedir. *Geleneksel yaklaşımlar*, veri akış diyagramları, varlık ilişki diyagramları, durum diyagramları ve ilişkisel veri tabanları işlenerek; *nesneye yönelik yaklaşımlar* ise kullanım senaryosu diyagramları, sınıf diyagramları, aktivite diyagramları, ardışıl diyagramlar ve işbirliği diyagramları kullanılarak vurgulanmıştır.

O . KALIPSIZ - A. BUHARALI - G. BİRİCİK
İstanbul, 2005

ŞEKİL LİSTESİ

Şekil - 1.1. Sistem özellikleri	17
Şekil - 1.2. Sistemde geri besleme	19
Şekil - 1.3. Bir sistem örneği: lokanta	19
Şekil - 1.4. Sistem hiyerarşisi	20
Şekil - 1.5. Hastane sistemi	21
Şekil - 1.6. Hastane sisteminin hiyerarşik alt sistemlere ayrılması	22
Şekil - 1.7. Hastane yönetim denetimi sistemi	23
Şekil - 2.1. Bilgi sistem türleri ve organizasyonlardaki bilgi sistemleri	26
Şekil - 2.2. İBS yapısı	26
Şekil - 2.3. YBS ile işlemsel bilgi sistemleri arasındaki veri ve bilgi alışverişi	27
Şekil - 2.4. Karar destek sistemi elemanları	29
Şekil - 2.5. Üst yönetim bilgi sistemi yapısı	29
Şekil - 2.6. Bilgi sistemleri sırasında veri etkileşimi ve bilgi alışverişi	31
Şekil - 3.1. Analistin problem çözümüne yaklaşımı	35
Şekil - 3.2. Sistem analistinin faydalandığı kaynaklar	36
Şekil - 4.1. Sistem geliştirmede klasik süreç	42
Şekil - 4.2. Model kurma yoluyla yazılım geliştirme aşamaları	43
Şekil - 4.3. Spiral model	45
Şekil - 5.1. Başabaş noktası grafiği	51
Şekil - 5.2. PERT diyagramı	52
Şekil - 5.3. Gantt çizelgesi	53
Şekil - 6.1. Sistemin gereksinim analizi aşaması	59
Şekil - 6.2. Görüşme kıyaslaması	61
Şekil - 6.3. Soru türlerinin karşılaştırılması	64
Şekil - 7.1. Analiz modelinin yapısı	67
Şekil - 7.2. Taslak veri akış diyagramı örneği	71
Şekil - 7.3. Birinci düzey veri akış diyagramı örneği	72
Şekil - 7.4. İkinci düzey veri akış diyagramı	73
Şekil - 7.5. Faturalama sisteminin ER diyagramı	74
Şekil - 7.6 Karar tablosu örneği (Öğrenci burs sistemi)	75
Şekil - 7.7. Karar ağacı	75
Şekil - 7.8. Veri akış sözlük giriş örneği	77
Şekil - 7.9. Veri deposu sözlük giriş örneği	77
Şekil - 7.10. Veri yapısı sözlük giriş örneği	78
Şekil - 7.11. Veri elemanı sözlük giriş örnekleri	78
Şekil - 7.12. İşlem sözlük giriş örneği	78
Şekil - 8.1. Sistemin tasarım aşamaları	81

Şekil - 8.2. Bilgisayar donanımı ve yazılım alımı adımları	83
Şekil - 8.3. Yapısal diyagram	88
Şekil - 8.4. VTOC şeması	89
Şekil - 8.5. Genel IPO	90
Şekil - 8.6. Ayrıntılı IPO örneği	91
Şekil - 8.7. Çıktı tasarımında amaçlar	92
Şekil - 8.8. Girdi tasarımında amaçlar	93
Şekil - 8.9. Veri saklama ortamı tasarımı amaçları	96
Şekil - 8.10. Arabirim tasarımında önemli olan unsurlar	97
Şekil - 10.1. Sınıf gösterimi	103
Şekil - 10.2. Sınıf özellikleri	104
Şekil - 10.3. İşlemler, nesnelerin birlikte çalışmalarını sağlarlar	105
Şekil - 10.4. Sınıflar arası bağlantı	106
Şekil - 10.5. Kümeleme ve oluşum ilişkileri	107
Şekil - 10.6. Genelleştirme	107
Şekil - 11.1. Kahve sitesi sistemi aktörleri	113
Şekil - 11.2. Kahve sitesi sistemi için kullanım senaryosu diyagramı	114
Şekil - 11.3. Kahve sitesi sistemi için sınıf diyagramı	118
Şekil - 12.1. Kahve sitesi sistemi için kredi kartı onayı tasarımı	125
Şekil - 12.2. İşbirliği diyagramında mesaj sıra numaraları	126
Şekil - 12.3. Ardışıl diyagramda mesajların sıralı gösterimi	127
Şekil - 12.4. Etkileşim diyagramlarında kendine mesajlar	127
Şekil - 12.5. Etkileşim diyagramlarında koşullu mesajlar	128
Şekil - 12.6. Etkileşim diyagramlarında karşılıklı dışlamalı mesajlar	129
Şekil - 12.7. Etkileşim diyagramlarında çoklu nesnelere mesajlar	130
Şekil - 12.8. Etkileşim diyagramlarında döngüler	130
Şekil - 12.9. Senaryoların gerçekleşmesi	131
Şekil - 12.10. Yeni sipariş sözleşmesinin tasarımı	132
Şekil - 12.11. Ürün ekleme işleminin ardışıl diyagramı	133
Şekil - 12.12. Özel kahve siparişi işleminin ardışıl diyagramı	133
Şekil - 12.13. Kahve sitesi için başlangıç işlemlerinin işbirliği diyagramı	134
Şekil - 12.14. Kahve sitesinin tasarım sınıf diyagramı	136
Şekil - 13.1. Sistem gerçekleştirme	141
Şekil - 14.1. Yazılım geliştirme sürecinde hatanın bulunması gideri ve olasılığı	147
Şekil - 14.2. Yazılım sınama basamakları	148
Şekil - 14.3. Onaylama testi	150
Şekil - 14.4. Program döngüleri	153
Şekil - 14.5. Kara kutu testi modeli	154
Şekil - 15.1. Yeni sisteme doğrudan geçiş	159
Şekil - 15.2. Yeni sisteme paralel geçiş	159
Şekil - 15.3. Yeni sisteme dereceli geçiş	159

Şekil - 15.4. Yeni sisteme prototip ile geçiş	160
Şekil - 15.5. Yeni sisteme dağıtık geçiş	160
Şekil - 16.1. Bilgi sistemleri ile organizasyonu yeniden düzenleme	165
Şekil - 16.2. Organizasyonlar ve bilgi sistemleri arasındaki ilişki	166
Şekil - 16.3. Uygulama yazılımlarının elde edilme alternatifleri	169
Şekil - 16.4. Yazılım geliştirme yaşam döngüsünde yazılım paketi kullanımı temel kavramları	170
Şekil - 17.1. Kurumsal kaynak planlaması (ERP) sistemindeki anahtar elemanlar	174
Şekil - 17.2. Genişletilmiş ERP sistemi yapısı	175
Şekil - 17.3. CRM stratejik yapısı	180
Şekil - 17.4. Temel CRM modülleri	181
Şekil - 17.5. Yeni ekonomi faaliyetinin tanımı	182

TABLO LİSTESİ

Tablo - 2.1. Bilgi sistemi türlerinin özellikleri	30
Tablo - 4.1. Sistem geliştirme süreci adımları	41
Tablo - 5.1. Aday sistemler matrisi	54
Tablo - 5.2. Fizibilite matrisi	54
Tablo - 5.3. Fizibilite matrisi örneği	55
Tablo - 7.1. Veri akış diyagramı sembolleri	68
Tablo - 7.2. Veri akış diyagramı geliştirilmesinde uyulması gereken kurallar	69
Tablo - 8.1. Var olan sistem ile önerilen sistemin iş yüklerinin karşılaştırılması	84
Tablo - 8.2. Yazılım değerlendirme kriterleri	85
Tablo - 8.3. Satıcı değerlendirmede kriterleri	86
Tablo - 8.4. Donanım edinme yöntemlerinin karşılaştırılması	86
Tablo - 11.1. "sipariş verme" senaryo dokümantasyonu	115
Tablo - 16.1. Temel kurumsal işlemler	167

KISIM I

SİSTEM VE BİLGİ SİSTEMLERİ

Birinci kısım üç bölümden oluşmaktadır. Bu kısım, sistem kavramını ve genel özelliklerini, bilgi sistemlerini ve taraflarını tanıtmayı hedeflemektedir. Sistem ve Bilgi Sistemleri Kısımı'nda yer alan bölümler kısaca;

- Bölüm 1:** Sisteme genel bir bakışla genel kavramı, temel özellikleri ve tarafları anlatılmıştır.
- Bölüm 2:** Kullanım amaçlarına göre bilgi sistemi türleri, özellikleriyle ve aralarındaki ilişkilerle ele alınmıştır.
- Bölüm 3:** Kullanıcılar, yöneticiler, programcılar, sistem destek personeli ve sistem analisti şeklinde bilgi sistemi taraflarının rolleri açıklanmıştır. Sistem analistinin sahip olması beklenen yetenek ve gerçekleştireceği görevlere değinilmiştir.

BÖLÜM 1

SİSTEM

Sistem, bir hedef veya amacı gerçekleştirmek üzere bir arada çalışan birbiriyle ilişkili parçalardan oluşan girdi ve çıktıları olan sınırları belirlenmiş bir bütündür. Bu tanıma göre sistem, bileşenlerden oluşmakta, bu bileşenler arasında bir ilişki bulunmakta ve belli bir amaca yönelmektedir. Sistem tanımında üç temel kavram vardır:

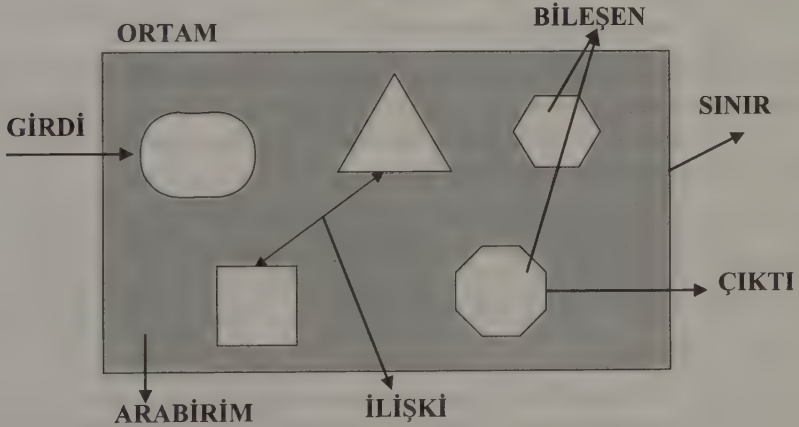
Bileşen: sistemin elemanıdır. Bileşen bir arabanın motoru gibi fiziksel olabileceği gibi bir işletmede pazarlama gibi bir faaliyet de olabilir.

İlişki: Sistemin bileşenleri arasındaki her tür ve yöndeki akış ilişki olarak adlandırılır. Bileşenler arasındaki ilişki ise zamansal, neden-sonuç, enerjinin korunumu, mantıksal veya matematiksel olabilir.

Amaç: İnsan yapısı olan her nesne belli bir gereksinimden doğmuştur. Temelde amaç bu gereksinimleri karşılamaktır. Amaçların gerçekleştirilmesi, insan-makine sistemleri yaratmak için harcanan ekonomik kaynakların ekonomik bir doğrulamasıdır.

1.1. Sistemin Özellikleri

Sistem yapısındaki temel özellikler, bileşenler, değişkenler, parametreler, ilişkiler, arabirimler, ortam, sınır, kısıtlar ve ölçütler olarak belirlenmektedir (Şekil-1.1).



Şekil-1.1. Sistem özellikleri

Bileşenler, sistemi oluşturan parçalardır.

Değişkenler, sistemin özellikleridir. Değişik koşullarda ve değişik sistem durumlarında farklı değerler alırlar. Değişkenler dört sınıfta tanımlanabilirler;

- Bağımsız ve bağımlı
- Denetlenebilir ve denetlenemez
- İçsel ve dışsal
- Girdi ve çıktı

Parametreler, analiz boyunca değiştirilemeyen sabit kalan değerlerdir. Örneğin, $y=3x+2$ gibi bir denklemde 3 ve 2 parametre iken y ve x birer değişkendirler.

İlişkiler, sistemin bileşenleri, değişkenleri ve parametreleri arasındaki bağlantılardır. Bunlar sistemin durumundaki değişimleri denetlerler. İlişkiler, sistemdeki farklı değişkenlerin değerlerinin birbirleri ile ve sistemin parametrelerinin değerleri ile nasıl ilişkili olduğunu gösterirler. İlişkiler; yapısal, işlevsel veya sırasal olabilirler.

Yapısal ilişkiler, bileşenleri ve bunların özelliklerini birbirine bağlayan ilişkilerdir.

İşlevsel ilişkiler, bileşen veya bileşenlerin davranışını belirleyen ilişkilerdir.

Sırasal ilişkiler, birbirine yalnızca zaman açısından bağımlı olan sistem olayları arasındaki ilişkilerdir. Öncelik, sonralık, eşzamanlılık gibi deyimlerle ifade edilirler.

Sınır, sistemin ortamını belirler.

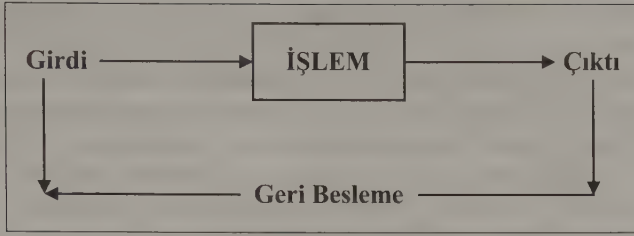
Arabirim, sistemin ortam ile veya her bir alt sistemin diğeri ile karşılaştığı yerdir. Arabirim işlevleri:

- Gizlilik
- İstenmeyen veriyi filtreleme
- Gelen giden mesajları kodlama ve kod çözme
- Ortam ile iletişimde hataları bulma ve düzeltme
- Ham veriyi özetleme veya ayrıştırlandırma

Kısıtlar, değişkenlerin değerleri veya kaynakların nasıl tahsis edileceği üzerindeki sınırlamalardır. Bu kısıtlar, tasarımcı tarafından koyulacağı gibi, sistemin doğasından da olabilir. Tasarımcı tarafından koyulan kısıtlar üzerinde oynanabilmesi, bunların sıkılaştırılıp gevşetilmesi mümkündür.

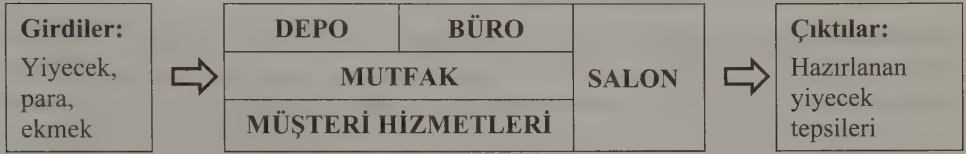
Ölçüt, yargılama standardı olarak tanımlanabilir. Ölçüt fonksiyonu ise sistemin hedeflerinin veya amaçlarının ve bunların nasıl değerlendirileceğinin bir durumudur. Buna göre ölçüt fonksiyonunun iki büyük önemi vardır. Birincisi, modelin tasarımı ve işletilmesi üzerinde büyük etkisi vardır. İkincisi, ölçütün yanlış tanımlanması, yanlış sonuçlar verecektir.

Ayrıca sistemler, çıktıları kontrol etme ve ölçme değerlendirme yoluyla girdiler ve sistem üzerinde iyileştirme yapmak için geri beslemeye (*feedback*) sahiptirler. (Şekil-1.2.)



Şekil-1.2. Sistemde geri besleme

Ortam, sistemle etkileşimli olan sistemin dışında kalan her şeydir. Örneğin Şekil-1.3'te müşteriler, yiyecek dağıtım şirketleri ve bankalar bu sistemin ortamını oluşturmaktadırlar.



Şekil-1.3. Bir sistem örneği: lokanta

1.2. Genel Sistem Teorisi

Genel sistem teorisiyle sistemlerde belirlenen özellikler, bilgi sistemlerinin yapısını da anlaşılmasına olanak sağlamaktadır.

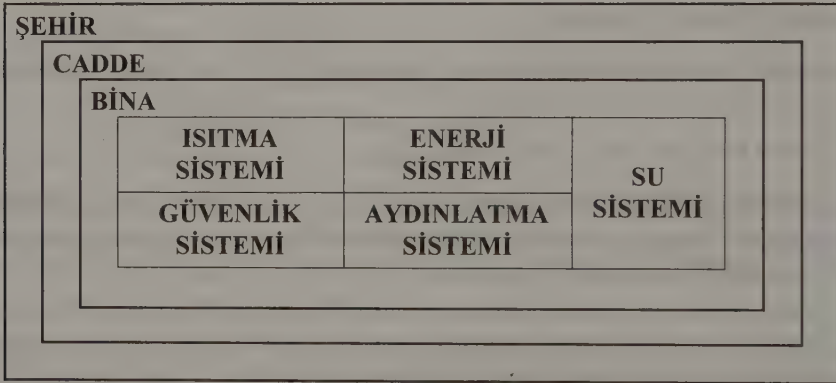
Genel sistem teorisinin başlıca özellikleri;

- Sistemler, girdileri çıktılara dönüştürür. Bilgi sistemi de veriyi bilgiye dönüştürür.
- Sistemler disiplinler arasıdır. Bir bilim dalında bulunan ürün, kural ya da yöntem başka bir bilim dalında kullanılabilir.
- Sistem elemanları arasında etkileşim vardır. Parçalar arasındaki iç bağımlılık nedeniyle sistemdeki parçalarda gerçekleşen bir işlem diğerlerini de etkilemektedir. Bir parçaya uygulanan bir işlem sırasında tüm sistem elemanları gözönünde tutulmalıdır. Örneğin patates kurtlarından kurtulmak için yapılan ilaçlamada kullanılacak kimyasal ilaçların insanlara olan yan etkisi ve toprağa etkisi gözönünde bulundurulmalıdır;
- Sistemler farklı elemanlardan oluşur. Örneğin taşımacılık sistemi genelinde demiryolu ve havayolu farklı alt sistemlerdir.
- Sistemler hiyerarşiktir. Sistemler alt sistemlerden oluşur. Sistemlerin üstünde de daha büyük sistemler vardır.
- Sistem ortama göre düzenlenmelidir. Bu özellik, entropi kavramı ile açıklanmaktadır. Entropi, sistemin ortam ile ilişkisini belirler.
- Sistem, amaç yönelimlidir. Her sistemin belirli bir amacı vardır.

Sistemin özellikleri gözönünde bulundurularak Şekil-1.3’de bir lokantanın sistem anlayışı içinde tanımı yapılmıştır. Her işletmede olduğu gibi lokantanın da en önemli girdisi sermayesidir. Lokantada yapılan yatırımla ürün (lokantadaki ürün yemekler) çıkarmak için gerekli olan malzemeler de bu sistemin diğer girdileridir. Satın alınan malzemelerin içinde dayanıklı tüketim malzemeleri depolanacağı gibi kullanılmak üzere doğrudan mutfağa alınan malzemelerde vardır. Mutfakta pişirilen yiyecekler salonda müşteriye sunulur. Salon, bu işletme için hem müşteri siparişlerinin alındığı hem de hizmetin sunulduğu bölge olması nedeniyle arz ve talebin bulunduğu noktadır. Ayrıca lokanta sistemi içinde alt sistemler olarak da nitelendirilebilecek büroda lokantanın işletimine yönelik işlemler gerçekleşirken müşteri hizmetlerinde müşteriye daha iyi hizmet verebilmek için çalışılmaktadır. Sonuçta ortaya çıkan ürün ise müşterinin talebine göre pişirilip sunulan yemeklerdir. Lokanta sisteminin sınırı lokantanın alanıdır.

1.3. Sistem ve Ortam

Sistemlerin içinde bulunduğu, işlev ve performansını etkileyen bir ortam vardır. Bu ortamın kendisi bir sistem olabilir. Bir sistem başka bir sistemin parçası ise alt sistemdir. Örneğin Şekil-1.4’de gösterilen büro sisteminin ısıtma sistemi, enerji sistemi, su sistemi, güvenlik sistemi ve aydınlatma sistemi bina sisteminin alt sistemlerini oluşturmaktadır. Bina sisteminin kendisi de içinde bulunduğu şehrin herhangi bir caddesinde yer alması nedeniyle aslında bir alt sistemdir.



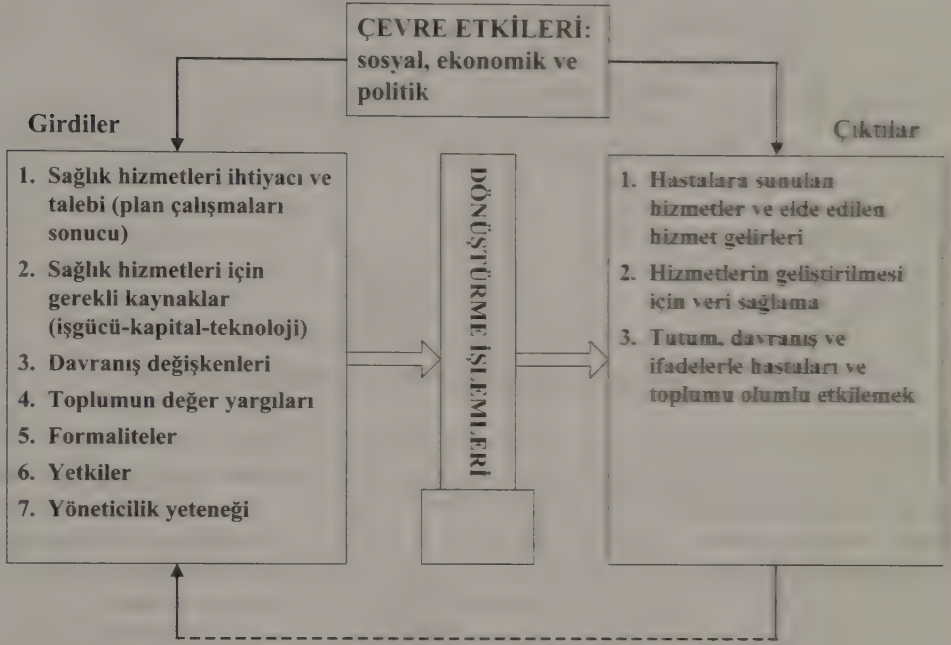
Şekil-1.4. Sistem hiyerarşisi

Sistem ortamının sistem mühendisleri tarafından anlaşılmasını gerektiren iki önemli neden vardır:

- Bazı durumlarda sistemin var olma nedeni ortamında değişiklik yapmaktır. Örneğin ısıtma sistemi ortamın ısınısını arttırmak veya azaltmak üzere vardır. Bir sistemin izlenmesi ortama bağlıdır.
- Bir sistem ve ortamı arasında doğrudan ya da genellikle dolaylı bir ilişki vardır. Örneğin bir binanın elektrik sisteminin, caddedeki elektrik kablosundaki kesintiden etkilenmesi gibi içinde bulunduğu bina dışında meydana gelen değişikliklerden de etkilenmektedir.

1.4. Bir Sistem Örneği Olarak Hastane Yapısı

Çağdaş yönetim biliminde, bir işletme veya organizasyona, sistem yaklaşımı açısından bakılmaktadır. Bu anlayışa göre, hastane de bir sistem olarak görülmektedir (Şekil-1.5).

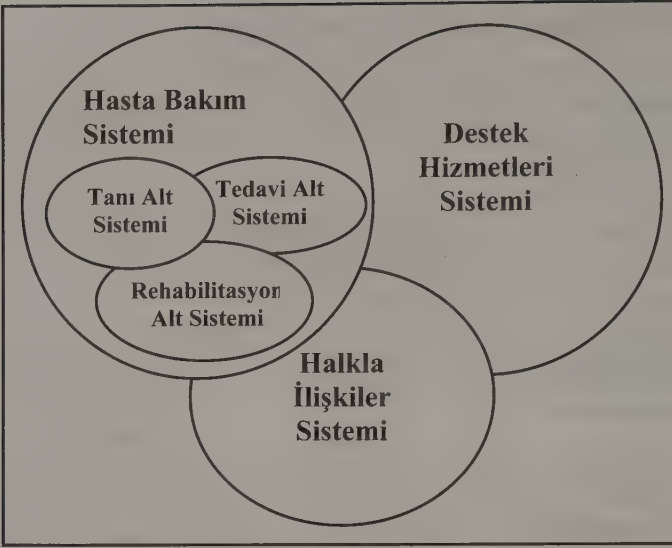


Şekil-1.5. Hastane sistemi

Burada mekanik, beşeri ve insan-makine sistemleri, hastanedeki dönüştürme işlemlerinin birer bileşeni ya da alt sistemleri olarak kabul edilebilir.

Hastane sisteminin işleyişi çok karmaşıktır. Bu durumda, hiyerarşik yapıya göre, basamaklar halinde, alt sistemlere ayırmak gerekmektedir. Örneğin, hastane sistemi, hasta bakımı sistemi, destek hizmetleri sistemi ve halkla ilişkiler sistemi olarak, üç alt sisteme ayrılabilir. Şekil-1.6'da gösterildiği üzere hasta bakımı alt sistemi de tanı-tedavi ve rehabilitasyon alt sistemleri halinde bölünebilir.

Hastane sistemi sürekli bir değişim halinde ve devingen olmasına karşın, yine de değişen koşullara uyum göstererek, dengelenmesi gerekmektedir. Değişen koşullarla uyum sağlayamayan bir sistem işlevlerini yerine getiremez ve verimli çalışmaz. Hastanedeki olaylar ve işlemler çok çeşitli ve değişkendir. Örneğin, hastaneye başvuran günlük hasta sayısı ve her birine yapılacak işlemler büyük farklılıklar gösterir. Bu nedenle, hastane sistemleri "olasılık" (probabilistik) niteliktedir.



Şekil-1.6. Hastane sisteminin hiyerarşik alt sistemlere ayrılması

Hastane sistemi ve bazı alt sistemlerinde “geri besleme” (*feedback*) eylemi de söz konusudur. Örneğin, hasta kabulü sistemi: “girdiler-dönüştürme işlemi-çıktılar”dan oluşan basit bir sistemdir. Girdi olarak; doktor ve ilk yardımdan gelen talepler, boş yatak durumu, yardımcı personel, araç-gereç ve malzeme yeterliliği söz konusudur. Dönüştürme işleminde; kabul memurlarının bu girdilere ve hastadan alınan bilgilere göre karar vererek, kabulü halinde yatağının belirlenmesi ile tanımlanmaktadır. Sistemin çıktısı; hastanın yatırılması ya da reddedilmesidir. Hastanın yatırılması halinde bu çıktı, hasta bakımı-tanı-tedavi-laboratuar gibi, hastanenin diğer alt servislerine girdi olarak işlem görecektir. Buna karşılık, “boş yatak sisteminde; çıktı olarak bir hastaya yer gösterince, sistemi yeni durumuna göre ayarlamak üzere, bu verinin geriye besleme yolu ile sisteme girdi olarak da vermek gerekmektedir. Böylece, geri beslemeli bir sistem; “girdiler-dönüştürme işlemi-çıktılar-geri besleme” aşamalarından oluşmaktadır.

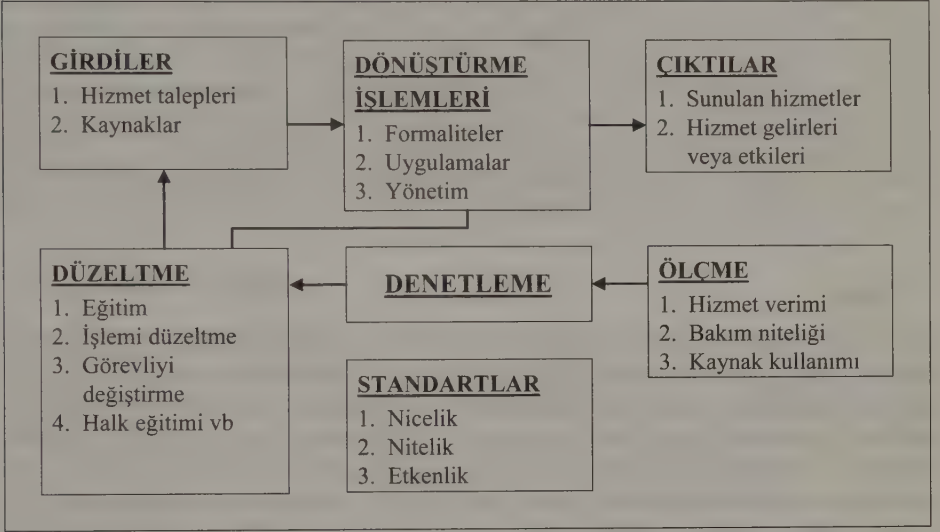
Tam otomatik aygıtlar dışındaki tüm hastane sistem ve alt sistemleri, çeşitli dış etmenlere karşı açıktırlar. Bu etmenler genel olarak; sosyal-ekonomik-politik-doğal çevre olarak gruplandırılabilir. Fakat çok çeşitli ve değişken oldukları gibi, birbirlerini de etkilemektedirler. Örneğin, hasta kabul sistemini; başvuran hastaların özellikleri, görevlilerin niteliği ve davranışları, hastanenin düzeyi, başvuranların ve görevlilerin gelir düzeyleri üst yöneticilerin politik etkileri, büroanın durumu ve çalışma ortamı vb. koşullar önemli ölçüde etkileyebilmektedir.

1.4.1. Hastane Yönetim Denetimi Sistemleri

Denetim işlevinin amacı, var olan durumun istenilen biçimde olup olmadığını araştırmak ve düzeltici önlemler almaktır. Bu tanıma göre denetim işlevi; üç aşamada gerçekleştirilmektedir:

- istenilen durumu gösteren standartların oluşturulması,
- mevcut durumun ölçülmesi,
- standartlardan sapmaların belirlenmesi ve düzeltici önlemlerin alınması.

Şekil-1.7’de hastane sistemine denetim işlevinin eklenmesiyle genelleştirilmiş, kendini kontrol edebilen (sibernetik) bir hastane sistemi gösterilmektedir.



Şekil-1.7. Hastane yönetim denetimi sistemi

Denetim sistemi, çıktıları ölçme, standartlarla karşılaştırma, düzeltme işlemlerinden oluşmaktadır. Ölçme işlemi, kurumca sunulan hizmetlerin ve harcanan kaynakların sürekli olarak nitelik ve nicelik bakımlarından değerlendirilerek, veri halinde toplanmasıdır. Sonra bu veriler, kurumun üretim ve hizmet amaçlarına göre önceden saptanmış nitel ve nicel standartlar ile karşılaştırılmakta ve denetlenmektedir. Ölçü ve değerlendirme sonuçlarının standartlardan sapma göstermesi halinde gerekli değişiklik ve düzeltmeleri yapmak amacı ile çalışmalar yürütülmektedir. Bu çalışmalar, hastane görevlilerinin eğitimi, halk eğitim programları, sağlık hizmetleri sunumunda işlem değişiklikleri, görevlilerin değiştirilmesi ve verimlerinin artırılması, disiplin önlemleri vb. konularını içermektedir.

“Toplam kalite kontrolü” adı verilen denetim işlevinin gerçekleştirilebilmesi için görülen hizmetlerin niteliği ve etkinliği bakımından standartlar konulması gereklidir. Hastanede sunulan hizmetler için standart geliştirmek kolay değildir. Bunun için

hastane yöneticileri, doktorlar ve diğer görevlilerin katılımıyla yoğun bir çalışma yapmak ve düşünce yürütmek gerekmektedir. Standart geliştirmede çeşitli yollar izlenebilir. Bu amaçla kurum içerisinde yöneticilerden, doktorlardan ve sistem bilimcilerden oluşan bir komisyon kurulabilir.

Yönetim denetiminin gerçekleştirilmesinde standartların yeri ve önemi bir hastane laboratuvarı sibernetik sistemi örneğinde daha belirgin olarak görülmektedir. Burada da girdiler - dönüştürme işlemleri - çıktılardan oluşan bir sistem ile bu sisteme bağlı bir denetim sistemi söz konusudur (Şekil-1.7).

Girdiler: Laboratuvara gelen test talepleri ile laboratuvarında bulunan kaynaklar (görevliler-donanım ve malzeme, teknoloji) Dönüştürme işlemleri: Örnek alma, test yapma, rapor yazma. Çıktılar: Test raporları ve özetleri, istatistik raporlar ve maliyet hesabı olarak sayılabilir. Denetim Sistemi: Ölçme (*sensor*), denetleme (*monitor*) ve düzeltme (*control process*) işlemlerinden ve standartlardan oluşmaktadır. Ölçme işlemlerinde; çeşitlerine göre test sayıları, nitelik denetimi verileri, kaynak kullanımı Denetleme işleminde; ölçüm sonuçlarının standartlarla karşılaştırılması. Standartlar olarak; sağlık teşkilatının koyduğu nitelik ve nicelik standartları, verimlilik (gider/ yara) oranları. Düzeltme işleminde; görev içi eğitim, uygulanan politika ve yönlemlerin gözden geçirilmesi, görevlendirme ve programlamada değişiklik vb. önlemlere başvurma söz konusu olmaktadır.

1.5. Özet

Sistem, bir amacı gerçekleştirmek üzere birbiriyle ilişkili bileşenlerden oluşan girdileri ve çıktıları olan ortam ile sınırları belirlenmiş bir bütündür. Genel sistem teorisine göre sistemler, hiyerarşik, disiplinler arası, girdileri çıktılarına dönüştüren, elemanları arasında etkileşim olan yapılardır. Sistemlerin içinde bulunduğu, işlev ve performansını etkileyen kendisi de sistem olan bir ortam vardır. Geri besleme; çıktıları, kontrol etme ve ölçme değerlendirme yoluyla girdiler ve sistem üzerinde iyileştirme yapma özelliğidir. Günlük hayatımızda karşılaştığımız kurumları sistem yapısı içinde değerlendirebiliriz.

1.6. Sorular

- 1.6.1) Sistem kavramını açıklayınız, Sistem ile arakesiti olan bilim dallarını tanıttınız.
- 1.6.2) Sistem özelliklerini belirtiniz.
- 1.6.3) Sistemin arabirim özelliğinin işlevlerini tanıttınız.
- 1.6.4) Genel sistem temel özellikleri nelerdir?
- 1.6.5) Üniversiteyi bir sistem yapısı çerçevesinde tüm özellikleri dikkate alarak tanımlayınız.
- 1.6.6) Soru 1.6.5 için sistem hiyerarşisini şekil çizerek belirtiniz.

BÖLÜM 2

BİLGİ SİSTEMLERİ

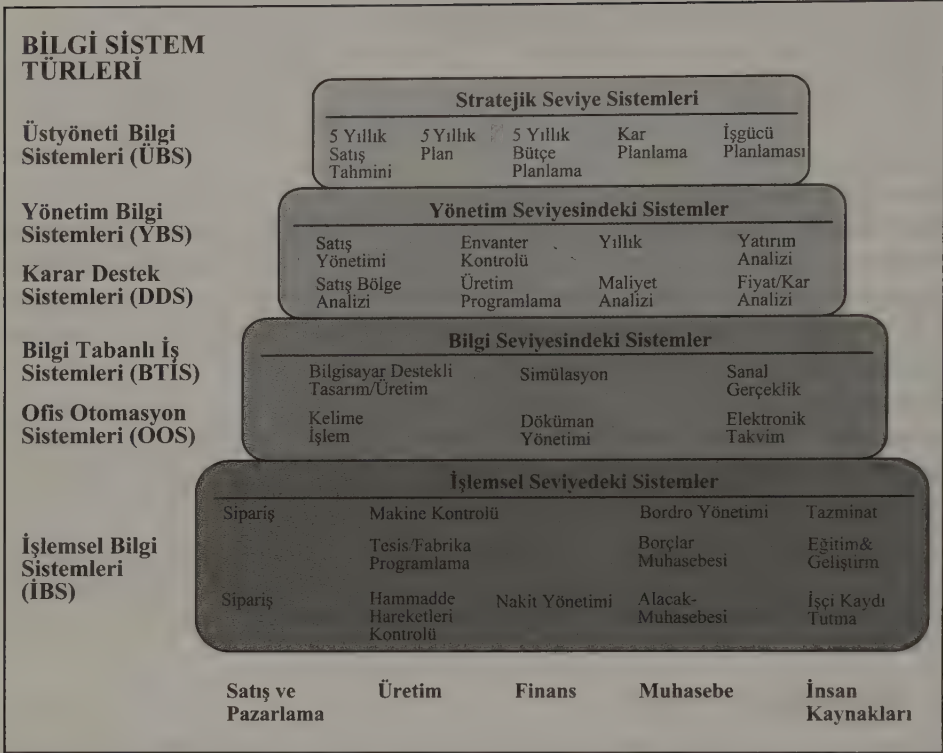
Bilgi sistemi, organizasyonda kontrolü ve koordinasyonu sağlayarak karar almada kullanılmak üzere bilgiyi toplamak, işlemek, saklamak ve dağıtmak için oluşturulan ilişkisel elemanlar kümesidir. Bilgi sistemi bunların dışında problemlerin analizinde, yeni ürünlerin oluşturulmasında ve karmaşık nesnelerin canlandırılmasında da hem yöneticilere hem çalışanlara yardımcı olur.

Bilgi sistemlerinin girişini oluşturan veri, organizasyonda ya da fiziksel çevrede oluşan olayları temsil eder ancak hiçbir şekilde kullanılabilir bir anlam ifade etmez. Ancak bir gözlem veya işlem sonucunda ortaya çıkan verilerin birbiriyle ilişkilendirilmesi ile elde edilen bilgi kullanılabilir anlamlıdır.

Bu bölümde, “Bilgi sistemleri nasıl geliştirilir?”, “Nasıl kullanılır?”, “Bileşenleri nelerdir?”, “Her bir bilgi sisteminin diğer bilgi sistemi tipi üzerinde önemi nedir?”, “Bilgi sistemi türleri nelerdir?” sorularına yanıt verilecektir.

2.1. Bilgi Sistemi Türleri

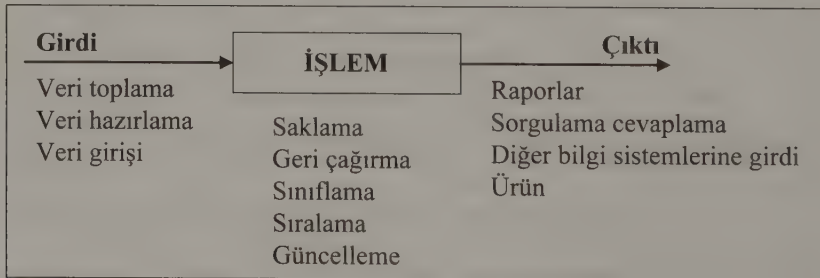
İşlemsel bilgi sistemleri (TPS: *Transaction Processing System*), yönetim bilgi sistemleri (MIS: *Management Information System*), ofis otomasyon sistemleri (*Office System*), karar destek sistemleri (DSS: *Decision Support System*), üst yönetim bilgi sistemleri (EIS: *Executive Information System*) ve bilgi tabanlı iş sistemleri (KWS: *Knowledge Work System*) olmak üzere altı temel bilgi sistemi türü vardır. Organizasyonda kullanıldıkları seviyelere ve destekledikleri işlemlere göre bilgi sistem türleri Şekil-2.1’de gösterilmektedir.



Şekil-2.1. Bilgi sistem türleri ve organizasyonlardaki bilgi sistemleri [LOUDON-2003]

2.1.1. İşlemsel Bilgi Sistemleri (İBS)

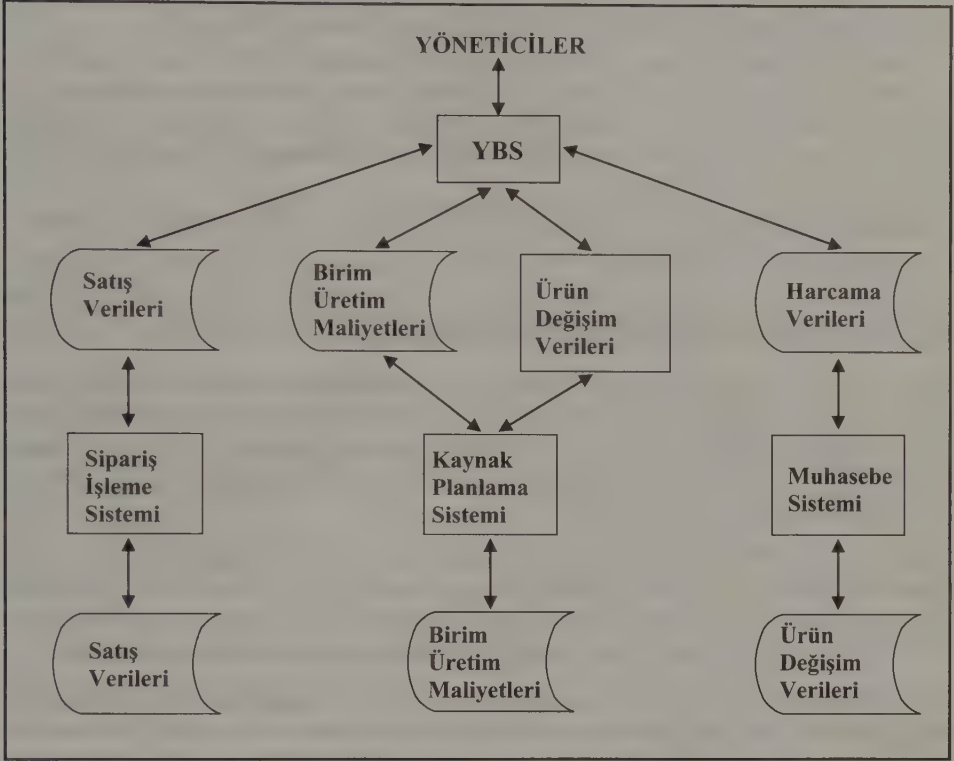
İBS, 1950'lerde başlayan bilgi işlem sistemi kavramının gelişmişidir. İBS, diğer bilgi sistemi tiplerine giriş ve temel oluşturmak amacıyla işlenmiş veriyi sınıflayan, saklayan, bakımını yapan, değiştiren, güncelleştiren ve geri çağıran bilgisayar destekli sistemdir. Satış izleme, envanter düzenleme, fatura hazırlama sistemleri gibi rutin işlemleri hızlı ve doğru gerçekleştiren sistemler işlemsel bilgi sistemi örnekleridir (Şekil-2.2).



Şekil-2.2. İBS yapısı

2.1.2. Yönetim Bilgi Sistemi (YBS)

YBS, bir kurum içinde planlama, kontrol etme ve karar vermeyi desteklemek üzere yöneticilere bilgi sağlayan işlemler dizisidir.



Şekil-2.3. YBS ile işlemsel bilgi sistemleri arasındaki veri ve bilgi alışverişi

Günümüzde bir kurum sadece bir bilgi sistemine sahipse bu sistem, yönetim bilgi sistemidir. Karar destek sistemi ve ofis otomasyon sistemi YBS kapsamındadır. Diğer bilgi sistemlerinin de bulunduğu bir ortamda teknoloji ve uygulamalar arasındaki denge daha iyi kurulmaktadır.

YBS, işlem, yönetim ve işlev olmak üzere üç boyutu birleştiren bir model ile açıklanabilmektedir:

İşlem boyutu; uygulama tabanı, işlemler, veri tabanı yönetim sistemi ve veri tabanını kapsamaktadır. En yaygın YBS uygulamaları, özet ve kıyaslamalı rapor üretimi ve veri analizleri sonucunda gerçekleştirilen karar desteğidir. YBS işlemleri, işlemsel bilgi sistemi işlemleri yanısıra hesaplama, özetleme ve iletişimden oluşmaktadır. Hesaplamalar, lineer programlama, eğilim analizi veya olasılık yöntemleri ile gerçekleştirilmektedir. YBS'den farklı olarak KDS'ler daha karmaşık yöntemleri kullanarak daha özel problemlere çözüm getirmektedirler.

Yönetim boyutu; farklı organizasyon seviyelerinde bulunan yöneticilerin aktivitelerini ve gereksinimlere olan bilgi özelliklerini dikkate almaktadır. Alt yönetimin işlemsel kontrol aktivitesini güncel gerçekleştirilmesine, orta yönetimin yönetim kontrollü kaynak paylaşmasına yöneliktir. Üst yönetimde gerçekleştirilen stratejik planlama ise kurumun hedefleri ile ilgilidir. Farklı yönetim aktiviteleri için gereken bilgi; kaynağı, büyüklüğü ve güncelliği ile karakterize edilmektedir.

Stratejik planlama için dış veri kaynağı, işlemsel kontrol için ise iç veri kaynakları önemlidir. Yönetimin üst seviyelerinde özetlenmiş bilgi, daha alt seviyelerinde ise ayrıntı gerekmektedir. Üst yönetimde uzun dönem yönelim hesaplamaları ve planlamalar amacıyla eski veriler de gerekmektedir. Alt yönetimdeki işlemsel kontrol için güncel veri kullanılmaktadır.

İşlevsel boyut; üretim, pazarlama, finans ve personel gibi çeşitli kurumsal işlevler için gereken bilgi farklılığını incelemektedir.

2.1.3. Ofis Otomasyon Sistemleri (OOS)

OOS, büro işlemlerinin elektronik olarak gerçekleştirilmesini sağlayan çok işlevli bütünlüklü bilgisayar destekli sistemlerdir. Kelime işlemciler, yazı hazırlama, kopyalama, saklama, düzenleme, yazdırma gibi işlemlerde sekreter, diğer büro çalışanları ve yöneticilere kolaylık sağlamaktadır. OOS ile kurumlarda belge hazırlama ve iletişim standartları oluşturulmaktadır.

Yerel iletişim ağları, büro çalışanların hızlı haberleşmesini ve bazı birimlerin ortak kullanımı bakımından gerekmektedir. Sistem içinde İtranet olanağı ile belirli gruplar için kurum içi haberleşme grupları oluşturulmaktadır. Ofis otomasyon birimlerinin kullanım kolaylığı nedeniyle daha ergonomik tasarlanmasına çalışılmaktadır.

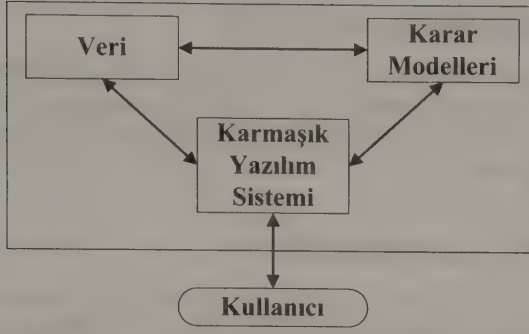
OOS, kelime işleme, tablolama, grafik çizimi paket programları, elektronik posta, faks, telekonferans gibi bilgisayar temelli iletişim sistemleri, kişisel bilgisayarlar ve büyük yer kaplayan belgelerin saklanması ve kolay geri çağırılması için mikro grafik birimlerini kapsamaktadır.

2.1.4. Karar Destek Sistemleri (KDS)

KDS, planlanmış ya da planlanmamış karar verme görevlerini desteklemek için kullanıcının veri ve karar modellerine kolay erişimini sağlayan etkileşimli bir sistemdir (Şekil-2.4). Karar ve modellere erişim karmaşık bir yazılım sistemi ile desteklenmektedir. Önceden yaratılmış modeller özel karar verme problemleri için özelleştirilmektedir.

KDS, karar verme sürecinin tüm aşamalarında destek sağlamaktadır. Bu aşamalar karar vermede gereken koşullar, karar için çözüm alternatifleri tasarlama ve özel bir alternatif seçimidir. KDS, farklı alanlardaki karar problemlerine esnek analiz yöntemleri ile çözüm getirmektedir. KDS'nin kullanım kolaylığı bulunmalıdır.

YBS bünyesinde olan karar desteği özet ve kıyaslamalı raporlardan elde edilen planlı bilgi akışı ile sağlanmaktadır. Modeller sadece belirli bir probleme cevap verecek şekilde hazırlanmıştır, esnek değildir.



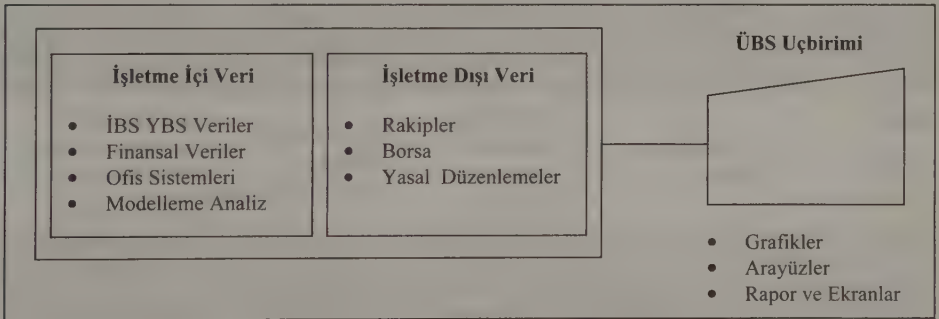
Şekil-2.4. Karar destek sistemi elemanları

Karar destek sistemleri, KDS araçları veya KDS üreticileri ile yaratılmaktadır. KDS üreticileri kullanım kolaylığı olan bir yazılım paketinde karar destek sistemi için önemli olan olanaklar dizisini birleştirmektedir. Bu olanaklar;

- işlemsel olmayan programlama dili,
- istatistiksel analizler,
- optimizasyon analizleri,
- olasılık analizleri,
- veri tabanı yönetimi,
- grafik gösterim,
- tahmin,
- finansal analizler,
- ne eğer (*what if*) analizi,
- hedef arama (*goal seeking*),
- rapor üretimi,
- gizlilik

2.1.5. Üst Yönetim Bilgi Sistemleri (ÜBS)

ÜBS (*Executive Information Systems, EIS*), organizasyonların stratejik seviyesinde kullanılır ve işletme dışı verileri kapsamak yanında, yönetim bilgi sistemleri ve karar destek sistemlerinden de işletme içi bilgileri alır. Üst yönetim bilgi sistemi, üst yönetimin, işletmenin performansını ve rakiplerin aktivitelerini takip edebilmesini, sorunları görebilmesini, fırsatları değerlendirmesini ve ileriye dönük tahminler yapabilmesini sağlar. Bu işlevleri yerine getirirken Şekil-2.5'te gösterildiği gibi hem işletme içinden hem de dışından verileri değerlendirilir.



Şekil-2.5. Üst yönetim bilgi sistemi yapısı

2.1.6. Bilgi Tabanlı İş Sistemleri (BTİS)

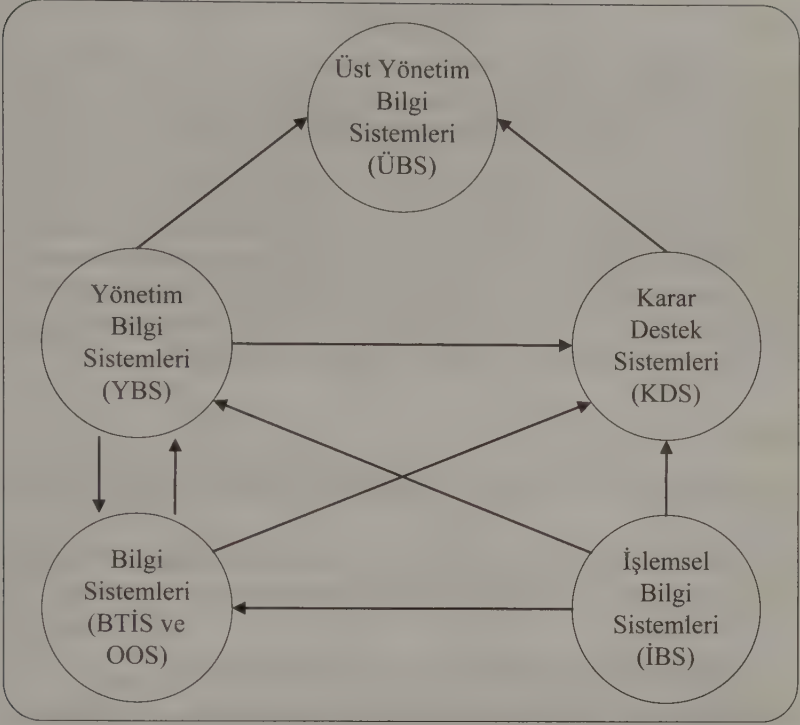
Bilgi tabanlı iş sistemi araştırmacının yerini almaz sadece araştırmanın sonuçlarını bilgi sistemiyle birleştirilerek daha verimli kullanılmasını sağlar. Bilgisayar destekli dizayn (*Computer Aided Design-CAD*) araçları bilgi tabanlı iş sistemine verilebilecek en iyi örneklerdendir.

2.2. Bilgi Sistemleri Arasındaki İlişkiler

Buraya kadar anlatılan bilgi sistemi türleri arasındaki girdi, çıktı, gerçekleştirilen işlemler ve kullanıcıları açısından farklılıklar Tablo-2.1 belirtilirken bu sistemle arasındaki etkileşim Şekil-2.6'da gösterilmektedir.

Tablo-2.1. Bilgi sistemi türlerinin özellikleri [LOUDON-2003]

Sistem	Giriş	İşlem	Çıktı	Kullanıcı
ÜBS, EIS	Dahili ve harici veri	Grafikler; simülasyonlar etkileşimli	Görsel; sorgulamalara cevaplar	Üst yönetim
KDS, DSS	Düşük yoğunlukta veri ya da analiz için optimize edilmiş veritabanları; analitik modeller ve veri analiz araçları	Etkileşimli; simülasyonlar analiz	Özel raporlar; karar analizleri; sorgulamalara cevaplar	Uzmanlar; yöneticiler
YBS, MIS	Özet kayıtlar; büyük miktarda veri; basit modeller	Rutin raporlar; basit modelleme; düşük düzeyde analiz	Özet ve istisna raporları	Orta kademe yönetim
BTİS, KWS	Tasarım özellikleri; bilgi tabanı	Modelleme; simülasyon	Modeller; grafikler	Uzmanlar; teknik personel
OOS, OFFICE SYS	Dokümanlar; planlar	Doküman yönetimi; planlama; e-posta	Dokümanlar; planlar; elektronik posta	Sekreterler; muhasebeciler; alt kademe yöneticiler
İBS, TPS	İşlemler; olaylar	Sıralama; listeleme; birleştirme; güncelleme	Detaylı raporlar; listeler; özetler	İşletim personeli; şefler



Şekil-2.6. Bilgi sistemleri sırasında veri etkileşimi ve bilgi alışverişi

Organizasyonlarda farklı amaçlar için hizmet veren farklı bilgi sistemleri bulunmaktadır. İşlemsel bilgi sistemleri, alt düzeydeki yöneticilerin satışlar, faturalar, ödemeler ve ham madde kullanımı gibi işlemlerin takip edilmesini sağlarlar. İşlemsel bilgi sistemleri, karar destek sistemleri ve yönetim bilgi sistemlerine girdi göndererek temel oluşturmaktadır. Yönetim bilgi sistemleri, sadece önceden belirlenmiş ve sistem içinde yapılandırılmış konular hakkında bilgi sağlar. Yönetim bilgi sistemlerinin bu eksikliğini karar destek sistemleri ve üst yönetim bilgi sistemleri tamamlamaktadırlar. Karar destek sistemlerinde istatistik analiz ve simülasyon gibi gelişmiş modeller kullanılmaktadır. Kurum içi ve dışı bilgileri alan üst yönetim bilgi sistemleri ise önceden tahmin edilemeyen değişken anlık sorulara cevap bulan ileriye dönük tahmin yapmayı sağlayan sistemlerdir. Üst yönetim bilgi sistemlerinde OLAP ve “Drill down” analizleri kullanılarak veri madenciliği gerçekleştirilir.

2.3. Özet

Bilgi sistemi, organizasyonda kontrolü ve koordinasyonu sağlayarak karar almada kullanılmak üzere bilgiyi toplamak, işlemek, saklamak ve dağıtmak için oluşturulan ilişkisel elemanlar kümesidir. Kullanılan temel bilgi sistemi tipleri hiyerarşik ve tarihsel gelişime göre, işlemsel bilgi sistemleri, yönetim bilgi sistemleri, ofis otomasyon sistemleri, karar destek sistemleri, üst yönetim bilgi sistemleri ve bilgi tabanlı iş sistemleridir. İşlemsel bilgi sistemleri, karar destek sistemleri ve yönetim bilgi sistemlerine girdi göndererek temel oluşturmaktadır. Yönetim bilgi sistemleri, sadece önceden belirlenmiş ve sistem içinde yapılandırılmış konular hakkında bilgi sağlar. Üst yönetim bilgi sistemleri ise önceden tahmin edilemeyen değişken anlık sorulara cevap bulan ileriye dönük tahmin yapmayı sağlayan sistemlerdir.

2.4. Sorular

- 2.4.1)** Bilgi Sistemi türlerini yazarak, Yönetim Bilgi Sistemini bir şekil yardımı ile açıklayınız.
- 2.4.2)** Aşağıdaki yönetim sorumluluklarının her birinin, Yönetim Bilgi Sistemi uygulaması içinde hangi yönetim düzeyindeki planlama veya karar olduğunu gerekçesi ile tanımlayınız.
- a) Üretim bandında çalışacak işçi sayısının belirlenmesi
 - b) Renault' un yeni bir araba tipi üretimine başlaması
 - c) İş Bankasının Beşiktaş ilçesinde bir şube daha açması
 - d) Üretim bandında hangi hata düzeyindeki ürünün reddedileceği
- 2.4.3)** Aşağıdaki analizleri açıklayarak her birinin, hangi bilgi sistemi tiplerinde kullanıldığını açıklayınız.
- a) OLAP
 - b) Simulasyon
 - c) Drill-Down
 - d) What-If
- 2.4.4)** Yönetim Bilgi Sistemi (MIS) özellikleri nedir, Üst Yönetim Bilgi Sistemi (EIS) ve Karar Destek Sistemleri (DSS) ile kıyaslayınız.
- 2.4.5)** Karar destek sisteminde kullanılan yöntem ve analizleri tanımlayınız.
- 2.4.6)** OLAP ve Data Mining (Veri Madenciliği) nedir, nerede, hangi sonuçları üretmek amacı ile kullanılır?

BÖLÜM 3

BİLGİ SİSTEMİ TARAFLARI

Sistem geliştirme yaşam döngüsü içinde farklı konumlardaki bireyler birarada çalışmaktadırlar. Birlikte çalışanların sayısı ve görevleri projenin büyüklüğüne ve türüne göre farklılık gösterse de genel olarak bir bilgi sisteminin tarafları,

- Kullanıcı
- Yönetici
- Programcı
- Bilgi sistem destek personeli
- Sistem analisti

olarak gruplandırılmaktadır.

3.1. Kullanıcı

Bir sistem kullanıcılar tarafından kullanıldığı sürece varlığını sürdürebileceği için kullanıcı, sistemin en önemli parçası durumundadır. Bu nedenle geliştirilen sistem kullanıcının ihtiyaçlarını karşılayabilmelidir. Sistem analizi süresince kullanıcıların (kullanıcı burada müşteridir) sistemden beklentileri tam olarak belirlenmeli ve sistemin nasıl başarıya ulaşacağı tespit edilmelidir.

Sistem analistleri ve tasarımcıları tarafından yapılan genel bir yanlış ile bütün müşteriler aynı kategoriye konulmamalıdır. Her kullanıcının bilgisayar bilgisi ve deneyimi aynı olmadığı gibi görev ve sorumlulukları sonucu kullanacakları sisteme yaklaşımları da farklı olacaktır.

3.2. Yönetici

Geliştirilen sistemin büyüklüğüne göre yöneticiyi;

- Proje yöneticisi
- Üst düzey (işletme) yöneticisi

olmak üzere iki farklı açıdan ele almak mümkündür.

Büyük ölçekli projelerde, projenin uzun süreli başarıya ulaşması için gerekli kriterlerin belirlenmesinden ve proje ekibinin idaresinden proje yöneticisi sorumludur.

Sistemin geliştirilmesi için gerekli olan kaynağı sağlayacak olan üst düzey yöneticisinin bilgi sistemine olan yaklaşımı sistemi kullanacak olan kullanıcılar üzerinde etkili olmaktadır.

3.3. Programcı

Yapısal analiz ve tasarımın ortaya atıldığı ilk yıllarda analist, sistem için gerekli bilgileri toplayarak sistemi geliştirip kodlayanı programcı olarak değerlendiriliyordu. Ancak zaman içinde analistin ve programcının sistem geliştirme yaşam döngüsündeki görevleri tam olarak ayrılmıştır. Zamanla yarışan ve giderek daha karmaşık yapıya sahip olan programların söz konusu olduğu günümüzde ise daha çok CASE (*Computer Aided Software Engineering*, Bilgisayar Destekli Yazılım Mühendisliği) araçları kullanarak kodun büyük bölümü otomatik olarak üretilmektedir. Ancak CASE araçlarının programcının yerini tam olarak aldığı söylenemez. Kod üreticileri sayesinde programcılar, zamanlarını optimizasyona ve üretilen kodun sisteme entegrasyonuna ayırmaktadırlar.

3.4. Bilgi Sistem Destek Personeli (Operasyonel Personel)

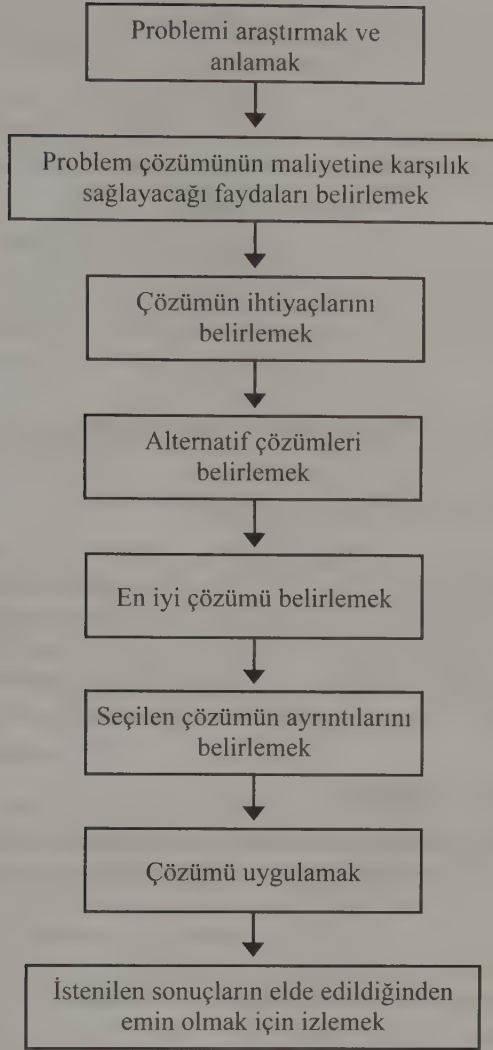
Sistemin sürekliliğini sağlamak amacıyla ağ iletişimi, donanım ve veri güvenliği, ilgili bilgisayar programlarının çalışmasından çıktıların düzenlenmesine kadar birçok konuda desteğin verilmesinden sorumludur. Sistem analisti, sistemin tarafları arasında anahtar rolü üstlense de sistemin başarılı olması tek başına sistem analistine bağlı değildir.

3.5. Sistem Analisti

Adından anlaşılacağı üzere sistem analisti, sistem geliştirme projesinin kilit üyesidir. Bu nedenle sistem analistinin görevleri ve yetenekleri bu bölümde ayrıntılı olarak anlatılmıştır.

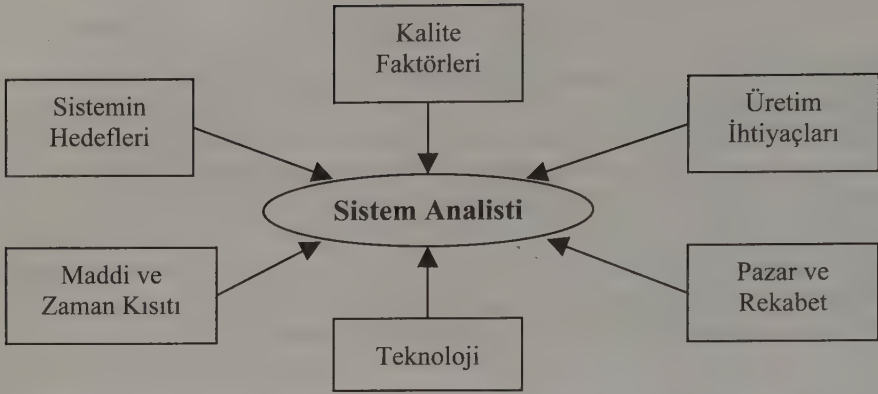
3.5.1. Sistem Analistinin Beceri ve Görevleri

Genel olarak sistem analistinin hem işletme yönetimi hem de bilgi sistemleri konusunda bilgi sahibi olması beklenir. Böylece sistem analisti, işletmenin karşılaştığı sorunları ve karşılaştığı iş fırsatlarını fark ederek bilgi sistemindeki ihtiyaçların belirlenmesini ve iş akışının oluşturulacak bilgi sistemine yansımaları kolayca sağlamaktadır. Sistem analistleri, işin ihtiyaçlarını ve sorunlarını ortaya koyarak, işin gelişmesi için bilgi teknolojilerini kullanılarak bilgi sisteminin nasıl uygulanacağına karar vermektedirler. Sistem analistinin öncelikli görevi mevcut problemleri fark edip problem çözümüne yönelik en uygun metodolojiyi kullanmaktır (Şekil-3.1).



Şekil-3.1. Analistin problem çözümüne yaklaşımı [SATZINGER ve ark.-2002]

Sistem analisti, çözümü ortaya koyarken müşterinin ihtiyaçlarını ve isteklerini belirlemelidir. Bütün bu işlemleri gerçekleştirmek için sistem analistinin yararlandığı kaynaklar arasında; teknolojinin son durumu, mevcut pazar ve bu pazardaki rekabet koşulları, üretim ihtiyaçları, mali ve zaman kısıtları yer almaktadır (Şekil-3.2). Ancak bu kaynaklardan verimli şekilde yararlanmak için sistemi analiz eden ve tasarlayanın, teknik ve iş bilgisinin yanı sıra analitik düşünme ve insan ilişkilerinde başarılı olması gibi farklı kabiliyetlere de sahip olması gerekmektedir. Geliştirilecek olan bilgi sisteminin bireyler tarafından kullanılacağı düşünülürse sistem analistinin, kullanıcıların işlerini nasıl yaptıklarını ve davranış biçimlerini tespit etmesi gerekmektedir. Sistem analistinin becerileri alt başlıklar altında incelenmektedir.



Şekil-3.2. Sistem analistinin yararlandığı kaynaklar [PRESSMAN-1987]

3.5.1.1. Analitik Düşünme Bilgi ve Beceri

Sistem geliştirme yaşam döngüsü problemin tanımla ile başladı ğı için, sistem analistinin öncelikle problemi tam olarak belirleyip tanımlayabilmesi gerekmektedir. Büyük resmi kaybetmeden sistemin parçalarını inceleyip aralarındaki ilişkiyi ortaya koymak tamamıyla analistin başarısına bağlıdır. Sistem analistinin görevi sadece problemi belirlemekle sınırlı değil, çözüm alternatiflerini belirleyerek en uygununun seçilmesini sağlamak ve bilgi teknolojilerini kullanarak sisteme uyarlayabilmektir.

3.5.1.2. Teknik Bilgi ve Beceri

Hiç kimsenin teknolojinin her alanında bilgi sahibi olması beklenemez ancak bilgi sistemi geliştirecek olan bir sistem analisti aşağıdaki konuların temeli hakkında bilgi sahibi olmalıdır:

- Donanım
- Programlama dilleri
- İşletim sistemleri
- Veri tabanı yönetim sistemleri
- Haberleşme protokolleri

Sistem içinde yer alacak farklı teknolojilerin ne için, nasıl kullanılacağı nın ve sisteme nasıl uyum sağlayacağı nın bilinmesi gerekir. Ayrıca sistem analistinin, sistem geliştirme için kullanılan araçlar ve teknolojiler hakkında da bilgi sahibi olması gerekmektedir. Bunlardan bazıları,

- CASE araçları (*Rational Rose, Visual Analyst vb.*),
- Sistem geliştirme için kullanılan yazılım paketleri (*Microsoft Access, VisualAge ve PowerBuilder vb.*),
- Bütünleşik geliştirme ortamları (*Integrated Development Environments-IDE*) (*Sun Forte Java, Microsoft Visual Studio .Net vb.*),

Bu araçlar, program üretimi, test, konfigürasyon yönetimi, yazılım kütüphanesi yönetimi, dokümantasyon desteği ve proje yönetimini sağlamaktadır.

Buraya kadar belirtilen teknik bilginin yanısıra sistem analistinin, sistem geliştirme projesinin nasıl planlanıp yönetileceği, sistem analizinin ve tasarımının, gerçekleştirilmesinin ve test işlemlerinin nasıl yapılacağı bilgisine de sahip olması gerekmektedir.

3.5.1.3. Yönetim ve İş Bilgi, Beceri

Mevcut sistemdeki sorunların tam olarak anlaşılabilmesi için sistem analistinin işin işleyişine ve organizasyon yapısına hakim olması gerekmektedir. Örneğin kurumun çalıştığı sektör, hedefleri, iş stratejileri, planları ve kurum kültürüne bağlı olarak değerleri vb. sistem analisti tarafından anlaşılmalıdır.

Sistem geliştirme sürecinde yer alan takımda sistem analistinin oynadığı rol düşünüldüğünde sistem analistinin aşağıda kısaca belirtilen yönetim bilgi ve becerisine sahip olması gerekir:

- Kaynak yönetimi
- Proje yönetimi
- Risk yönetimi
- Değişim yönetimi

3.5.1.4. İnsan İlişkileri

Sistem geliştirme yaşam döngüsü boyunca sistem analistleri, işletmenin diğer çalışanları ile takım çalışması içindedirler. Çalışanlara, onların becerilerine ve bilgi teknolojilerine karşı yaklaşımlarına dair fikir sahibi olmalıdır ki yeni sistemin adaptasyonunda karşı bir dirençle karşılaşmasın. Dolayısıyla sistem analisti, insanlarla rahat iletişim kuran, onları dinleyerek sorunlarını ve ihtiyaçlarını tam olarak ortaya koyma yeteneğine sahip olmalıdır. Ayrıca sistem analistinin organizasyonun bütün yönetim kademelerinde ve farklı pozisyonlarda çalışanlarla iletişim içinde bulunması ona tüm organizasyonu etkilemesi için önemli bir fırsat vermektedir. Bu sebeple sistem analisti, sistem geliştirme sürecinde proje ilerlemelerini belgelemeli ve diğer kişilere belirlediği toplantı planı çerçevesinde aktarmalıdır.

3.6. Özet

Bilgi sisteminin tarafları, kullanıcı, yönetici, programcı, bilgi sistem destek personeli ve sistem analisti olarak gruplandırılmaktadır. Sistem analisti, sistem geliştirme projesinin temel üyesidir. Genel olarak sistem analistinin hem işletme yönetimi hem de bilgi sistemleri konusunda bilgi sahibi olması beklenir. Böylece sistem analisti, işletmenin karşılaştığı sorunları ve karşılaştığı iş fırsatlarını fark ederek bilgi sistemindeki ihtiyaçların belirlenmesini sağlayacak ve iş akışını oluşturulacaktır. Sistem analistleri, işin ihtiyaçlarını ve sorunlarını ortaya koyarak, işin gelişmesi için bilgi teknolojilerini kullanarak bilgi sisteminin nasıl uygulanacağına karar vermektedirler. Analitik düşünme bilgi ve becerisi - teknik bilgi - yönetim ve işletmecilik bilgi, becerisi - insan ilişkileri yeteneği sistem analisti için önemli olan temel yeteneklerdir.

3.7. Sorular

- 3.7.1) Bilgi sistemi geliştirmede kullanıcıların rolü nedir?
- 3.7.2) Geliştirdiğiniz bir bilgi sistemi çalışması için problem çözümü adımlarınızı yazarak, ideal yaklaşım ile kıyaslayınız.
- 3.7.3) Sistem Analistinin sahip olması gereken yetenekleri yazınız.

Kısım II

BİLGİ SİSTEMİ GELİŞTİRME SÜRECİ

Bu kısımda sistem geliştirme süreci adımları fizibilite, gereksinim analizi ve sistem şeklinde incelenmiştir.

Bölüm 4: Sistem geliştirme yaşam döngüsünün adımları ve bu adımların düzenlenmesi için kullanılan modeller tarihsel gelişim süreci sırasına göre açıklanmıştır.

Bölüm 5: Bir bilgi sisteminin geliştirme sürecinin ilk adımları olan ön inceleme ve fizibilite çalışmasının nasıl yapılacağı anlatılmıştır.

Bölüm 6: Veri ve süreçler hakkında ayrıntıları elde etmek için gereksinim analizi yöntemi ve izlenecek stratejiler belirtilmiştir.

Bölüm 7: Geleneksel yaklaşımla sistem analizi gerçekleştirmek için kullanılan temel araçlar olan veri akış diyagramı, karar tablosu ve veri sözlüğü örneklerle anlatılmıştır.

Bölüm 8: Bir bilgi sistemini tasarlamak için gerekli bilgiler bu bölümde aktarılmıştır.

BÖLÜM 4

SİSTEM GELİŞTİRME SÜRECİ VE MODELLERİ

Bilgi sistemlerinin geliştirilmesi için izlenen sürece sistem geliştirme yaşam döngüsü denir. Farklı yaklaşımlar bu sürecin aşamalarına farklı isimler de verse temel olan adımlar, bu adımların amacı ve çıktıları Tablo-4.1’de verilmiştir:

Tablo-4.1. Sistem geliştirme süreci adımları [İnternet Kaynağı-1]

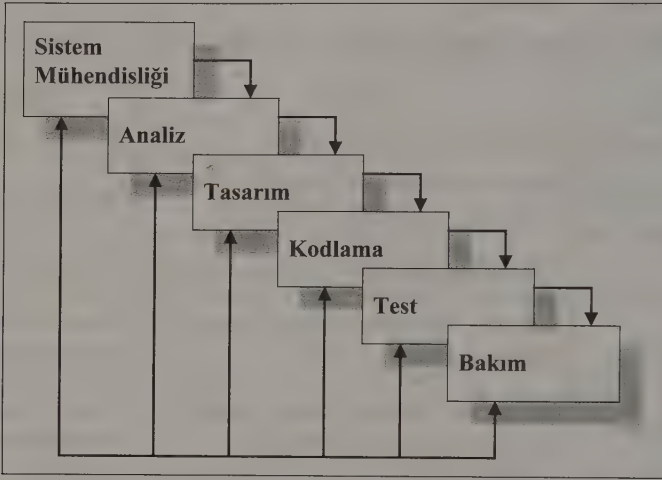
ADIM	İŞLEM	ÇIKTILAR
Problemin Tanımı	Problemi ortaya koymak	İhtiyaçlar belirlenir
Fizibilite Çalışması	Projenin kapsamı ve hedefleri ortaya konarak olabilirliğini belirlemek	Fizibilite çalışması raporu
Analiz	Problemin çözümlerini ortaya koymak	Çözümün lojik modeli
Genel Tasarım	Sistemin nasıl gerçekleştirileceğini belirleme	Sistemin maliyeti ve üst düzey tasarımı
Ayrıntılı Tasarım	Genel tasarımda belirlenen sisteme ait alt sistemlerin tanımlanması	Sistemin özellikleri ve ayrıntılı tasarım
Gerçekleştirme	Programı yazma, yükleme ve sınama	Çalışan sistem ve dokümantasyon
Bakım	Sistemin bakımını yaparak desteklemek	Çalışan Sistem

Sistem geliştirme süreci bir model halinde gösterilerek yönetimi, kaynak dağıtımı, gider denetimi ve sistem kalitesini yükseltmeye temel oluşturmaktadır. Sistem geliştirme yaşam döngüsü adımlarının sıralanması ve işlenmesine göre farklı sistem geliştirme modelleri oluşturulmuştur. Kullanılan başlıca modeller arasında *klasik süreç*, *model oluşturma*, *RAD (Rapid Application Development) modeli* ve *evrimsel süreç* yer almaktadır. Yeni bir yöntem olarak tartışılan aykırı programlamanın (XP) kullanımı da son zamanlarda artırmıştır.

4.1. Klasik Süreç

Klasik süreç (*Waterfall Modeli*), bilgisayar sistem mühendisliği olarak, ana sistemin analizinden başlayıp yazılım gereksinim analizleri, tasarımı, kodlanması, kullanım ve bakımı aşamalarında bir süreç halinde gerçekleştirilmektedir. Şekil-4.1’de gösterildiği üzere her aşamada elde edilen sonuçların rapor halinde sunulması ve gerekirse geri besleme yoluyla düzeltilmesi beklenmektedir.

Sistemin geliştirilmesinde uygulanan klasik süreç, eskiden beri yaygın olarak kullanılan bir yöntemdir. Ancak, projenin başlangıcında, müşteri gereksinimlerinin bütünü ile ortaya konmasındaki güçlükler ve eksikliklerin görülmesi; projenin çok uzun bir süreçte tamamlanabilmesi ve müşterinin beklemeye zamanı olmaması; başta yapılabilen kaba hataların bazen ancak sınama ve hatta kullanma aşamasında ortaya çıkabilmesi; uygulamada bu modeldeki sıraya tam olarak uyulmaması, yinelemeler yapılması konularında bazı güçlüklerle karşılaşmaktadır. Buna karşın yine de rasgele ve gelişigüzel bir sistem geliştirmeye kıyasla, çok tutarlı ve güvenli bir yöntem olarak sayılmaktadır.



Şekil-4.1. Sistem geliştirmede klasik süreç

4.2. Model Oluşturma

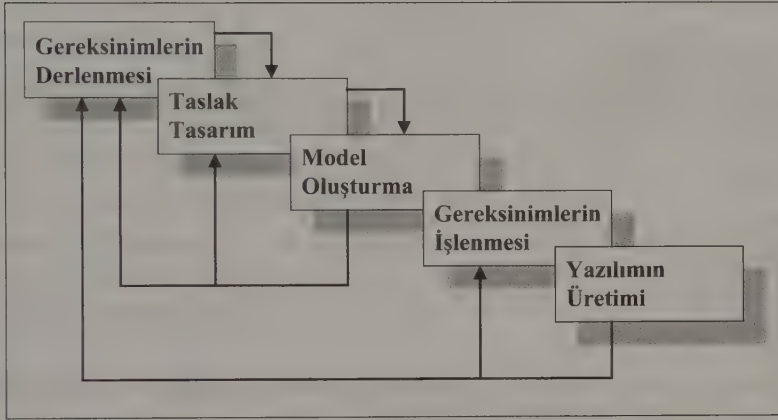
Sistem geliştirmenin klasik sürecinde müşteri, sistem konusundaki amaç, hedef ve gereksinimlerini genel hatları ile bildirmektedir. Ancak girdi, işlem ve çıktı gereksinimlerini tanımlaması beklenememektedir. Sistem analisti, bu bilgilere dayanarak var olan sistemin algoritmalarının etkenliğini, işletim sisteminin uygunluğunu ya da insan-makine etkileşim biçimini yargılamak ve yeni öneriler getirmek durumundadır. Bu halde, klasik geliştirme sürecinde söz konusu güçlükleri aşmak için, bir sistem modeli (*prototip*) oluşturmak yoluna gidilmektedir.

Bu durumda sistemin geliştirilmesi, müşterinin gereksinimlerini belirtmesi, bir tasarım taslağı hazırlanması, model oluşturma-gereksinimlerinin işlenmesi, yazılımın üretilmesi aşamalarında tamamlanmaktadır (bkz. Şekil-4.2).

Her sistem geliştirme yönteminde olduğu gibi burada da sistem analisti ile müşteri bir araya gelerek, sistemin amaç ve hedeflerini belirlemekte, bu hedeflere ulaştıracak gereksinimleri saptamaktadır. Bu bilgilere dayanarak, sistem analisti bir taslak tasarım hazırlamaktadır. Taslak tasarım, müşterinin (kullanıcının) kolayca anlayabileceği biçimde, örneğin girdi yaklaşımları ve çıktı formatları halinde düzenlenmekte ve birlikte

üzerinde tartışılmaktadır. Taslak tasarım üzerinde çalışılarak, bir sistem modeli oluşturulmaktadır. Model, sistem analisti ve kullanıcı tarafından birlikte değerlendirilerek işlenmektedir. Böylece, geliştirilecek sistemin gereksinimleri saptanmakta ve karşılanmaktadır. Bu çalışma, müşterinin bütün ihtiyaçlarının yerine getirilişine kadar yinelenerek sürdürülmektedir.

Model (*Prototip*) yardımıyla sistem geliştirme yönteminde de, modelin kurulması ve üzerinde düzeltmeler yapılması müşteri ile birlikte ve onun önünde gerçekleştirildiği için yerli yersiz müdahale etmesine ve yapılan işi küçük görmesine neden oluşu; sistem analistinin modeli oluştururken basitlik ya da kolaylık nedeni ile kullandığı algoritmalar ve işletim sistemi veya programlama dilinden daha sonra benimseyip vazgeçmemesi, bu uygunsuz öğe ve yöntemlerin de geliştirilen sistem içerisinde kalma olasılığı vb. sorunlar ileri sürülmektedir. Buna karşın, sistem geliştirilmesinde modelden yararlanma, yine de etkili bir yöntem olarak görülmektedir. Böylece, müşteri gereksinimleri ve onayı daha kısa sürede sağlanmakta ve sistem analisti kalite ve bakım konularına ağırlık verebilmektedir.



Şekil-4.2. Model kurma yoluyla yazılım geliştirme aşamaları

4.3. RAD Model

RAD model (*Rapid Application Development*), klasik süreç (*Waterfall*) modelin geliştirme süresinin kısaltılmış ve hızlandırılmış şeklidir. Hızlı geliştirme, bileşen temelli yapılanma (*Component based construction*) yaklaşımı ile gerçekleştirilir. RAD aşamaları:

- **İş Modelleme:** İş sürecinde hangi bilgiyi kim üretiyor? Üretilen bu bilgi nereye gidiyor ve kim tarafından işleniyor?
- **Veri Modelleme:** Veri kaynakları arasındaki iletişim (bilgi akışı) nasıl?
- **Süreç Modelleme:** Veri kaynaklarının, veri akışı gerektiren işlemlerin arasında tanımlanması.

- **Uygulama Üretimi:** 4. kuşak teknik modüllerinin yeniden kullanımı (*reuse*), yazılım otomatik araçlarının kullanımı.
- **Test Etme:** RAD yeniden kullanımı desteklediği için pek çok modül test edilmiştir. Test süresi kısadır, ancak yeni modüller test edilir ve test sürecinin önemli kısmı arabirim testleri için kullanılır.

RAD, yeniden kullanılabilir program bileşenlerinin geliştirilmesini vurgular. Yeniden kullanılabilirlik (*reusability*), nesne (*object*) teknolojisinin gereğidir.

4.4. Evrimsel Süreçler

Bu modeller çevrimseldir (*iteratif*). Sistemlerin daha tamamlanmış versiyonlar şeklinde hazırlanmasını sağlarlar. Evrimsel modeller:

- Artımlı (*incremental*) Model
- Spiral Model

olarak incelenebilir.

4.4.1. Artımlı Model

Prototip oluşturma sürecinin çevrimsel (*iteratif*) yaklaşımıyla *Waterfall* modelin elemanlarını birleştirir. Her artım,

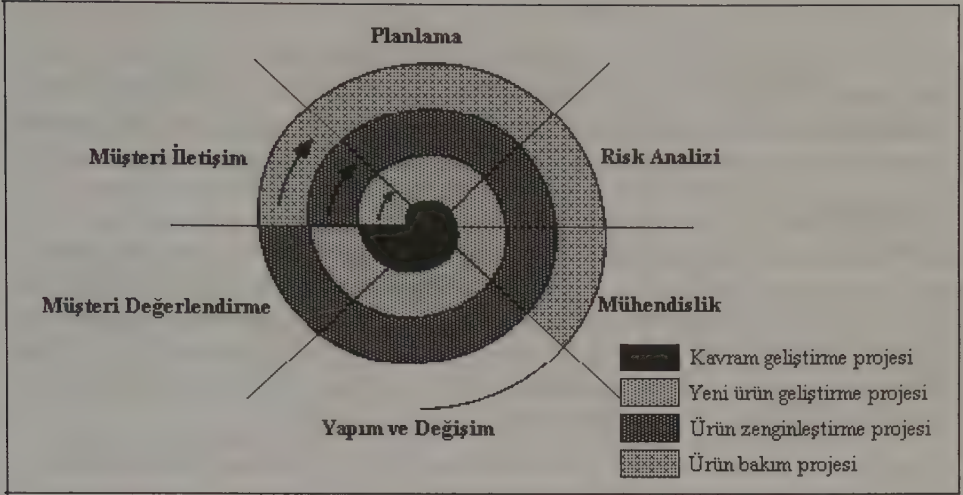
- Analiz
- Tasarım
- Kodlama
- Sınama/Test

adımlarında gerçekleştirilir.

Prototip oluşturmada her bir artımda üretilen ürün dağıtılır ve kullanıcıların değerlendirme yapabileceği bir platform sağlanır.

4.4.2. Spiral Model

Spiral Model, Boehm tarafından 1988'de geliştirilmiştir. Bu modelde de prototip oluşturma sürecinin iteratif yapısı ve *Waterfall*'un kontrollü ve sistematik yapısı birleştirilmiştir. Bu modele göre yazılım, artımlı yayımlar (*release*) dizisi olarak hızlı geliştirilir. İlk iterasyonlar prototiptir; daha sonrakiler ise dağıtımı yapılan versiyonlar şeklindedir.



Şekil-4.3. Spiral model

Spiral model Şekil-4.3'te de gösterildiği gibi altı aşamadan oluşmaktadır:

- **Müşteri İletişim:** Geliştirici ve müşteri arasındaki etkin iletişimi sağlama.
- **Planlama:** Proje için zaman ve kaynak yöntem tanımı yapma
- **Risk Analizi:** Teknik ve yönetim risklerini belirleme
- **Mühendislik:** Uygulamanın tanımının yapılması (tasarım)
- **Yapım ve Değişim (Construction & Release):** Gerçekleştirme test, kuruluş ve kullanıcı desteği (belgeleme ve eğitim)
- **Müşteri Değerlendirme**

4.5. Aykırı Programlama (Extreme Programming-XP)

Çevik yaklaşımlar, artan müşteri taleplerini daha hızlı karşılamak amacıyla geleneksel sistem geliştirme modellerine alternatif olarak ortaya çıkmıştır. Çevik yaklaşımlar, yazılım geliştirme safhasında geç ortaya çıkan gereksinim değişimlerini çabuk karşılamayı esas alan modellere verilen genel bir isimdir. Bir çevik yaklaşım olan aykırı programlama (XP) üzerinde ise halen bir sistem geliştirme modeli olup olmadığı tartışmaları yapılmaktadır. [KARLIDERE ve KALIPSIZ-2003]

XP, 4 temel değere ve bunların çevresinde oluşturulmuş 12 pratiğe sahiptir. XP'nin temel değerleri:

1. İletişim

Proje paydaşları arasındaki iletişim, bir projenin başarısını doğrudan etkileyen en önemli unsurlardan biridir. İletişim planı çerçevesinde kimin kiminle, nasıl ve ne zaman iletişime geçeceği belirlenmelidir. Aykırı programlama ise diğer sistem geliştirme modellerinden farklı olarak güçlü bir iletişim için planlama oyunu, eşle programlama (*pair programming*), günlük kısa toplantılar, kısa aralıklı yayımlar (*release*) ve müşterinin ekip üyesi olması gibi XP pratiklerini geliştirmiştir.

2. Basitlik

XP’de sistem geliştirmenin her aşamasında basitlik anlayışı hakimdir. XP, bir problemi çözerken sadece o anki gereksinimleri karşılayan en basit çözümün kullanılması gerektiği savunur. Böylece ileride gerekli olabilir düşüncesiyle karmaşık çözümler üretmek için harcanacak iş gücü ve yüksek riskten kaçınılmış olunur.

3. Geri Besleme

İlk bölümde de anlatıldığı üzere sistem yapısındaki geri besleme, sistemin geliştirilmesi sürecinde de yer almaktadır. Bu geri besleme sayesinde projedeki gerekli değişiklikler zamanında yapılarak ihtiyaçlara daha hızlı cevap verilebilir. Geri besleme, proje ekibi içindeki bireyler arasında olabildiği gibi sistemle bireyler arasında da olabilir. Kısa aralıklı yayımlar ve planlama oyunu gibi XP pratikleri, müşteri ile yazılım ekibinin arasındaki geri beslemenin sürekliliğini sağlar.

4. Cesaret

XP’nin dördüncü değeri olan *cesaret*, ancak yukarıdaki diğer değerlerin sağlandıktan sonra proje ekibine fayda sağlar. Aksi durumda sadece cesaret, yarardan çok zarara neden olur. XP’de grup çalışması daha ön plana çıktığı için proje üyelerinin motivasyonun yüksek olması daha fazla önem kazanmaktadır.

XP, bu dört ilkenin etrafında oluşturulmuş 12 uygulamaya sahiptir:

- 1. Sistem Metaforu:** Ekip üyelerinin proje hakkında ortak bir çerçeveden bakabilmesini ve hedefleri kavramasını sağlamak için sistemin işleyişini genel olarak anlatan metafor kullanılarak hikayeleştirilir.
- 2. Ekip Üyesi Müşteri:** Proje paydaşlarından biri olan müşteri, yazılım geliştirme ekibinin bir üyesi olarak kabul edilir. Ancak ticari bir paket sisteminin geliştirilmesi örneğinde olduğu gibi müşterinin ekibe katılmasının mümkün olmadığı durumlarda analistlerin bir grubu müşteri rolünü üstlenebilir.
- 3. Kısa Aralıklı Yayımlar:** XP, proje paydaşları için oluşturulan yayımlar arasındaki zamanın mümkün olduğunca kısa olması gerektiğini savunur. Böylece müşteriden alınacak sürekli geri beslemeyle istenilen sistemi çabuk şekilde teslim etmek amaçlanmaktadır.
- 4. Planlama Oyunu:** Planlama oyununda müşterinin belirlediği öncelikleri, sisteme dair kısıtları ve teknik tahminleri kriter olarak alıp planlayarak en kısa zamanda bir sonraki yayımın kapsamı belirlenir.
- 5. Basit Tasarım:** Yukarıda anlatıldığı gibi basitliği savunan XP yaklaşımı, sistem tasarımının da mümkün olduğunca basit şekilde yapılmasını öngörür.
- 6. Ortak Kod Mülkiyeti:** Yazılan kodun mülkiyeti, yazılım geliştirme ekibinin tüm üyelerine aittir. Kısacası ekibin üyesi herhangi bir kodu kendi yazmamış dahi olsa, kod üstünde gerekli değişiklikleri yapabilir. Bu pratiğin başarısı kodlama standartlarının tam olarak belirlenmesine bağlıdır.

7. Kodlama Standartları: Yazılım ekibi arasındaki iletişimin kolaylaşması ve ortak kod mülkiyetinin uygulanması bu pratiğe bağlıdır. Tüm ekip üyelerinin, kodlarını belirlenen standartlar çerçevesinde yazması önem kazanmaktadır.

8. Eşle Programlama: Eşle programlamada kod, iki ekip üyesinin aynı zamanda aynı bilgisayarda birlikte çalışmasıyla yazılır; Bir programcı, sürekli kodu yazarken diğeri yazılan kodu gözden geçirip, yazılanların sisteme uygunluğu, nasıl ya-
nılaştırılabileceği vb. konular üzerine odaklanır. Eşle programlama, sürekli kodu gözden geçirmeye (*code review*) ve incelemeye (*inspection*) olarak sağladığı için hataların azaltılmasında oldukça faydalıdır.

9. Test: XP'nin sunduğu hızlı geliştirme sürecinde test süreci daha da önem kazanmaktadır. Bu nedenle yazılım geliştiriciler (analist, programcı vb.), daha kodlanmaya başlamadan önce test senaryolarını hazırlar.

10. Sürekli Tümleştirme: Yazılan kodlar yeni özelliklerin eklenmesinden sonra, çok kısa aralıklarla hatta günde birkaç kere tümleştirilir ve test edilir.

11. Devamlı Yeniden Tasarım: Testi gerçekleştirilen her birimden sonra yeni bir işlev veya modül eklendikçe mevcut koddaki karışıklıklar temizlenir. Böylece basitlik ilkesinin devamlılığı sağlanmış olur.

12. Devam Ettirilebilir Hız - 40 saat / hafta: XP, motivasyona ve verimliliğe önem verdiği için çok zorunlu haller dışında fazla mesaiye karşıdır. XP mantığına göre haftada 40 saat çalışmak standarttır ve birkaç hafta üst üste fazla mesai yapılması verimliliğin en büyük düşmanıdır.

4.6. Özet

Sistem geliştirme yaşam döngüsü, problemin tanımı-fizibilite çalışması-sistem analizi-sistem tasarımı-gerçekleştirme ve bakım olmak üzere bilgi sistemlerinin geliştirilme süreci temel adımlarından oluşmaktadır.

Fizibilite çalışması, sistem geliştirme projesinin kapsamı ve hedeflerini tanımlayarak yapılabilirliğini belirlemektir. Sistem analizi, problemin çözümlerinin ne olduğunu ve sistemin çalışma sürecini ortaya çıkarmaktır. Genel sistem tasarımı, sistemin nasıl gerçekleştirileceğinin, ayrıntılı sistem tasarımı ise, genel tasarımda belirlenen sisteme ait alt sistemlerin tanımlanmasıdır. Gerçekleştirme aşamasında, tasarıma uygun kodlama yapılır ve tüm sistem sınanır. Bakım aşaması ise sistemin hatasız kullanımı için yapılan destek işlemleridir.

Klasik süreç (*waterfall*)-model oluşturma (*prototip*)-hızlı uygulama geliştirme (RAD) - evrimsel süreç (artımlı ve spiral) ve aykırı programlama (XP) temel sistem geliştirme süreci modelleridir.

4.7. Sorular

- 4.7.1) Yazılım geliştirme modellerinden hangisi risk yönetimini kapsar?
- 4.7.2) Spiral Model’de hem *Waterfall* (doğrusal ardışık), hem de prototip modelin nasıl birarada bulunduğunu açıklayınız.
- 4.7.3) Aşağıdaki sistemlerin geliştirilmesini yönetmek için temel olarak kullanılan en uygun yazılım süreç modelini ve neden seçtiğinizi yazınız.
- Bir arabadaki fren kontrol sistemi
 - Var olan sistemin yerine geçecek üniversite muhasebe sistemi
 - İstasyonlardaki yolcuların tren zamanlarını öğrenecekleri etkileşimli bir sistem
- 4.7.4) Büyük bir yazılım şirketinde proje yöneticisi olarak görevlisiniz. Yaygın olarak kullanılan *word processing* yazılımının yeni versiyonunu geliştirme projesini yönetiyorsunuz. Rekabet nedeni ile proje bitim süresi belirlenmiş ve duyurulmuş. Bu projede hangi yazılım geliştirme modeli kullanılır, niçin?
- 4.7.5) Sistem analizi ve tasarımı (bilgi sistemi geliştirme) süreci adımlarını bir şema yardımı ile kısaca tanıtınız.

BÖLÜM 5

ÖN İNCELEME VE FİZİBİLİTE ÇALIŞMASI

Fizibilite çalışmasındaki amaç projenin olabilirliğini araştırmaktır. Fizibilite çalışması sonucu genel hatlarıyla projenin planı ve tahmini bütçesi ortaya konur.

Fizibilite aşamasında sistem üzerinde genel olarak bir ön inceleme yapılır. Ön incelemede amaç projenin olabilirliğini belirlemektedir. Bu aşamada,

- Projeye (yeni sisteme) gerçekte ihtiyaç var mı?
- Sistemin gerçekleştirilmesi için neye ihtiyaç var?
- Ne kadar süreye ihtiyaç var?
- Tahmini bütçe nedir?
- Yararları ve zorlukları nelerdir?

sorularına cevap bulunmaya çalışılır.

5.1. Fizibilite (Olabilirlik) Türleri

Geliştirilmesi düşünülen sistem için altı farklı fizibilite çalışması söz konusudur.

5.1.1. Teknik Fizibilite

Sistemin teknik açıdan içerdiği riskler ve teknik olanaklar araştırılır. Bu değerlendirme hem cihaz hem de insan gücünün sınırlarını test etmeli ve önerilen sistemin gerçekleştirme aşamasında kullanılacak kaynaklarına karar vermelidir.

Kullanılacak olan teknolojinin yeni veya eski olması durumunda karşılaşılabilecek riskler göz önüne alınmalıdır. Örneğin yeni üretilen bir yazım uygulamasının var olan işletim sistemleriyle karşılaşılabileceği sorunlar ya da eski bir cihazın kullanılmasında ortaya çıkabilecek sorunlar düşünülmelidir. Sistem geliştirilirken ilk kez kullanılacak olan programlama diline programcıların ne kadar hakim olduğu belirlenmelidir.

5.1.2. Ekonomik Fizibilite

Yeni projenin ekonomik riski araştırılır. Geliştirilecek olan sisteme ait bütün maliyetler hesaplanarak maliyet - yarar analizi yapılır. Bu çalışma sonucunda ekonomik açıdan önerilen sistemin uygun olup olmadığına karar verilir.

Gerçekleştirilecek sisteme ait maliyetler içinde örneğin kullanılacak donanım ve yazılım harcamaları, personel harcamaları (sistem analisti, sistem tasarımcısı, programcı vb. çalışanların adam saat ücretleri) yer alabilir.

Bu maliyetlerin yatırım yapıldıktan ne kadar süre sonra geri dönüşü sağlanacağı çeşitli yarar-maliyet analizleri ile hesaplanabilir. Bu hesaplar arasında bugünkü değer (*net*

present value-NPV), iç verim oranı (*internal rate of return-IRR*), başabaş noktası (*break-even*) ve geri ödeme süresi analizleri yer alır. Hangi yöntem kullanılırsa kullanılсын asıl amaç bugün yapılan birim yatırımın gelecekte kaç birim olacağını hesaplamaktır.

Bugünkü Değer Yöntemi: Farklı dönemlerdeki nakit giriş ve çıkışlarının arasındaki ilişkileri paranın zaman değerini dikkate alarak inceler. Bu yöntemde, projenin bütün nakit giriş ve çıkışları bugünkü değere indirilir ve aralarında bir karşılaştırma yapılır. Projenin hesaplanan net nakit girişlerinin bugünkü değeri yatırım harcamalarının bugünkü değerinden fazla ise proje kabul edilir, değil ise reddedilir. Yöntemde, projenin farklı dönemlerde oluşan nakit giriş ve çıkışlarını aynı zaman bazına indirgemek amacıyla, belirli bir iskonto oranı kullanılır. İskonto oranının belirlenmesi amacıyla çok çeşitli yöntemler kullanılmaktadır. Bunların hepsi de birbirinden farklı sonuçlar vermektedir. Ancak, en pratik olan yol, ekonomideki sermaye piyasasına hakim bir tek sektör olmadığı için, bunlardan ağırlıklı olan sektörlerin uyguladığı faiz oranlarının ortalamasının alınmasıdır.

Bir yatırımın gelecekte oluşacağı öngörülen nakit akışlarının bugünkü değere indirilmesi, aşağıdaki formül aracılığıyla yapılabilir.

$$V = Q_1 / (1+i) + Q_2 / (1+i)^2 + \dots + Q_n / (1+i)^n$$

Burada, $Q_1, Q_2, \dots, Q_n$ yıllara itibariyle oluşacak nakit akışlarını; i , iskonto oranını; V , indirgenmiş nakit akışları toplamını ifade etmektedir.

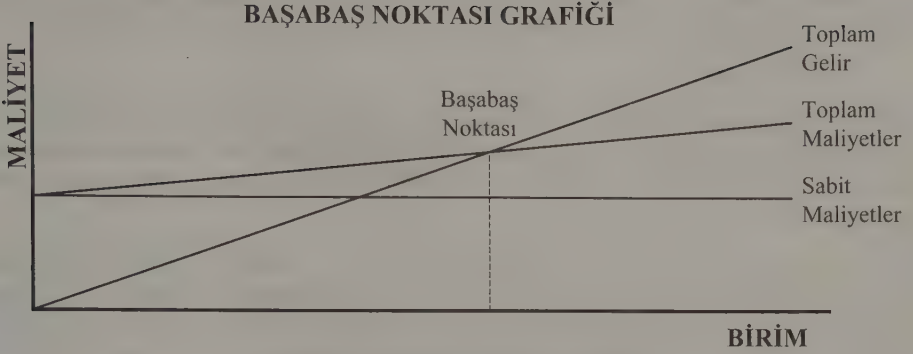
Bu formül yardımıyla hesaplanan V değeri yatırımın başlangıç yatırım tutarından büyükse yatırım projesi kabul edilebilir, değilse reddedilir.

İç Verim Oranı Yöntemi: Yöntem, paranın zaman değerini ve yatırımın ekonomik ömrünü de dikkate almaktadır. Bu yöntem, her yatırım projesinin özelliğine göre bir karlılık oranı belirlemektedir. İç verim oranı, herhangi bir yatırıma ilişkin nakit çıkışları ile yatırım maliyetini birbirine eşitleyen iskonto oranıdır. Bu yöntemin esasını yatırımın maliyetinin, yatırımın yıllar itibariyle elde ettiği toplam gelirlere eşitliğini sağlayacak iskonto oranının hesaplanması oluşturur. İç verim oranı;

$$x = \sum_{t=1}^T (R_t / (1+d)^t)$$

bağıntısıyla ifade edilir. Burada T , yatırımın yıl bazında ömrünü; x , yatırımın toplam maliyetini; R_t , t yılındaki net getiriyi; d , iskonto oranını göstermektedir.

Başabaş Noktası Yöntemi: Bir projenin işletme aşamasına geçtikten sonra toplam satışları ile toplam giderlerinin birbirlerine eşit olduğu satış tutarı, satış miktarı veya kapasite kullanım oranını bulmak için kullanılan bir yöntemdir. Sistemin geliştirilmesi için yapılan harcamalarla sistemin maliyeti ve sistemden sağlanacak yararlar/gelirler Şekil-5.1'de gösterildiği gibi aynı grafik üzerinde gösterilir. Bu grafikte iki eğrinin çakıştığı nokta başabaş noktasıdır. Bu noktadan sonra sistemin geliri maliyetinden yüksek olacaktır.



Şekil-5.1. Başabaş noktası grafiği

Geri Ödeme Süresi Yöntemi: Bu yöntem yatırımın yapılıp yapılamayacağını anlamak için en çok kullanılan yöntemlerden biridir. En basit şekliyle hesaplama, gerekli yatırım ve yatırımın işletmeye açılması sonucu elde edilen yıllık yarar arasındaki ilişkiye dayanır. Bu yıllık yarar, yatırım miktarına bölünerek geri ödeme süresi elde edilir. Yatırım, brüt veya net olarak alınabilir. Bazı durumlarda başlangıç yatırımı yerine ortalama yatırım kullanılır.

Geri ödeme yöntemi bir oran kavramı değil, bir zaman kavramıdır. Başlangıçtaki nakit yatırımlarının, kaç yıl içerisinde elde edilebileceğini belirler. Dolayısıyla temel formülü şu şekilde ifade edilmektedir.

$$\text{Geri Ödeme Süresi} = \text{Yatırım Tutarı} / \text{Yıllık Nakit Akışı}$$

5.1.3. Zaman Fizibilitesi

Belirlenen süre çerçevesinde önerilen projenin tamamlanıp tamamlanamayacağı araştırılır. Bu amaçla günümüzde PERT ve Gantt gibi pek çok teknik kullanılmaktadır.

PERT (*Program Evaluation and Review Techniques*), ilk kez 1954 yılında NASA tarafından POLARIS füzesinin yapımında kullanılmıştır. Zaman, araç, maliyet ve kontrol gereklerine en iyi şekilde cevap veren PERT, aralarındaki ilişki ve tarih sayesinde bir nihai mamulün ki burada söz konusu olan bir sistemin oluşmasını sağlayan bir çok faaliyetin ağ şeklinde düzene konmasından meydana gelen bir yöntemdir.

PERT ağlarıyla planlamada dört tür süre tahmini söz konusudur. Bunlardan birincisi normal şartlarda işlerde gecikme ve aksama olmaksızın bir işlemin tamamlanması için gerekli olan en kısa süredir. Buna **iyimser süre** de denilir. İkincisi ise aksama ve gecikmeleri de dikkate alan en uzun süredir ve **kötümser süre** de denir. Normal gecikme ve aksamaları kabul eden **en yaklaşık süre** iyimser süreyle kötümser süre arasında bulunur. Her işlem için beklenen süre aşağıdaki şu formülle bulunur.

$$S_b = (S_i + S_e + S_k) / 6$$

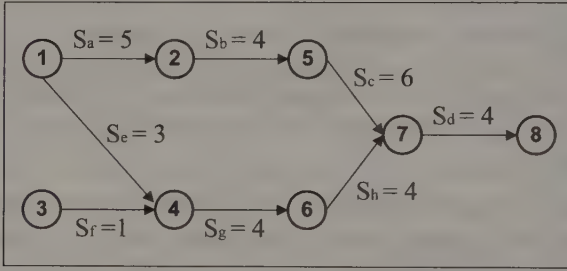
Burada S_b beklenen süreyi, S_i iyimser süreyi, S_e en yaklaşık süreyi, S_k ise kötümser süreyi ifade eder. Beklenen süreyi hesaplayabilmek için ağırlıklı ortalama alınır ve sonuç

6'ya bölünür. Beklenen sürenin hesaplanması varyansı ise, yapılan öngörünün değişme derecesini gösterir.

$$\text{Varyans} = (S_k - S_i) / 3$$

PERT ağı hazırlarken, olayların mantıksal bir sıra takip edeceği, bir olaya ancak kendinden önceki bütün işlemler bitirildikten sonra ulaşılacağı, bir işlemin kendisinden önceki bütün olaylar meydana gelmeden tamamlanamayacağı gerçeğini sürekli olarak göz önünde tutmak gerekecektir.

PERT'in uygulanmaya başlandığı yıllarda Amerikan Dupont firması da bir inşaat programında uygulamak üzere **CPM**'i (*Critical Path Method*) geliştirmiştir. Biri askeri diğeri de özel firmada geliştirilmekle birlikte PERT ve CPM birbirlerine çok benzemektedirler. PERT dört farklı süre hesaplarken CPM ortalama süreyi hesaplamaya çalışır.



Şekil-5.2. PERT diyagramı

Şekil-5.2'deki örnek PERT diyagramında izlenebilecek 3 farklı yol vardır:

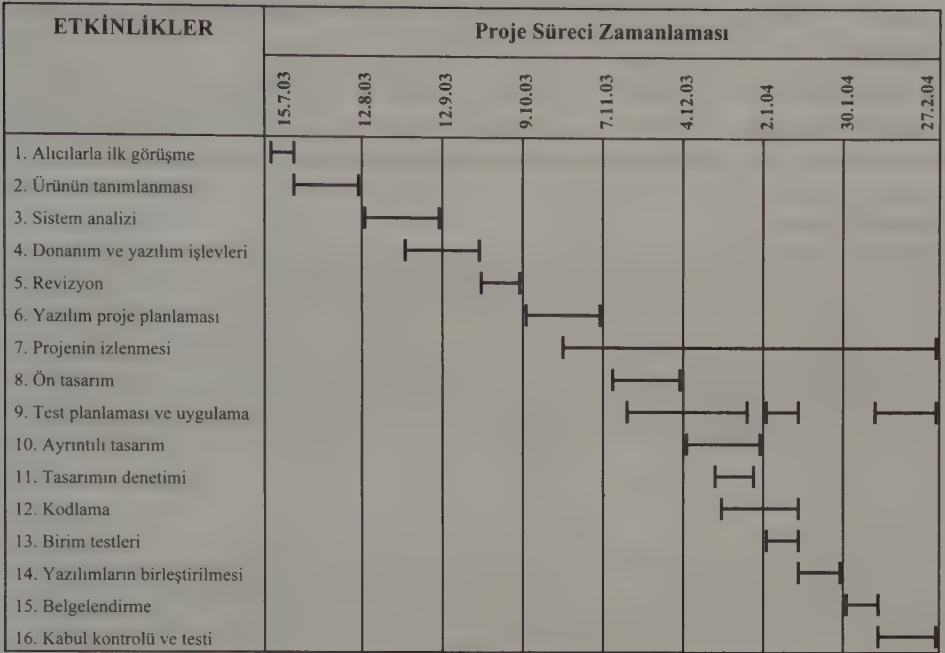
1.yol : 1-2-5-7-8 = 19 hafta

2.yol : 1-4-6-7-8 = 15 hafta

3.yol : 3-4-6-7-8 = 13 hafta

Gantt çizelgesi, problemin tanımından sistemin işletilmesine kadar olan süre içerisinde gerçekleştirilecek olan tüm aktivitelerin ne zaman başlayıp ne zaman biteceği gösterilir; bkz. Şekil-5.3. Gantt çizelgesinde ayrıca sistemin geliştirme sürecindeki önemli durak noktaları (*milestone* - kilometre taşları da) gösterilir. Günümüzde Proje zamanlaması için, her kişisel bilgisayar sisteminde kullanılabilecek olan otomatik proje çizelge aracı da geliştirilmiştir. Örneğin, kişisel bilgisayarlar için geliştirilen "*MS Project*" çizelge düzenleme aracı, proje verisinin bir işlem ağı (PERT veya CPM) formunda verilmesi ile otomatik olarak;

- Yapılacak işleri tarih sırası ile gösteren, durak yerlerini ve iş yoğunluğunun az olduğu işlemleri belirleyen bir Gantt çizelgesi,
- Yöneticinin proje sürecini izleyebilmesi için, her işlemin süresini en erken ve en geç başlama ve bitme tarihlerini gösteren bir görev durumu çizelgesi
- İşlem sırasına göre görev paylaşımını gösteren bir çizelge
- Üzerinde kritik yolu koyu çizgilerle belirleyen bir işlem ağı çıktıları elde edilebilmektedir.



Şekil-5.3. Gantt çizelgesi

5.1.4. Sosyal Fizibilite (Olabilirlik)

Önerilen sistemin kullanıcılar tarafından kabul edilip edilmeyeceği araştırılır. Bu sebeple önerilen sistemden direk olarak etkilenecek kullanıcı grupları kontrol edilir.

5.1.5. Yönetim Fizibilitesi

Yönetimin yeni sisteme bakış açısı ve destek düzeyi belirlenir. Oluşturulan yeni sistem, genelde organizasyondaki bilginin paylaşımını ve bilgiye ulaşımını daha hızlı ve kolay kılmak amacıyla gerçekleştirilmektedir. Bilginin güç olduğu organizasyonlarda ise üst yönetim, gücünü kaybetmek istemediği için yeni sisteme karşı olumsuz yönde etkisi olabilir. Bu açıdan sistem geliştirilirken üst yönetimin tam desteğini almak sistemin varlığını uzun süre sürdürmesi için gereklidir.

5.1.6. Yasal Fizibilite

Önerilen sistemin yasalara göre uygunluğu araştırılır. Patent, lisans, ilgili çalışma yasası, işletmenin yapmış olduğu anlaşmalar, uluslararası ticaret hukuku vb. yasal konular açısından önerilen sistemin uygun olup olmadığı araştırılır.

5.2. Sisteme Yönelik Farklı Çözüm Önerilerinin Değerlendirilmesi

Sistem analisti, sisteme yönelik farklı çözüm önerilerini değerlendirmek için herbiri için ayrı ayrı bir önceki bölümde anlatılan fizibilite çalışmasını gerçekleştirir. Bu bö-

lümde de çözüm önerilerinin hangi kriterlere göre nasıl değerlendirildiği üzerinde durulacaktır.

İlk yöntem olan aday sistemler matrisi ile yapılan değerlendirmede sistemlerin temel özellikleri açısından benzerlikleri ve farklılıkları ortaya konulur. Matrisin sütunları aday sistemleri temsil ederken satırlar, özellikler için kullanılır. Karşılaştırılması yapılacak temel özellikler arasında teknoloji, ara birim, veri, işlem ve coğrafik yapı yer almaktadır (Tablo-5.1).

Tablo-5.1. Aday sistemler matrisi

	Önerilen Sistem 1	Önerilen Sistem 2	Önerilen Sistem 3
Teknoloji Sistemin teknik alt yapısı			
Ara Birim Sistemin kullanıcılarla ve diğer sistemlerle olan iletişimi			
Veri Sisteme hangi verilerin nasıl girileceği ve buna göre elde edilen çıktılar			
İşlemler Sistemdeki veriler üzerinde hangi işlemlerin gerçekleştirileceği			
Coğrafik Yapı İşlem ve verilerin sistem içinde dağılımı			

Sistem analisti, sistem çözümlerini değerlendirmeden önce çözümde herhangi bir sınırlamanın/kısıtın olup olmadığı belirlenmelidir. Çünkü sisteme yönelik çözümde yer alan kısıtlar, yapısal kararlar üzerinde önemli rol oynamaktadır. Örneğin önerilen sistemde oluşturulması düşünülen teknolojik yapı, ilişkisel veri tabanı veya istemci/sunucu ağ yapısını zorunlu kılabilir.

İkinci yöntem **fizibilite matrisinde** ise her sistem önerisi için yapılan fizibilite çalışmalarına göre değerlendirilir. Tablo-5.2’de genel yapısının gösterildiği bu matriste de ilk yöntemde olduğu gibi sütunlar, çözüm önerilerini temsil ederken satırlar, fizibilite tiplerini gösterir. Bu matrisin son satırında ise çözüm önerilerinin sıralaması gösterilir.

Tablo-5.2. Fizibilite matrisi

	Önerilen Sistem 1	Önerilen Sistem 2	Önerilen Sistem 3
Teknik Fizibilite			
Ekonomik Fizibilite			
Zaman Fizibilitesi			
Sosyal Fizibilite			
Yönetim Fizibilitesi			
Sıralama			

Her fizibilite çalışmasının genel değerlendirmedeki yüzdelik oranı ile her çözüm önerisinin ilgili fizibilite çalışmasından aldığı puan matriste gösterilerek her çözümün aldığı toplam puan son satırda gösterilir. Tablo-5.3'de tam olarak yapılan bir fizibilite matrisi örneği gösterilmektedir.

Tablo-5.3. Fizibilite matrisi örneği

	Ağırlık	Önerilen Sistem 1	Önerilen Sistem 2	Önerilen Sistem 3
Teknik Fizibilite	%30	Kullanılan ürünün son sürümü yalnızca 6 haftadır piyasada. Dolayısıyla ürünün gelişimi risk taşımaktadır. Sistemle ilgili gerekli entegrasyonları yapmak için C++ bilen elemana ya da mevcut elemanların bu konuda eğitime ihtiyaç var. Puan : 50	Sayısı yeterli olmayan mevcut teknik personelin yalnızca Powerbuilder deneyimi olmasına karşın proje yöneticisi, sistemi MS Visual Basic'e çevirmenin basit olduğuna ve VB bilen eleman bulmanın Powerbuilder'e göre daha kolay ve ucuz olduğuna inanmaktadır. Puan : 95	Şirket bünyesinde MS SQL Sunucu kullanılmaktadır. Ancak istemci/sunucu veri tabanı yönetimi sektöründeki rekabettten ötürü Powerbuilder'in yeni versiyonlarının SQL Sunucu ile ne kadar uyumlu olacağı belli olmaması risk taşımaktadır. Puan : 60
Ekonomik Fizibilite	%30	Yaklaşık 350.000 \$'a mal olan ve geri dönüşümü 4,5 yıl olan sistemin net getirisi 210.000 \$'dır Puan : 60	Yaklaşık 418.040 \$'a mal olan ve geri dönüşümü 3,5 yıl olan sistemin net getirisi 306.748\$'dır Puan : 85	Yaklaşık 400.000 \$'a mal olan ve geri dönüşümü 3,3 yıl olan sistemin net getirisi 325.500 \$'dır Puan : 90
Zaman Fizibilitesi	%10	3 aydan az Puan : 95	9-12 ay Puan : 80	9 ay Puan : 85
Operasyonel Fizibilite	%30	Sadece servis üyelerinin ihtiyaçlarını karşılamakta ve var olan iş süreçlerini yazılıma göre güncellenmesi gerekmektedir. Puan : 60	Kullanıcının tüm ihtiyaçlarını karşılamaktadır. Puan : 100	2. sistem ile aynı Puan : 100
Sıralama	%100	60,5	92	83,5

5.3. Özet

Bilgi sistemi geliştirme sürecinin ilk adımı olan fizibilite analizinde amaç, projenin kapsamı ve hedeflerini tanımlayarak yapılabilirliğinin saptanmasıdır. Fizibilite çalışmasında, kurumun yeni sisteme olan ihtiyacı, sistemin gerçekleştirilmesi için gerekli olan araçlar, ortam, tahmini süre ve bütçe belirlenir. Ayrıca yeni sistemin getireceği yararlar ve sistemin kuruluşu sırasında ortaya çıkacak zorluklar ve riskler değerlendirilir.

Temel fizibilite analizi tipleri, sistemin teknik açıdan içerdiği riskler ve teknik olanakların araştırıldığı teknik fizibilite ve geliştirilecek olan sistem için gerekli olan tüm maliyetlerin hesaplanarak maliyet - yarar analizinin gerçekleştirildiği ekonomik fizibilitedir. Zaman, sosyal, yönetim ve yasal fizibilite tipleri de farklı boyutlarda sistemin yapılabilirliğini desteklemektedir.

5.4. Sorular

- 5.4.1)** a) *Milestone* b) *Payback* c) *Kaynak Seviyeleme* kavramlarını ve kullandıkları yeri açıklayınız.
- 5.4.2)** “MS Project” kullanarak, hazırladığınız en son yazılım için geliştirme sürecinin tüm alt adımlarını gösteren bir Gantt diyagramı oluşturunuz.
- 5.4.3)** Bir yazılım geliştirme sürecindeki görevler, bu görevin gerçekleştirilmesi için gerekli süre ve bu görevin başlaması için bitmesi gereken diğer görevlerin verildiği aşağıdaki tabloya göre, *Gantt* diyagramı (*Activity Bar Chart*) hazırlayınız.

GÖREV	SÜRE (GÜN)	BİTECEK GÖREV
T1	8	
T2	15	
T3	15	T1
T4	10	
T5	10	T2, T4
T6	5	T1, T2
T7	20	T1
T8	25	T4
T9	15	T3, T6
T10	15	T5, T7
T11	7	T9
T12	10	T11

BÖLÜM 6

SİSTEM ANALİZİ -GEREKSİNİM ANALİZİ-

Sistem analizi, ana sistem öge ve işlevlerinin ele alınarak, ayrıntılı olarak tanımlanmasıdır. Ayrıntılı incelemedeki amaç maddi, teknolojik, zaman vs açısından en uygun çözümü bulmaktır. Bu aşama sonucunda üretilen raporda kullanıcının bütün ihtiyaçları, var olan sistemin durumu, seçilen en uygun çözümün değerlendirmesi ve var olan sistemin nasıl iyileştirilebileceğine dair öneriler yer almaktadır. Bu inceleme,

- Gelecekteki hedeflerin ayrıntılandırılması
- Bilgi kaynaklarının ve ihtiyaçlarının belirlenmesi

olmak üzere ikiye ayrılabilir.

Ayrıntılı incelemeyi başarıyla yapabilmek için öncelikle aşağıda belirtilen unsurların yerine getirilmesi gerekmektedir:

- Bir önceki aşama olan fizibilite raporlarını gözden geçirmek
- Bütün teknik terimleri incelemek
- Ayrıntılı inceleme planı yapmak
- Proje grubu içinde görev dağılımını yapmak
- Yapılan planının gerçekleştirilmesi için denetleme yapmak

Sistem analizinin başarıya ulaşması için sistemin çok iyi anlaşılması; bunun için de sistem analistinin, uzman bir araştırmacı gibi davranması gerekmektedir. Bu nedenle sistemin nasıl çalıştığını ve çalışma maliyetinin ne kadar olduğunu anlamak sistem analizinin en önemli bölümünü oluşturmaktadır. Sistem analisti, var olan sisteme çok iyi hakim olmalıdır. Sistem analisti, var olan sistem hakkında aşağıdaki konularda bilgi sahibi olmalıdır:

- Var olan bilgi kaynakları
- Var olan sistemde kullanılan donanım ve yazılım
- İşlemlerde kullanılan bilgi
- İç ve dış bilginin birbirini nasıl etkilediği
- İşlenecek olan bilginin miktarı
- Sistem döngüsü ve süresi
- Arşivlenen bilgi ve arşivlemede kullanılan araçlar
- Kullanılan raporların formatı
- Sistem personeli
- ve tüm bunların maliyeti.

6.1. Sistem Analizi Yöntemleri

Sistem analizini gerçekleştirirken iki farklı yöntemden biri uygulanabilir. Birinci yöntem bilişim alanının, işlevlerinin, ara birimlerinin, kısıtlarının, kabul kriterlerinin belirlenip akış diyagramları halinde gösterilmesidir. Böylece sistem özellikleri ortaya konmuş olmaktadır. Ancak, bu yöntemin uygulanabilmesi için, gereksinimler daha önce müşteri ile birlikte saptanmalıdır.

İkinci yöntem sistem için bir örnek model (*prototip*) oluşturmak ve bu örnek üzerinde müşteri ile tartışarak bunu gereksinimlere uygun hale getirmektir. Bu yöntemde, gereksinimler önceden saptanmayıp, model üzerinde müşteri ile birlikte kararlaştırılmaktadır. Gereksinimlerin karşılanmasına ya da kaynak kısıtlamalarına göre sistem tasarımının özellikleri; sistem öğelerinin kabaca analizi ile de gereksinimleri karşılama derecesi belirlenmektedir.

Sistem gereksinimlerinin analizi ve tanımlanması, sistem analisti ve müşterinin ortak çalışması ile gerçekleştirilmektedir. Gereksinimlerin belirlenmesi ile “sistem gereksinimleri özellikleri” ortaya konulmuş olmakta ve bir belge haline getirilmektedir.

Sistem gereksinimleri özellikleri veya sistem modeli, sistem analisti ve müşterinin işbirliği ile tamamlanıp gözden geçirildikten sonra, buna göre “sistem geliştirme planı” da tekrar elden geçirilerek gerekli düzeltmeler yapılmalıdır. Böylece, sistem geliştirme planı ile gereksinim analizi aşamaları birlikte ve iç içe yürütülmüş olduğu için, ikisine birlikte “tanımlama aşaması” adı da verilmektedir. Toplu olarak bu aşamanın gerçekleştirilmesi Şekil-6.1’de şematik olarak gösterilmiştir.

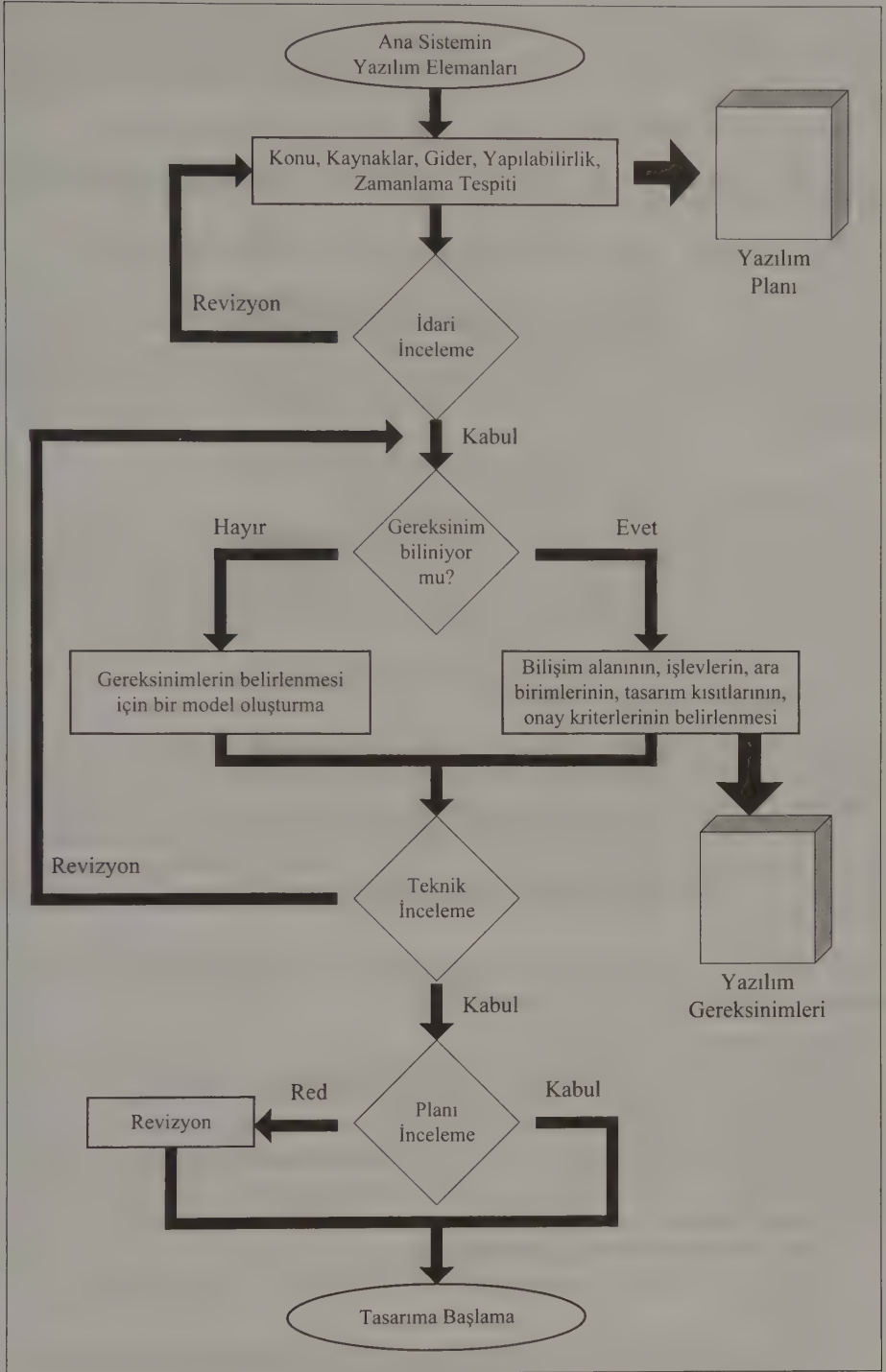
İdari inceleme kapsamında dikkat edilmesi gereken unsurlar aşağıda belirtilmiştir:

Organizasyondaki küçük değişikliklerin bile bilgi sisteminin ihtiyaçları üzerinde büyük etkileri olabilir. Bu nedenle sistem geliştirme süreci boyunca bu ihtiyaçların işletmenin genel stratejileriyle uyuşup uyuşmadığı düzenli olarak kontrol edilmelidir.

Karar verme sürecinde deneyim, bilgi ve geleceği önceden görebilme yeteneğinin birlikte önem kazandığı göz önünde bulundurulmalıdır.

Rekabetin güçlü olduğu bir çevrede daha iyi sistemlerin seçileceği unutulmamalıdır.

Yeni bir bilgi sisteminin sağlayacağı finansal yararlar gösterilmelidir. Çünkü işletmelerde, bir yatırım kararı geliştirme ve işletme maliyeti göz önünde tutularak yapılan finansal analizlerin sonucuna göre verilir. Başlıca finansal analizler Fizibilite Bölümü’nün Ekonomik Fizibilite kısmında anlatılmaktadır.



Şekil-6.1. Sistemin gereksinim analizi aşaması

İnsan kaynaklarındaki değişiklik personel sayısını, becerisini ve iş yükünü etkileyecektir. Hatta bazı durumlarda insan kaynaklarında değişiklik olmasa da sistem geliştirme süreci boyunca daha fazla iş yüküne ve personel becerisine ihtiyaç duyulabilir.

Ayrıntılı ve hızlı raporlama daha esnek ve verimli bir yönetim sağlayacaktır.

Bilgi akışını iyileştirme (optimize etme) ve gerekli bilgilerin güncelleştirilmesi kullanıcıların daha fazla bilgilendirilmesini sağlar.

Sistem uygulanması var olan sisteme göre daha hızlı ve iyileştirilmiş olmalıdır.

Bilgi sistemini oluştururken teknolojik ihtiyaçların da göz önünde tutulması gerekmektedir. Kullanılan bilgi teknolojisi sistemde yer alan bilginin hacmi ile uyumlu olmalıdır. Sistemin doğru çalışması ilk karşılanması gereken unsur olmasına rağmen kullanıcı dostu olmayan ara birimler, bilginin yanlış şekilde tutulması da sistemin başarısızlığına etken olan faktörlerdir.

6.2. Araştırma ve Bilgi Toplama Yöntemleri

Sistem analizinin en zor aşamalarından biri olan araştırma sürecinde var olan sistemin yanlış veya eksik anlaşılması, yeni kurulacak olan sisteme de yansımaktadır. Araştırma süreci boyunca hangi bilginin nasıl toplandığı önemli bir rol oynamaktadır. Bu bölümde bilginin nasıl toplanacağına dair araştırma metotları incelenecektir.

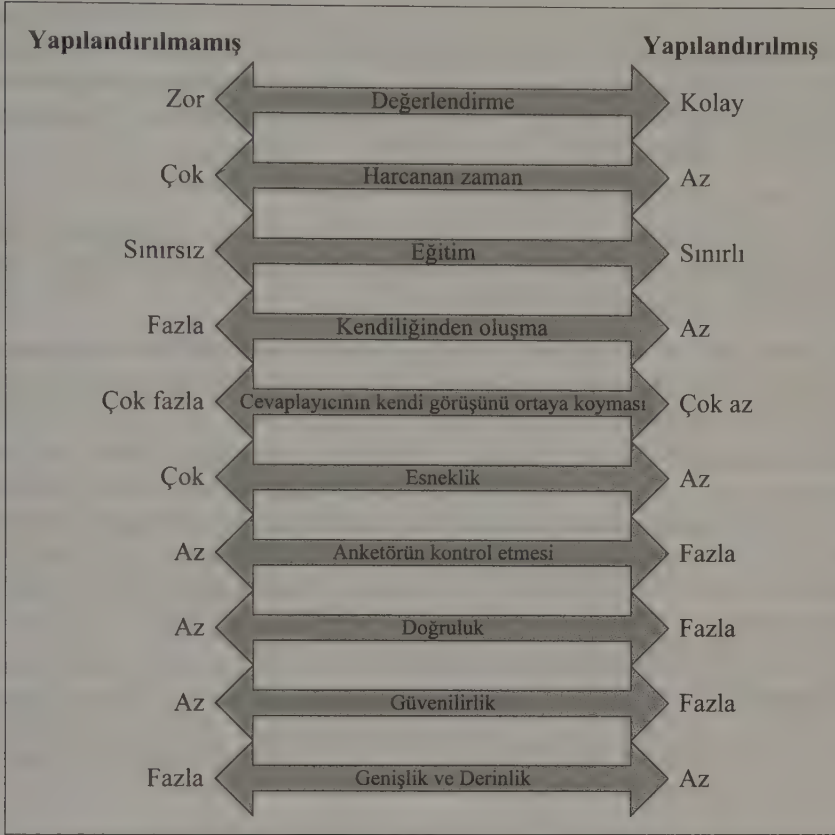
6.2.1. Kişisel Görüşme Yöntemi

Bireylerle karşılıklı **görüşme**, bilgi toplama yöntemlerinde en sık kullanılanıdır. Kişisel görüşme, sistem analistinin kullanıcı ile işyerinde karşı karşıya gelmesi ve soru sormasıdır. Bu nedenle görüşmeden önce sistem analistinin mutlaka karşı taraftan randevu almalı ve önceden hazırlıklı olmalıdır. Çünkü sistem analisti işine ne kadar profesyonel yaklaşırsa görüşmeci de aynı tavır içinde olacaktır. Görüşmeye gitmeden önce görüşmenin başlangıç konuşması ve konuşulacak başlıca konular belirlenmiş şekilde hazır olunmalıdır.

Sistem analisti, aşağıdaki maddeler halinde belirtilenleri yerine getirmesi durumunda görüşmeyi önceden planlayarak yönlendirebilir.

- Görüşme amaçlarını saptamak.
- Gereken veriyi elde etmek için kiminle görüşüleceğini saptamak.
- Görüşmenin zamanını ve yerini belirlemek.
- Görüşülecek kişiye toplantının amacını, süresini ve gerekli dokümanlarını bildirmek.
- Görüşme sırasında sorulacak soruları ve soru türlerini belirlemek; bu türler anket yöntemi bölümünde açıklanmaktadır.
- Görüşmeden sonra en kısa zamanda görüşme sonuçlarını değerlendirerek yorumlamak.
- Görüşme sırasında ulaşılan sonuçları formal biçimde yazarak, görüşülen kişiye ve sorumlu olan yönetici ve ilgililere göndermek.

Şekil-6.2’de yapılandırılmış ve yapılandırılmamış görüşmenin karşılaştırması yer almaktadır. Görüldüğü üzere daha önceden belli bir yapıya oturtulmuş görüşmeyi hem değerlendirmek daha kolay hem de tamamlanması için daha az zamana ihtiyaç duyulmaktadır.



Şekil-6.2. Görüşme kıyaslaması [KENDALL-2004]

Görüşme süresince konuşmacının söyledikleri yazılı olarak ya da sesli kayıt yapılarak not alınmalıdır. Bu notlar ileride sistem analistine sistemi tasarlariken yardımcı niteliğinde olacaktır. Görüşme sırasında sistem analistinin, konuşma biçimi, görüşmeciye karşı hitabı ve hareketleri görüşmecinin güvenini kazanacak şekilde olmalıdır. Görüşmenin sonunda ise sistem analisti, görüşmenin başlıca noktalarını, daha sonra konuşulacak ya da hiç değinilmemiş konuları belirtip eğer gereklyse gelecek görüşme için randevu tarihini belirlemelidir.

6.2.2. Anket Yöntemi

Birinci dereceden kaynaklara başvurarak yapılan diğer veri toplama yöntemi ise **anket** yapmaktır. Anket, insanlardan düşünceleri, duyguları, sağlıkları, planları, inançları ve sosyal, eğitimsel, ekonomik durumları hakkında bir bilgi toplama yöntemidir.

Anket formu, cevaplayıcıya sorulmak üzere hazırlanan sorulardan ve yanıtlar için gerekli alanlardan oluşan bir soru listesidir. Cevaplayıcı ile bu iletişimi sağlarken hatalara veya yanlış anlamalara yer vermemek için anketi düzenlerken önce neyin nasıl sorulacağına karar vermek gerekir. Bu nedenle anket formu hazırlarken uyulması gereken bazı ilkeler vardır. Bunlar:

Açıklık: Sorular tam ve açık olmalı, eksik olmamalıdır.

Hatırlatma: Cevaplayıcıya hafızayla ilgili sorulan sorularda, hakkında bilgi istenen olayın üzerinden geçen zaman, olayın cevaplayıcı bakımından önemi ve diğer hatırlatıcı olaylar dikkate alınmalıdır.

Cevap verme arzusu oluşturma: Cevaplayıcının işbirliğini sağlamak için anket formunda başlangıç sorularının cevaplayıcının özel yaşantısıyla ilgisi bulunmayan, cevaplaması kolay ve ilgi çekici olmasında yarar vardır.

Hataya engel olma: Cevap hatalarına engel olmak için bazı durumlarda dolaylı sorular sormak gerekir. Örneğin kişiye aylık gelirini doğrudan sormak yerine gelir grupları verilir ve bunlardan hangisine girdiği sorulur. Anket formunun başında bir eleme sorusunun olmayışı da hataya neden olabilir. Örneğin İnternet kullanıcıları ile ilgili bir anketin başında “hiç İnternet kullanıp kullanmadığı” sorulabilir. Eğer cevaplayıcı bir internet kullanıcısı değilse anketteki diğer sorular yönetilmez ve anketin baştaki amacına uygunsuzsa sadece neden İnternet kullanılmadığı sorulur.

İfade kolaylığı: Cevaplayıcının sorulara doğru cevap vermesini sağlamak için şekil, resim vb görsel öğelerle soruya ilişkin açıklama yapılabilir. Örneğin, kullanıcı arabirimlerinde kullanılacak renkler, katalogdan gösterilirse cevaplayıcı bunlar arasından hoşuna gideni daha rahat seçebilecektir.

Cevaplayıcıyı koşullandırma: Sorular, yanıt vereni koşullandırmamalıdır. Örneğin, “Microsoft İnternet Explorer mı kullanıyorsunuz?” sorusu cevaplayıcıyı ya evet ya hayır alternatifinin seçilmesinin beklendiği yönünde bir kaniye ulaşmasına yol açar. Şartlandırmanın yok edilmesi için “Hangi Web tarayıcısını kullanıyorsunuz?” şeklinde sorulmalıdır.

Anket sorularının yazımı anket sürecinin muhtemelen en önemli aşamasıdır. Sorular, anketin yapı taşlarıdır. Soruların istenilen bilgiyi elde etmedeki performansı anketin sonuçlarına diğer bütün anket süreci aşamalarından daha fazla etki eder. Dolayısıyla anket sorularının yazımına büyük özen gösterilmelidir. Anket sorularını iki genel grupta toplamak olasıdır:

- Açık uçlu sorular
- Kapalı uçlu sorular

Açık uçlu sorular, yanıt verenden kendi ifadelerinden oluşan bir cevap ister.

Açık uçlu soruların olumlu yanları:

Açık uçlu sorulardan elde edilen cevaplar, kapalı uçlu sorulardan çıkarılan kuru sayılara ve sonuçlara can katabilirler; önceden hazırlanmış cevaplardan alınamayacak bilgileri sağlayabilirler.

Açık uçlu soruların en büyük avantajı cevaplayıcılara, önceden hazırlanmış seçeneklerin etkisi olmaksızın, kendi bakış açılarını kullanarak cevap vermelerine izin vermesidir. Bu yüzden cevaplayıcıların gerçek görüşlerini daha yakından betimleyen cevaplar ortaya çıkarabilirler. Açık-uçlu sorulara verilen cevaplar cevaplayıcıların endişelerinin hem içeriğini hem bu içeriğin ne kadar öncelikli olduğunu ortaya koyar, daha derinlikli bilgiler elde edilmesini sağlar.

Açık uçlu soruların biraz gözden kaçan önemli bir avantajı cevaplayıcılara duygusal açıdan yüklü konularda (çalışan sayısında yapılan azaltmalar, performans değerlendirme sistemi gibi) duygularını ifade etmeleri ve boşaltmaları imkanını vermesidir.

Açık uçlu soruların olumsuz yanları:

Açık uçlu sorulardan oluşan anketin tamamlanması çok zaman alabileceğinden sonlarda bulunan soruların yorgunluğa bağlı olarak yeterince üzerinde durulmayabilir. Ayrıca bazı cevaplayıcılar, cevapların gerektirdiği çabadan dolayı soruların bazılarını veya tümünü cevapsız bırakabilir.

Konu hakkında az bilgisi olanların cevapları yetersiz olabilmektedir.

Verilen cevapların işlenmesi ve analizi için kapalı uçlu sorulara göre daha fazla zaman ve uğraş gerektirmektedir.

Sayısal bilginin elde edilmesi kolaylaştırmak için cevaplayıcıdan, bir konuyla ilgili en iyi ve en kötü üç şeyi veya konunun nedeni olduğunu düşündüğü üç-beş gibi önemli noktayı belirtmesi istenebilir. Ancak bu da cevaplayıcı sınırlandırabilir.

Kapalı uçlu sorular, cevaplayıcıdan sabit bir cevap seçenekleri setinden bir seçeneği seçmelerini ister. Çoktan seçmeli, iki cevaplı ve derecelmeli kapalı uçlu sorulara örnektir.

Kapalı uçlu soruların avantajları:

Cevaplayıcılar kapalı uçlu soruların cevaplanmasını anlatımsal bir cevaba göre çok daha hızlı ve kolay bulurlar. Bir rakamı veya bir seçeneği işaretlemek bir paragraf yazmaya göre daha rahattır.

Kodlanmaları ve bilgisayara veri girişlerinin yapılması oldukça sorunsuzdur.

Kapalı uçlu sorular, cevap olasılıklarını anketin amaçlarıyla ilgili seçeneklerle sınırlarlar.

Herkese aynı bakış açısını ileterek cevaplayıcıların soruyu ve olası cevapları yorumlama girişimlerini daha güvenilir kılarlar. Ortak bakış açısından dolayı anketi cevaplayanların soruları aynı şekilde okumuş ve yorumlamış olmaları olasılığı çok büyüktür.

Kapalı uçlu sorular cevaplayıcılara bilgiyi hafızalarından geri getirme işlemlerinde yardımcı olan ipuçları sağlar. Verilen cevap seçenekleri listesi cevaplayıcılara unut-

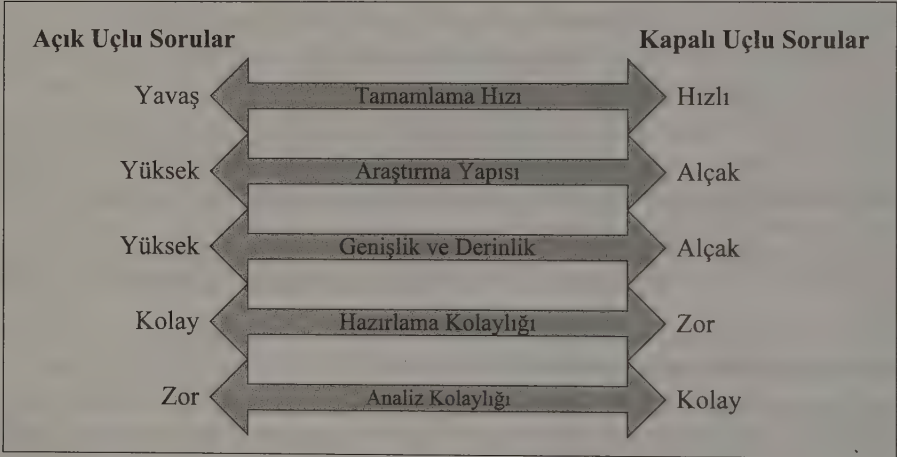
muş olabilecekleri bilgileri hatırlatır. Örneğin; “Son bir yıl içinde şirketin düzenlediği hangi eğitimlere katıldınız?” soru kapalı-uçlu bir soru olarak verilirse, şirkette verilmiş olan eğitim programlarının bir listesini sağlayacak ve diğer hallerde cevaplayıcıların hatırlamalarının çok zor olacağı bilgileri sunacaktır.

Kapalı uçlu soruların dezavantajları:

Kapalı uçlu sorulara sıkça getirilen bir eleştiri, cevaplayıcıları konuyla ilgili gerçek duygularını yansıtmayabilecek cevap seçenekleri arasından bir seçim yapmaya zorladığı şeklindedir. Cevaplar verilmiş olan özellikli seçeneklere bağlı kalır. Kişileri gerçek cevaplarına en benzeyen sabit bir cevabı seçmeye zorlamak cevaplayıcılar arasında bulunan ince ama bazen önemli farklılıkları her zaman tespit edemez.

Kapalı uçlu sorular kişileri bir cevapları olmadığı halde, hatta bazen soruyu anlamadıkları halde bir cevap vermeye iter.

Bir anketteki soruların tümü veya tümüne yakını kapalı uçlu ise, cevaplayıcılar, kendi görüşlerini kendi kelimeleri ile ifade etmelerine izin verilmediği için engellenmiş hissedebilirler. Açık uçlu sorularla kapalı uçluları karşılıklı olarak değerlendirecek olursak kapalı uçlu soruların değerlendirilmesinin daha kolay olmasına karşın hazırlanmasının daha zor olduğu görülmektedir (Şekil-6.3).



Şekil-6.3. Soru türlerinin karşılaştırılması [KENDALL-2004]

Veri toplamada seçilen sorunun tipi kadar soruların anket formunda diziliş sırası da önemlidir. Sorular ya genelden özele ya da özelden genele doğru şeklinde sıralanır. Ancak önce cevaplayıcının direnişi ile karşılaşmamak için ilk yöntem en çok tercih edilebilir. Ayrıca ilk soruların basit, fazla düşünmeyi gerektirmeyen kısa cevaplı olup gitikçe daha zor soruların sorulması bir önceki sorunun, cevaplayıcıyı, onu izleyen soruya hazırlayıcı nitelikte olması gerekir. Böylece hem cevaplayıcının ilgisi ve işbirliği sağlanmış; hem de sorular arasında bağlantı kurularak cevaplayıcı ani konu değişiklikleriyle karşılaşmamış olur.

6.2.3. Araştırma Raporu

Bilgi toplama sürecinde elde edilen sonuçlar bir rapor haline getirilip ilgililere sunulur. Teknik elemanlara sunulan **teknik raporda**;

- Bunu yürüten kişi veya kuruluş adının yer aldığı başlık sayfası
- İçindekiler cetveli
- Araştırmanın amacı
- Araştırmada kullanılan metodoloji (veri toplama yöntemi, veri toplama işlemi, analiz ve yorum)
- Genel sonuç ve öneriler
- Ekler (kullanılan veri toplama formlarının kopyaları, tablolar, grafikler ve bibliyografya).

yer alır. Ancak rapor genel müdüre verilecekse teknik rapora göre daha genel olan **özet rapor** hazırlanır. Özet rapor, teknik rapor tamamlandıktan sonra hazırlanır ve teknik raporun önemli kısımlarını içerir:

- Araştırmanın amacı
- Kullanılan yöntem
- Araştırmada elde edilen sonuçlar
- Teknik ekler.

6.3. Özet

Sistem analizi, problemin çözümlerinin ne olduğunu ve sistemin çalışma sürecini ortaya çıkarmaktır. Sistem kullanıcılarının gereksinimleri, var olan sistemin durumu, seçilen en uygun çözümün değerlendirmesi ve var olan sistemin nasıl iyileştirilebileceğine yönelik öneriler yer almaktadır.

Sistem gereksinimlerinin analizi ve tanımlanması, sistem analisti ve müşterinin ortak çalışması ile gerçekleştirilmektedir.

Sistem analizi çalışmasının ilk adımı olan sistem hakkında bilgi toplama, görüşme veya anket yoluyla sağlanmaktadır. Görüşme sırasında sorulacak sorular, açık ya da kapalı uçlu olarak hazırlanmaktadır. Sistem analizi kapsamında, bilgi toplama aşamasından sonra yorumlama, analiz ve modelleme yapılmaktadır.

6.4. Sorular

6.4.1) Yazılım Gereksinim Analizi sırasında yapılan en önemli hata ve eksikler nelerdir?

6.4.2) Bir otel için var olan bilgi sisteminin incelenerek, yeni bir bilgi sisteminin kuruluş sürecinde gerçekleştirilecek sistem analizi ve tasarımı çalışmalarında bulunduğunuzu varsayarak; bu çalışmanın tümünde yapmanız gereken iş adımlarını belirtiniz ve her bir adımı kapsamı, amacı ile açıklayınız.

6.4.3) Yukarıdaki çalışmanın ön inceleme sürecinde otel yöneticisine sorulacak bir açık, bir kapalı uçlu soru hazırlayınız.

6.4.4) Yukarıdaki çalışmanın ön inceleme raporunu hazırlayınız.

6.4.5) XYZ şirketi temizlik ürünleri toptan satış ve perakende dağıtımını yapmaktadır. Firma sahibi depodaki envanteri yönetmek için bir bilgi sistemi geliştirmek istemektedir. Bu sistemin ayrıca telefon, elektronik posta veya satış temsilcileri aracılığı ile alınan müşteri siparişlerini izlemesi istenmektedir. Siz, yukarıdakilerden başka bir şey bilmediğiniz bu sistem hakkında bilgi toplamak ve sistem yapısını tartışmak üzere şirket sahibine hangi soruları sorarsınız? (Soru türlerini belirterek açıklayınız.)

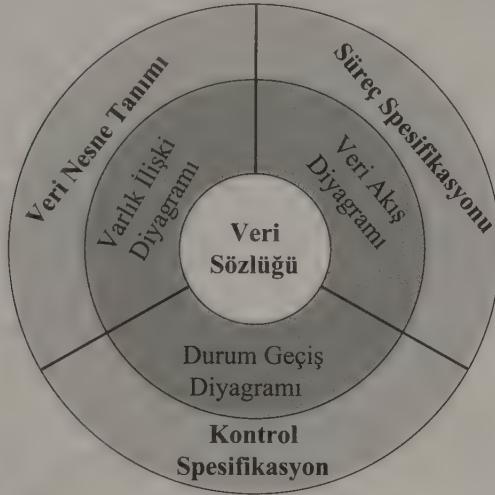
BÖLÜM 7

SİSTEM ANALİZİ MODELİ -GEREKSİNİM ANALİZİ-

Sistem analizi süresince oluşturulan model aşağıdaki üç amacı yerine getirmelidir:

- Müşteri ihtiyaçlarını açıklamak
- Yazılım tasarımının nasıl oluşturulacağına dair temeli oluşturmak
- Yazılım, bir kere oluşturulduktan sonra belirlenen ihtiyaçları karşılayıp karşılamadığını onaylayan unsurları belirlemek.

Bu amaçları yerine getirmek için yapısal analiz süresince oluşturulan analiz modeli Şekil-7.1'de gösterilmiştir:



Şekil-7.1. Analiz modelinin yapısı [PRESSMAN-1997]

Modelin ortasındaki veri sözlüğü, yazılım tarafından kullanılan ve üretilen tüm veri nesnelerinin tanımını içerir. Merkezin etrafındaki üç farklı diyagramdan **varlık ilişki diyagramı** (*Entity Relationship Diagram*, ERD), veri nesneleri arasındaki ilişkiyi açıklar. Her veri nesnesinin özelliklerinin açıklandığı ERD, veri modelleme aşamasında kullanılır.

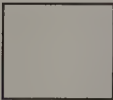
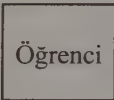
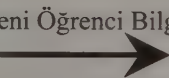
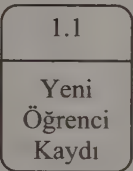
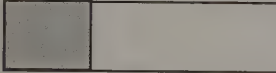
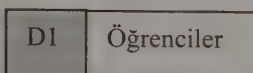
Sistem içinde her verinin nasıl taşındığı ve bu veri akışını sağlayan fonksiyonların neler olduğu **veri akış diyagramında** (VAD, *Data Flow Diagram*, DFD) tarif edilir. VAD, sistem analizi süresince temeli oluşturan fonksiyonların modellenmesindeki kullanılan bilgiyi sağlar.

Durum geçiş diyagramı (*State-Transition Diagram, STD*) sistem dışındaki olaylar sonucunda sistemin nasıl hareket ettiğini gösterir. Bu amacı gerçekleştirmek için STD’de sistemin her farklı davranışı (ki buna **durum** denilir) ve bir durumdan başka duruma geçerken olan biçimi gösterilir. Dolayısıyla STD davranışsal modellemenin temelini oluşturmaktadır.

7.1. Veri Akış Diyagramı

Veri akış diyagramında sistemin varlıkları, süreçleri, sistemdeki veri depoları ile bunlar arasında verinin nasıl aktığı gösterilir. Bu nedenle veri akış diyagramında temelde dört simge bulunmaktadır. Tablo-7.1’de gösterildiği gibi dörtgen, varlığı; ok işareti, verinin akış yönünü; oval köşeli dörtgen süreci; bir köşesi açık dörtgen ise veri deposunu temsil etmektedir.

Tablo-7.1. Veri akış diyagramı simgeleri

Simge (Gane-Sarson simgesi)	Anlam	Örnek
	Varlık	
	Veri Akışı	
	İşlem	
	Veri Deposu	

Sisteme ait VAD, bu simgelerin kombinasyonudur. Bir VAD’de birden fazla varlık olabileceği gibi veri akış oklarını üst üste çakışmasını önlemek için aynı varlığın birden fazla kullanılması olasıdır. VAD’de ok iki uçlu değildir; okun ucu verinin ulaşması gereken hedefi gösterir. Bu kaynak, bir veri deposu olabileceği gibi bir varlık veya işlem de olabilir. Her okun yalnız bir kaynağı ve hedefi vardır.

Sistemde yer alan işlemler hiyerarşik yapı göz önüne alınarak numaralandırılır (işlemlerin nasıl bir hiyerarşik yapıya yerleştirileceği ilerideki bölümlerde anlatılacaktır). Her işlemin kendine ait özel numarası yuvarlak köşeli dörtgenin üstüne yazılırken işlemin yerine getireceği işlevini belirtecek şekildeki adı alt kısma yazılır.

En son simge olan ucu açık dörtgen belirtildiği üzere veri deposunu temsil etmektedir. Bu veri deposu, teyp, disket, CD vb fiziksel veri saklama nesneleri olabileceği gibi veri tabanındaki tablo isimleri de olabilir. VAD'da her veri saklama deposunun kendine ait özel/tekil bir numarası olmalıdır. Bu numaralandırma genelde D1, D2, D3, ... şeklinde devam eder.

7.1.1. Veri Akış Diyagramı Kuralları

Sistem analisti, VAD ile sistemi rahatlıkla okuyup hakim olabilmektedir. Her analistin aynı VAD'dan aynı sonuçlara ulaşması ve doğru kararları verebilmesi için geliştirilen VAD'in belli kurallar çerçevesinde geliştirilmesi gerekmektedir. Başlıca kurallar Tablo-7.2'de gösterilmiştir. Ancak yine de unutulmamalıdır ki VAD'nın sadece aşağıdaki tablodaki kurallara göre geliştirilmesi sistemdeki veri akışını tamamiyle doğru yansıttığı anlamına gelmez. Kurallara uygun olması teknik açıdan doğru olabilir ama doğru olması VAD'nı geliştiren sistem analistinin başarısına bağlıdır.

Tablo-7.2. Veri akış diyagramı geliştirilmesinde uyulması gereken kurallar

Kural	Yanlış ✗	Doğru ✓
İşlemin sadece çıkışı olamaz.		
İşlemin sadece girişi olamaz.		
İşlem girişleri istenen çıkışı verecek kadar yeterli olmalıdır.		
Her veri deposu bir işlemle ilgili olmalıdır.		
Veri deposu bir varlıkla doğru- dan ilişkide olamaz.		

Tablo-7.2'nin devamı.

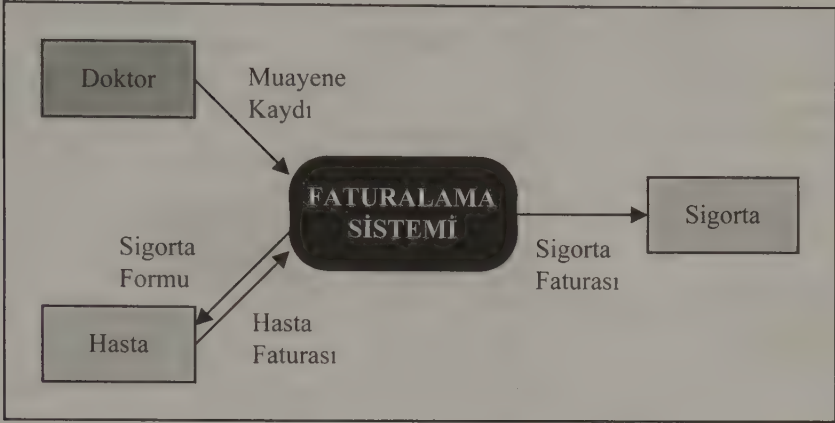
Kural	Yanlış ✗	Doğru ✓
Veri akışı oku iki yönlü olmaz. Bir işlemle veri deposu arasında karşılıklı veri akışı varsa farklı tek yönlü oklarla gösterilmelidir.		
Bir işlemden farklı iki işleme gidecek olan aynı veri, aynı yönde iki uçlu okla gösterilmelidir.		
Veri, hiçbir işlemten geçmeden çıktığı işleme doğrudan dönmemez.		
Veri akış okları üzerinde gösterilen veri, sadece isim formatında olmalıdır.	Siparişi Doğrula → Zaman Çizelgesini Tamamla →	Doğrulanmış Sipariş → Tamamlanmış Zaman Çizelgesi →

7.1.2. Veri Akış Diyagramı Düzeyleri

Veri akış diyagramlarında belli bir sıra izlenir. Ön inceleme sonucunda taslak veri akış diyagramı oluşturulur. Taslak veri akış diyagramı, temel alınarak birinci düzey ve ikinci düzey ayrıntılı veri akış diyagramları çizilir. Ancak bazı kaynaklarda ayrıntılı veri akış diyagramları birinci ve ikinci diye adlandırılırken bazılarında da sıfırıncı ve birinci düzey olarak adlandırılır.

Sistem bir bütün olarak düşünülürse taslak veri akış diyagramı, sistemle veri kaynakları arasındaki ilişkiyi gösterip hangi kaynaklardan veri akışı olduğunu belirtmek için kullanılır.

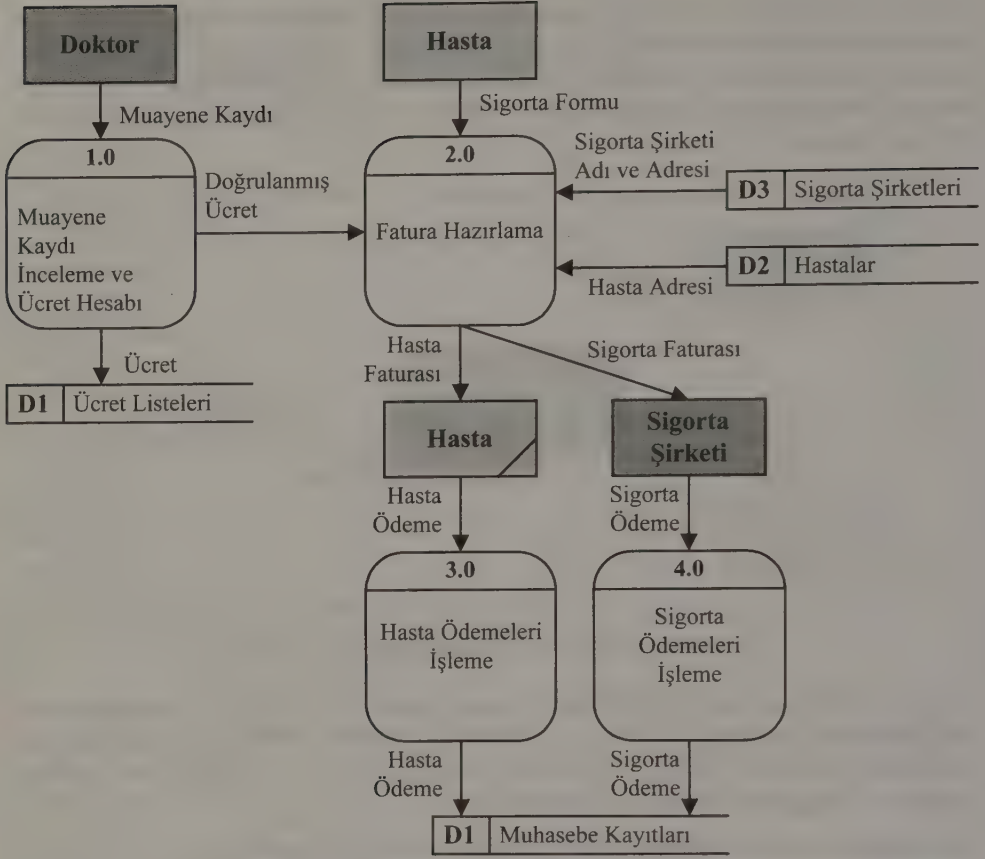
Şekil-7.2'deki taslak veri akış diyagramı, bir kliniğin faturalama sistemine aittir. Bu sistemde doktorlardan gelen hastanın muayene kaydı ve hastadan gelen sigorta formu verileri faturalama sistemine girerken, sistemde gerçekleştirilen işlemler sonucunda hastanın bağlı bulunduğu sigorta şirketine ve hastanın kendisine fatura bilgileri sunulmaktadır.



Şekil-7.2. Taslak veri akış diyagramı örneği

Birinci düzey veri akış diyagramında sistem içindeki alt sistemler, bu sistemlerde gerçekleştirilen işlemler, ilişkili veri kaynakları ve bu kaynaklarla işlemler arasındaki ilişkiler gösterilir. Böylece sistemdeki tüm alt sistemlerin birbiriyle ve dış kaynaklarla ilişkisi genel olarak belirlenmiş olur.

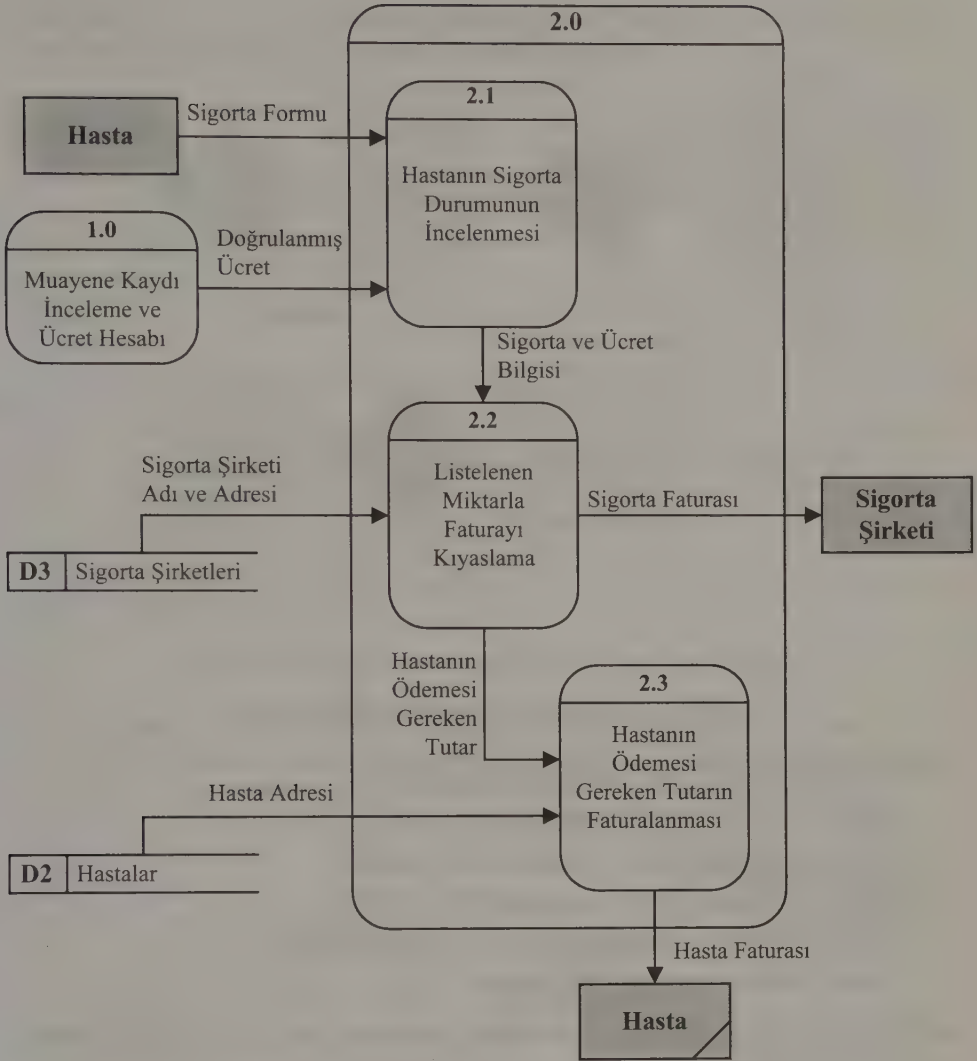
Faturalama sistemi örneğinin birinci düzey veri akış diyagramı Şekil-7.3'de görüldüğü gibi dört alt sistemden oluşmaktadır. VAD'de numaralandırılan her alt sistemin hangi kaynaklarla ve alt sistemle ilişkili olduğu genel olarak görülmektedir. Bu sistemde hastaya ait özel bilgilerin daha önce hasta dosyasına kaydedildiği kabul edilmektedir. Bu nedenle faturalama sisteminde hastadan sadece sigorta formuna ait bilgiler alınmaktadır. Veri akış diyagramlarını okumanın en kolay yolu soldan sağa doğru okumaktır. Veri akış diyagramında veri akış oklarının birbirini kesmesini ve karmaşıklığı önlemek amacıyla aynı kaynak diyagramın farklı yerlerine yerleştirilebilir. Ancak bu kaynağın aslında farklı bir kaynak olmadığını göstermek amacıyla (Şekil-7.3'de gösterilen diyagramda hasta kaynağında olduğu gibi) kaynağa ait olan dikdörtgenin sağ alt köşesine eğik bir çizgi çizilir.



Şekil-7.3. Birinci düzey veri akış diyagramı örneği

İkinci düzey veri akış diyagramında ise bir önceki basamakta çizilen veri akış diyagramında detaylandırılmayan alt sistemlerde gerçekleştirilen işlemler ayrıntılı olarak gösterilir. Sistem tasarımı kapsamında birinci düzey veri akış diyagramındaki her alt sistem için ikinci düzey veri akış diyagramı çizilir. Bu düzeydeki veri akış diyagramının amacı tek bir alt sistemle veri kaynakları arasındaki ilişkiyi göstermektedir.

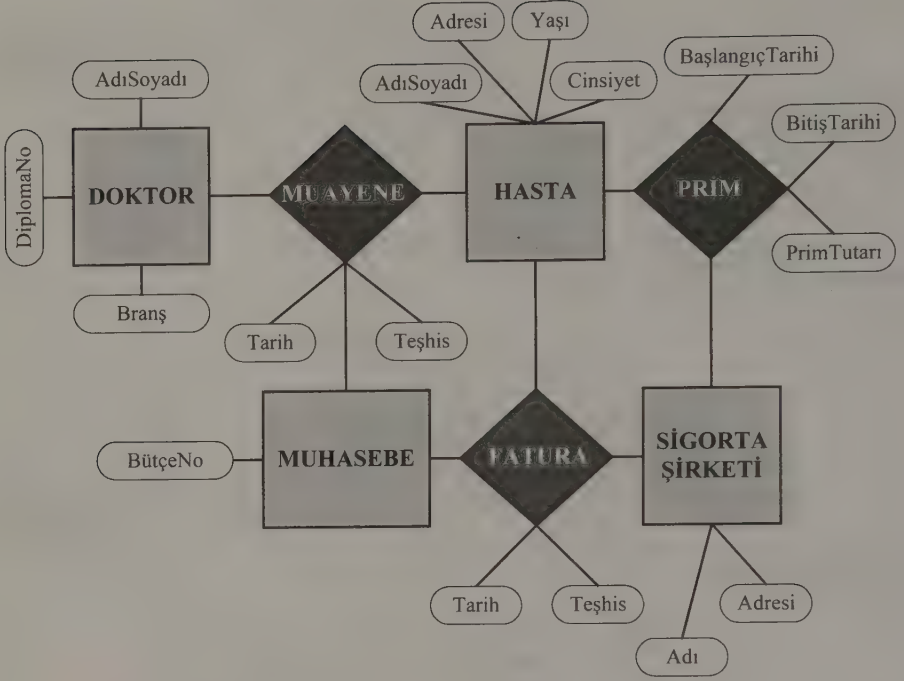
Faturalama sisteminde yer alan fatura hazırlama sisteminin ikinci düzey veri akış diyagramı Şekil-7.4’de gösterilmektedir. Alt sisteme ait işlemler sistemin numarasına göre numaralandırılırlar.



Şekil-7.4. İkinci düzey veri akış diyagramı

7.2. Varlık İlişki Diyagramı

Bilgi sisteminde yer alan veri nesneleri arasındaki ilişkiler, varlık ilişki diyagramında (*Entity Relationship Diagram*, ERD veya ER) grafiksel olarak tarif edilir. Varlık ilişki diyagramında kullanılan standart yapısına göre belli öğeleri ve kuralları vardır. Bu kurallar çerçevesinde veri akış diyagramı bölümünde anlatılan “Faturalama Sistemi”nin ER diyagramı Şekil-7.5’de çizilmiştir.



Şekil-7.5. Faturalama sisteminin ER diyagramı

Faturalama Sistemi için uygulanan gösterim biçiminde;

- **Dikdörtgen**; sistemdeki doktor, hasta vb. varlıkları,
- **Çift kenarlı dikdörtgen**; sistemdeki zayıf varlıkları,
- **Eşkenar dörtgen**; varlıklar arasındaki ilişki setini,
- **Elips**; ilişkiye veya varlığa ait özellikleri,
- **Çizgi**; varlıklar arasındaki ilişki türünü

ifade eder. Bu öğeler arasında çizginin başlangıcı ve bitişi ilişkinin türüne göre değişir.

7.3. Karar Tabloları

Karar tabloları, karmaşık karar mantığını belirlemek için bir mekanizma sağlamaktadır. Karar tabloları, bilgi işlem uygulamalarında yaygın olarak kullanılmaktadır. Bir karar tablosu; durumlar, kurallar, işlemler, kararlar olarak dört ana bölüme ayrılmaktadır (Şekil-7.6). Durumların kurallara uygun olanları (E, evet), olmayanları (H, hayır) olarak işaretlenmektedir. Bu işaretleme sonucuna göre, işlem seçenekleri için kararlar verilmektedir. Karar tablolarında satır ve sütun sayıları belirlidir. Sütun sayısı, durum satır sayısına göre;

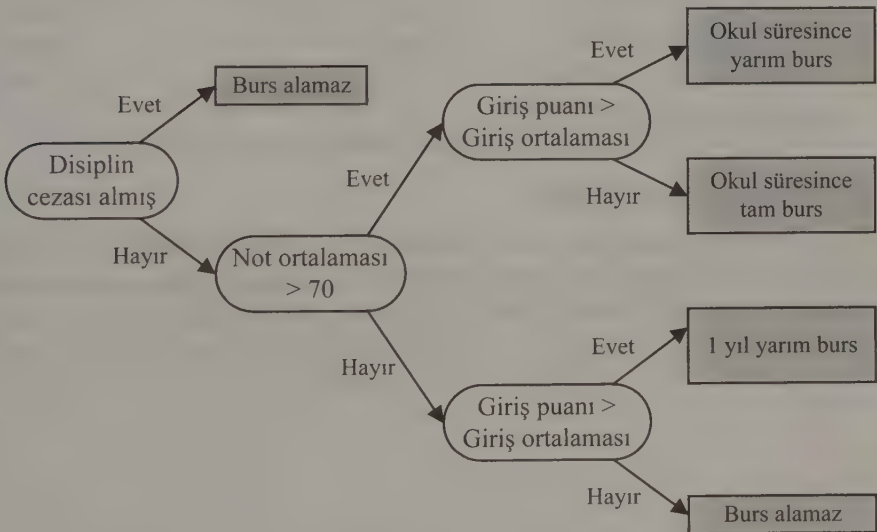
$$\text{Sütun Sayısı} = 2^{\text{Durum Sayısı}}$$

olarak bulunur. Ancak, gerekmeyen sütunlar ya da karar yapısında etkisi olmayan sütunlar kullanılmadığı için sütun sayısı daha az olabilir.

		KURALLAR				
		1	2	3	4	5
D U R U M	Disiplin cezası almış	E	H	H	H	H
	Not ortalaması > 70	H	E	H	E	H
	Giriş Puanı > Giriş ortalaması	H	E	E	H	H
İ Ş L E M	Okul süresince tam burs		X			
	1 yıl yarım burs			X		
	Okul süresince yarım burs				X	
	Burs alamaz	X				X

Şekil-7.6. Karar tablosu örneği (Öğrenci burs sistemi)

Şekil-7.6’da öğrenci burs verme sisteminin karar tablosunda disiplin cezası alan bir öğrenci kesinlikle burs alamayacağı için diğer durumlara bakmaksızın değerlendirme yapılır. Bu nedenle kolon sayısı yukarıda verilen formüle göre daha azdır.



Şekil-7.7. Karar ağacı

Şekil-7.7’de de görüldüğü gibi bazı durumlarda cevap ne olursa olsun aynı durum sorusu karar ağacında tekrarlanmaktadır. Bu örnekte öğrencinin alacağı bursu belirlemek için “not ortalamasının 70’den büyük olma” durumunun hem “evet” hem de “hayır” kollarında “giriş puanının giriş ortalamasından yüksek olma” durumu sorulmalıdır. Dolayısıyla daha karmaşık yapılar için karar tablosu, karar ağacına göre daha avantajlıdır.

Yapısal karar sürecinde karmaşık dallanmalar söz konusu olduğunda karar ağaçları kullanılır. Ancak işletme yönetiminde kullanılan karar ağaçlarından farklı olarak analizde kullanılan karar ağaçlarında olasılıklar yoktur. Çünkü sistem analizinde ağaç yapısının kullanılmasının amacı yapısal karar sürecindeki tüm şartları ve faaliyetleri belirleyip organize etmektir.

7.4. Veri Sözlüğü

Veri sözlüğü (*data dictionary*), bilişim maddelerinin içeriğini dil kurallarına uygun olarak tanımlamak amacı ile düzenlenmektedir. Sadece veri akış diyagramı çizmekle, bilişim alanının analizi tamamlanmış olmamaktadır. Kısa isimler olarak belirlenen veri akışı ve işlemlerin bir sözlük içerisinde tanımlanması gerekmektedir.

Veri sözlüğü veri akış diyagramında, işlem spesifikasyonlarında ve sözlükte geçen her verinin tanımlarından oluşturulmaktadır. Bileşik veriler, öğelerine göre ve basit veriler de anlamlarına göre tanımlanmalıdır. Böylece bir veri sözlüğü veri akışlarının, dosyaların ve veriye uygulanan işlemlerin tanımlarından oluşmaktadır. Bir veri sözlüğünde beş tipte bilgi girişi bulunmaktadır:

- Veri Akış Sözlük Girişi
- Veri Deposu Sözlük Girişi
- Veri Yapısı Sözlük Girişi
- Veri Elemanı Sözlük Girişi
- İşlem Sözlük Girişi

7.4.1. Veri Akış Sözlük Girişi

Veri akış sözlük girişinde (*Data Flow Dictionary Entry*) veri akış diyagramındaki her veri akışı açıklanır (Şekil-7.8). Bu akışın nereden nereye olduğu, verinin yapısının ne olduğu (*kompozit* [K] ya da *elemanter* [E]) gibi bilgileri açıklar.

Veri Akış Sözlük Girişi

Veri Akış Adı	: FATURA
Tanım	: Müşteriye fatura edilecek doküman için gerekli bilgiler
Nereden	: 1.1 Faturayı Hazırla
Nereye	: 1.2 Fatura numarasını hazırla
Veri Yapıları	: Fatura Detayları (K) Müşteri Detayları (K)
Açıklama	:

Şekil-7.8. Veri akış sözlük giriş örneği [internet kaynağı-2]

7.4.2. Veri Deposu Sözlük Girişi

Veri deposu sözlük girişinde (*Data Store Dictionary Entry*) veri akış diyagramındaki birim veri kayıtları açıklanır. Eğer veri akış diyagramında aynı veri deposu birden fazla varsa tek veri deposu girişi kullanılır. Veri depo adı veri akış diyagramında görülen ile aynıdır. Şekil-7.9’da veri deposu sözlük girişi örneği verilmiştir.

Veri Deposu Sözlük Girişi

Veri Depo Adı	: SATIŞ SİPARİŞ FORM DOSYASI
Tanım	: Satış sipariş formlarının saklandığı arşiv dosyasıdır
Veri Yapıları	: Satış sipariş kaydı
Miktar	: Günde yaklaşık 100 kayıt
Erişim	: Sipariş bölümü personeli
Açıklama	:

Şekil-7.9. Veri deposu sözlük giriş örneği [internet kaynağı-2]

7.4.3. Veri Yapısı Sözlük Girişi

Veri yapısı sözlük girişinde (*Data Structure Dictionary Entry*) veri akışında ve veri kayıtlarında yer alan her bir birim veri yapısı açıklanır. Veri yapısı sözlük girişinde bulunan kompozit olan her veri elemanı için veri yapısı sözlük girişinin olması gerekir. Şekil-7.10’da bir örnek verilmiştir.

Veri Yapısı Sözlük Girişi

Veri Yapı Adı	: SATIŞ SİPARİŞ KAYDI
Tanım	: Müşterinin mal siparişi için kullandığı satış sipariş formu
Veri Elemanları	: MusteriNo (E)
	SiparisNo (E)
	SiparisTarihi (K)
	* ParcaNo (E)
	* Miktar (E)
	* BirimFiyat (E)

Şekil-7.10. Veri yapısı sözlük giriş örneği [internet kaynağı-2]

7.4.4. Veri Elemanı Sözlük Girişi

Veri elemanı sözlük girişinde (Data Element Dictionary Entry) veri yapısında yer alan her bir birim veri elemanı açıklanır. Eğer veri elemanı, sadece bir değer içeriyorsa bu durumda sadece bu değerlerden oluşan bir kod tablosu da oluşturabilir. Şekil-7.11’de bir örnek verilmiştir.

Veri Elemanı Sözlük Girişi

Veri Elemanı Adı	: MusteriNo
Tanım	: Müşteriyi tanımlayan numara
Tip	: Numerik
Uzunluk	: 4
Değer Aralığı	: 0001-6999
Diğer Detaylar	:

Şekil-7.11. Veri elemanı sözlük giriş örnekleri [internet kaynağı-2]

7.4.5. İşlem Sözlük Girişi

İşlem sözlük girişinde (Process Dictionary Entry) Şekil-7.12’de gösterildiği üzere veri akış diyagramındaki işlemlerin girdileri, süreci ve çıktıları açıklanır.

İşlem Sözlük Girişi

İşlem Adı	: 2.0 Sipariş Satış Verisini Gir
Girdi	: Satış siparişleri
İşlem Tanımı	: ... yap, eğer değilse yap vb.
Çıktı	: Girilmiş satış siparişleri

Şekil-7.12. İşlem sözlük giriş örneği [internet kaynağı-2]

7.5. Özet

Sistem analizi aşamasında oluşturulan gereksinim modelinde, veri nesne tanımı, süreç ve kontrol spesifikasyonu bulunmaktadır.

Veri modelleme aşamasında kullanılan varlık ilişki (ER) diyagramı, veri nesneleri arasındaki ilişkiyi tanımlar. Sistem içinde verinin nasıl taşındığı ve bu veri akışını sağlayan fonksiyonların, alt fonksiyonların neler olduğu veri akış diyagramında tarif edilir. Ön inceleme sonucunda taslak veri akış diyagramı oluşturulur. Taslak veri akış diyagramı, temel alınarak birinci düzey ve ikinci düzey ayrıntılı veri akış diyagramları çizilir. Durum geçiş diyagramı sistem dışındaki olaylar sonucunda sistemin nasıl hareket ettiğini gösterir.

Karar tabloları, karmaşık karar mantığını belirlemek için bilgi sistemi geliştirme uygulamalarında yaygın olarak kullanılmaktadır. Veri sözlüğü, yazılım tarafından kullanılan ve üretilen tüm veri nesnelerinin tanımını içerir.

7.6. Sorular

- 7.6.1) Gereksinim analizi çalışmasında gerçekleştirilmesi gereken işlemler nelerdir?
- 7.6.2) ABC Spor Ltd.Şti.'de kurulacak Müşteri Destek Sisteminde gerçekleşmesi istenen olaylar:
- 7.6.3) Müşterinin sipariş vermesi-müşterinin ve satış biriminin mal satışı yapılabilirliğini kontrol etmesi-müşterinin - siparişini değiştirmesi veya iptali - yönetim veya müşterinin sipariş durumunu kontrol etmesi-sipariş gönderme-müşterinin mal iadesi-müşterinin katalog istemesi-istemesi-pazarlama biriminin müşteriye promosyon malzemesi ve katalog göndermesi-müşteri muhasebe bilgilerinin güncellenmesi-stok mal bilgilerinin ve kataloğun güncellenmesi (ekleme, silme, fiyat değişikliği) - özel ürün promosyonları- yönetime ara dönem raporları göndermedir. Bu sistem için;
 - a) Taslak, özet (Context) veri akış diyagramı,
 - b) Birinci düzey ayrıntılı veri akış diyagramı,
- 7.6.4) Yeraltı treninin otomatik bilet makinesinin işleyişi aşağıdaki gibidir:

“Makine kullanıcısı gidilecek yeri, bilet tipini (sadece gidiş, gidiş-dönüş, aynı gün gidiş-dönüş) ve indirim kartı olup olmadığını belirtir; makine bilet fiyatını görüntüler. Yeterli para atıldığında, makine, üzerinde gidilecek yer-bilet tipi-indirimli olup olmadığı, fiyat ve tarih basılmış olan bilet verir. Eğer bütün para atılmışsa, bozuk para geri verir. Gün sonunda, her tipteki bilet sayısı ve her varış istasyonuna alınmış olan bilet sayısı gibi istatistik bilgi kaydetmektedir. Bu istatistik bilgi gün sonunda istasyon yönetimi tarafından kontrol edilmekte, haftada bir özetlenerek, ana bilgisayar sistemine transfer edilmektedir.”

Bu sistem için bilgi sistemi geliştirme sürecinde;

 - a) Birinci düzey ayrıntılı veri akış diyagramını
 - b) İkinci düzey ayrıntılı veri akış diyagramını
 - c) ER diyagramını

çiziniz.

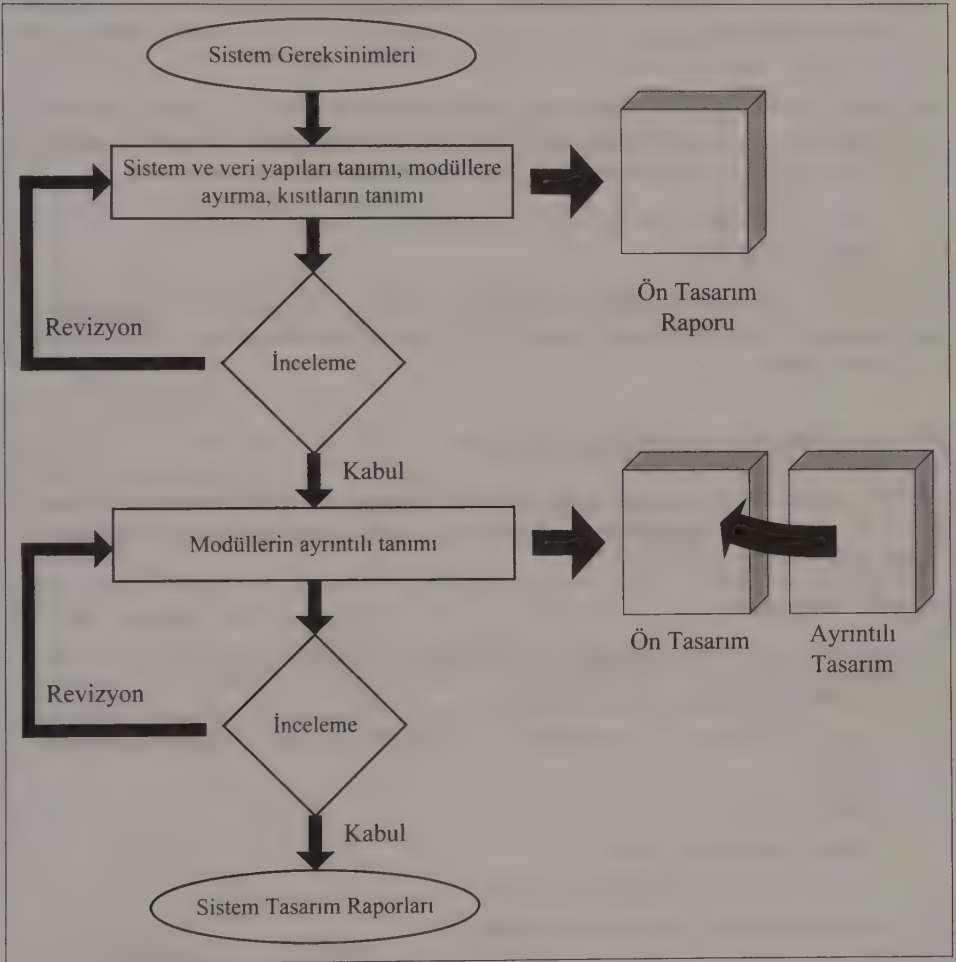
BÖLÜM 8

SİSTEM TASARIMI

Belirlenmiş olan bir dizi gereksinimin, ana sistem içerisinde işlevsel bir öge haline getirilmesi, başka bir deyimle, yazılım şekline dönüştürülmesidir. Sistem tasarımı,

- Ön tasarım,
- Ayrıntılı tasarım,

olarak iki aşama halinde gerçekleştirilmektedir (Şekil-8.1).



Şekil-8.1. Sistemin tasarım aşamaları

Ön tasarımı, sistemin belirlenmiş olan amaç ve hedeflere nasıl ulaştırılabileceğine ilişkin tanımlar geliştirilmektedir. Bunun için; sistemin işlevleri tanımlanmakta ve modül adı verilen bağımsız öğelere ayrılmakta, veri yapıları oluşturulmakta, modül arabirimleri kurulmakta, kısıtlar belirtilmektedir. Bir rapor halinde düzenlenen ön tasarım, incelenerek kabul edilmekte ya da tekrar düzeltilmektedir.

Ayrıntılı tasarımı; ön tasarım aşamasında oluşturulan modüller alt modüllere ayrılmakta ve ayrıntılı olarak tanımlanmaktadır. Kütükler, ekran görüntüleri ve rapor biçimleri tasarlanmakta, programlar için ayrıntılı planlar düzenlenmektedir. Böylece hazırlanan tasarım raporu incelemeye sunulmaktadır.

Ayrıntılı tasarım raporu kabul edilince program planları bir bilgisayar dilinde kodlanarak dış belleklere aktarılmaktadır.

Geliştirme sürecinin her aşamasında “kalite kontrolü” yapıldığı gibi, ayrıca son aşamada da “kabul muayenesi” anlamında bir testten geçirilmektedir. Bu amaçla, sistem projesinin planlanmasında bir “sistem denetleme planı” ve son aşamada da kabul muayenesi için bir “test planı” düzenlenmektedir.

Test; önce her bir modülün, sonra da bir bütün halinde sistemin testten geçirilmesi ve onaylanması ile tanımlanmaktadır. Sistemde kullanıcılar tarafından bulunan hataların çoğunun nedeni problemi ifade etme ve anlama hatasıdır.

8.1. Genel Tasarım

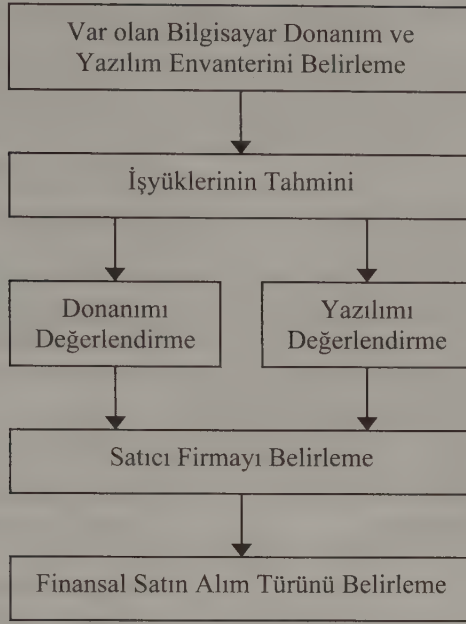
Genel tasarım, sistemin alt yapısını belirleme ve sistem mimarisinin oluşturulmasını kapsamaktadır. Genel tasarımı sistem analizinde oluşturulan sistem gereksinimleri girdi olarak alınır.

8.1.1. Sistem Altyapısını Belirleme

İşletmede var olan donanımın, yazılımın ve ağ yapısının belirlenmesiyle başlayan Şekil-8.2’de şematik olarak gösterilen süreçte amaç önerilen sistem çözümüne uygun alt yapının ortaya konmasıdır.

Envanter belirleme;

- Sistem analisti, var olan sistemin alt yapısının durumunu görmek için;
- Donatım modelini ve üreticisini
- Donatım durumunu (kullanılmıyor, bakıma ihtiyacı var, çalışır durumda vb. şeklinde)
- Donatım yaşını
- Donatım planlanan yaşını
- Donatım işletme içindeki fiziksel yerini
- Donatım sorumlu çalışanı veya bölümü
- Donatım finansal durumunu (işletmenin kendi malı, kiralık ve leasing yapılmış vb. şeklinde) belirlemelidir.



Şekil-8.2. Bilgisayar donanımı ve yazılım alımı adımları

Envanter listesi elde edildikten sonra önerilen sistem için gerekli olan alt yapıyla kıyaslaması yapılır. Bu bilgiyle yeni alt yapının sistem ihtiyaçlarını nasıl karşılayacağı planlanır.

İş yüklerinin tahmini;

Sistem analisti, sistemin var olan ve planlanan iş yüklerini hesaplayarak donanımın altyapı kapasitesinin bu iş yüklerine göre yeterlilik durumunu belirler. Hesabın tam olarak yapılmasıyla işletmenin sahip olduğu donanımın her yeni sistemin kurulmasıyla gereksiz yere büyümesi ve karmaşıklaşması önlenabilir.

Tablo-8.1’de belirlenen işe göre var olan sistemle önerilen sistem arasında zaman ve maliyet karşılaştırılması yapılmaktadır. Tablodan da anlaşıldığı üzere bu işletmede halen elle yürütülen işin bilgisayar ortamında yapılması önerilmektedir. Var olan sistemde bu işin nasıl yapıldığı anlatılırken önerilen bilgisayar tabanlı sistemde nasıl yapılacağı belirtilip maliyet/saat, harcanan insan ve bilgisayar zamanı karşılaştırılmaktadır.

Tablo-8.1. Var olan sistem ile önerilen sistemin iş yüklerinin karşılaştırılması

	Mevcut Sistem	Önerilen Sistem
İş	Bayi satışlarının aylık raporu	Bayi satışlarının aylık raporu
Yöntem	Dosya takibi	Bilgisayar tabanlı takip
Personel	Bayi yöneticisi	Bilgisayar uzmanı
Maliyet / Saat	30.000.000 TL	15.000.000 TL
Ne Zaman? Nasıl?	Günlük: Her bayi için faturaların dosyalanması Aylık: Hesap makinası kullanılarak günlük kayıtların özetlenmesi ve rapor hazırlanması	Günlük: Fatura bilgilerinin girilip toplamın alınacağı programın çalıştırılması Aylık: Raporlamayı yapıp yazdıran programın çalıştırılması
Gerekli Olan İnsan Zamanı	Günlük: 20 dakika Aylık: 8 saat	Günlük: 4 dakika Aylık: 20 dakika
Gerekli Olan Bilgisayar Zamanı	Yok	Günlük: 4 dakika Aylık: 20 dakika

Donanım değerlendirme;

Bilgisayar donanımının değerlendirilme sorumluluğu yönetim, kullanıcı ve sistem analistine aittir. Çünkü sistem için gerekli donanımı belirlemek sistem analistinin görevi olsa da bu donanımdan tam kapasite yararlanılması ve değerlendirilmesi yönetime ve son kullanıcılara bağlıdır. Bu nedenle sistem analisti, sistem için gerekli bilgisayar donanımının satın alımından önce yönetimi bilgilendirmeli ve kullanıcıyı da bu konuda eğitmelidir.

Bundan önceki adımda tahmin edilen iş yüklerine ve mevcut bilgisayar envanterine göre değerlendirme yapılarak projenin ihtiyaçları belirlenir. Gerçekleştirilmesi muhtemel sistemi konfigürasyonuna göre satıcılardan alınacak bilgi bu aşamada önem kazanmaktadır. Sonuç olarak iş yüklerinin farklı sistemler üzerinde simülasyonu yapılarak karşılaştırması yapılır. Bu kıyaslama sürecinde sistem analistinin ve kullanıcının göz önünde bulundurması gereken kriterler;

İşlemin gerçekleşmesi için geçen ortalama süre (Bu süreye giriş verisinin sisteme girilmesi ve çıkışı elde etme de dahildir.)

- Sistemin toplam kapasitesi (Herhangi bir problem oluşmadan aynı zamanda kaç işlem gerçekleştirildiği.)
- CPU'nun atıl zamanı
- Önerilen belleğin büyüklüğü

Değerlendirme sonucunda var olan sistem donanımı, hiç değiştirilmeden yeni sistemde de kullanılacağı gibi yenilenebilir ya da küçük değişiklikler yapılarak kullanılabilir.

Yazılım değerlendirme;

Günümüzde paket yazılımların sayısının giderek artması sonucu donanım seçiminde olduğu gibi yazılım seçiminde de sistem analisti satıcılarla karşı karşıya gelmektedir. Bu nedenle sistem analisti, Tablo-8.2’de gösterilen kriterlere göre yazılım alım kararını üst yönetimle beraber vermelidir.

Tablo-8.2. Yazılım değerlendirme kriterleri

Yazılım Gereksinimleri	Yazılım Özellikleri
Performans Etkinliği	• İhtiyaç duyulan tüm işleri yerine getirme • Tüm işleri istenildiği şekilde yerine getirme • İyi tasarlanmış ekran çıktıları • Yeterli kapasite
Performans Verimliliği	• Hızlı cevap zamanı • Verimli giriş • Verimli çıkış • Verimli veri saklama • Verimli yedekleme
Kullanım Kolaylığı	• İyi tasarlanmış kullanıcı arabirimi • Yardım menüleri • Esnek arabirim • Yeterli geri besleme • Hata düzeltme
Esneklik	• Girdi seçeneği • Çıktı seçeneği • Diğer yazılımlarla birlikte kullanılma
Dokümantasyon Kalitesi	• İyi organizasyon • Yeterli çevrim içi kaynak • Web sitesinde sık sorulan soruların(FAQ) bulunması
Üretici Desteği	• Teknik destek • Ürün güncellemesinin Web sitesi üzerinden yapılabilmesi

Satıcı firmayı belirleme;

Satıcı değerlendirmedeki en önemli etken satıcının kullanıcıya sunduğu hizmetlerdir. Bu hizmetleri Tablo-8.3’de görüldüğü gibi dört temel grupta değerlendirmek mümkündür.

Tablo-8.3. Satıcı değerlendirilmede kriterleri

Temel Satıcı Hizmetleri	Satıcının Sunduğu Hizmetler
Donanım Desteği	• Ürün kalitesi • Garanti
Yazılım Desteği	• Toplam yazılım ihtiyacı • Özel programlama gereksinimi • Garanti
Kuruluş ve Eğitim	• Zaman programına uyma • Eğitim • Müşteri hizmetleri
Bakım	• Rutin bakım işlemleri • Acil durumdaki cevap verme süresi

Ancak bilgisayar donanımının markasını ve modelini değerlendirmek, sadece fiyatları karşılaştırarak en uygun olanını seçmekten ibaret değildir. Gelecekte ihtiyaç duyulduğunda sisteme yeni eklemelerin yapılabilmesi, sistemin gerektiğinde geliştirilmesi durumunda diğer markalarla uyumlu olması, belleğin gerektiğinde artırılabilmesi ve üretici firmanın güvenilir olması da değerlendirme aşamasında göz önünde bulundurulması gereken diğer kriterlerdir.

Satın alma şeklini belirleme;

İşletme finansal durumuna bağlı olarak bilgisayar donanımı satın almak yerine leasing yapabilir veya kiralayabilir. Her üç yöntemin de avantajları olduğu gibi dezavantajları vardır (Tablo-8.4). İşletme yönetimi bu kararı verirken sadece bütçesine göre değil uzun dönem stratejilerine göre hareket etmelidir.

Tablo-8.4. Donanım edinme yöntemlerinin karşılaştırılması

	Avantajları	Dezavantajları
Satın Alma	• Uzun dönemde leasing ve kiralamadan ucuz olması • Sistemin değiştirilebilmesi • Vergi indirimi sağlaması • Tam kontrol	• İlk ödemnin yüksek olması • Eskime riski • Seçimin yanlış yapılması sonucu yatırımın batma riski • Tam sorumluluk
Leasing	• Sermayenin bağlanmaması • Finanslamaya ihtiyaç olmaması • Kira bedelinden düşük olması	• Sözleşme bittiğinde işletmenin sisteme sahip olamaması • Ödemenin gecikmesi durumunda yüksek faiz oranı • Satın almadan daha pahalı olması

Kiralama	• Sermayenin bağlanmaması • Finanslamaya ihtiyaç duyulmaması • Sistemin kolayca değiştirilmesi • Genelde bakım ve garantinin fiyat kapsamında olması	• İşletmenin sistemin gerçek sahibi olmaması • Satıcı kendi riskini üstlendiği için kira bedelinin çok yüksek olması
-----------------	---	---

Satın alıp almama kararındaki en belirgin etkenlerden biri de sistemin planlanan süresidir. Aşağıdaki örnekteki kiralama ve leasing maliyetlerini göz önüne alındığında sistemin üç yıldan daha fazla kullanılması planlanıyorsa donanımı satın almak daha cazip olmaktadır.

Örnek:

Aynı donanım için

Aylık kira : 230,00 YTL

Aylık leasing : 210,00 YTL

(Leasing için) İlk ödeme : 700,00 YTL

Satın alma : 7.700,00 YTL

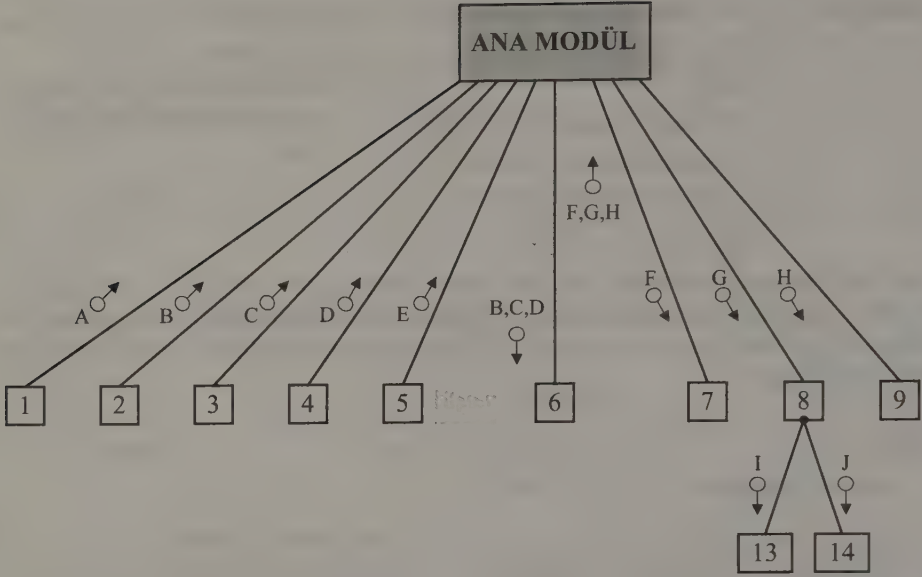
olarak kabul edilirse.

*Buna göre 3 yıl için kiralama bedeli 8.280,00 YTL; leasing bedeli 8.260,00 YTL (210,00 * 36 + 700,00 = 8.260,00) olarak belirlenir.*

Ancak sistem üç yıldan daha kısa ömürlü olacaksa bu durumda leasing işletme için iki açıdan yararlı olacaktır. Birincisi bilgisayar teknolojisi çok hızlı değiştiği için satın alma durumunda işletmenin yatırımı da o ölçüde hızlı eskiyecektir. İkincisi ise işletme bilgisayar donanımına yatırması gereken bütçeyi başka bir alanda değerlendirmeye olanağı bulacaktır.

8.1.2. Sistem Mimarisi

Mimari tasarımda, programın ana yapısal öğeleri arasındaki ilişkiler belirlenmektedir. Burada asıl amaç; modüler (bölümlere ayrılmış) bir program yapısı geliştirmek ve modüller arasındaki ilişkilerin denetimini göstermektir. Ayrıca; program yapısı ve veri yapısı birbirine karıştırılmakta, verinin program içerisinde akışını sağlayacak olan ara birimler tanımlanmaktadır. Sistem mimarisi, yapı diyagramı (*structure chart*) ile açıklanabilir (Şekil-8.3).



Şekil-8.3. Yapısal diyagram

Diyagramdaki karelerde sistemdeki modüllerin isimleri bulunur. Diyagramda yukarıdan aşağıya doğru alt modüllere inilir. Bu durumda diyagramın en üst seviyesinde sistem yer alır. Ancak sistem çok büyük olduğunda sistemi oluşturan her ana modül için ayrı bir diyagram çizilebilir. Diyagramda, etkileşimde olan modüller birbirine çizgiyle bağlanır. Bir modülün altında yer alan elmas şekli, üst modülün elmas işaretini bağlı olan modüllerden sadece bir alt modülü çağıracağını gösterir. Modüllerin birbiriyle sağladığı veri iletişimini, bir ucunda daire olan ok temsil edilir. Şekil-8.3'deki yapısal diyagramda modüller numarayla modüller arasında taşınan veri ise bir harfle temsil edilmiştir. Okun ucundaki dairenin içinin dolu olması modüller arasındaki iletişimin mesaj veya kontrol parametresi (*flag*) olduğunu gösterir.

İyi bir gösterim, sistem öğeleri arasındaki ilişki ve etkileşimlerin kavranmasını kolaylaştıracak için sistem tasarımının şematik olarak açıklanması, büyük bir önem taşımaktadır. Buna karşılık kötü bir gösterim, uygun bir tasarımı bile karmaşık ve anlaşılmaz bir hale getirebilmektedir.

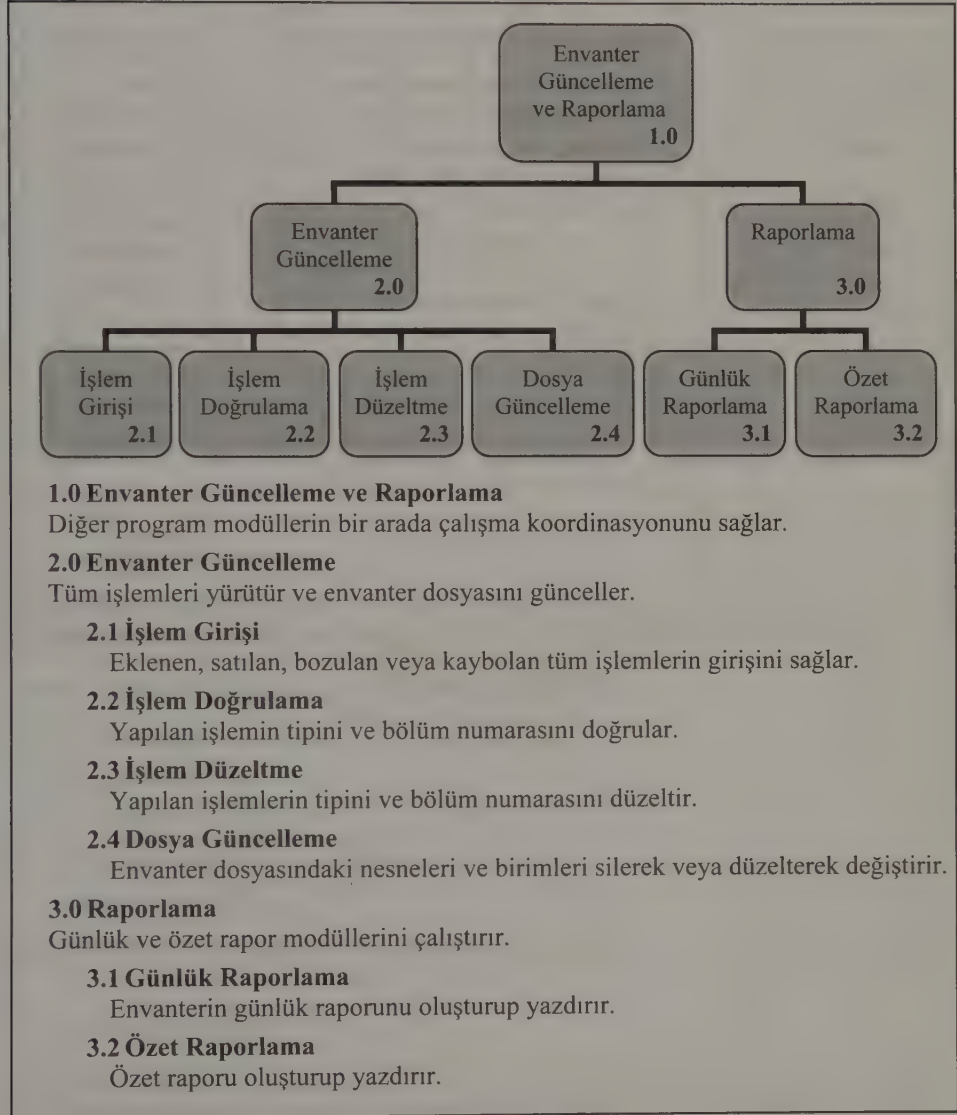
- Tasarım spesifikasyonları aşağıdaki gibi gruplandırılmaktadır:
- Ön tasarım spesifikasyonları; yazılım sisteminin genel özelliklerini ve ilişkilerini göstermektedir.
- Mimari tasarım spesifikasyonları; sistemin yapısını ve kuruluşunu belirlemektedir.
- Ayrıntılı tasarım spesifikasyonları; modüller içerisindeki kontrol akışını, veri gösterimini ve diğer algoritmik ayrıntıları içermektedir.

Tasarım notasyonlarından biri olan HIPO (*Hierarchy and Input-Process-Output*) diyagramı, sistemdeki her fonksiyonun birbiriyle olan ilişkisini belirleme amacıyla kullanılır. Sistem analisti, kullanıcıdan topladığı verilere göre sistemin temel fonksi-

yonlarını belirledikten sonra bu temel fonksiyonların alt fonksiyonları tanımlayarak fonksiyon hiyerarşisini oluşturur.

Görsel bir yöntem olan HIPO'daki standartlaştırılmış semboller sayesinde sistemin fonksiyonlar daha kolay anlaşılabilir. HIPO yönteminde 3 temel diyagram vardır;

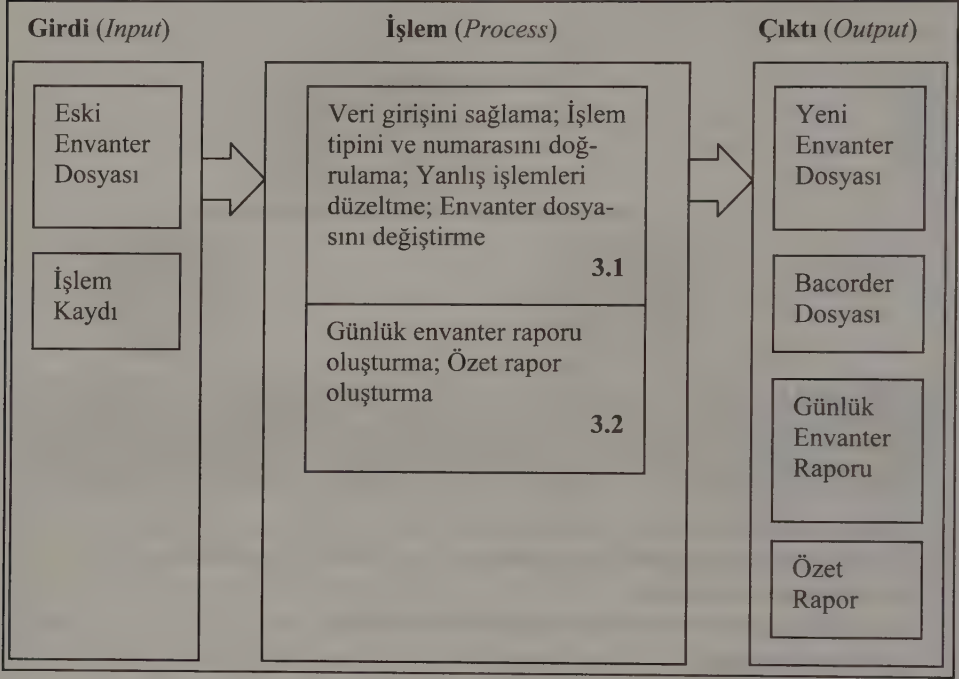
- Görsel İçerik Tablosu (*Visual Table of Contents-VTOC*)
- Genel IPO (*Input Process Output*) Diyagramı
- Ayrıntılı IPO Diyagramı



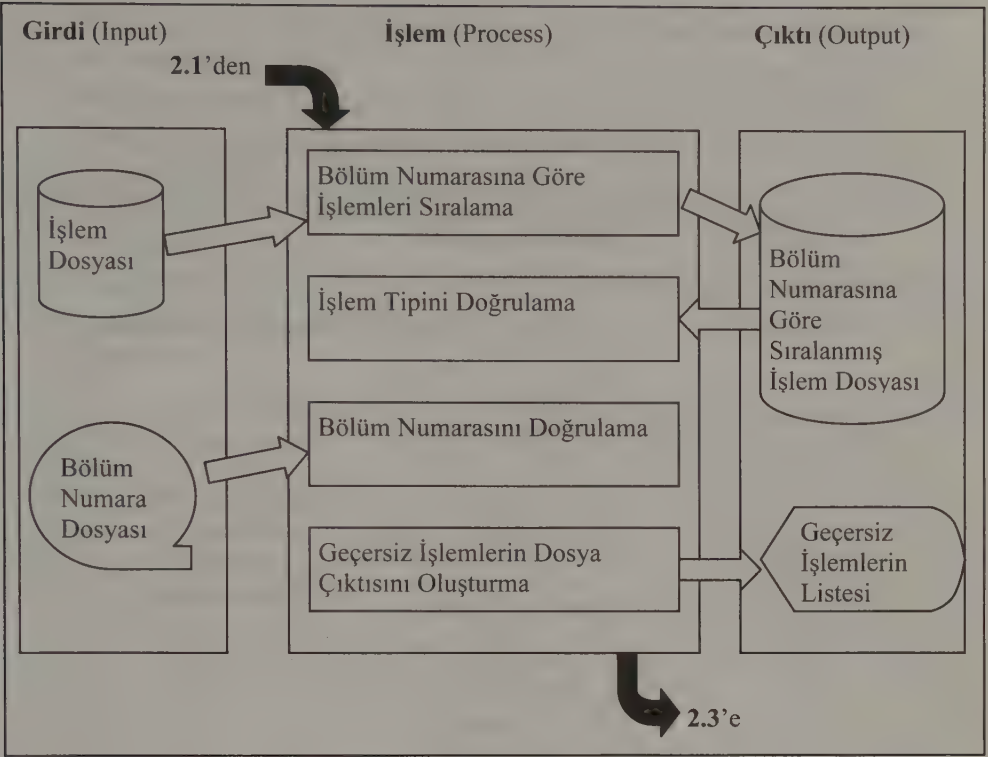
Şekil-8.4. VTOC şeması

Şekil-8.4’de de görüldüğü gibi her fonksiyon hiyerarşik yapıdaki yerine göre numaralandırılır. Bu aşamadan sonra diyagramda yer alan her fonksiyon için Şekil-8.5’de bir örneği verilen genel IPO (*Input-Output-Process*) diyagramı ve Şekil-8.6’da ayrıntılı IPO diyagramı tanımlanır.

HIPO diyagramı, sadece sistemdeki fonksiyonları belirlemek için kullanılır. Dolayısıyla sistemin fonksiyonel olmayan gereksinimlerini bu diyagramda belirtilmemektedir.



Şekil-8.5. Genel IPO



Şekil-8.6. Ayrıntılı IPO örneği

8.2. Ayrıntılı Tasarım

Tasarım sürecinde ilk iş, çıktıların belirlenmesidir. Böylece girdi ve saklama ortamları, veri tabanı gereksinimleri ortaya konmuş olmaktadır.

Ayrıntılı tasarım kapsamında;

- çıktı tasarımı
- girdi tasarımı
- veri tabanı tasarımı
- program tasarımı
- ara birim tasarımı

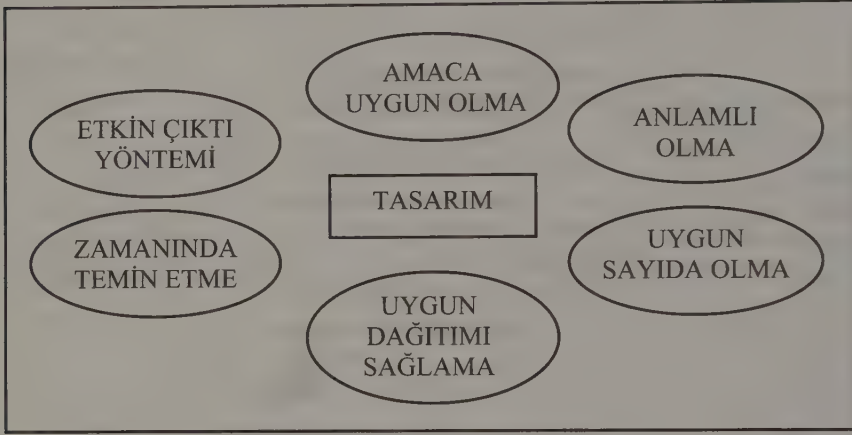
gerçekleştirilir. Bu farklı tasarım maddeleri aşağıda ayrıntılı olarak anlatılmaktadır.

8.2.1. Çıktı Tasarımı

Çıktı, bilgi sisteminin kullanıcılara verdiği bilgidir. Sistem analistleri, en yararlı çıktıyı elde etmek üzere kullanıcılar ile etkileşimli çalışmalar yapmaktadırlar. Çıktı tasarımında;

- belirlenen amaca hizmet etme
- kullanıcı için anlamlı olma
- uygun miktarda olma
- çıktıların hangi kullanıcılara dağıtılacağına doğru belirlenmesi
- çıktının zamanında sağlanması (günlük, aylık, yıllık veya koşula bağlı raporlar)
- doğru çıktı yönteminin (ortamının) seçilmesi

amaçları göz önünde bulundurulmalıdır (Şekil-8.7).



Şekil-8.7. Çıktı tasarımında amaçlar

Çıktıların kullanıcıları etkileyecek şekilde yönlendirilmesi

- bilgilerin, alfabetik - kronolojik veya maliyete göre sıralanması
- kabul edilebilir sınırların belirlenmesi
- grafik tipi - rengi ve ölçeğinin ayarlanması

ile gerçekleştirilmektedir. Bazı renklerin özel anlamları vardır. Örneğin, işletmecilikte bütçe değerlerinin kırmızı ile görünmesi problem belirtir. Bir bütünün paylaşım oranlarını ortaya çıkarmak için pasta (pie) grafik tipi, birden fazla veri grubunu kıyaslamak için ise çubuk (bar) grafik tipi daha uygundur. Basılı raporlar ve ekran görüntüleri olarak hazırlanan çıktıların tasarımı sürecinde, kullanıcı etkileşimini sağlamak amacıyla, prototip üzerinde çalışılmaktadır. Yönetim bilişim sistemi ve karar destek sistemlerinde, ekran çıktılarının kullanıcıyı etkileyecek şekilde yönlendirilmesi önem kazanmaktadır.

8.2.2. Girdi Tasarımı

Girilen verinin kalitesi, çıktı olarak elde edilen bilginin kalitesini belirler. Bilgi işlemede, 'Hatalı veri girerseniz, hatalı çıktı alırsınız (Garbage In Garbage Out, GIGO)' kuralı geçerlidir. İyi tasarlanan girdiler;

- **Etkinlik:** form ve ekran görüntülerinin yönetim bilgi sisteminde belli bir amacının olması
- **Doğruluk:** sistem analizinde tanımlanan tüm işlemleri yerine getirmesi
- **Kullanım kolaylığı:** bilgi girişi kullanıcılarının fazla zamanını almaması ve ergonomik olması
- **Uyumluluk:** Bir formda diğerine ya da ekran görüntüsüne geçişte düzenin değişmemesi
- **Basitlik:** Gereksiz ayrıntıya yer verilmemesi ve karmaşık olmaması
- **Çekicilik:** ekran ve form yapılarının güzel görünmesi amaçlarını sağlamalıdır (Şekil-8.8).



Şekil-8.8. Girdi tasarımında amaçlar

Tasarım aşamasında karar verilmesi gereken kavramlardan biri de kodlamadır. Kodlamayı, gereksinim analizinden itibaren göz önünde bulundurmalı tasarım aşamasında ise tamamen kesinleştirilmesi gerekmektedir. Etkin giriş ortamının sağlamak için etkin kodlama sisteminin varlığı şarttır. Kodlamadan kastedilen bilgisayar ortamında bilgileri daha rahat tutmak ve dolayısıyla daha kolay ulaşmak için kullanılan sembollerdir.

İyi bir kodlamada dikkat edilmesi gereken durumlar ise;

- Kodlamanın mümkün olduğunca kısa olması gerekir. Ege girilecek bilgiler çok kısa tanımlamalarla ifade edilemiyorsa anlamlı bölümlerinden “-“, “/” ve ya “.” ile ayırarak kolay okunur ve hale girilir duruma getirilebilir.
- Kodlama, kolay değişmeyecek şekilde olmalıdır. Koda başlamadan önce kesin olarak karar verilmiş olmalıdır.
- Tekrarlanmaması, tekil (*unique*) olmalıdır.
- Karışıklığa yol açmamak için birbiriyle karıştırılabilecek sembollerden kaçınmak gerekmektedir. Örneğin “I” (büyük I harfi) ile “1” (bir) ya da “Z”

(büyük z harfi) ile “2”(iki) vb. Bu durumu önlemek için çoğunluğu karakter olan kodlama sisteminde bu harflere yer verilmemelidir.

- Kodlamada kısaltmalar varsa anlamlı olmalıdır.
- Araya eleman eklemeye izin verecek şekilde değişikliğe izin verilebilmelidir.

Veri girişinin doğru tasarlanmasının esas anlamı kodlamanın doğru yapılmasıdır. Veri girişinde ve bilginin izlenmesinde kolaylık sağlamak ve bilgiyi sınıflandırmak kodlamanın başlıca iki amacını oluşturmaktadır. Sadece belli bir işleme bir numara verilerek izlemeyi sağlamak amacıyla kodlama yapılabilir. Örneğin siparişleri birbirinden ayırmak için sıralı sipariş numarası belirlemek buna bir örnektir. Dolayısıyla buradaki kod sipariş numarasıdır.

Bir derginin abonelerinin muhasebe numarası olarak belirlenen bir kod için iki haneli şehir kodu (karakter ya da sayı olabilir), kişinin soyadının sessiz harflerinden oluşan 2 ya da 3 harf, yaşadığı semtin posta kodu, üye olduğu derginin ilk 3 harfi gibi her bir parçası belli bir amaca göre oluşturulan bir kod da kullanılabilir. Eğer bilgiyi oluşturmak için çok fazla sınıf olmayacaksa sadece ilk harf de kullanılabilir. Kodlama yaparken hepsi harflerden veya sayılardan oluşsa da belli anlamlara göre grupta yapılmalıdır.

Önemli unsurlardan biri de kodların karışmasını önlemek için sıralama sonucunun doğru şekilde olmasını sağlamaktır. Örneğin vergi grupları arasında bir tasnif yapmak istenirse burada tarih bilgisinin hangi formatta girildiği önemlidir.

Örnek: Tarih bilgisi ilk sütundaki gibi yapılırsa hatalı bir sıralama elde edilirken bu hata ikinci sütundaki gibi yapıldığı takdirde giderilmektedir.

aaa	dd	yy
eki	16	86
kas	26	85
nis	04	87

yy	aa	dd
85	11	26
86	10	10
87	04	04

Bilgi girişi tasarımı, genellikle kağıt üzerindeki bilgi toplama formları ve ekran yapıları için gerçekleştirilmektedir. Form tasarımı oluşturulmasında;

- kolay doldurulmasına,
- tasarım amacını karşılmasına,
- doğru tamamlamayı sağlamasına,
- güzel görünmesine
- dikkat edilmelidir. Ekran tasarımında ise;
- gereksiz bilgi bulunmaması, pencereler içinde girilecek bilgilerle ilgili açıklamalar olması gibi **kolay kullanım** ve **basitlik**,
- bilgi toplama formları ve diğer ekran görüntüleri ile **uyumluluk**,

- ekrandan ekrana geçme ve başlık kolonunu sabit tutarak diğer kolonları kaydırma şeklinde **hareket kolaylığı**,
- tüm ekranların belli bir düzene uygun hazırlanması ve imleç yapısı, font tipinin seçimi ile **çekici ekran tasarımı**

önem kazanmaktadır.

Ekran Tasarımında Renk Kullanımı: Ekran üzerinde, zemin ve kalem renklerinin doğru belirlenmesi, bilgi girişini kolaylaştıran etmenlerdendir. Fazla parlak ve koyu renklerin kullanımından kaçınarak, önemli alanlar parlak hale getirilmelidir. Zemin ve kalem rengi, zıt renklerden seçilmelidir. En kolay okunan 5 renk grubu;

- sarı üzerine siyah
- beyaz üzerine yeşil
- beyaz üzerine mavi
- mavi üzerine beyaz
- siyah üzerine sarı,

iken en zor okunan gruplar ise, yeşil üzerine kırmızı ve kırmızı üzerine mavi olarak belirlenmiştir.

Doğru veri giriş birimi tasarımını gerçekleştirmek amacıyla;

- etkin kodlama,
- etkin ve verimli veri girişi,
- doğruluk ve geçerlilik (*validation - verification*) testleri ile kalitenin sağlanmasına

önem verilmektedir.

8.2.3. Veri (Yapısı) Tasarımı

Veri tasarımı, veri yapısının tanımlanmasını konu almaktadır. Veri yapısının tasarlanması, sistem tasarımında ilk olarak ele alınan, en önemli görevdir. Veri yapısının program yapısı üzerindeki etkisi ve işlemlerin karmaşıklığı nedenleriyle veri tasarımı, sistem tasarımının kalitesini önemli ölçüde etkilemektedir.

Veri tasarımında ilk iş, gereksinim analizi ve spesifikasyonu aşamasında tanımlanmış olan veri varlıklarının (veri yapılarının) mantıksal gösterimlerinin seçilmesidir.

Veri yapısı tasarımında;

- verinin kullanıcı istediği anda kullanılabilir olması,
- doğru ve uyumlu olması, veri bütünlüğünün sağlanması,
- güncelleştirmenin etkin yapılabilmesi,
- bilgi geri çağırmanın amaca uygun ve etkin gerçekleştirilmesi

amaçlanmaktadır (Şekil-8.9). Ayrıca, saklanan veriden elde edilen bilginin, yönetim, planlama, kontrol ve karar vermede yararlı olacak şekilde düzenlenmesi gerekmektedir.



Şekil-8.9. Veri saklama ortamı tasarımı amaçları

8.2.4. Kullanıcı Arabirim Tasarımı

Kullanıcı arabirimleri, sistemin kullanıcıyla iletişime giren elemanıdır. Arabirimler; doğal dil arabirimleri, soru cevap sorgulamalar, menüler, girdi/çıkı formaları, komut dili şeklinde farklı görünümde olabilirler. Bilgisayar arabirim yazılım sistemleri kullanıcıya;

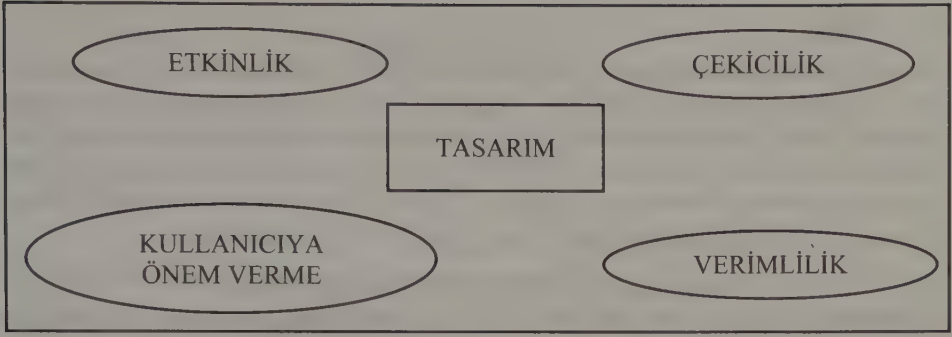
- bilgisayarın girdiyi kabul edip etmediğini
- doğru formda olup olmadığını
- işlemde gecikme olacağını (beklemesi gerektiğini)
- işlem veya isteğin tamamlandığı ya da tamamlanmadığını

belirten bilgiler gönderirler.

Arabirim tasarımcısının görevi, insan yetenekleri ve bilgisayar gücünün birbirini tamamlayacak şekilde bir arada çalışmasını sağlamaktır. Arabirim tasarımında;

- etkinlik (kullanıcıların gereksinimlerine uygun olarak sisteme erişmelerini sağlama),
- verimlilik (hataları azaltma, veri giriş hızını artırma),
- kullanıcıların görüşlerinin alınabilmesi,
- ergonomik olması

amaçlanmaktadır (Şekil-8.10). Kullanıcı görüşünü alabilmek için prototip oluşturularak etkileşim kolaylaştırılmaktadır. Bir kullanıcı arabiriminin bir defada doğru olarak tasarlanmasının genellikle mümkün olmaması nedeniyle, prototip oluşturma, test etme ve iteratif tasarım kullanarak değişikliği planlamaya gereksinim bulunmaktadır. Arabirim tasarımında iterasyon sayısı arttıkça, arabirimin kullanılabilirliği artmaktadır.



Şekil-8.10. Arabirir tasarımında önemli olan unsurlar

İnsan-bilgisayar arabirimleri, diğer mekanik cihazları denetlemek üzere geliştirilen arabirimlere benzerdir. Belirli bir mekanizmayı kontrol etmek üzere geliştirilen arabirimler, mühendislik ve kullanıcı görev modeli olmak üzere iki farklı modele göre tasarlanmaktadır. Mühendislik modeli, sistemi tanıyan kişilerin sistemin tüm işlevlerine ulaşabilmek ve tam denetim altında tutabilmek için kullanacakları arabirimleri geliştirmek amacıyla kullanılmaktadır. Kullanıcı görev modeli ise, kullanıcının gerçekleştirmesi gereken işlevler için sisteme kısa yollar, yönlendirmeler ve yardım menüleri ile kolay erişebilmesini sağlayan, hata yapmayı önleyen ve hızlı öğrenilen arabirimler geliştirmek üzere kullanılan modellerdir. Bu arabirimleri geliştirmek, daha karmaşık ve performansı azaltır niteliktedir.

Arabirimlerde oluşan problemlerden bazıları;

- Bilginin düzensiz görünümde yığınlar şeklinde olması
- Kullanıcının yürütmesi gereken çeşitli görevler arasındaki geçişin uyumlu olmaması
- Ekran üzerindeki komutlarda kullanılan terminolojinin karmaşık olması
- Sistem tarafından verilen hata mesajlarının açıklayıcı olmaması
- El kitaplarının anlaşılabilir derecede karmaşık olması

olarak belirlenmiştir.

8.3. Özet

Sistem tasarımı, sistemin nasıl gerçekleştirileceğinin tanımlanmasıdır. Genel tasarım, sistemin alt yapısını belirleme ve sistem mimarisinin oluşturulmasını kapsamaktadır. Genel tasarımda sistem analizinde oluşturulan sistem gereksinimleri girdi olarak alınır. Sistem altyapısını belirleme, işletmede var olan donanımın, yazılımın ve ağ yapısının belirlenmesiyle başlayan bu süreçte amaç önerilen sistem çözümüne uygun alt yapının ortaya konmasıdır.

Ayrıntılı sistem tasarımı ise, genel tasarımda belirlenen sisteme ait alt sistemlerin tanımlanmasıdır. Ayrıntılı tasarım kapsamında, çıktı tasarımı, girdi tasarımı, veri tabanı tasarımı, işlem tasarımı, ara birim tasarımı gerçekleştirilmektedir.

8.4. Sorular

- 8.4.1) Tasarım notasyonlarından hangisi yukarıdan aşağıya doğru yazılım geliştirmede tasarımın gösteriminde kullanılmaktadır?
- 8.4.2) Arabirim tasarımında ne amaçlanır?
- 8.4.3) Yedinci bölümün sorularında anlatılan ABC Spor Ltd.Şti. ve yeraltı treninin otomatik bilet makinesinin her biri için;
 - a) Yapı diyagramını hazırlayınız ve bu diyagramların yazılım geliştirme sürecinin hangi adımlarında kullanıldığını yazınız
 - b) HIPO diyagram paketini hazırlayınız.

KISIM III

NESNEYE YÖNELİK YAKLAŞIMLA SİSTEM ANALİZİ VE TASARIMI

Bu kısım, geleneksel yöntemlerden farklı olarak nesneye yönelik sistem analizi ve tasarımı konusuna yoğunlaşmıştır. Nesneye yönelik temel kavramlara da yer verilmiştir.

- Bölüm 9:** Genel olarak nesneye yönelik yaklaşımda sistem analizi ve tasarımı anlatılmıştır.
- Bölüm 10:** Nesneye yönelik temel kavramlar örneklerle incelenmiştir.
- Bölüm 11:** Sınıf ve kullanım senaryosu modellemeleri çerçevesinde analizin nasıl yapılacağı açıklanmıştır.
- Bölüm 12:** Tasarım kalıpları tanıtılmıştır. Bölüm 11’de analizi yapılan sistem bu kalıplara ve temel prensiplere uygun olarak tasarlanmıştır.

BÖLÜM 9

NESNEYE YÖNELİK YAKLAŞIMA GENEL BAKIŞ

Sistem geliştirmede nesneye yönelik yaklaşım, Tümlleştirilmiş Modelleme Dilinin (UML: *Unified Modeling Language*) standartlaştırılmasıyla 1990'larda yaygınlaşmaya başlamıştır. Klasik yaklaşımın tersine, veri merkezli, sınıf modelleri üzerinde geliştirilen bir yaklaşımdır.

Nesneye yönelik analiz ve tasarım yöntemleri kullanıldığında, kodların ve modüllerin yeniden kullanılabilirliği kolaylaşmakta, geliştirme süresi kısaltmakta, üretkenlik artmakta, yazılım kalitesi yükselmekte, anlaşılabilirlik kolaylaşmaktadır.

Nesneye yönelik yaklaşım ile yazılım geliştirilirken çeşitli süreç modelleri uygulanabilir. Deneyimler sonucunda ise, kabul gören en iyi özellikler bir araya getirilerek, modelin yapısına uygun tümlleştirilmiş yazılım geliştirme süreci (*Unified Process* - UP) ortaya atılmıştır. UP, genel anlamda tekrarlı ve artımlı bir süreç yapısı çizer. Tek bir model, analiz, tasarım, gerçekleştirme ve test aşamalarında sürekli işlenir, değiştirilir ve iyileştirilir.

Geleneksel yazılım geliştirme metotlarının eksikliklerini mümkün olduğunca azaltan nesneye yönelik yaklaşım, birkaç yeni problemi de beraberinde getirmektedir.

- Sistem analizi, yüksek seviyeli bir soyutlama üzerine kurulmuştur. Sistemin gerektirdiği çözüm ilişkisel bir veritabanı kullanımını gerektiriyorsa, kavram ve uygulama arasında belirgin farklar oluşur. Tasarımdan gerçekleştirmeye geçiş, ancak nesneye yönelik veya nesne-ilişkisel bir veritabanı sistemi kullanılabilirse kolaylaşır.
- Süreç içinde aşamaları birbirinden kesin olarak ayıran kilometre taşlarını belirlemek güçtür. Bu da, yazılım ölçümlerinin yapılabilmesini zorlaştırabilir. Ancak projenin küçük parçalara bölünerek yönetilmesi ve yinelemeli-artımlı süreç, bu güçlüğün üstesinden gelinmesini sağlar.

Gelişen ihtiyaçlar doğrultusunda yeni yazılım ve uygulamaların gerçekleştirilmesi giderek daha karmaşık bir hal almaktadır. Klasik yazılım geliştirme metotları, bu gibi durumlarda yetersiz kalmaktadırlar. Kullanıcı etkileşimli, olay tabanlı yazılımlar geliştirebilmenin zorluğunu ortadan kaldıracabilecek çözüm, nesneye yönelik yazılım geliştirme metotlarının kullanımınıdır.

Nesneye yönelik sistem analizi ve tasarımında UML bir standart olarak yaygın kullanılmaktadır. *Booch*, *Rumbaugh* ve *Jacobson* yöntemlerinin (OMG) bileşkesini esas alır. Ancak kapsamı daha geniştir. Aslında UML, bir yöntem değil, bir modelleme dilidir. Bu dil ile tasarım anlaşılır bir biçimde ortaya koyulabilir. Notasyon, ağırlıklı

BÖLÜM 10

NESNEYE YÖNELİK TEMEL KAVRAMLAR

Gerçek dünyada nesneler, belli sorumlulukları yerine getiren varlıklar olarak karşımıza çıkar. Yazılımda ise, değişik hizmetler veren ve özellikleri olan bir varlık olarak düşünülebilir. Programcı gözüyle bakıldığında ise nesne, veri ve fonksiyonlardan oluşan bir modülden başka bir şey değildir.

Bilgi sistemi geliştirmenin amacı, bir problemi çözmektir. Nesneye yönelik yaklaşımda problemi oluşturan nesneler, gerçek hayattaki yapılarına benzer şekilde modellenmeye çalışılır. Modellenen nesneler özellikleri ve davranışları olan birimlerdir. Problemi çözmek üzere bir sorumluluğu yerine getirmek amacıyla tasarlanırlar.

Gerçek hayattan örneklerle bu özellik ve davranışlar daha iyi anlaşılabilir. Gerçek hayatta gördüğümüz bir ağaç, nesne olarak düşünülebilir. Ağacın özelliklerine örnek olarak yaprak türü, gövde uzunluğu, meyveleri, çiçekleri verilebilir. Ağaç, büyür, yaprak verir, çiçek açar, meyve verir. Bunlar da, ağaç nesnesinin davranışlarıdır.

Nesneler, birbirlerinden kimlik bilgileri ile ayrılırlar. Aynı yemek takımına ait iki tabak, tıpatıp aynı özelliklere sahip nesneler olsalar da, birbirlerinden farklıdırlar. Üretim sırasında her birine bir seri numarası verilseydi, ikisi de farklı seri numaralarına sahip olurlardı.

Nesneye yönelik yaklaşımın hedefi, gerçek hayatta nesnelerin sahip olduğu durum, davranış, kimlik gibi özellikleri örneklemeye, modellemeye çalışmaktır. Sistemler, bu görüş üzerine dayandırılarak gerçekleştirilir.

10.1. Sınıf

Sınıf, aynı özellik ve işlemlere sahip nesne kümesi için bir tanımlayıcıdır. Nesnelerin oluşturulması için, bir şablon olarak düşünülebilir. Bu sınıftan türetilen her nesne, sınıfta tanımlanan özellikleri karşılayan değerlere sahiptir. Türetilen nesneler, sınıfta tanımlanan işlemleri de yapabilirler. Dolayısıyla her nesne, bir sınıfın örneğidir (*instance*).

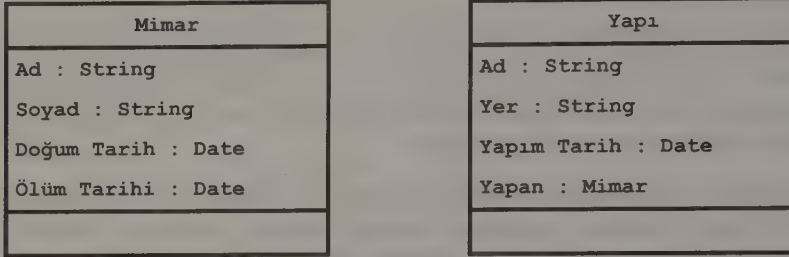
Bir sınıfın UML gösterimi, Şekil-10.1'deki gibi üç bölümü olan bir kutu şeklindedir. En üstteki bölümde, sınıfın adı bulunur. Ortada, sınıfın özellikleri yer alır. Alttaki bölümde ise, işlemler için gerekli tanımlamalar yapılır.

Sınıf Adı
Özellikler
İşlemler() :

Şekil-10.1. Sınıf gösterimi

10.1.1. Özellikler

Özellik, bir tip-değer çiftidir. Sınıflarda özellik tip tanımları yapılır, nesnelerde ise bu özellikler değer taşırlar. Şekil.10.2’de özellik isimleri ve tiplerinin belirtildiği bir gösterim yer almaktadır.



Şekil-10.2. Sınıf özellikleri

Bir sınıfın özellikleri sadece basit veri tiplerinden oluşmayabilir. Bir özellik, başka bir sınıftan tanımlanmış nesneler de olabilir. Şekil GÖKSEL-2’deki “Mimar” sınıfı basit veri tiplerine sahipken, “Yapı” sınıfının “Yapan” isimli özelliği “Mimar” sınıfı tipindedir. Sınıflar arasındaki bu ilişki, “sahip olma” (has a) ilişkisi olarak adlandırılır.

Nesneler, birbirleriyle mesajlar yoluyla haberleşirler. Mesaj, sınıfta tanımlı bir işlemin harekete geçirilmesini sağlar. Bu işlem de, nesnenin içindeki özelliklerin değerlerine erişip onları kullanarak mesaja bir cevap üretir. Bunun sağlanabilmesi için işlemler, diğer nesnelerin kullanımına açık olmalıdırlar. Ancak bir nesneyi kullanmak için, içyapısının bilinmesi gerekli değildir. Nesnenin dışarıya sunduğu hizmetlerin (sorumluluklar) nasıl kullanılacağını bilmesi yeterlidir.

Nesnenin özelliklerine ise, direk erişim yapılmaması veri tutarlılığı açısından oldukça önemlidir. Bunu sağlamak için sınıflar yazılırken bazı üyeler gizlenir, dışarıdan erişimleri engellenir. Bir sınıfın üyelerine erişim için üç etiket bulunmaktadır.

Public (açık): Üyeler diğer nesnelerin kullanımına açıktır. UML’de üyenin başına koyulan “+” işareti ile gösterilir.

Private (özel): Üyeler sadece o sınıfın nesneleri tarafından kullanılabilir. UML gösterimi, “-” işaretidir.

Protected (korunmalı): Üyeler sadece o sınıfın nesneleri ve alt sınıflarından türeyen nesneler tarafından erişilebilir. UML gösterimi “#” işaretidir.

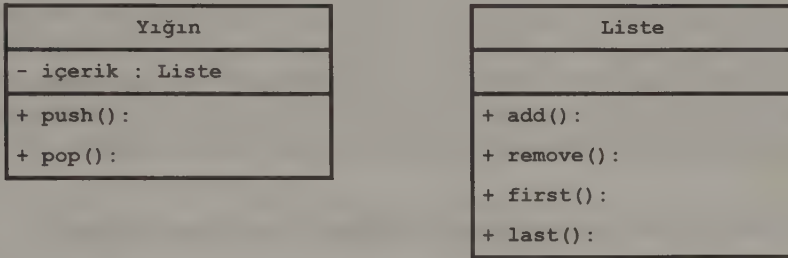
Bir sınıfta yer alan açık elemanlar, o sınıfın dışarıya verdiği hizmetleri oluşturur. Bu yüzden işlemler genellikle açık, özellikler de genellikle özel görünebilir olarak tanımlanırlar. Bu sayede sınıfın kullanıcıları, sınıfın içyapısındaki özel üyelerle ilgilenmezler. İyi tasarlanmış bir sınıf iyi hizmet verebilecek kadar açık, ayrıntılarla uğraştırmayacak ve sınıfa zarar görmesine izin vermeyecek kadar kapalı olmalıdır.

10.1.2. İşlemler

Nesne, bilgi saklayan özelliklerinin yanısıra bu özellikler üzerinde işlem yapmaya yardımcı olan işlemleri de içerir. Bir işlemi gerçekleştirilen yordama ise, “metot” adı verilir.

Metot, kendisiyle aynı isimde olan mesaj ile aktif hale gelir. Metodun sahip olduğu parametrelerin değerlerin alacağı değerler de mesajda yer alır. Bir sınıf içerisinde farklı parametrelere sahip aynı isimli metotlar bulunabilir.

Nesne yönelimli bir programda, kullanıcıdan her an değişik olaylar gelebilir. Bu olaylar, nesnelere gönderilen mesajları oluşturur. Bir işi gerçekleştirmek için, birçok nesne aralarında mesajlaşarak işbirliği yaparlar. Şekil-10.3’de verilen “Yığın” (*stack*) sınıfı, “push()” işlemi yapabilmek için “Liste” sınıfının “add()” hizmetinden yararlanır.



Şekil-10.3. İşlemler, nesnelerin birlikte çalışmalarını sağlar

İşlemlerin görünebilirliği de, özelliklerin görünebilirliği ile aynı esaslara dayanmaktadır. Nesne yönelimli bir programda dış dünyaya hizmet eden ve haberleşen metotlar açık görünürlüğe sahiptir. Sınıfın kendi içyapısında kullandığı işlemler de özel görünebilir olarak tanımlanırlar ve dışarıdan erişilemezler.

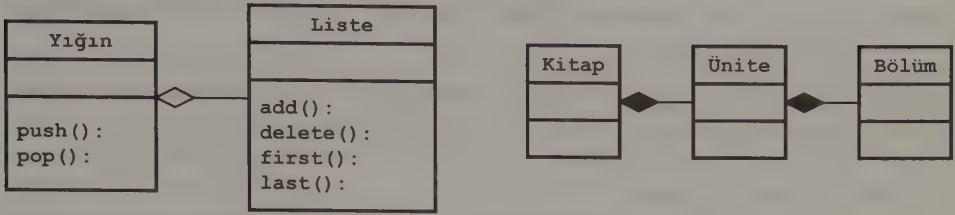
10.2. Sınıflar Arası İlişkiler (Relations)

Nesneye yönelik bir uygulamanın analiz aşaması, temel olarak sınıfların ve sınıflar arasındaki ilişkilerin belirlenmesi esasına dayanır.

10.2.1. Bağlantı (Association)

Bağlantı, sınıflar arası ilişkilerin bir çeşididir. Sınıflardan türeyen nesneler arasında hizmet alışverişini sağlar. Birbirleriyle haberleşecek nesneler, bu bağlantıyı kullanırlar. Nesneler arası mesajlar bağlantılara uygun olarak iletilir.

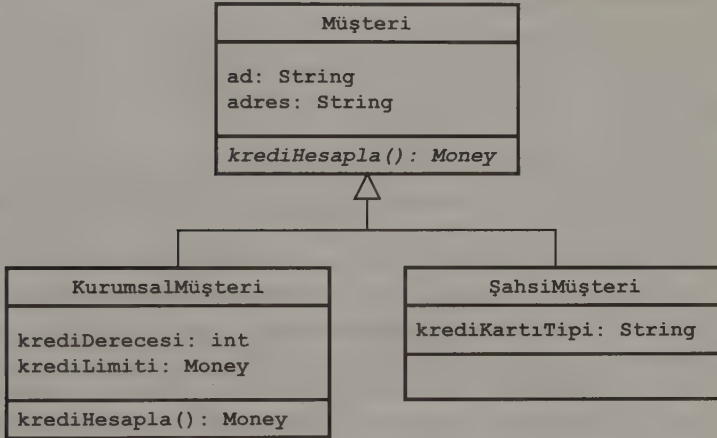
Bağlantılarda genellikle iki sınıf yer alır; ancak tek bir sınıf da kendi üzerinde bağlantıya sahip olabilir. Şekil-10.4’de örnek bağlantılar görülmektedir. Yönetici, çalışan sınıfa ait bir nesnedir ve birçok çalışanı yönetebilir. Ayrıca, yönetici aynı sınıfa ait olmasına rağmen yöneticisi yoktur. Fakat diğer çalışan nesnelerinin bir yöneticisi vardır.



Şekil-10.5. Kümeleme ve birleştirme ilişkileri

10.2.3. Genelleştirme (Generalization)

Genel tanımlar içeren ana bir sınıf ile daha özel ayrıntılara sahip alt sınıflar arasındaki ilişki türüne *genelleştirme* denilir. Alt sınıf, üst sınıftan türetilir. Üst sınıfın özellikleri ve işlemleri, alt sınıflarda aynen kullanılabilir. Böylece tanımlanmış olan özelliklerin tekrar tanımlanmaları gereği ortadan kalkar; tasarlanan sınıfların tekrar kullanılabilirliği artar. Bu duruma kalıtım (*inheritance*) adı verilir. UML’de genelleştirme, bağlantı çizgilerinin ucuna içi boş bir üçgen çizilerek gösterilir. Şekil-10.6’da, araç sınıfı ve bunun alt sınıfları görünmektedir. Araç sınıfı, tüm özellik ve işlemlerini alt sınıflara miras bırakır. Bu özellikler, alt sınıflarda gösterilmez.



Şekil-10.6. Genelleştirme

10.2.4. Çok Şekillilik (Polymorphism) ve Soyut Sınıf (Abstract Class)

Çok şekillilik, farklı sınıflardan yaratılmış olan nesnelerin, aynı mesaja farklı tepkiler vermesi olarak tanımlanabilir. Alt sınıfın miras aldığı bir işlem, genellikle olduğu gibi kullanılır. Örneğin “Müşteri” sınıfında tanımlı olan bir `krediHesapla()` metodu, “ŞahsiMüşteri” alt sınıfında kullanılır. Ancak, alt sınıfın ihtiyaçlarına uygun olarak bu işlemlerin tekrar tanımlanması gerekebilir ve buna geçersiz kılma (*overriding*) adı verilir. Alt sınıflarda yeniden tanımlanan işlemler, bu halleriyle kullanılırlar. Bu du-

rumda Şekil-10.6'daki "KurumsalMüşteri" sınıfında "krediHesapla()" işleminin çok şekilli olduğu söylenebilir.

Soyut sınıf, üzerinden doğrudan nesne tanımlanamayan sınıftır. Ancak kendinden türeyen alt sınıflardan nesne oluşturulabilir. Çok şekilliliğin sağlanabilmesi için, üst sınıftaki işlem soyut tanımlanmalıdır. Böylece alt sınıfta yeniden tanımlanan işlem, çok şekillilik özelliği kazanır. Bu sayede "krediHesapla()" metodu çağrıldığında, "Müşteri" tipindeki bir işaretçi o anda hangi nesneyi gösteriyorsa, onun ait olduğu sınıftaki işlem çalışır. Soyut sınıfının UML tanımı, sınıftan ya da soyut işlemden önce yazılan "<<abstract>>" anahtar kelimesidir. Bir diğer gösterim türü de, soyut öğelerin eğik (*italic*) yazılmasıdır. Şekil-10.6'da "Müşteri" sınıfında yer alan "krediHesapla()" metodunun, soyut olarak tanımlandığı görülebilir.

Bir sınıf içerisinde tanımlı olan bir işlem, aynı isimle ancak farklı tip veya sayıda değişik parametreler ile tanımlanabilir. Bu duruma, aşırı yükleme (*overloading*) adı verilir. Aşırı yükleme ile bir sınıf içindeki işlemlerin çok şekilliliği sağlanır. Örneğin, "Aritmetikİşlem" sınıfında bir toplama işlemi tanımlanırsa, aldığı parametrelere göre bu işlem birçok farklı şekilde bulunabilir.

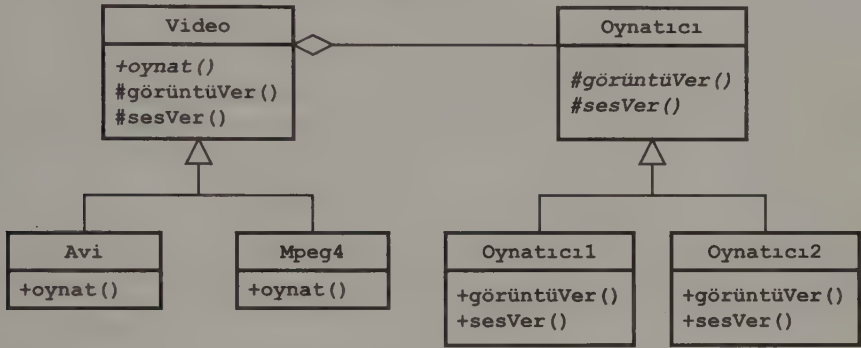
- integer topla(integer a, integer b)
- float topla(integer a, float b)
- float topla(float a, integer b)
- float topla(float a, float b)

10.3. Özet

Nesne, değişik hizmetler veren ve özellikleri olan bir yapıdır. Özellikler, tip-değer çiftleridir. Değerler, basit veya karmaşık veri tipinde olabilir. Özellikler, dışarıdan erişimin kontrolünü sağlamak için "sarmalanırlar" (*encapsulation*). İşlemler, özellikler üzerinde işlem yapan ve dış nesnelere hizmet veren metotlardır. Sınıf ise, aynı özellik ve işlemlere sahip nesne kümesi için bir tanımlayıcıdır. Nesneler, ait oldukları sınıflardan türetilirler. Birbirleriyle haberleşecek olan nesneler, bağlantı (*association*) ilişkisini kullanırlar. Sınıflar arasındaki parça-bütün ilişkileri ise kümeleme (*aggregation*) ve birleştirmedir (*composition*). Kümeleme ilişkisini oluşturan nesneler kendi başlarına kullanılabilirken, birleştirme ilişkisindeki nesneler uygulamada tek başlarına kullanılamazlar. Genel tanımlara sahip bir ana sınıf ile özel detaylara sahip alt sınıflar arasındaki ilişki ise genelleştirme (*generalization*) olarak tanımlanır. Üst sınıftan türeyen alt sınıf, üst sınıfın özellik ve işlemlerine sahiptir. Farklı sınıflardan yaratılan nesnelerin aynı mesaja farklı tepkiler vermesi, çok şekillilik (*polymorphism*) olarak adlandırılır. Çok şekilliliğin sağlanabilmesi için üst sınıftaki işlem soyut (*abstract*) tanımlanır. Soyut sınıf, üzerinden direkt olarak nesne tanımlanamayan sınıftır. Ancak türeyen alt sınıflardan nesne oluşturulabilir.

10.4. Sorular

- 10.4.1)** İyi bir nesne modelinde özelliklerin çoğunun özel (private) ve işlemlerin çoğunun açık (public) olmasının sebebi nedir?
- 10.4.2)** Geçersiz kılma (overriding) ve çok şekillilik (polymorphism) arasındaki ilişki nedir?
- 10.4.3)** Kümeleme ve birleştirme arasındaki fark nedir?
- 10.4.4)** Değişik video formatlarını oynatabilecek bir program isteniyor. Bunun için, önceden hazırlanmış iki farklı oynatıcı kütüphanesinden (Oynatıcı1, Oynatıcı2) yararlanılıyor. Video nesneleri, kendilerini nasıl oynatacaklarını biliyorlar (+oynat()). Oynatıcı kütüphane nesneleri ise nasıl görüntü ve ses üreteceklerini biliyorlar (görüntüVer(), sesVer()). Video nesnesi, kendini oynatmak için oynatıcı kütüphane nesnesini kullanmaktadır.



- 10.4.5)** Yukarıda verilen sınıf diyagramında

- kümelemeyi (aggregation)
- genelleştirmeyi (generalization)
- soyut sınıf ve metotları (abstract classes and methods)
- sarmalamayı (encapsulation: [private, public, protected])

örneklendirerek, istenen bu programda ne amaçla kullanıldıklarını anlatınız.

BÖLÜM 11

NESNEYE YÖNELİK SİSTEM ANALİZİ

Yazılım geliştirme yaşam döngüsü açısından nesneye yönelik sistem analizi, geleneksel yapısal yaklaşım ile aynı aşamalara sahiptir. Planlama, analiz, tasarım, gerçekleştirme aşamaları ve aşamalarda gerçekleştirilen aktiviteler de aynıdır. Yapısal sistem geliştirmede işlemlerin tanımlanması ve modellenmesi üstünde durulurken, nesneye yönelik yaklaşım nesneler ve etkileşimleri vurgular. Her iki yaklaşım, geliştirme aşamalarında modelleme üstünde dursalar da, değişik açılardan ele alırlar. Örneğin sistem analisti, veri akış diyagramları yerine, etkileşim diyagramları kullanır. Bundan başka, geleneksel yaklaşımda kullanılan varlık-ilişki diyagramları ile nesneye yönelik yaklaşımdaki sınıf diyagramları, farklı gösterimler kullanmalarına karşın, birçok ortak kavram içerirler. Nesneye yönelik yaklaşımda prototip oluşturma miktarı ve seviyesi, geleneksel yaklaşıma oranla daha fazla, daha detaylıdır. İki yaklaşım arasındaki temel fark, sistem aktivitelerini göstermek için kullanılan diyagramların farklılığı ve çeşitliliğidir.

Nesneye yönelik yaklaşımda uygulamanın gereksinimlerini tanımlamak için birbiriyle ilişkili ancak bağımsız iki diyagramdan yararlanılır. Gereksinimlerin tam bir tanımını yapmak için bu iki diyagram bir arada kullanılır. Diyagramlar genellikle paralel hazırlanırlar.

Sınıf diyagramı, bir sistemdeki tüm nesnelerin sınıf tanımlarını ve ilişkilerini göstermek için kullanılır. Aynı zamanda her sınıfın özellikleri ve işlemleri de bu diyagramda belirtilir.

Kullanım senaryosu diyagramları ise, yeni sistemdeki kullanımların, senaryoların belirlenmesi için kullanılır. Genel olarak, sistemin nasıl kullanılacağını modeller. Sistemdeki olaylar tablosunun bir özet gösterimi şeklindedir. Bu sayede, sistemin gerçekleştirmesi gereken fonksiyonlar belgelenmiş olur. Tek, anlaşılabilir bir kullanım senaryosu diyagramı ile sistem modellenebileceği gibi, daha küçük diyagramlardan oluşmuş bir kullanım senaryosu modeli de oluşturulabilir.

Tasarlanacak olan sistem analiz edilirken temel amaç, çözülecek problemin doğru, mantıklı, anlaşılır ve test edilebilir bir modelini oluşturmaktır. Problemin çözülmesi, analiz aşamasında gerçekleşmez. Buradaki hedef, problemin tam olarak anlaşılmasıdır. Nesneye yönelik yaklaşıma uygun olacak şekilde gerçek dünyayı oluşturan kavramsal sınıflar belirlenir. Yazılım nesneleri gibi tasarıma yönelik detaylarla bu aşamada ilgilenilmez.

11.1. Kullanım Senaryosu Modellemesi (Use-Case Modeling)

Bir sistemin davranışı, dış olaylara verdiği tepkilerdir. UML’de, sistemin dışarıdan görülebilen ve test edilebilen davranışları, kullanım senaryoları ile gösterilir. Senaryolar, anlamlı bir sonuca ulaşmak için aktör ile sistem arasında gerçekleşen olaylardır. Aktör ise bir senaryoyla etkileşime geçen kimse ya da şey, kısaca sistemin kullanıcısıdır. Bir insan, başka bir sistem olabilir. Aktör, senaryodan kullanabileceği bir sonuç bekler.

Kullanım senaryosu modellemesi ile aktörler ve senaryolar, ek tanımlamalar ve şartlar ile birlikte görselleştirilir. Sadece bir diyagram değil, tüm sistemin beklenen davranışını açıklayan bir modeldir. Kullanım senaryosu modellemesi kolay anlaşılır, müşteri ile iletişimi kolaylaştırır. Gerçeklenen sistemin isteklere uygun olup olmadığı, senaryolar ile test edilebilir. Nesneye yönelik analiz için uygun bir başlangıç oluşturur.

11.1.1. Aktörler (Actors)

Senaryo ya da aktörlerden ilk olarak hangisinin belirleneceği, sistem analistinin vereceği bir karardır. Aktörler UML’de kullanım senaryosu diyagramları çizilirken çöp adamlarla ifade edilir. Ancak, aktör özelliklere ve işlemlere de sahip olabileceğinden, normal bir sınıf şeklinde de gösterilebilir.

Nesneye yönelik sistem analizi ve diyagramları örneklemek için, bir alışveriş sitesi örneği verilmiştir. Güncel bir uygulama olan Internet’te alışveriş için uygulamanın yapısı, teknik altyapı, alışveriş sırasında gerçekleştirilen işlemler üzerinde durulmuştur. Sitenin genel yapısı, aşağıda özetlenmiştir;

- Site, kahve satışı yapmaktadır.
- Hazır kahveler satın alınabileceği gibi, müşteri dilediği çeşitleri harmanlayarak özel kahve siparişi de verebilmektedir.
- Bir alışverişte birden fazla çeşit kahve siparişi verilebilir.
- Siparişi tamamlamak için, müşteri teslimat ve ödeme formlarını doldurur.
- Kredi kartı ve banka havalesi ile ödeme yapılabilir.
- Arka planda, müşterinin bilgileri ve ödemesi doğrulanır.
- İstenen özelliklerdeki sipariş, kahve sağlayan depodan alınır, fatura basılır ve müşteriye gönderilir.

Aktörleri ve senaryoları belirlemek için, öncelikle sistemin sınırları belirlenmelidir. Tasarlanacak olan sistemin sınırları dâhilinde nelerin olup olmayacağı, belirlenecek aktör ve senaryoları değiştirir. Bunu için ilk olarak sistemin genel akışı adım adım özetlenir;

- Müşteri, hazır bir kahve seçer. Özellikleri ile fiyatı da gösterilir.
- Müşteri, istediği kahveleri harmanlayarak kendine özel kahve oluşturabilir. Fiyat da seçilen kahvelere göre yeniden hesaplanır.

- Müşteri, 1. ve 2. adımları tekrarlayarak alışverişine devam eder.
- Müşteri, siparişini siteden verir. Siparişi tamamlamak için teslimat ve fatura bilgileri, ödeme detaylarını içeren bir form doldurur.
- Sipariş sisteme girilince, müşteri ve ödeme bilgileri doğrulanır.
- İstenen özelliklerdeki kahve, sağlayıcı depodan alınır, faturası basılır.
- Sipariş, fatura ile birlikte müşteriye gönderir.

Bu tariflere göre temel olarak iki aktör bulunmaktadır. Bunlar, Şekil-11.1’de gösterilmiştir.



Müşteri



Kahve Sağlayıcı Depo

Şekil-11.1. Kahve sitesi sistemi aktörleri

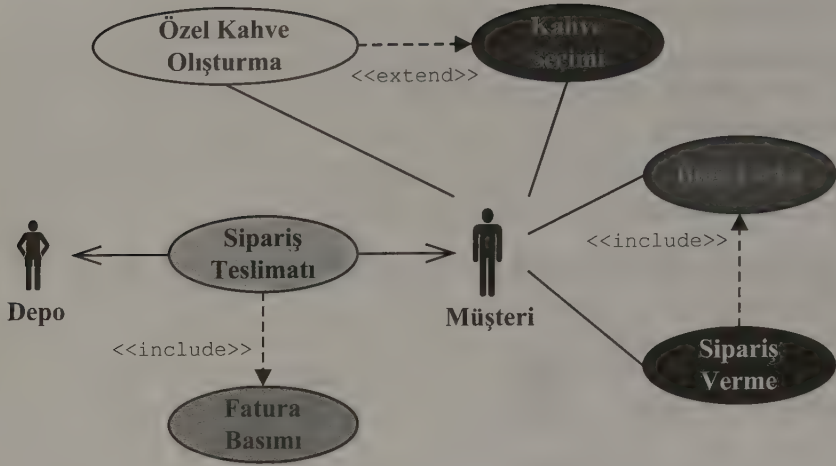
11.1.2. Kullanım Senaryoları

Senaryo, bir aktöre fonksiyonel bir değer sunan bir bütündür. Herhangi bir senaryo ile ilişkisi bulunmayan bir aktör, sistemde gereksizdir. Ancak, hiçbir aktörle ilişkide olmayan senaryolar bulunabilir. Bu tip senaryolar, kullanım senaryosu modellemesinde ana senaryolara yardımcı olurlar. Aktörlerin görevlerinin saptanması ile kullanım senaryoları da çıkarılmış olur. Kullanım senaryoları, fonksiyonel gereksinimlerin incelenmesi ile de bulunabilir. Fonksiyonel bir gereksinim, birçok durumda bir senaryo olarak modellenir.

Kullanım senaryoları belirlenirken, sadece ihtiyaçlar ve gereksinimler listelenmemelidir. Sistemin aktörlere karşı olan sorumlulukları düşünülmelidir. Genel olarak, sistemin bittiğinde nasıl çalışacağı, ne yapacağı belirtilmelidir.

Kullanım senaryolarını belirlemek üzere, aktörleri ve senaryoları birleştirilebilir. Daha önceden kahve sisteminin genel akışı adım adım açıklanmıştı. Buradaki her bir adımda, sistemde olması gereken senaryolar ve etkileşimde oldukları aktörler belirlenir.

Kullanım senaryolarının UML gösterimi elips şeklindedir. Senaryonun adı, elipsin içine ya da altına yazılabilir. Senaryolarla ilişkide olan aktörler de diyagramda yer alır. Genelde aktör ve senaryoların etkileşimde olması beklenir. Ancak, bir senaryo başka bir senaryoya da ilişkili olabilir. Bir aktörün direk olarak bir başka aktörle etkileşimde olması ise önerilmez. Aralarında mutlaka etkileşimi sağlayan bir senaryo bulunmalıdır. Şekil-11.2’de, kahve sitesi sisteminin kullanım senaryosu diyagramı görülmektedir.



Şekil-11.2. Kahve sitesi sistemi için kullanım senaryosu diyagramı

Kullanım senaryoları arasında üç farklı ilişki tanımlanabilir.

İçerme (Include-Use): Bir senaryoda kullanılan başka bir senaryoyu belirtir. Örneğin Şekil-11.2’de Sipariş verme senaryosunun içinde, müşterinin ödeme ve teslimat bilgilerini girmesi gereklidir.

Genişletme (Extend): Senaryolar doğal akışlarına göre hazırlanırlar. Özel durumlarda sapmalar olabilir. Bu gibi senaryolar da genişletme ilişkisi ile gösterilir. Şekil-11.2’de “kahve seçimi” senaryosunu, “özel kahve hazırlama” senaryosu genişletmektedir.

Özelleştirme (Specialization): Sınıflar arası türeme ilişkileri gibi, senaryolar da türetilir (Ödeme için kredi kartıyla ödeme, havale ile ödeme gibi).

Her senaryo, bir senaryo dokümanı ile ayrıca detaylandırılmalıdır. Bu sayede, bir aktör herhangi bir senaryoyu etkinleştirdiğinde sistemin neler yapacağını açıklanmış olur. Senaryo dokümanının yapısı değişmekle birlikte, tipik bir tanımlamada;

- Senaryo adı
- İlgili aktörler (Birinci aktör, ilgililer ve beklentileri)
- Senaryonun gerçekleşebilmesi için gerekli ön koşullar
- Senaryo tamamlandığında sistemin ulaşacağı durumu tanımlayan son koşullar
- Senaryonun temel akışı
- Özel durumlarda yürütülecek alternatif akışlar

yer alır. Kullanım senaryosu dokümanlarından daha sonra gerçekleştirilen sistemin kullanıcı elkitablarının hazırlanması sırasında da yararlanır. Tablo-11.1’de kahve sitesi sisteminin, “sipariş verme” senaryo dokümantasyonu örnek olarak verilmiştir. Genellikle bu dokümanlar bu kadar kısa ve özet değil, daha açıklayıcı biçimde yazılır.

Tablo-11.1. “sipariş verme” senaryo dokümantasyonu

Senaryo KS1	Sipariş verme
Birinci Aktör	Müşteri
İlgililer ve Beklentileri	<i>Kahve Sitesi:</i> Müşterinin siparişi doğru ve eksiksiz kaydedilmeli, ödeme ve teslimat bilgileri doğru alınmalı. <i>Depo:</i> Siparişteki kahve miktarları doğru ve eksiksiz olmalı.
Ön Koşullar	Müşteri alışveriş sitesinde olmalı, sunulan seçenekler ile istediği kahveyi oluşturmalıdır.
Son Koşullar	Ödeme ve teslimat bilgileri doğrulanmıştır. Müşterinin siparişi sistem veritabanına teslimat yapılmak üzere kaydedilmiştir.
Ana akış	1. Müşteri sipariş etmek istediğini belirtir. 2. Müşteri teslimat için adını ve adresini, teslimat bilgilerinden farklıysa fatura bilgilerini girer. 3. Müşteri ödeme şeklini seçer ve gerekli ödeme bilgilerini girer. 4. Sistem sipariş için benzersiz bir tanımlayıcı kod üretir, müşteri hesap bilgileri ile birlikte kaydeder.
Alternatif akış	<i>2a-3a:</i> Müşteri gerekli tüm bilgileri sağlamadan siparişi tamamlar. Sistem bir hata mesajı vererek eksik olan bilgilerin tamamlanmasını ister.

11.1.3. Kullanım Senaryolarında Sözleşmeler (Contracts)

Kullanım senaryoları, birçok uygulamanın modellenmesinde genellikle yeterli olmaktadır. Ancak bazı senaryolar, karmaşık işlemler içerebilir. Bu gibi durumlarda anlaşılabilirliği arttırmak için sözleşme yazılır.

Sözleşmeler, ön koşullar sağlandığında sistemin işlemesi ile uygulama nesnelerinin alacağı son durumları tarif eden yapılardır. Senaryolarda aktörler ile sistemin etkileşimi modellenirken, sözleşmelerde sistemin içindeki nesnelerdeki değişimler de belirtilir. Sözleşmelerde özellikle sistemdeki nesnelerin alacağı son durumlar belirtilir.

11.2. Sınıf Modellemesi

Nesneye yönelik analizde sistemin iç durumu sınıf modeli ile tanımlanır. Bu modellemede sınıflar ile birlikte özellikleri, işlemleri, bağlantılar, bütünleşmeler, bileşimler, genelleştirmeler de yer alır. Sınıf diyagramı ile tüm bu öğeler görselleştirilir.

Nesneye yönelik sistem analizi sırasında kullanım senaryosu modellemesi ile sınıf modellemesi, birbirine paralel olarak ilerler. Her iki model de, diğerine destek bilgisi sağlar. Kullanım senaryoları ile sınıfların belirlenmesi kolaylaşırken, sınıflar da göz-

den kaçırılan kullanım senaryolarının belirlenmesine yardımcı olur. Bu iki model ile uygulama alanının modeli oluşturulmuş olur. Tasarlanacak sistem tüm yönleriyle anlaşılır.

Sınıf modellemesinde yer alan sınıflar, gerçek dünyayı oluşturan kavramsal nesnelerdir. Çözüm için yaratılacak olan yazılım nesneleri bu aşamada düşünülmez. Ancak modelde yer alan kavramsal sınıflar, tasarlanacak yazılım sınıflarına kılavuzluk eder. Sınıfları belirlemek için, kullanım senaryolarında yer alan isimlerden ve isim tamlamalarından yararlanılabilir. Örnek olarak KS1 kullanım senaryosundaki isimler ve tamlamaları verilmiştir.

1. **Müşteri sipariş** etmek istediğini belirtir.
2. Müşteri **teslimat** için adını ve adresini, teslimat bilgilerinden farklıysa **fatura** bilgilerini girer.
3. Müşteri **ödeme şeklini** seçer ve gerekli **ödeme bilgilerini** girer.
4. Sistem sipariş için benzersiz bir tanımlayıcı kod üretir, müşteri hesap bilgileri ile birlikte veritabanına kaydeder.

Bunun gibi tüm kullanım senaryolarındaki isimler bulunarak aday sınıflar elde edilebilir. İlk aşamada tüm sınıfların doğru olarak belirlenmesi beklenemez. Gereksiz sınıflar elde edilebilir. Sınıfların tespit edilmesi, artımlı bir süreçtir. Bu yüzden, sınıfların tam olarak belirlenmesi ve aday sınıfların gerçekleştirilecek sistemde yer almaları zaman içinde değişebilir. Gereksinimlerde yer alan bir kavramın gerçekte bir sınıf olarak gerçekleştirilip gerçekleştirilmeyeceği, birkaç soru ile kesin olarak belirlenebilir.

- Kavramla ilgili veri saklanması gerekiyor mu?
- Değişik değerler alabilecek farklı özellikleri var mı?
- Kavramdan birçok nesne türeyebilir mi?
- Uygulamanın kapsama alanı içinde mi?
- Kavramın sınırları iyi çizilmiş mi, tanımı tam yapılabilir mi?

Uygulamaya özel olarak, cevaplanması gereken sorular değişebilir ve artabilir. Bu gibi sorulara yanıt vermek ilk bakışta kolay değildir. Uygulamanın gereksinimleri hakkında geniş bilgiye sahip olmayı gerektirir. Bu aşamada hata yapılsa bile, tasarım aşamasında düzeltilebilir. Analiz sırasında önemli olan problemi düzgün biçimde anlamak, modellemek ve ifade etmektir.

11.2.1. Özelliklerin Belirlenmesi

Bir sınıfın özellikleri, yaratılan nesnelere özgü değerler alan verilerdir. Sistem analisti, uygulama ile ilgili kavramsal sınıfları belirlerken, kullanım senaryolarını ilgilendiren özellikleri de oluşturmalıdır. Özellikler genellikle basit veri tiplerinden oluşur. Eğer özellik olarak düşünülen kavram, daha karmaşık tipte ise, büyük ihtimalle ayrı bir sınıf adaydır. Birden fazla alandan oluşan, üzerinde işlem yapılan, kendi niteliklerine sahip olan kavramlar, bir sınıfın özelliği değil ayrı bir sınıf olarak ifade edilmelidir.

11.2.2. Bağlantıların Belirlenmesi

Bağlantılar, nesnelerin işbirliği ilişkilerinin çıkarılmasına yardımcı olmaktadır. Sistem gerçekleştirildiğinde sınıflar arasındaki bağlantılar, bu sınıflardaki belirli özellikler ile temsil edilir. Analiz aşamasında ise, bağlantılar çizgiler ve çoğullama simgeleri ile gösterilir.

Bağlantıların belirlenmesi için, sınıfların belirlenmesinde olduğu gibi kullanım senaryolarından yararlanılır. Senaryolarda yer alan fiiller, olası bağlantılardır. Sistemin amacı dışındaki bağlantılar gereksizdir. Sistemin tasarımı ile ilgili bağlantılar dikkate alınmalıdır. Ayrıca senaryoda yer alan faaliyetler de bağlantı değil, etkileşimi ifade eder. Bu yüzden bağlantı, bir ya da birkaç işlemi değil bir ilişkiyi temsil etmelidir. İki sınıf arasında birden fazla yol varsa, genellikle çözümde fazlalık ilişki olduğu söylenebilir. Bu gibi durumlarda türetilen ilişki elenebilir. Ancak her durumda bu gerekli değildir (çocuk-baba ve baba-kardeş ilişkisi yerine çocuk-amca ilişkisi de kullanılır). Gereğinden fazla bağlantı, sınıf modellemesinin anlaşılabilirliğini azaltır.

Kümeleme ve birleştirme ilişkileri ile sınıflar arası sahiplik bilgileri daha iyi bir şekilde oluşturulabilir. Bunların da modelde belirtilmesi yararlıdır ancak analiz aşamasında fazla ayrıntı vermek gereksizdir.

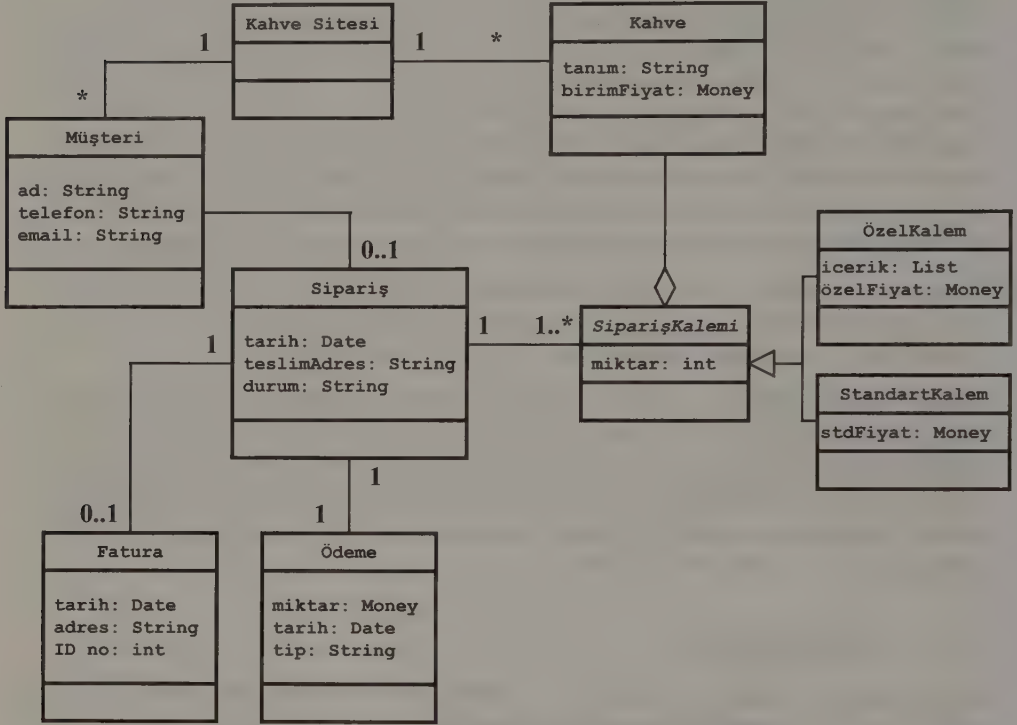
Genelleştirmeler, bir yazılımdaki yeniden kullanılabilirliği önemli ölçüde artırmaktadır. Ayrıca sınıf modellemesini oldukça basitleştirmekte ve anlaşılır hale getirmektedir. Basitlik, diyagram görünümünde olmasa da, sistemin anlamsal görünümünde etkilidir. Genelleştirmeler ile genellikle soyut sınıfları da içeren fazladan sınıfların oluşturulması gerekir. Ancak uygun soyutlamalar ile oluşturulan hiyerarşik düzen yönetim ve takip kolaylığı sağlar.

11.2.3. Sınıf Diyagramı

Sınıf diyagramı, kullanım senaryoları ile analizi yapılan sistemin uygulama modelini gösterir. Bu diyagramda kavramsal sınıflar, aralarındaki ilişkiler ve sınıfların nitelikleri yer alır. Şekil-11.3'de kahve sitesi sistemi için sınıf diyagramı gösterilmiştir. Sınıfların henüz işlemleri bulunmamaktadır. İşlemlerin belirlenmesi, sınıfların sorumluluklarının belirlenmesi demektir ve nesneye yönelik yaklaşımda tasarım safhasını oluşturur. Sınıf diyagramına işlemler de eklendiğinde, sistemin davranışı da gösterilmiş olur. Şekil-11.3, kahve sitesi sistemi için bütün bir çözüm olmasa da, temel bilgileri içermektedir. Daha fazla özellik, sistemin ihtiyaçları ve analiz doğrultusunda eklenebilir.

Kahve Sitesi sınıfı, uygulamanın ana sınıfı olarak düşünülebilir. Sitede bulunan *Müşteri* ve *Kahve* sınıflarının sahibi olacaktır. Bir müşterinin hiç *Siparişi* olmayabilir. Bunun yanında müşteri birden fazla sipariş de verebilir. Bir *Siparişin* tek bir *Ödemesi* olabilir. *Siparişin* durum özelliğine bağlı olarak, *Faturası* olmayabilir. Bir siparişin en fazla bir faturası olabilir. Bir siparişte bir veya birden fazla *Sipariş Kalemi* yer alabilir. Burada *Sipariş Kalemi* soyut sınıf olarak tanımlanmıştır. Uygulamada kendi üzerinden bir nesne yaratılamayacaktır. Bu durumda uygulamada bir siparişin kalemleri, ya *StandartKalem* ya da *ÖzelKalem* olacaktır. *Kahve* sınıfı *Sipariş*

Kalemi sınıfına kümeleme ilişkisi ile bağlıdır. Başka bir deyişle *Kahve*, *Sipariş Kalemi* sınıfından türeyen nesnelerde yer alan bir nesne olacaktır. Kullanım senaryosu modellemesinde *ÖzelKalem* sınıfının birçok kahve nesnesinden oluştuğu görülmektedir. Bu yüzden *ÖzelKalem* sınıfı içinde *Kahveleri* tutan bir liste yapısına ihtiyaç duyulmuştur.



Şekil-11.3. Kahve sitesi sistemi için sınıf diyagramı

11.3. Özet

Uygulamanın gereksinimlerini tanımlamak için paralel olarak geliştirilen sınıf ve kullanım senaryosu (*use-case*) modellemeleri yapılır. Kullanım senaryosu modellemesi, sistemdeki kullanımların, senaryoların belirlenmesini sağlayarak sistemin nasıl kullanılacağını modeller. Böylece sistemin beklenen davranışını açıklar. Senaryolar, anlamlı bir sonuca ulaşmak için sistemle aktör arasında gerçekleşen olaylardır. Aktör ise, senaryoyla etkileşime geçen kimse ya da şey, kısaca sistemin kullanıcısıdır. UML’de modelleme, kullanım senaryosu diyagramı ile yapılır. Bunun yanında her senaryo için bir doküman da hazırlanır. Karmaşık işlemlerden oluşan senaryoların anlaşılabilirliğini arttırmak için sözleşme de yazılabilir.

Sistemin iç durumu, sınıf modellemesi ile belirtilir. Sınıflar ile özellikleri, işlemleri, bağlantılar, bütünleşmeler, bileşimler, genelleştirmeler de sınıf diyagramı ile gösterilir. Sınıf modellemesinde özellikler ve bağlantılar belirlenirken kullanım senaryolarından yararlanılır. Modelde yer alan sınıflar, gerçek dünyayı oluşturan kavramsal nesnelerdir. Yazılım sınıfları bu aşamada düşünülmesi de, tasarıma kılavuzluk eder. Analiz sürecinde amaç problemin doğru, mantıklı, anlaşılır ve test edilebilir modelini oluşturmaktır. Problemin çözümü, tasarım sürecine bırakılır.

11.4. Sorular

11.4.1) Sistem analizinde kullanım senaryosu modellemesinin rolü ve önemi nedir?

11.4.2) Üniversite öğrenci bilgi sistemini yazan geliştirme grubu, kayıt alt sistemini modellemek istemektedir. Mevcut sistem incelendiğinde, aşağıdaki bilgiler elde edilmiştir:

- Öğrenciler, sistemi kullanarak derse kayıt olurlar.
- Öğretmenler, sınıf listelerini görebilirler.
- Öğrenciler normal kayıt yapamazlarsa, danışmanları onları ekle-sil haftasında geç kayıt eder.
- Derse kayıt olmak için de, sınıf listesi görmek için de sisteme giriş yapılması gereklidir.

Kayıt alt sistemi için kullanım senaryosu diyagramını (*use-case diagram*) oluşturunuz.

11.4.3) İnternet üzerinden video kiralama ve satış hizmetleri sunan bir sitede müşteriler, siteye üye olarak alışveriş yapabilmektedir. Müşteriler satın almak yerine videoları kiralamak isterlerse, kullanıcı kimlik doğrulanması ve kontrolü işlemleri de yapılır. Kiralama işleminde ayrıca süresini aşan işlemler de kontrol edilerek, gecikme cezası tahsil edilmektedir. “Video Kiralama” senaryosu için, kullanım senaryosu dokümantasyonunu oluşturunuz.

11.4.4) Bir grup yarış arabasının yapım ve bakımı ile ilgilenen bir şirket için gerçekleştirilecek olan bir bilgi sisteminde yer alan varlıklarla ilgili bilgiler aşağıda verilmiştir. Bu varlıkların sınıf modellemesini gerçekleştiriniz.

- Yarış arabalarında özellikleri aşağıda verilmiş olan motor ve şanzımanlar kullanılmaktadır.
- Şanzımanlar, “otomatik” ve “manuel” olmak üzere iki çeşittir. Her ikisinin de ayrı özellikleri ve işlemleri vardır. Ancak bir arabada sadece bir tek “şanzıman” bulunur. Bir şanzıman hiçbir arabada kullanılamayacağı gibi, birçok arabada da kullanılabilir.
- Her motorda bir adet “radyatör”, tam 8 adet “piston”, tam 40 adet “supap” bulunmaktadır. Radyatör, piston ve supaplar, ısı ve diğer etkenler nedeniyle sadece tek bir motorda kullanılabilir, sökülüp başka bir motora takılamazlar. Bir arabada tek bir “motor” bulunur. Bir motor hiçbir arabada kullanılamayacağı gibi, birçok arabada da kullanılabilir.
- “Yarış arabaları” dışında, şirkete bağlı çalışan pilotlar da bulunmaktadır. Pilotlar hakkında da verilerin tutulması gerekmektedir. Hazırlanan bir yarış arabasını hiçbir “pilot” kullanmayabilir veya bir araba birçok pilot tarafından kullanılabilir. Bir pilot hiçbir arabayı kullanmayabileceği gibi, birçok arabayı da kullanabilir.

BÖLÜM 12

NESNEYE YÖNELİK SİSTEM TASARIMI

Tasarım aşamasında analizi yapılarak modellenen sistemin mantıksal çözümü oluşturulur. Kavramsal sınıflar baz alınarak yazılım sınıfları belirlenir. Asıl önemli olan, sınıfların aralarındaki etkileşimlerin belirlenmesidir. Bu sayede sistemin parçalarının bir arada nasıl çalışacağı, hangi sorumluluğun hangi sınıfa ait olacağı ortaya konulur. Yazılım nesnelere metotların eklenmesi ve istekleri yerine getirmek üzere mesajların oluşturulması ile nesnel tasarım gerçekleştirilir. Kısacası, nesneye yönelik tasarımın amacı sorumlulukların ilgililere atanmasıdır. Atanan sorumlulukları yerine getirmek üzere sınıflarda metotlar oluşturulur. Bir sorumluluğu gerçekleştirmek için bir nesne başka nesnelerle işbirliği içinde çalışabilir.

Nesneye yönelik sistem tasarımı sırasında nesnelerin birlikte çalışmalarını göstermek amacıyla etkileşim diyagramları çizilir. Bunlar işbirliği (*collaboration*) ve ardışıl (*sequential*) diyagramlarıdır. Etkileşim diyagramları ile birlikte tasarlanan sistemin son durumunu gösteren tasarım sınıf diyagramı da çizilir.

Nesneye yönelik tasarımda gerekli olmamakla birlikte kullanılabilecek bir diğer şekil, durum diyagramıdır. Durum diyagramı ile her nesnenin durumu ve davranışı tanımlanır. Her sınıfın bir durum grafiği vardır. Gerçekleştirilen sistemdeki mantıksal tüm durumlar, durum grafiklerinde faaliyet durumları olarak belirtilmiştir. Her sıradaki bu faaliyet birimleri, metotlardır.

12.1. Tasarım Kalıpları

Yazılım sınıflarının belirlenmesinde ve onlara uygun sorumlulukların atanmasında tasarım kalıpları yaygın olarak kullanılmaktadır. Nesnel tasarımda sınıfları ve sorumlulukları belirlemeden önce tasarım kalıplarının incelenmesi yararlıdır.

Nesneye yönelik bir yazılımın tasarımında, genel prensipler ve akla yatkın çözümlerin bir araya getirilmesi ile bir repertuar oluşturulmuştur. Sıklıkla karşılaşılan, tanımlanabilir bir probleme sistematik bir çözüm getiren prensiplere, tasarım kalıpları adı verilir. Tasarım kalıplarına problemine getirdiği çözümü belirten, kolay hatırlanan ve iletişimi kolaylaştıran isimler verilir. Nesneye yönelik tasarımda kalıplar, tanımlanmış bir problemin en iyi çözümünü sağlar. Tasarımın kalbi olan sorumlulukların atanmasında izlenmesi gereken temel yolları tarif eder. *E. Gamma*, *R. Helm*, *R. Johnson* ve *J. Vlissides* tarafından ortaya konulan kalıplar literatürde GoF kalıpları (*Gang of Four*) olarak bilinir. Toplam 23 adet olan GoF tasarım kalıpları yerine, genel olarak nesnelere sorumluluklarını atarken baz alınan genel sorumluluk atama prensiplerinden bahsedilecektir. Tasarım kalıpları gibi genel sorumluluk atama prensipleri de çözülecek problemi bir soru olarak görmekte, kalıbı ise sorunun cevabı olarak sunmaktadır.

12.1.1. Yaratıcı (Creator)

Bir sınıftan nesne yaratma sorumluluğunun kime verileceğini belirleyen kalıptır. Nesneye yönelik sistemlerde nesne yaratımı sıklıkla yapılan işlemlerden biridir. Yaratıcı kalıbına uygun davranıldığında tasarım az bağımlı, daha anlaşılır ve yeniden kullanılabilir olur. Aşağıda verilen koşullar geçerli olduğunda B sınıfına A sınıfından nesne yaratma sorumluluğu verilir:

- B, A nesnelerini içeriyorsa (kümeleme ilişkisi dahil)
- B, A nesnelerinin kaydını tutuyorsa
- B, A nesnelerini sıkı bir biçimde kullanıyorsa
- A'nın yaratılması için gerekli verilere B sahipse (B, A'nın yaratılmasında bilginin uzmanıysa)

Kahve sitesinde, *Sipariş Kalem*i nesnesini kimin yaratacağı örnek olarak verilebilir. Uygulama modeli incelenirse bir *Sipariş*in birçok *Sipariş Kalem*i içerdiği görülür. Bu yüzden sipariş kalemlerini yaratma sorumluluğu siparişindir.

12.1.2. Bilginin Uzmanı (Expert)

Nesnelere sorumlulukları atamanın temel prensibidir. Bir sorumluluğun o bilginin uzmanına, onu yerine getirecek veriye sahip olan sınıfa atanmasını tanımlar. Sorumluluklar doğru atandığında sistemin anlaşılabilirliği artar, bakımı ve genişletilebilmesi kolaylaşır, uygun ve yeniden kullanılabilir çözümler üretilebilir.

Bir sorumluluğun atanmasında, öncelikle uzman kalıbına uygun olarak bu bilgiye sahip olan yazılım sınıfı aranır. Henüz böyle bir sınıf yoksa kavramsal sınıflar incelenir ve uygun sınıf tasarım modeline alınır.

Kahve sitesinde bir siparişin toplam bedelinin bilinmesi sorumluluğunu almak için en uygun sınıf, *Sipariş* sınıfıdır. Ancak *Sipariş* tek başına bu bilgiye sahip değildir. Sipariş kalemünün toplam bedellerini bilme sorumlulukları kendilerindedir. Bir sipariş kaleminde birden fazla kahve yer alabilir. Bu durumda *Sipariş Kalem*i, kendi bedelini hesaplayabilmek için *Kahve* nesneleri ile işbirliği yapmalıdır.

12.1.3. Az Bağımlılık (Low Coupling)

Diğer sınıflarda olabilecek değişimlerden etkilenmemeyi ve tekrar kullanılabilirliği tanımlayan tasarım kalıbıdır. Bağımlılık, bir birimin diğerlerine ne kadar bağlı olduğu, haklarında ne kadar bilgi sahibi olduğu ve işleri yerine getirebilmek için ne kadar diğer birimlere bağlı olduğunu gösteren bir ölçüttür. Fazla bağımlılık, aşağıdaki sebeplerden ötürü tercih edilmez.

Bir sınıftaki değişim bağımlı diğer sınıfları da etkiler. Sınıfları birbirlerinden ayrı olarak (izole) anlamak zordur. Bağımlılıktan ötürü sınıfı yeniden kullanmak zordur.

Kahve sitesi incelendiğinde, hem *Sipariş* hem de *Ödemenin Müşteri* ile ilişkili olduğu görülür. Gerçek dünyada da durum böyledir. Yaratıcı kalıbına göre *Ödemeyi*

Müşterinin yaratması gerekir. Ancak bu durumda *Müşteri*, *Ödeme* hakkında bilgiye sahip olmalıdır. Oysa bu sorumluluk *Sipariş* sınıfına atanırsa, bağımlılık azaltılmış olur. Bir çözümün uygulanması diğer bir kalıba tam olarak uygun olmayabilir. Tasarım kalıpları birlikte kullanılarak en uygun tasarım çözümü oluşturulmalıdır.

12.1.4. Denetçi (Controller)

Kullanıcı arabiriminin arkasında durarak sistem olaylarını alma ve kontrol etme sorumluluğunu alacak bir sınıfa ihtiyaç duyulmaktadır. Tüm sistemi veya alt sistemi temsil eden bir sınıf denetçi olarak kullanılabilir (*facade controller*). Daha büyük sistemlerde ise, her bir kullanım senaryosunu temsil eden bir sınıf denetçi olarak atanır. Uygulamalarda yer alan “window”, “applet”, “frame” gibi yapılar denetçi sınıflar değildir. Bunlar sadece sistem olaylarını ilgili denetçiye aktarırlar. Denetçi nesneleri sistem olaylarını algıladıktan ve kontrol işlerini yaptıktan sonra bu olayları ilgili işlemleri yapacak olan nesnelere yönlendirirler.

12.1.5. İyi Uyum (High Cohesion)

Nesneye yönelik bir programda nesneler, kendilerine atanan sorumlulukları yerine getirmekle yükümlüdür. Bir sınıfa atanan sorumlulukların birbiri ile ilgili olması ve belirli bir konuya yoğunlaşması beklenir. Eğer bir sınıf çok fazla iş yapıyorsa ve sorumlulukları birbirinden farklı ise, sınıfta uyum kötüdür. Bu durumda sınıfın anlaşılabilirliği azalır, tekrar kullanım zorlaşır. Birlikte çalıştığı çok fazla sınıf olacağı için, sınıf değişimlerden daha çok etkilenir. Karmaşıklığın idare edilebilmesi için sorumluluklar, sınıf içinde iyi uyum olacak şekilde atanmalıdır.

Kahve sitesi örneğinde az bağımlılık kalıbında incelenen durum iyi uyum kalıbı için de geçerlidir. Gerçek hayatta *Ödemeyi Müşteri* gerçekleştirir. Ancak uygulama açısından bakıldığında *Ödeme* hakkında bilginin uzmanı *Sipariş* sınıfıdır. *Müşterinin Ödeme* ile ilgili sorumlulukları taşıması yerine, zaten bilgiye sahip olan *Sipariş* sınıfını bu sorumluluğu taşıması iyi uyum kalıbına uygun olacaktır.

12.1.6. Çok Şekillilik (Polymorphism)

Birbiriyle ilgili ancak tiplere göre değişiklik gösteren davranışları tasarlarlarken çok şekilli metotlar kullanmak, değişen durumlara kolaylıkla uyum sağlayabilecek bileşenler yaratmayı sağlar. Tiplere bağlı alternatif durumlarda ilgili metot koşullu değişimlerle test edilerek çağrılabilir. Ancak bu durumda değişen koşullarda, bu testi gerçekleştiren sınıfın da yapısının değiştirilmesi ve yeniden derlenmesi gerekecektir. Bunun önüne geçebilmek için davranışın değişebileceği tiplere sorumluluk, çok şekilli metotlar kullanarak atanır.

Nesneye yönelik bir yazılımın kolay monte edilebilir bileşenleri, istemci-sunucu mantığı gibi düşünülebilir. Bir parçanın sisteme kolay monte edilebilmesi, sunucunun sistemden çıkarılması halinde onun yerine sisteme başka bir parça eklendiğinde kullanıcıların bundan etkilenmemesi anlamına gelir. Kahve sitesinde kredi kartından

onay almak için farklı programlar kullanılabilir. Gelecekte yeni bir kredi kartı tipi de sisteme eklenebilir. Bu problemi çözmek üzere yazılım ile kredi kartı onay programları arasında bir ara yüz oluşturulur. Tüm kredi kartı tiplerinde yer alan ve aslında aynı işleri yapan onay fonksiyonları çok şekillidir. Eğer sistem tek bir kredi kartı sistemi kullansaydı, çok şekilliliğe gerek olmadan sadece tek bir uyarlayıcı ara sınıf koymak yeterli olurdu.

12.1.7. Yapay Sınıf (Pure Fabrication)

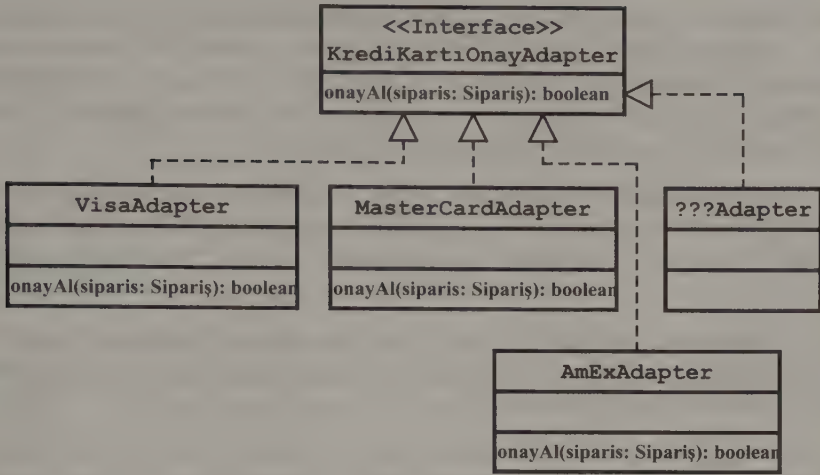
Daha önceden bahsedilen uzman kalıbının getirdiği çözümler, iyi uyum ve az bağımlılık kavramları ile çelişebilir. Bu durumda bağımlılıkları azaltıyorsa ve tekrar kullanılabilirliği artırıyorsa gerçek dünyada var olmayan yapay sınıflar yazılımda yer alabilir. Gerçek dünyadaki varlıklar ile onları yazılımda temsil eden unsurlar arasında benzerlik bulunmaktadır. Bazı durumlarda sorumlulukların atanması diğer kalıplara ters düşebilir. Bu gibi durumlarda temel olarak birbirine benzeyen işlevler bir sınıf olarak toplanarak gerçek dünyada var olmayan yapay sınıflar yaratılabilir.

Nesneye yönelik sistem tasarımında yazılım sınıflarını tasarlamak için gerçek dünyadaki kavramlardan yararlanılmaktadır. Ancak uyum ve bağımlılık gibi problemlerin üstesinden gelmek için benzer işlevler, gereğinden fazla kullanılmamak şartıyla bir sınıf içinde toplanabilir. Yapay sınıfları gereğinden fazla kullanmamak esastır. İşlevsel programcılar sistemi gerçek dünya varlıklarına ayırmak yerine işlevlere göre ayırırlar. Çok fazla kullanılan yapay sınıflar yazılımı gerçek dünyadan uzaklaştırır ve anlaşılabilirliğini azaltır. Ayrıca sınıflar arası bağımlılığı da artırır.

12.1.8. Dolaylılık-Arabirim (Indirection)

Kolaylıkla değişebilecek yapılar arasındaki bağımlılığı azaltmak, yeniden kullanılabilirliği artırır. Değişebilecek iki birim, bileşen veya servis arasındaki sorumluluklar bir arabirim nesnesine atanarak dolaylılık sağlanır.

Uygulamalarda dolaylılığı arabirim nesneleri sağlar. Çok şekillilik özelliği de olan bu nesneler dış değişimlerden etkilenmemeyi sağlar. Örnek olan kahve sitesinde, kredi kartı onaylama hizmeti dışarıdan alınmaktadır. Değişik kredi kartı kuruluşlarının değişik onay mekanizmaları bulunmaktadır. Ancak uygulama açısından önemli olan sadece kredi kartı onay sorumluluğudur. Bu durumda çok şekilli bir arabirim nesnesi ile bunu gerçekleyen kredi kartı sistem adaptörleri kullanılarak uygun tasarım yapılabilir. Bahsedilen tasarım Şekil-12.1'de verilmiştir.



Şekil-12.1. Kahve sitesi sistemi için kredi kartı onayı tasarımı

12.1.9. Değişimlerden Korunma (Protected Variation)

Dikkat edilmesi gereken tasarım prensiplerinin sonuncusu değişimlerden korunmayı esas alır. Bir sistemdeki her bir alt sistem hatta her bir nesnede değişimler ve kararsızlıklar olabilir. Bunların diğer birimlere etkisi minimum seviyede tutulmalıdır. Birimlerdeki değişikliklerin birbirlerini en az etkilediği tasarım, değişimlere açık ve kararsız noktaların etrafında sorumlulukların ara yüzlere atanmasıdır. Bu sayede birimler değişimlerden korunabilir, kararlı sistemler oluşturulabilir.

Dolaylılık kalıbında örnek olarak verilen çok şekilli arabirim nesneleri, kahve sitesi sistemini dışta olabilecek değişimlere karşı korumaktadır. Kredi kartı onay sistemleri değişse de, yenileri de eklense tasarlanan sistem bu değişimlerden etkilenmeyecektir. Sistem için gerekli olan onay alma sorumluluğu gerçekleştirilecektir.

Değişimlerden korunma prensibi nesneye yönelik birçok kavramın da temelini oluşturur. Nesneye yönelik temel kavramlar başlığı altında incelenen sarmalama (*encapsulation*), veri gizleme, çok şekillilik (*polymorphism*) gibi kavramlar bu prensip sonucu oluşmuştur.

12.2. Etkileşim Diyagramları

Nesneye yönelik tasarım, senaryoları gerçeklemek üzere sorumlulukların ilgili birimlere atanması ve nesneler arası işbirliğinin oluşturulmasıdır. Senaryoları gerçeklemeden önce, nesneler arasındaki etkileşimlerin nasıl modellendiğini incelemek gereklidir. UML’de etkileşim modellemesi iki diyagram ile yapılır. İşbirliği (*collaboration*) diyagramı ile birlikte ardışıl (*sequential*) diyagram, etkileşim diyagramları olarak isimlendirilirler.

İşbirliği diyagramı, hangi nesnelerin birlikte çalışacağını, nasıl mesajlaşacağını, bir durumu ya da fonksiyonu nasıl gerçekleştireceklerini belirtir. Kullanım senaryosu diyagramında belirtilen tüm durumlar için, ayrı işbirliği diyagramları çizmek gerekir. İşbirliği diyagramında nesneler arası etkileşim graf yapısında gösterilir. Nesneler grafın düğümlerini, mesajlar ise bağlantılarını oluşturur.

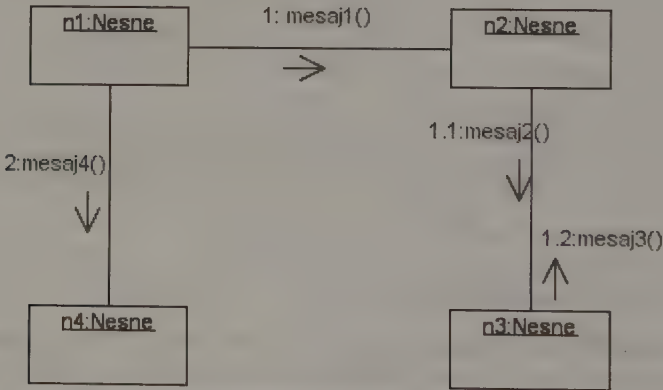
Ardışıl diyagram, işbirliği diyagramında verilen bilginin daha detaylı açıklamalarından oluşur. Nesneler arasındaki mesajlaşmaların sırası, diyagramdaki diziliş sırası ile betimlenir. Mesaj sırası, yukarıdan aşağı doğru devam eder. Nesneler ise yan yana gösterilirler.

İşbirliği diyagramı az yer kaplar. Ayrıca mesaj dallanmaları ile döngüler anlaşılır biçimde gösterilir. Ardışıl diyagram ise çok yer kaplar. Buna karşın mesajların zaman içinde oluş sıraları, işbirliği diyagramlarına göre çok daha anlaşılırdır.

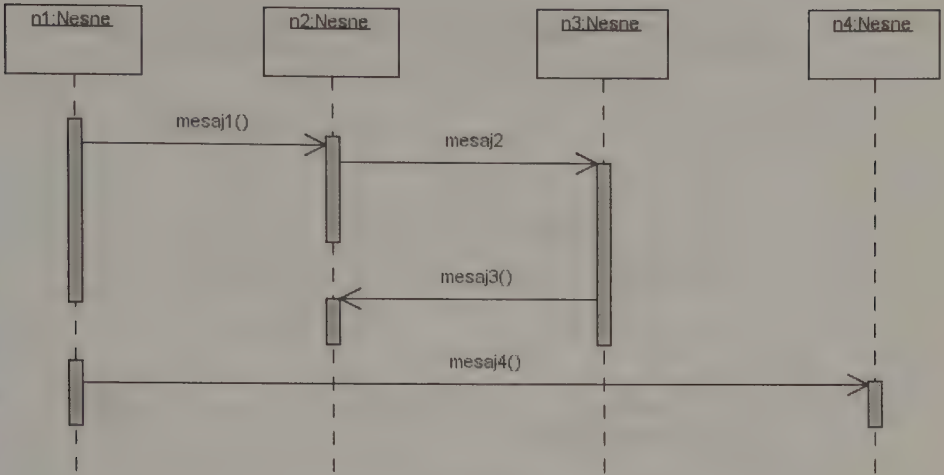
Etkileşim diyagramlarının her ikisinde de aynı kavramlar, farklı şekillerde gösterilebilir. İlerleyen bölümlerde bu kavramlar, her iki diyagram için de örnek çizimlerle açıklanacaktır.

12.2.1. Mesaj Sıra Numaraları

İşbirliği diyagramlarında mesajlar, yollandıkları sıraya göre numaralandırılırlar. Diyagramdaki ilk mesaja numara verilmez. Bir mesajın yollanması sebebiyle yollanan mesajlara, bağlı alt numaralar verilir. Ardışıl diyagramlarda ise sıra numarası kullanılmaz. Mesajlar yollandıkları sırada alt alta çizilir. Şekil-12.2’de işbirliği diyagramında mesaj sıra numaraları gösterilmiştir. Şekil-12.3’de ise aynı örnek için ardışıl diyagramda mesajların sırası görünmektedir.



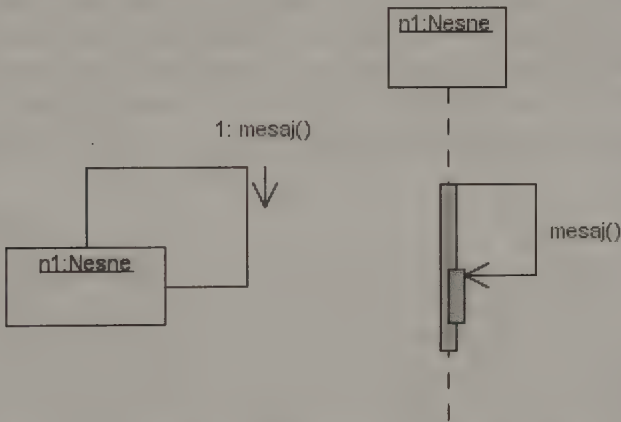
Şekil-12.2. İşbirliği diyagramında mesaj sıra numaraları



Şekil-12.3. Ardışıl diyagramda mesajların sıralı gösterimi

12.2.2. Kendine Mesaj

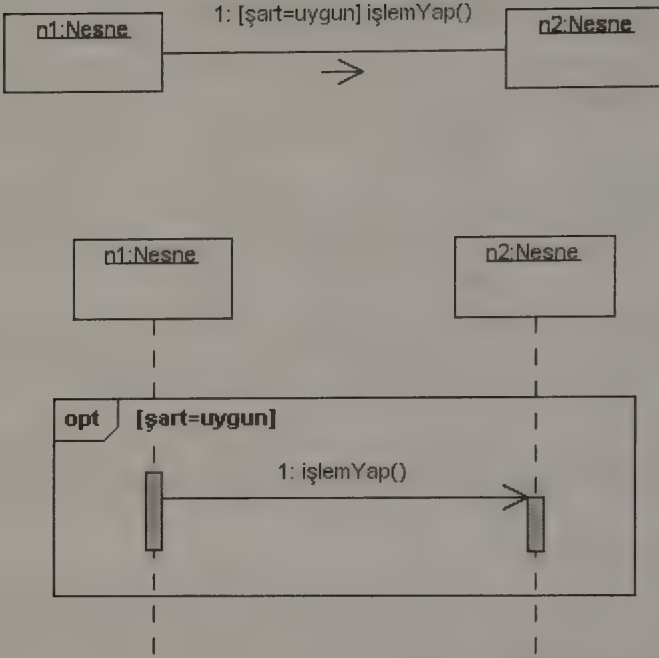
Tasarımda bir sorumluluğu yerine getirmek üzere nesne kendi metotlarından birini çağırabilir. Bir nesnenin kendi metotlarını çağırmasına kendine mesaj denir. Etkileşim diyagramlarının her iki çeşidinde de kendine mesaj gösterilebilir. Şekil-12.4’de örnek mesajlar görülmektedir.



Şekil-12.4. Etkileşim diyagramlarında kendine mesajlar

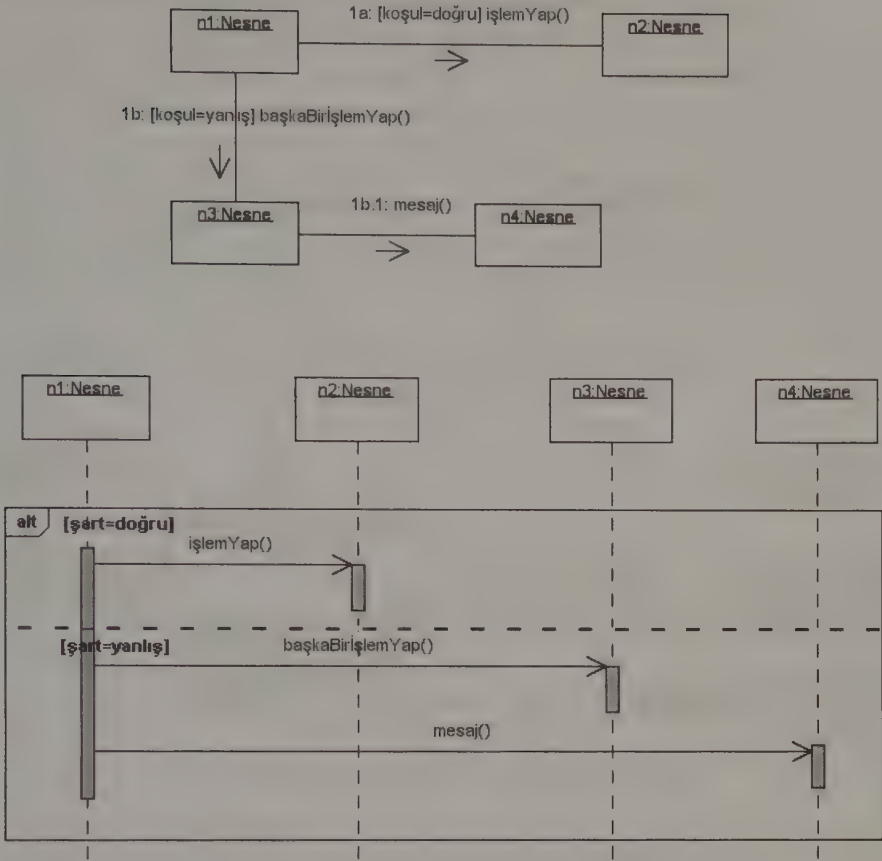
12.2.3. Koşullu Mesajlar

Bir mesajın sadece belirli bir koşul gerçekleştiğinde gönderilmesi gerekebilir. İşbirliği diyagramında bu durum mesajın başına eklenen şart ile gösterilir. Ardışıl diyagramlarda ise “opt” çerçevesi kullanılır. Şekil-12.5’de etkileşim diyagramlarında koşullu mesaj örnekleri yer almaktadır.



Şekil-12.5. Etkileşim diyagramlarında koşullu mesajlar

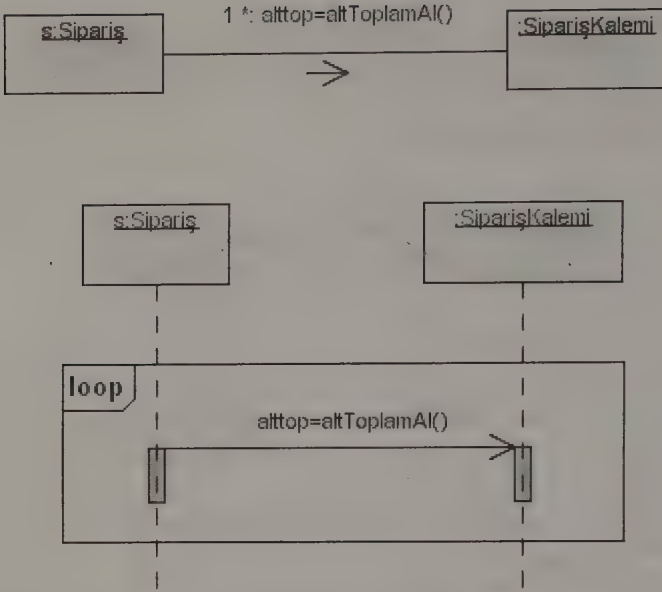
Karşılıklı dışlamalı (*mutually exclusive*) mesajlarda ise birbirini tümleyen iki durum söz konusudur. İşbirliği diyagramında tümleyen durumlar şart olarak mesajların başında gösterilir. Ardışıl diyagramlarda ise iki parçalı “alt” çerçevesi kullanılır. Şekil-12.6’da etkileşim diyagramlarında karşılıklı dışlamalı mesaj örnekleri görülmektedir.



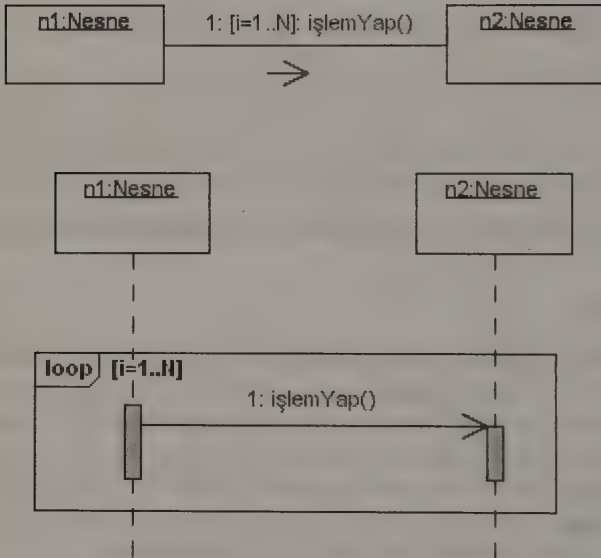
Şekil-12-6. Etkileşim diyagramlarında karşılıklı dışlamalı mesajlar

12.2.4. Döngüler

Bir işlemin yapılması için bir nesne grubuna mesaj gönderilmesi gerekebilir. Bu gibi durumlarda nesneye yönelik dillerde yer alan iteratör yapıları kullanılır. Bir nesne grubuna yollanan mesaj ya da mesajlar işbirliği diyagramında “*” simgesiyle, ardışıl diyagramda ise bir “loop” çerçevesi içinde gösterilir. İşbirliği diyagramında çoklu nesneler ardı ardına çizilen kutular ile gösterilir. Şekil-12.7’de çoklu nesnelere gönderilen mesaj örneği verilmiştir. Döngü koşulu istenirse koşullu mesajlarda olduğu gibi belirtilebilir. Örnek gösterimler Şekil-12.8’de bulunabilir.



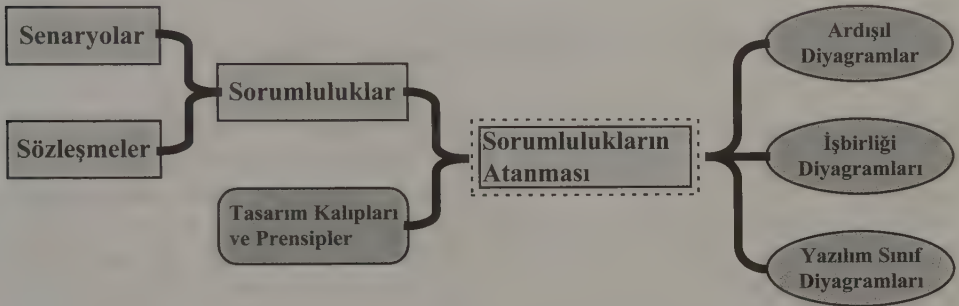
Şekil-12.7. Etkileşim diyagramlarında çoklu nesnelere mesajlar



Şekil-12.8. Etkileşim diyagramlarında döngüler

12.3. Kullanım Senaryolarının Gerçeklenmesi

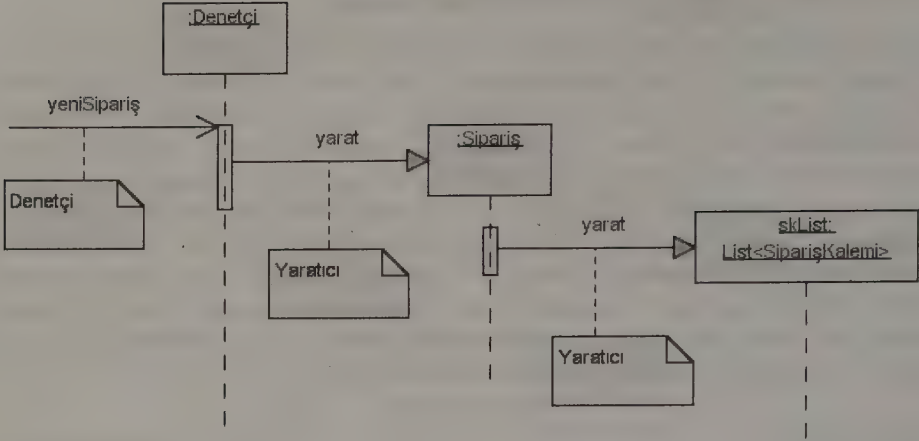
Analiz aşamasında hazırlanan kullanım senaryoları, tasarım modelinde işbirliği içinde çalışan nesneler şeklinde gerçekleşirler. Tasarımcının bir kullanım senaryosunda yer alan durumları modelleyerek anlatması ile senaryolar gerçekleşir. Gerçekleme için önceki başlıkta incelenen diyagramlardan yararlanılır. Bunun yanında öğrenilen tasarım kalıpları ve prensipleri de, doğru tasarım yapılmasına yardımcı olur. Tasarımdaki en önemli girdi ise kuşkusuz gerçekleştirilecek olan senaryolar ve varsa bunlar için yazılan sözleşmelerdir. Senaryoların gerçekleşmesi ile işbirliği içinde çalışan nesnelere sorumluluklar atanır. Bu aşamada nesneye yönelik tasarım genel hatlarıyla tamamlanmıştır. Tasarım, UML kullanılarak etkileşim ve sınıf diyagramları ile ifade edilir. Senaryoların gerçekleşmesi (nesnel tasarım) adımıyla kullanılan girdiler ve elde edilen çıktılar Şekil-12.9'da görülmektedir.



Şekil-12.9. Senaryoların gerçekleşmesi

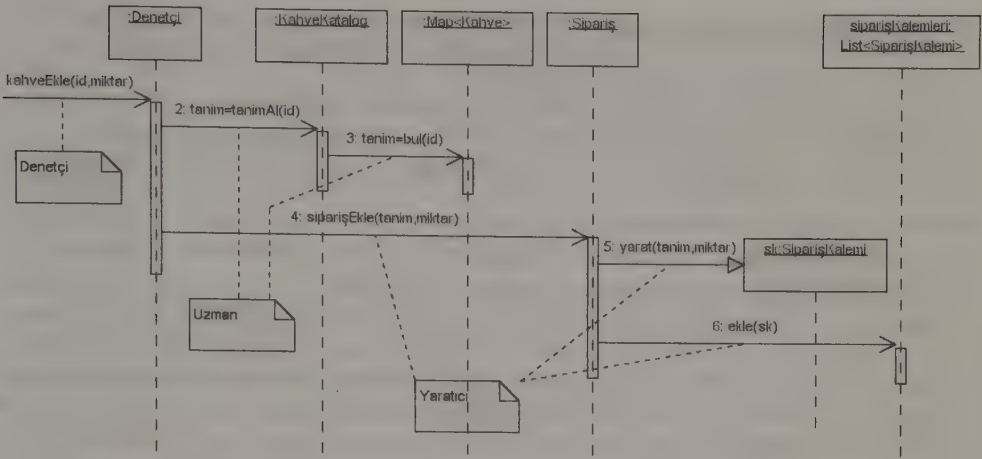
Senaryolar gerçekleşirken sorumlulukları atamak üzere ilk bakılması gereken yer senaryolar ve onlara bağlı olarak yazılan sözleşmelerdir. Analiz aşamasında hazırlanan sınıf diyagramı incelenen sistemi tasarlamak için temel oluşturmaktadır. Her bir senaryo ve sözleşme tek tek ele alınarak tasarlanır.

Müşterinin yeni bir sipariş vermesi, senaryoya bağlı bir sözleşme olarak yazılabilir. Yeni sipariş işleminin sonunda yeni bir *Sipariş* nesnesi ile boş bir *SiparişKalemi* listesi yaratılmalıdır. Bu işleri yapmak içinse sistemde bir yetkili olması gereklidir. Şekil-11.3'deki sınıf diyagramı incelenirse böyle bir sorumlunun olmadığı görülür. İncelenen tasarım kalıp ve prensipleri göz önüne alındığında sorumluluğu alacak bir denetçi nesnesine ihtiyaç olduğu açıktır. Denetçi kalıbına uygun olarak yeni sipariş verme işleminin sorumlusu *Denetçi* nesnesidir. Yaratıcı kalıbına uygun olarak, *Sipariş* nesnesini yaratma sorumluluğu *Denetçi*nindir. Yine yaratıcı kalıbına göre *SiparişKalemi* listesini yaratması gereken nesne *Sipariş* olmalıdır. Örnek olarak tasarlanan sözleşmeye ait ardışıl diyagram Şekil-12.10'da verilmiştir.

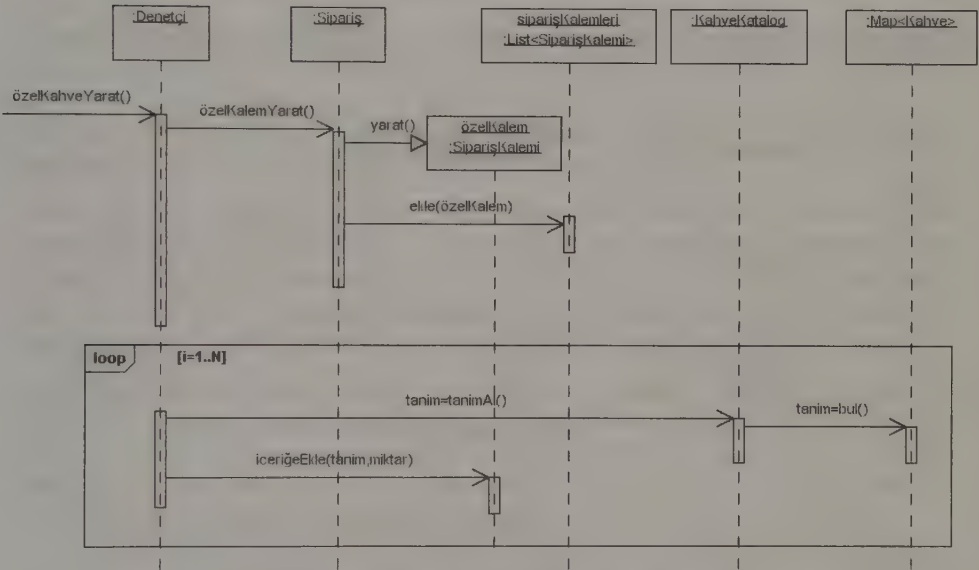


Şekil-12.10. Yeni sipariş sözleşmesinin tasarımı

Sipariş senaryosunda ürün ekleme işlemi de bir sözleşme olarak ele alınabilir. Yeni bir *SiparişKalemi* yaratılarak *Sipariş* ile ilişkilendirilmelidir. *SiparişKalemi*nde yer alacak *Kahve* nesnesinin de alınarak ilişkilendirilmesi gerekmektedir. Analiz sınıf diyagramı incelendiğinde *Kahve* nesnelerini bir arada tutan, ilgili *Kahvenin* bulunmasını sağlayacak bir sınıf olmadığı görülür. Bu durumda bütün *Kahve* nesnelerini içerecek olan bir *KahveKatalog* nesnesine ihtiyaç duyulmaktadır. *Kahve* ekleme işleminin sorumluluğu, denetçi kalıbı göz önüne alınarak yaratılan *Denetçi* nesnesine verilebilir. *Siparişe* *Kahveyi* ekleyebilmek için *Denetçinin* *Kahve* nesnesinin referansına sahip olması gereklidir. *KahveKatalog* nesnesinden ilgili *Kahve* nesnesini alır. *Denetçi* aldığı referansı *Sipariş* nesnesine aktararak yeni bir *SiparişKalemi* yaratılmasını ister. *Sipariş* de bir *SiparişKalemi* nesnesi yaratarak onu listesine ekler. Bu sayede standart bir *Kahvenin* siparişi tamamlanır. Ürün ekleme işlemine ait ardışıl diyagram Şekil-12.11’de görülmektedir.



Şekil-12.11. Ürün ekleme işleminin ardışıl diyagramı



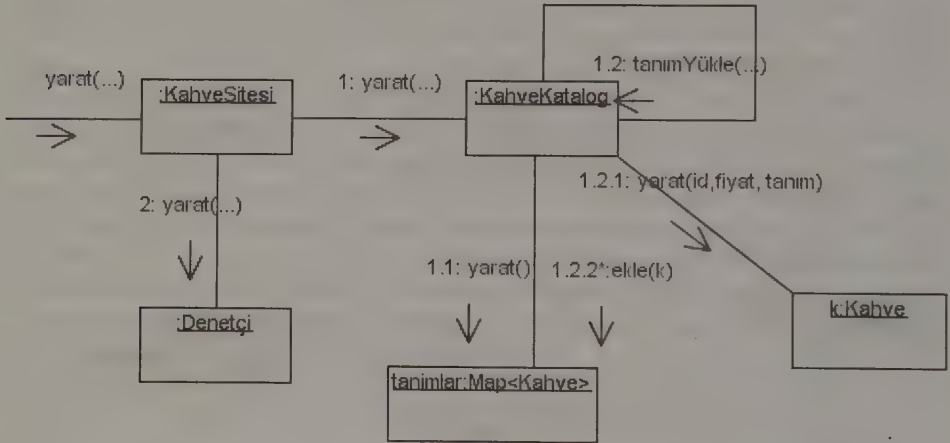
Şekil-12.12. Özel kahve siparişi işleminin ardışıl diyagramı

Özel kahve oluştururken bir sipariş kaleminde birden fazla *Kahve* yer alabilir. Bu durumda kullanıcı istediği kadar *Kahve*yi istediği miktarlarda seçerek bir sipariş kalemine ekleyebilir. Yaratılan *SiparişKalemi* nesnesi ise bir tanedir. *SiparişKalemi*nde yer alan içerik listesinin doldurulması işlemi, kullanıcı istediği sürece devam edecektir. Elde edilen *SiparişKalemi*, *Sipariş* ile ilişkilendirilecektir. Özel kahve siparişi işleminin ardışıl diyagramı Şekil-12.12’de verilmiştir. Sistemdeki diğer senaryolar ve bunlarla ait sözleşmeler de kurallar göz önünde tutularak modellenmelidir.

12.3.1. Başlangıç İşlemleri

Tasarlanan sistem ilk çalışmaya başladığında yapılması gereken işlemler vardır. Başlangıç nesnesi ve buna bağlı nesnelerin yaratılması gibi işlemlerin ayrı bir kullanım senaryosu olarak yazılması yararlıdır. Sistemin başlangıcında yapılacak olan işleri ve görevleri belirlemek için en uygun zaman, tasarımın en son aşamasıdır.

Program çalışmaya başladığında yaratılacak olan ilk nesne belirlenir ve içerdiği diğer nesneleri yaratma, aralarındaki bağlantıları sağlama gibi sorumluluklar verilir. Başlangıç nesnesi tüm programın çalışmasını denetleyen bir nesnedir. Başlangıç nesnesi yaratıldığında ve başlangıçla ilgili diğer işlemler de yerine getirildiğinde sistem harekete geçer. Diğer bir deyişle başlangıç nesnesi doğrudan içerdiği nesneleri yaratır ve aralarındaki bağlantıları kurar. Tasarlanan sistemin denetimi atanan başlangıç nesnesinde olmayabilir. Başlangıç sorumluluklarına ana program ya da sistemin ara yüz nesneleri de sahip olabilir. Sistemin kullanıcı arayüzü ile olan ilişkisi de Denetçi nesnesi üzerinden sağlanır. Bu sayede tasarlanan sistemin kullanıcı arayüzünden bağımsız olması da sağlanır. Şekil-12.13’de kahve sitesi sistemi için tasarlanan başlangıç işlemlerine ait işbirliği diyagramı görülmektedir.



Şekil-12.13. Kahve sitesi için başlangıç işlemlerinin işbirliği diyagramı

12.3.2. Görünürlük Tipleri

Senaryolar gerçekleşirken nesneler işbirliği içinde çalışırlar. Bu amaçla nesneler birbirleri ile mesajlaşırlar. Bir nesnenin diğer bir nesneye mesaj yollaması, o nesnedeki bir metodu çağırmasıdır. Nesnenin mesaj yollayabilmesi için yolladığı nesneyi tanıması, bilmesi gereklidir. Bunun için o nesnenin referansına sahip olmalıdır. Görünürlük, bir nesnenin diğer bir nesneye erişilebilmesidir. Nesneler arasında dört çeşit görünürlük tipi bulunmaktadır.

Özellik Görünürlüğü (*Attribute visibility*): B nesnesi A nesnesinin bir özelliğidir. Bu durumda A, B'ye sahiptir. A ve B var oldukça görünürlük devam eder.

Parametre Görünürlüğü (*Parameter visibility*): B nesnesi, A nesnesinin bir metodunda parametre olarak yer alır. Görünürlük sadece o metodun içinde ve çalıştığı sürece geçerlidir. Metod bitince görünürlük ortadan kalkar. Geçerliliğin daha az olması sayesinde nesneler arası bağımlılık da azalır. Ancak genellikle parametre görünürlüğü özellik görünürlüğüne dönüşür. Örneğin A nesnesine parametre olarak gelen B nesnesi, A nesnesinin yarattığı bir C nesnesinde bir özellik olarak atanır.

Yerel görünürlük (*Local visibility*): B nesnesi, A nesnesinin bir metodunda yerel bir değişken olarak yer alır. Görünürlüğün geçerliliği metodun çalışma süresi kadardır.

Global görünürlük (*Global visibility*): B nesnesi ile A nesnesi aynı uzayda yer aldığında olan görünürlük tipidir. A ve B nesneleri var oldukça görünürlük devam eder. Nesnelerin global değişkenlere atanması ile gerçekleştirilebilen global görünürlük çok tercih edilmez ve nesneye yönelik dillerin tümü tarafından da desteklenmez.

12.4. Tasarım Sınıf Diyagramı

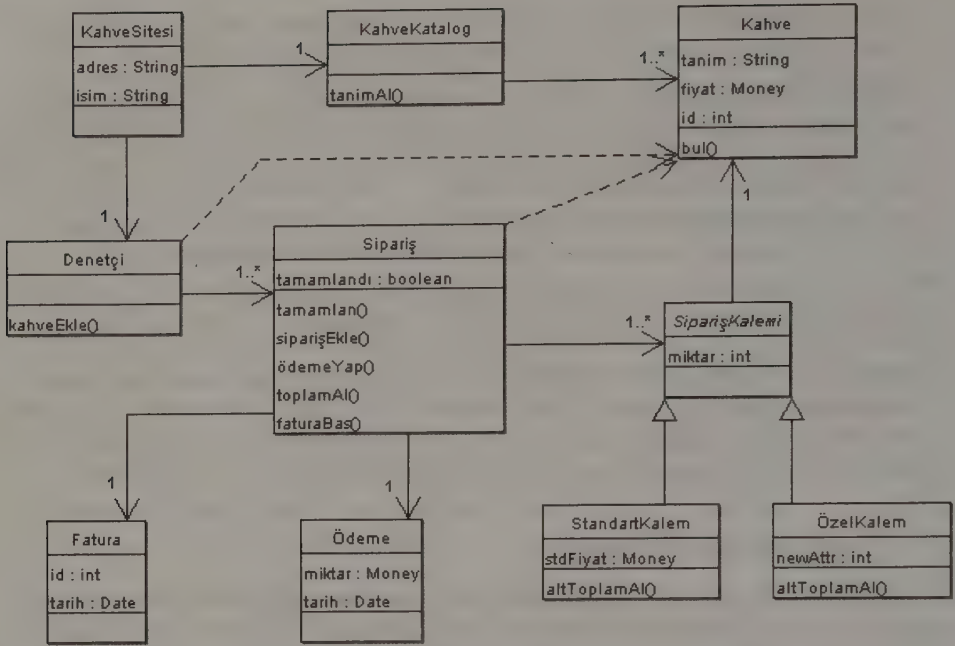
Senaryoları gerçeklemek üzere etkileşim diyagramlarının oluşturulmasına paralel olarak yazılım sınıflarını gösteren tasarım sınıf diyagramı da çizilir. Analiz aşamasında çizilen sınıf diyagramı kavramsal sınıfları ve temel özellikleri gösterirken, tasarım sınıf diyagramında yazılım sınıfları, bunların özellikleri, özelliklerin tipleri, metotları, metotların parametreleri ve erişim hakları gösterilir.

Tasarım sınıf diyagramında yazılım sınıfları arasındaki ilişkiler ve bağımlılıklar yönlü olarak gösterilir. Genellikle nitelik görünürlüğü sonucunda oluşan bağlantı, mesaj yollayan nesneden yollanan nesneye doğru çizilen bir ok ile belirtilir. Bu sayede yazılım sınıfları arasında yer alan özellik görünürlükleri belirtilmiş olur. Sınıflar arasındaki bağlantılar belirlenirken, çizilen etkileşim diyagramlarından yararlanılır.

Yazılım sınıfları arasında bağımlılık, özellik görünürlüğü dışındaki görünürlük tipleri sonucunda oluşur. Sınıf diyagramında kesik çizgili ok ile gösterilir. Örneğin Şekil-12.11'de *Denetçi*, *KahveKatalogdan* *Kahve* tanımını alırken *Kahve* tipinden bir parametre alır. Bu durumda *Denetçi*, *Kahveye* bağımlıdır. Şekil-12.14'de tasarlanan kahve sitesinin sınıf diyagramı görülmektedir.

Tasarımın sonunda sınıflar ve aralarındaki etkileşimler, işbirlikleri, bağlantılar belirlenmiş olur. Sadece metotların nasıl çalışacaklarını belirleyen algoritmaları belli değildir. Etkileşim diyagramlarına eklenecek küçük not ve açıklamalar ile yapılması gereken işlemler belirtilebilir. Bu sayede programcılara yol gösterilmiş olur.

Kodlama aşamasında sınıf tanımları sınıf diyagramı, metotları da etkileşim diyagramları göz önüne alınarak oluşturulur. Sınıfların gerçekleşmesine bağımlılığı en az olan sınıftan başlamak kolaylık sağlar. Gerçeklenen her bir sınıfın sonunda testinin de yapılması gereklidir. Test edilecek olan sınıftan nesne yaratılır, mesajlar gönderilerek içerdiği tüm metotlar çalıştırılır. Gerçeklenen sınıfın doğru sonuçlar ürettiği sınanarak, tasarıma uygunluğu kontrol edilmelidir.



Şekil-12.14. Kahve sitesinin tasarım sınıf diyagramı

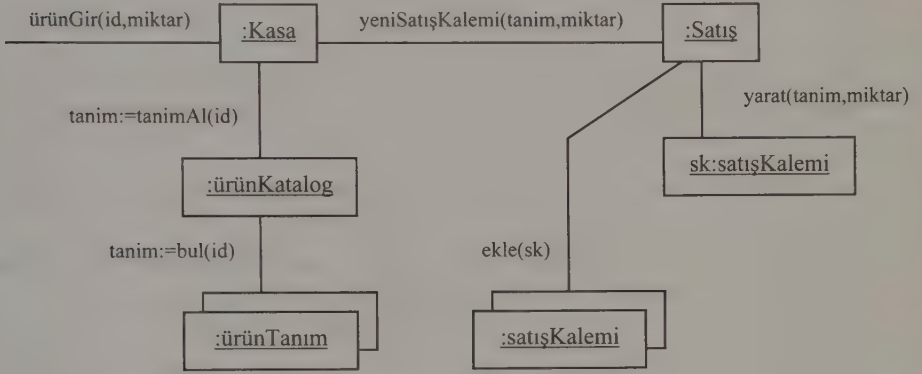
12.5. Özet

Analizi yapılarak modellenen sistemin mantıksal çözümü tasarım aşamasında oluşturulur. Kavramsal sınıflardan yola çıkılarak, yazılım sınıfları belirlenir. Esas iş ise, etkileşimlerin belirlenmesi ile nesnelere sorumlulukların atanmasıdır. Sorumlulukları yerine getirmek üzere sınıflarda metodlar oluşturulur. Etkileşim diyagramları ile tasarlanan sistemin nesnelerinin işbirliği modellenmesi resmedilir. Bunlarla birlikte sistemin son durumunu gösteren tasarım sınıf diyagramı da çizilir.

Yazılım sınıflarının belirlenip sorumlulukların atanması aşamasında, genel prensipler ve akla yatkın çözümlerin bir araya getirilmesiyle oluşturulan tasarım kalıplarından yararlanılır. Tasarım kalıpları sorumlulukların atanmasında izlenmesi gereken temel yolları tarif eder. Kullanım senaryolarında yer alan durumların incelenmesiyle sorumluluklar bulunur. Sorumlulukların kalıplara uygun biçimde modellenerek anlatımı ile senaryolar gerçekleşir. Gerçeklenen senaryolar ile de, işbirliği içinde çalışan nesnelere sorumlulukları atanır. Tasarımın tam olması için, tüm senaryoların gerçekleşmesi gereklidir. Bunların yanında, sistemin başlangıcında yapılması gereken işler de en sonda ayrı bir senaryo grubu olarak ele alınıp tasarlanabilir.

12.6. Sorular

- 12.6.1)** GoF tasarım kalıplarının en yaygın olarak kullanılanlarını araştırınız.
- 12.6.2)** UML'in birinci ve ikinci versiyonunda, diyagramların çiziminde ne gibi farklılıklar bulunduğunu araştırınız.
- 12.6.3)** Bir mağazanın satış sisteminde yer alan ürün satış senaryosuyla ilgili olarak, kasa da ürün okutma işi aşağıda verilen işbirliği (*collaboration*) diyagramı ile tasarlanmıştır. Kasa ilk olarak ürün tanımını öğrenir. Bu tanımı ve miktarı, satış nesnesine aktarır. Satış nesnesi, girilen ürünü satış kalemi olarak listeye ekler.



- Mesajların sıra numaralarını belirleyiniz.
 - Mesajların yönlerini çiziniz.
 - “ürünGir” mesajı ve “kasa” nesnesi hangi kalıba uygundur? Nedenleri ile açıklayınız.
 - “yeniSatışKalemi” mesajı, “satış” nesnesi ve “yarat” mesajı hangi kalıba uygundur? Nedenleri ile açıklayınız.
 - “tanımAl” mesajı ve “ürünKatalog” nesnesi, hangi kalıba uygundur? Nedenleri ile açıklayınız.
- 12.6.4)** Aşağıda gerçekleştirilen sistemle ilgili kod parçacıkları verilmiştir. Buradaki “tanım” nesnesinin bulunduğu görünürlük tiplerini nedenleri ile kısaca açıklayınız.

```

yeniSatışKalemi(ürünTanım tanım, int miktar){
    ...
    sk = new satışKalemi(tanim, miktar);
    ...
}
// satışKalemi kurucu metodu
satışKalemi(ürünTanım tanım, int miktar){
    ...
    ürüntanımlı = tanım;
    ...
}

```


KISIM IV

SİSTEM GERÇEKLEŞTİRME

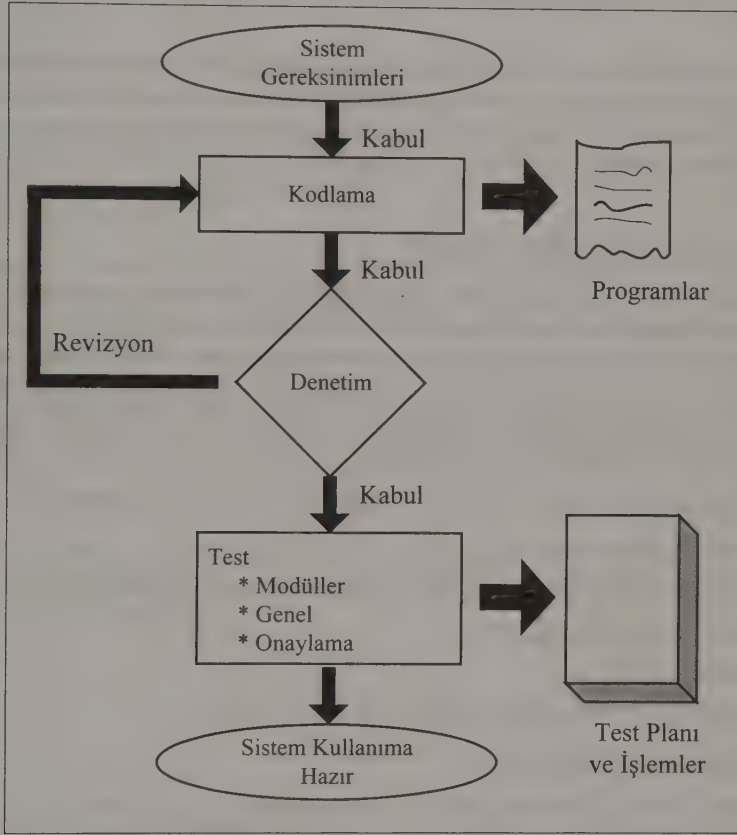
Dördüncü kısımdaki bölümler, geleneksel ya da nesneye yönelik yaklaşımla analiz edilen ve tasarlanan sistemin gerçekleştirilmesine genel bir bakış açısı sağlamaktadır.

- Bölüm 13:** Sistemi gerçekleştirmek için programlama dili seçimiyle başlayan kodlama sürecini anlatmaktadır.
- Bölüm 14:** Güvenilir ve bakım yapılabilir bilgi sistemlerini geliştirmek için gerekli olan kalite özellikleri ve standartları tanıtarak yazılım sınaması teknikleri açıklanmaktadır.
- Bölüm 15:** Mevcut sistemden yeni sisteme geçiş yöntemlerini olumlu ve olumsuz yanlarıyla anlatmaktadır.

BÖLÜM 13

KODLAMA

Sistem gerçekleştirme Şekil-13.1’de görüldüğü gibi yazılımın kodlanması ile başlayıp test aşaması sonrası sistem kullanıma hazır hale gelir. Tasarım aşamasında yazılım fiziksel olarak tanımlanıp diyagramlar ya da geçersiz kodlama (*pseudocode*) ile gösterildikten sonra, makine tarafından okunabilecek bir program halinde düzenlenmektedir.



Şekil-13.1. Sistem gerçekleştirme

Bilgi sisteminin yapısı durağan olmadığı için kodlamanın bazı ilkelere uyularak yazılımın belirli bir düzende yapılması test ve bakım aşamaları için de büyük önem taşımaktadır. Dolayısıyla uygulamanın gerektirdiği güncel değişikliklerin ilgili yazılıma aktarılması için gerekli değişiklikleri yapmak gerekir. Belli bir düzene sahip

olmayan kodlarda bu değişiklikleri yapmak hem zaman hem de maliyet kaybına yol açmaktadır.

Kaynak programdan beklenen; kodlamanın spesifikasyonlara uygun olarak düzenlenmesi ve bu uygunluğunun kolayca izlenebilmesi, test edilmesi ve gerektiğinde değiştirilmesidir. Bunun için, her ne kadar yazılım kod stili için kabul edilmiş genel bir standart yoksa da kodlama program ekibi tarafından önceden belirlenen standartlara uygun olarak kodlamanın olduğunca açık ve doğru biçimde yapılmalıdır. Kaynak programın niteliği, düzenli bir kodlama tekniğine ve kodlama biçimine bağlı bulunmaktadır.

Kodlama aşamasına geçilmeden yapılması gereken diğer husus ise kodlama için kullanılacak programlama dilinin ve geliştirme platformunun belirlenmesidir. Kodlama için kullanılacak program dili yazılımın amaç ve görevlerine uygun olarak seçilmelidir.

Kaynak programın bakımı ve sürdürülmesinde en önemli husus, tasarımın kaynak programa dönüştürülmesindeki kolaylık özelliğidir. Ayrıca programlama dilinin giriş ve iç açıklamalarının özellikleri de kaynak programın bakım ve sürdürülmesini büyük ölçüde etkilemektedir.

13.1. Programlama Dilinin Seçimi

Programlama dilinin seçiminde, yazılım mühendisliği yönünden uygunluğu ve özelliği dikkate alınmalıdır. Programlama dilinin kararlaştırılmasına, sorunun tanımlanması ve gereksinimlerin analizi aşamasında başlanmalıdır. Büyük projelerde çoğu kez, belirli bir tek programlama dili, gereksinimleri karşılamaya yetmeyebilir. Bu durumda, gereksinimleri karşılayabilecek diller, liste halinde belirlenmeli ve birbirleriyle uyumu göz önünde bulundurulmalıdır.

Kullanılacak programlama dillerinin seçiminde ve kararlaştırılmasında;

- Genel uygulama alanı
- Algoritma, bilgi işlem ve veri yapısı açılarından karmaşıklık derecesi
- Yazılımın kullanılacağı ortam
- Uygulama koşulları
- Yazılım geliştirmede görevli personelin bilgi düzeyi

ölçütlerinden yararlanılmaktadır.

Programlama dilinin seçiminde, dil özellikleri dışında, proje koşullarının sınırladığı çok sayıda faktör de etkili olmaktadır. Örneğin, bazı kuruluşlar belirli bir programlama dili üzerine önemli bir yatırım yapmış olabileceği gibi programcıların bir programlama dilindeki tecrübesi oldukça fazla olabilir. Ya da müşteri belirli bir programlama dilinin kullanılmasını ön koşul olarak ileri sürmüş olabilir. Sahip olunan derleyici, veri tabanı yönetim sistemi, işletim sistemi ve yardımcı araçlar yüzünden de dil kısıtlanmış olabilmektedir.

13.2. Yapısal Kodlama

Yapısal kodlama (*structured coding*), programın uygulanması sırasında düzenli bir kontrol akışının sağlanmasıdır. Düzenli bir kontrol akışı, ancak bir girişi ve bir çıkışı bulunan **yapısal programlama** ile sağlanabilmektedir. Fakat bu durum, program etkinliği bakımından bazı sorunlara neden olmaktadır.

13.3. Kodlama Biçimi

Kodlama biçimi (*coding style*), programcının tasarımda tanımlanmış olan işlemleri kod olarak ifade şeklidir. Daha öncede belirtildiği gibi kodlama biçimi için genel standart olmasa da kodlama düzeni proje ekibi tarafından ortaya konmalıdır.

Kodlamanın anlaşılır olabilmesi için kodlama biçiminin en azından aşağıdaki özelliklere sahip olması beklenir:

Açıklama satırları; bir program parçasının anlaşılabilir olması için gereklidir. Örneğin programın başlangıcındaki açıklamada modülün adı, işlevi, amacı, kodlayanın adı, ilk oluşturulma tarihi, son düzenleme tarihi, girdi ve çıktılar, kabuller vb. genel tanıtıcı bilgiler yer alabilir. Kod açıklamaları ise daha çok program içinde yer alan koşullu deyimler, döngü deyimleri gibi denetim yapılarının öncesinde ve/veya sonrasında kullanılmaktadır.

Kod yazım düzeni; de programın anlaşılabilir olmasında önemli bir etkidir. Programın okunabilirliğini arttırmak amacıyla kod satırları arasında ve kod satırının kendi içinde gerekli boşluğu bırakılabilir. Ayrıca satırbaşı, parantez, sözcük ve satır aralıkları ile açıklama bloklarının sınırları, okumayı kolaylaştırıcı şekilde düzenlenebilir.

Anlamlı isimlendirme yapılarak veri tabanı tabloları, değişkenler ve program modül isimlerinde anlamayı zorlaştırmayacak şekilde aşırı kısaltmadan çekinmek gerekir. İsimlendirme yöntemi de proje ekibi tarafından tasarım aşamasında belirlenerek bir standart oluşturulmalıdır.

Kaynak programın verimliliği doğrudan, ayrıntılı tasarım aşamasında belirlenen algoritmaların etkinliğine bağlıdır. Kodlama biçimi sadece işletim hızı ve iç bellek ihtiyacını etkileyebilmektedir.

13.4. Özet

Sistemin gerçekleştirme aşamasında, sistem tasarımına uygun kodlama, sına ve kullanıma hazırlık işlemleri gerçekleştirilir. Kaynak programdan beklenen; kodlamanın tasarım spesifikasyonlarına uygun olarak düzenlenmesi, kolayca izlenebilmesi, test edilmesi ve gerektiğinde değiştirilmesidir.

Kullanılacak programlama dillerinin seçiminde ve kararlaştırılmasında, genel uygulama alanı, algoritma, bilgi işlem ve veri yapısı açılarından karmaşıklık derecesi, yazılımın kullanılacağı ortam, uygulama koşulları, yazılım geliştirmede görevli personelin bilgi düzeyi ölçütlerinden yararlanılmaktadır.

Yapısal kodlama, bir girişi ve bir çıkışı bulunan düzenli bir kontrol akışının sağlanmasıdır.

13.5. Sorular

- 13.5.1) Sistem gerçekleştirme süreci adımlarını ve aralarındaki ilişkiyi gösteriniz.
- 13.5.2) Kullanılacak programlama dillerinin seçiminde ve kararlaştırılmasında hangi ölçütlerden yararlanılmaktadır?
- 13.5.3) Kodlamanın anlaşılır olabilmesi için kodlama biçiminin hangi özelliklere sahip olması gerekir?

BÖLÜM 14

YAZILIMDA KALİTE VE TEST TEKNİKLERİ

Kaliteli yazılım; önceden belirlenmiş olan işlevsel gereksinimlere, yazılım geliştirme standartlarına ve bir yazılımdan beklenen bütün özelliklere tamamen uygun bir yazılımdır.

14.1. Yazılım Kalitesi Faktörleri

Yazılım kalitesini oluşturan faktörler şunlardır;

- **Doğruluk**, spesifikasyonlara uygunluk ve müşteri isteklerini karşılama derecesi.
- **Güvenirlilik**, tasarlanan işlevleri istenilen duyarlılıkta yerine getirme olanağı.
- **Verimlilik**, programın işlevlerini yerine getirebilmesi için kullandığı bilgi işlem kaynaklarının miktarı.
- **Güvenlik**, yetkisiz kişilerin yazılıma müdahalesini veya veri girişini önleme olanağı.
- **Kullanışlılık**, öğrenme, işletme, girdi hazırlama ve çıktı yorumlamada kolaylık derecesi.
- **Hata bulma kolaylığı**, hatanın yerini bulma ve düzeltme kolaylığı.
- **Esneklik**, yazılımda değişiklik yapma kolaylığı.
- **Sınama kolaylığı**, yazılımın doğruluğunun sınanmasındaki kolaylık.
- **Taşınabilirlik**, programın farklı donanımlarda ve yazılım sistemi ortamlarında kullanılma olanağı.
- **Tekrar kullanılabilirlik**, yazılımın tamamının ya da bir bölümünün farklı bir uygulamada kullanma olanağı.
- **Bağlanabilirlik**, bir sistemin başka bir sisteme bağlanma olanağı.

Bu faktörler (F_i) doğrudan ölçülmeyip, ancak bazı özelliklere (m_j) bağlı birer doğrusal regresyon denklemi;

$$F_i = c_1m_1 + c_2m_2 + c_3m_3$$

biçiminde ya da faktör analizi (diskriminant fonksiyonu) ile kestirilebilmektedir.

14.2. Kalitenin Güvence Altına Alınması

Kalitenin güvence altına alınması;

- Yazılım sisteminin geliştirilmesinin yüksek bir kalitede gerçekleştirilmesi,
- Ölçüm yoluyla kalite derecesinin belirlenmesi

yolları ile sağlanmaktadır.

Yazılım geliştirme sürecinde kaliteyi yükseltmek için;

- Gereksinim analizi aşamasında sistemin kolayca kullanılıp öğrenilir olması, istenmeyen hallerin planlı olarak değiştirilmesine çaba gösterilmesi
- Modüllere ayırmada taşınabilirlik ve bağlanabilirlik özelliklerine önem verilmesi
- Ayrıntılı tasarımda doğruluk faktörüne ağırlık verilmesi
- Sınama aşamasında doğruluk ve olduğunca performans vb. diğer faktörlerin ölçümüne çaba gösterilmesi

öğütlenmektedir.

Kalitenin belirlenmesi için de;

- Doğrudan, objektif kalite ölçüleri kullanmak (örneğin, HALSTEAD ve McCABE ölçüleri, performans ölçümü)
- Kaliteye koşut olarak değişen, dolaylı ölçülerden yararlanmak (örneğin, doğruluk ile korelasyon halindeki test uygulamaları)
- Uzmanların kişisel takdirine başvurmak

yolları söz konusudur.

14.3. Yazılım Kalitesinin Sağlanması

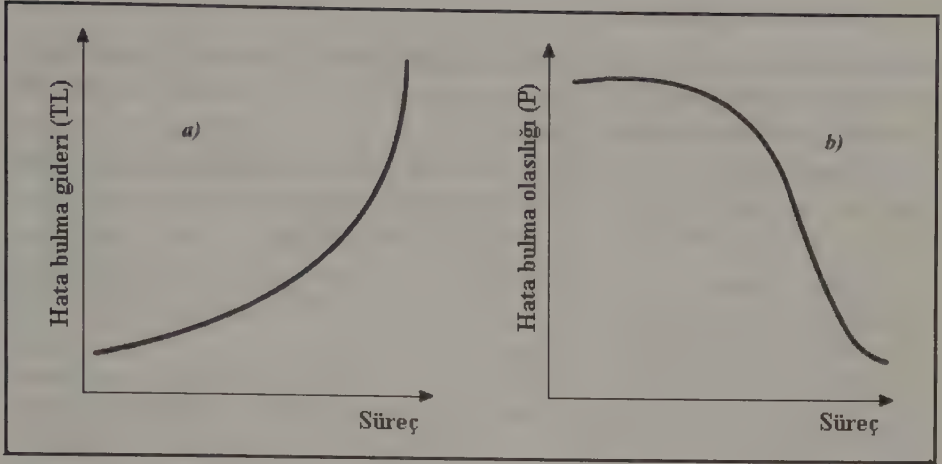
Bir yazılım ürünü, çok sayıda ve bütünleşik alt programlardan oluşmaktadır. Yazılımın geliştirilmesi de, çok sayıda sistem analistinin katkısı ile, uzun bir sürede ve basamaklar halinde gerçekleştirilmektedir. Yazılım ürününün kullanılması aşamasında ortaya çıkan hata ve eksiklerin giderilmesi, donanımın değiştirilmesi ya da bilgisayar sisteminin geliştirilmesi hallerinde de yazılımın yeni durumuna uydurulması gerekmektedir (yazılımın bakımı ve onarımı). Bu işlemler sırasında çeşitli hatalar yapılabilmekte ve yazılım kalitesi yetersiz olabilmektedir.

Yazılım kalitesini sağlamak için, hata ve eksiklerin anında bulunması ve düzeltilmesi büyük önem taşımaktadır. Çünkü bir basamakta yapılan hata, sonraki basamakları da etkilemektedir. İleri basamaklarda bu hatanın bulunması olasılığı giderek azalmakta, düzeltilmesi için gerekli emek ve gider de zamanla orantılı olarak artmaktadır (Şekil-14.1).

Yazılım kalitesi,

- Geliştirmenin planlanması aşamasında kalite kontrolü yöntem ve araçlarının belirlenmesi,
- Geliştirme sürecinin durak noktalarında yapılanların gözden geçirilmesi,
- Kaynak programın sınanması

aşamalarında gerçekleştirilmelidir.



Şekil-14.1. Yazılım geliştirme sürecinde hatanın bulunması gideri ve olasılığı

Bu işlemlere, “üretim sürecinde kalite kontrolü” adı verilmektedir. Ayrıca, yazılım ürünün alıcıya teslimi sırasında da “kabul muayenesi” yapılmaktadır.

14.4. Yazılımda Kalite Kontrolünün Planlanması

Yazılım geliştirme planının düzenlenmesi sırasında, ayrıca “yazılımı inceleme planı” (*software verification plan*) ve “kabul muayene planı” (*acceptance test plan*) hazırlanmaktadır.

Yazılım inceleme planı, incelenecek gereksinimler, tasarım inceleme ve onaylama, kaynak programı sinama planı ve belgeleri onaylama planı konularını içermektedir. Ayrıca bütün bunların gerçekleştirilmesi için uygulanacak teknikler ve kullanılacak araçlar da yazılım inceleme planı kapsamında belirlenmelidir.

Kaynak programı sinama planı, sinama amaçlarını, sinama kriterlerini, sistem birleştirme planını, modüllerde uygulanacak yöntemleri ve kullanılacak özel test programlarını belirtmelidir. Bu plan yazılım ürününün gereksinimlere uygunluğu göstermek için yapılacak işlemleri tanıtmaktadır.

Sinama amacı; verilen sürede belirli koşullar altında hatasız olarak yazılım geliştirme sürecinin saptanmasıdır. Sinama kriterleri; hata oranı, denetlenecek mantıksal yol yüzdesi vb. olarak kararlaştırılabilir. Sistemi birleştirme planında; strateji, zamanlama, programcıların sorumluluğu belirlenmektedir. Yöntem olarak da; izleme, inceleme, denetleme, statik analiz, dinamik analiz, onaylama biçimleri söz konusudur.

Kabul muayene planı, onaylanacak gereksinimler, gereksinimlerin test programları, test programlarında beklenen sonuçlar ve testlerin yeterliliği bölümlerinden oluşmaktadır. Kabul muayene planı, yazılım tasarımı basamağından hazırlanmaya başlanmakta ve üretim tamamlanıncaya kadar geliştirilmektedir.

14.5. Yazılımın Sınanması

Sınama (*testing*), bir programdaki hataları bulmak amacı ile yapılan işlemlerdir. Sınama, yazılımın fonksiyonel, performans, dayanıklılık ve yapısal açılardan yeterliliğini denetlemek için gerçekleştirilir;

Fonksiyonel sınama, tipik bir işletme ortamında örnek girdi değerleri ile alınan çıktı değerlerinin beklenen sonuç ile karşılaştırılması biçiminde yürütülmektedir. Burada uç değerler de verilerek fonksiyonel sınırlar da denenmelidir.

Performans sınamada, değişik durumlardaki yanıt süresi, alt programların işlem süresindeki payları, iç ve dış belleklerin kullanımı ve iletişim hızı ölçülür. Böylece sistemin dar boğazları belirlenir.

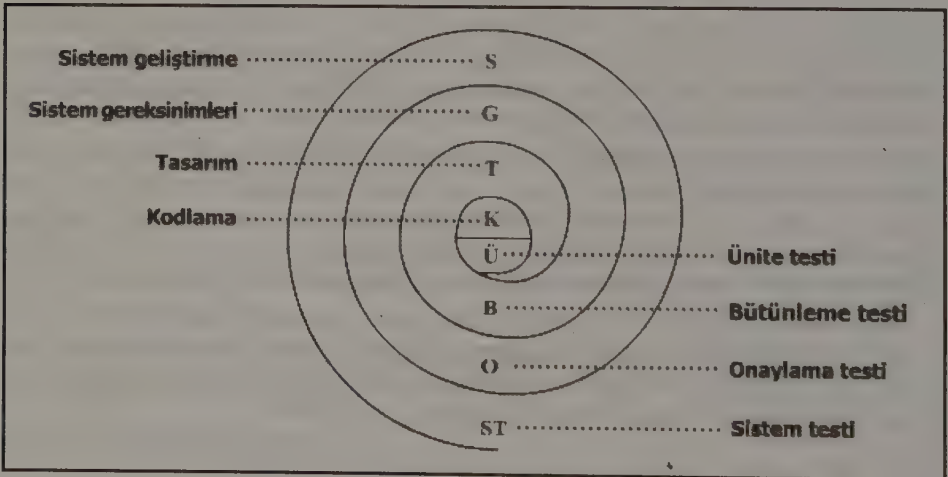
Dayanıklılık sınama, sisteme gereğinden çok terminal bağlamak, girdi ve tanıyıcı yüklemek, bir iletişim hattını devreden çıkarmak vb. Yollarla gerçekleştirilmektedir. Böylece sistemin zayıf ve kuvvetli tarafları belirlenmekte, aksama durumlarında düzeltilme olanakları saptanmaktadır.

Yapısal sınama, yazılım sisteminin iç işletim mantığı sınanmaktadır. Belli alt programlar çağrılarak bunların mantıksal yolları incelenip değerlendirilir.

Sistem üzerinde sırasıyla fonksiyonel, performans ve dayanıklılık testleri uygulanmakta ve bunları tamamlamak üzere de yapısal test gerçekleştirilmektedir. Yapısal testin tasarımı, sistem geliştirildikten sonra ve önceden hazırlanan test planına göre düzenlenmektedir.

14.5.1. Yazılım Sınama Basamakları

Yazılımın sınanması; ünite testi, bütünleşme testi, onaylama testi ve sistem testi basamakları halinde gerçekleştirilmektedir. Böylece sınama işlemi, yazılım geliştirme-nin son basamağı olan kodlamadan geriye doğru yürütülmektedir (Şekil-14.2).



Şekil-14.2. Yazılım sınama basamakları

14.5.1.1. Ünite Testi

Ünite (birim) testi, yazılım tasarımının en küçük birimi olan modül üzerinde uygulanmaktadır. Ayrıntılı tasarım tanımlarına dayanılarak, modül içerisindeki hataları bulmak üzere, önemli kontrol yolları sınanmaktadır. Saydam kutu testi olarak uygulanan bu işlem, çok sayıdaki modül üzerinde, paralel olarak yürütülmektedir.

Ünite testinde; modülün arabirimi, veri yapısı, kontrol yapıları arasındaki ana yollar, hata arama yolları ve modül sınırları sınanmaktadır.

Modül arabirim testinde, verinin giriş ve çıkışının uygunluğu denetlenmektedir. Burada; parametre sayısının argümanlara (işlem yapılan veri maddelerine) eşitliği ve denkliği, modül çağırma argümanlarının niteliklerinin ve OPEN (aç) deyimlerinin doğruluğu, format tanımlarının girdi/çıkı deyimlerine uygunluğu, ara bellek büyüklüğünün kayıt büyüklüğüne denkliği, dosyaların açılıp kapanması işlemleri, çıktı metninin doğruluğu da denetlenmektedir.

Modül veri yapısının sınanmasında, algoritmik işlemlerin her basamağında geçici olarak saklanan verinin bütünlüğünü koruduğu denetlenmektedir. Bu denetim için; kayıt, başlama, değer, değişken adı, veri tipi, kuyruk listedeki boşluk ve taşmalar (*underflow*, *overflow*) ve adresleme hatalarını bulacak bir test program (*test case*) tasarlanmaktadır. Bu arada, modüldeki genel veri denetlenmektedir.

Kontrol yapıları arasındaki anayolların denetiminde, hesaplama, karşılaştırma ve kontrol akışı hataları aranmaktadır. Bu amaçla da test programları kullanılmaktadır. Hesaplama hatalarına; aritmetik işlemlerin öncelik sırasında, tamsayılı bayağı kesir işlemlerinde, doğruluk derecesinde, tanımların simgesel gösterimlerinde rastlanmaktadır. Karşılaştırma ve kontrol akışı, birbirine bağlı bulunmaktadır. Genellikle kontrol akışı değişikliği, karşılaştırmadan sonra yapılmaktadır. Test programı ile farklı veri tiplerinin karşılaştırması, mantıksal işlemler ve sıralamalar, eşitlikler, değişkenlerin karşılaştırılması, döngünün tamamlanması, yinelemelerde çıkış yeri, döngü değişkenleri konularındaki hatalar saptanabilmektedir.

İyi bir yazılımda hata hallerinin kestirilmesi, hata bulma yollarının kurulması, böylece hataların izlenmesi ve giderilmesi beklenmektedir. Hata arama yollarının sınanmasında, hata tanımlarının tutarlılığı, hata bildirilerinin doğruluğu, hatalı haldeki işlemin doğruluğu, hata tanımlarının yeterliği konuları denetlenmektedir.

Modül sınırlarında genellikle; n-elemanlı ve n-boyutlu dizilerin işlenmesinde, bir döngünün i'ninci (i.) yinelenmesinde ya da en büyük ve en küçük değerlerde hatalar oluşmaktadır. Test programları ile veri yapısının, kontrol akışının ve veri değerlerinin işletilmesi sırasında sınır değerler verilerek, bu hatalar bulunabilmektedir.

Ünite testi, kaynak program düzenlenerek gözden geçirilip sözdizimi hataları düzeltildikten sonra uygulanmaktadır. Önce, yazılım tasarımına dayanarak, yukarıda açıklanan hata çeşitlerini bulabilecek test programları hazırlanmaktadır. Bu programlardan beklenen sonuçlar da belirlenmektedir.

14.5.1.2. Bütünleme Testi

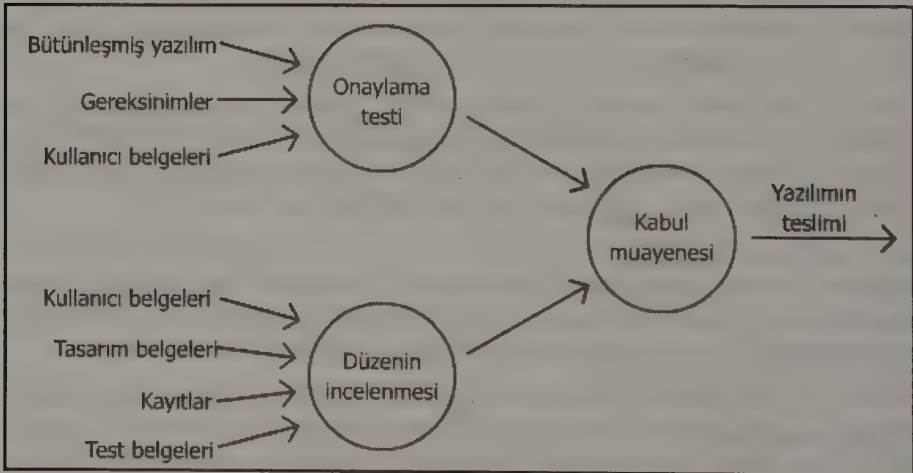
Modüller bağımsız olmayıp, birbirlerine ya da bir donanım ögesine veya işlemciye bağlı olmalıdır. Bu bağlantı, “yazılım arabirimi” (*software interface*) ile sağlanmaktadır. Modüllerin birleştirilmesi sırasında veri kaybı, dikkatsizlik nedeni ile birbirini ters etkileme, alt fonksiyonların birleştirilmesiyle beklenen ana fonksiyonun gerçekleşmemesi, her birinde göze alınabilen hata toleranslarının eklenerek büyümesi, genel veri yapılarının sorun yaratması söz konusudur. Bu hata ve sorunları bulup gidermek için, modüllerin birleştirilerek ana programın oluşturulmasında bütünleme testi uygulanmalıdır.

Bütünleme testi;

- **Bütün olarak (bing bang) sinama:** Modülleri gruplar ya da bütün halinde birleştirildikten sonra gerçekleştirilmektedir. Bu yöntem, karışıklığa yol açmakta ve sakıncalı bulunmaktadır.
- **Arttırmalı sinama:** Modüller teker teker birbirine bağlanmaktadır. Böylece hatalar kolayca ayrılıp düzeltilebilmektedir. Arttırmalı sinama, yukarıdan aşağıya ve aşağıdan yukarıya olarak iki ayrı şekilde uygulanmaktadır.

14.5.1.3. Onaylama Testi

Bütünleme testi sonunda, yazılım bir paket halinde derlenmiş, arabirim hataları bulunmuş ve düzeltilmiş olmaktadır. Bundan sonra, onaylama testi yapılmaktadır. Bu testte, yazılımın müşteri ve kullanıcı beklentilerini gerçekleştirme olanağı denetlenmektedir. Bu amaçla; onaylama testi, düzenlik testi ve kabul muayenesi olarak yürütülmektedir (Şekil-14.3).



Şekil-14.3. Onaylama testi

Onaylama testi, bütünleştirilmiş yazılım ve hazırlanan kullanıcı belgeleri “yazılım gereksinimleri spesifikasyonu” ile karşılaştırılarak gerçekleştirilmektedir. Bunun için, bir dizi kara kutu testleri uygulanmaktadır. Uygulama, daha önce hazırlanmış olan test planına göre yürütülmektedir. Bu aşamada bulunan hata ve eksikler belirlenmektedir. Fakat düzeltme işlemi genellikle, müşteriye danışarak yerine getirilmektedir.

Yazılım düzeninin gözden geçirilmesi (*configuration review*); yazılım düzenini oluşturan bütün elemanların uygun biçimde geliştirildiği, düzenlendiği ve bakım-onarım aşaması için gerekli ayrıntıyı içerdiğini onaylamak amacı ile yapılmaktadır. Bu işleme “kesin hesap” (*audit*) adı verilmektedir.

Kabul muayenesi; müşteri tarafından yazılımın tamamının kullanıcı gözü ile incelenmesidir. Bu inceleme, proje yöneticisi tarafından kullanıcı gözü ile (alfa testi) ve sonra da müşteri tarafından (beta testi) iki aşamada gerçekleştirilmektedir.

14.5.1.4. Sistem Testi

Bilgisayar sistemi, donanım ve yazılım alt sistemlerinden oluşmaktadır. Bu nedenle, yazılım alt sisteminin kendi başına sınanması yeterli değildir; bilgisayar sistemi içerisinde de denetlenmelidir. Bu denetimin, sistem mühendisinin görevi olmasına karşın, yazılım mühendisi de donanım ile yazılım arabirim sorunlarını önceden kestirmek ve karşılamak, sistem testine de katılmak durumundadır.

Sistem testinin amacı; sistemin bütün öğelerinin uygun olarak bir araya getirildiğinin ve her birinin işlevini tam olarak gerçekleştirebildiğinin onaylanmasıdır. Yazılıma dayalı sistemlerde sistem testi; düzeltme testi, güvenlik testi, dayanıklılık testi, performans testi biçimlerinde uygulanmaktadır.

Düzeltilme testi, yazılımı çeşitli yollarla hata yapmaya zorlamak ve bu hataların düzeltilbildiğini göstermektedir.

Güvenlik testi, sistemin zararlı dış müdahalelerden ve bilgi hırsızlığından korunabildiğinin kanıtlanmasıdır.

Dayanıklılık testi, sistemin miktar, frekans ya da hacim bakımından anormal biçimde yüklenmesi hallerindeki dayanıklılığını ölçmek amacı ile düzenlenmektedir.

Performans testi, gerçek zamanlı sistemlerde yazılım işlem süresinin bilgisayara dayalı sistem ile uyumluluğunun sınanmasıdır. Performans sınanması, her test basamağında uygulanmaktadır. Hatta ünite testinde de “saydam kutu” şeklinde yetenek testi yapılabilir. Ancak, sistemin performansı tam olarak sistemin bütünleştirilmesinden sonra anlaşılabilir.

14.5.2. Hata ve Eksikliklerin Giderilmesi

Sınama sonucu saptanan hata ve eksiklerin nedenlerinin bulunup, düzeltilmesi gerekmektedir. Hataları giderme (*debugging*) adı verilen bu işlemde, belirtiler ile nedenlerinin karşılaştırılması, sonra da hataların düzeltilmesi yoluna gidilmektedir.

Nedeni bilinen hatalar düzeltilerek, giderilebilmektedir. Fakat bazı hataların nedeni bulunamamaktadır. Bu duruma aşağıdaki haller yol açmaktadır;

- Belirti ve nedenin, programın değişik yerlerinde olması,
- Belirtinin, düzeltilen başka bir hata yüzünden geçici olarak kaybolması,
- Belirtinin hatadan ileri gelmemesi,
- Belirtinin gözden kaçan bir insan hatasından oluşması,
- Belirtinin işlem sorunu olmaktan çok, zamanlama sorunu olması,
- Gerçek zamanlı sistemlerde girdi koşullarının doğru olarak yinelenmesi güçlüğü,
- Belirtinin kesintili olması

Hata nedeninin doğrudan bulunamaması halinde, bilimsel yöntemler uygulanarak, keşfedilmeye çalışılmaktadır. Burada genellikle; tümdengelim (*deduction*), tümevarım (*induction*) ve geriye dönüş (*backtracking*) yöntemleri uygulanmaktadır. En çok kullanılan geriye dönüş yönteminde, kaynak programda hatanın bulunduğu yerden nedeni buluncaya kadar geriye gidilmektedir.

Sinama yoluyla hatalar bulunup düzeltildiği gibi, yazılımın spesifikasyonlara uygun olarak işlevlerini yerine getirdiği ve performans gereksinimlerini gerçekleştirdiği de kanıtlanmış olmaktadır. Ayrıca, bir bütün halinde yazılımın güvenilirliği ve diğer kalite özellikleri de ortaya konmaktadır. Ancak, sınamalar sonunda yazılım bütün hata ve eksiklerden arındırılmış olmamakta, yine de bulunamamış veya düzeltilememiş hatalar içerebilmektedir.

Sinama sırasında toplanan sonuçlar da ayrıca bilimsel yöntemle değerlendirilmektedir. Değerlendirme sonunda hata oranları bulunmaktadır. Böylece, hataların oluşum olasılığı ve yazılımın güvenilirliği kestirilebilmektedir (yazılım güvenilirlik modeli).

14.5.3. Sinama Yöntemleri

Fonksiyonel, performans ve dayanıklılık testlerine, sistemin dış spesifikasyonlarına ve gereksinimlerine dayandırıldığı için **kara kutu testi** (*black box testing*) adı verilmektedir. Buna karşılık, yapısal denetimde modül düzeyinde programın deyimleri ya da dalları sınanarak iç yapısı incelenmektedir. Bu şekilde uygulanan sinama yöntemine de **saydam kutu testi** (*white box, glass box testing*) denilmektedir.

14.5.3.1. Saydam Kutu Testi

Saydam kutu testi, modül düzeyinde uygulanmakta ve programın iç yapısını denetlenmektedir (ünite testi). Test programı ile yazılımdaki koşul ve döngü durumları çalıştırılarak, mantıksal yollar sınanmaktadır. Programın tasarlandığı ve beklendiği yapıda olup olmadığını sınamak için, değişik noktalarında inceleme yapılmaktadır. Fakat bu yöntemde, programın tamamını sınamak olanaksızdır. Sadece sınırlı sayıdaki çok önemli mantıksal yollar seçilmekte ve bu yolların sınanması ile yetinilmektedir.

Saydam kutu testinde, işlemsel (*procedural*) tasarımın kontrol yapısı kullanılmaktadır. Bu test ile;

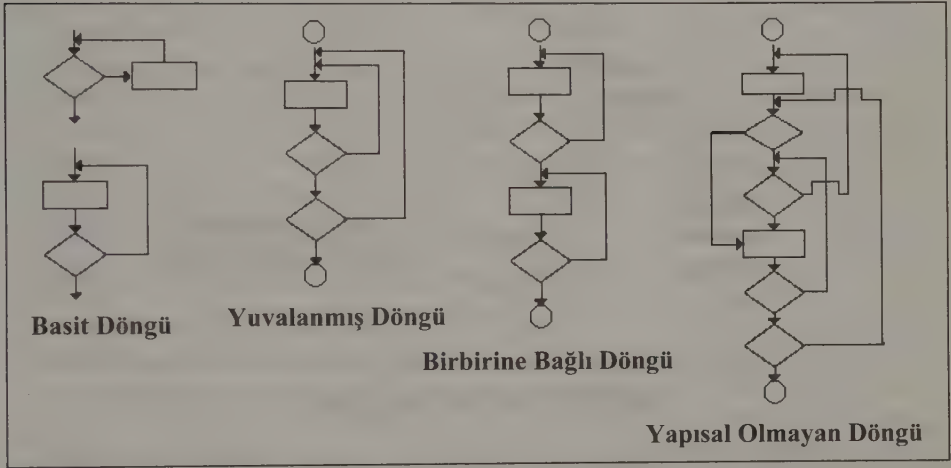
- Bir modüldeki bütün bağımsız yolların en az bir kez çalışacağı garanti edilmekte,
- Bütün mantıksal kararların “doğru” ve “yanlış” durumları denenmiş olmakta,

- Bütün döngülerin kendi içinde ve çevresinde işlerliği sağlanmakta,
- İç veri yapıları denenerek, geçerliliği güvence altına alınmaktadır.

Saydam kutu testinin uygulanmasında, temel yollar testi ve döngü testi teknikleri kullanılmaktadır.

Temel yollar testi, işlemsel tasarımın mantıksal karmaşıklığını ölçmek ve bu ölçüye göre uygulama yolları için bir temel grup oluşturmak esasına dayanmaktadır. Bu grubu denemek için bir test programı düzenlemektedir. Test programları, test sırasında programdaki her deyimini en az bir kez uygulayarak denemektedir.

Döngü testinde, bir döngü içerisinde bir giriş ve bir çıkışlı olarak soyutlanmış bulunan yollar da, birer döngü halinde ayrıca sınanmaktadır. Döngü testinin amacı; döngü içerisindeki başlama hatalarının, indeksleme ve artırma hatalarının, döngüyü sınırlama hatalarının bulunmasıdır. Test sonunda, döngü yapısının geçerliliği onaylanmış olmaktadır. Döngüler; basit, iç içe yuvalanmış, birbirine bağlı ve yapısal olmayan şeklinde olabilmektedir (Şekil-14.4).



Şekil-14.4. Program döngüleri

14.5.3.2. Kara Kutu Testi

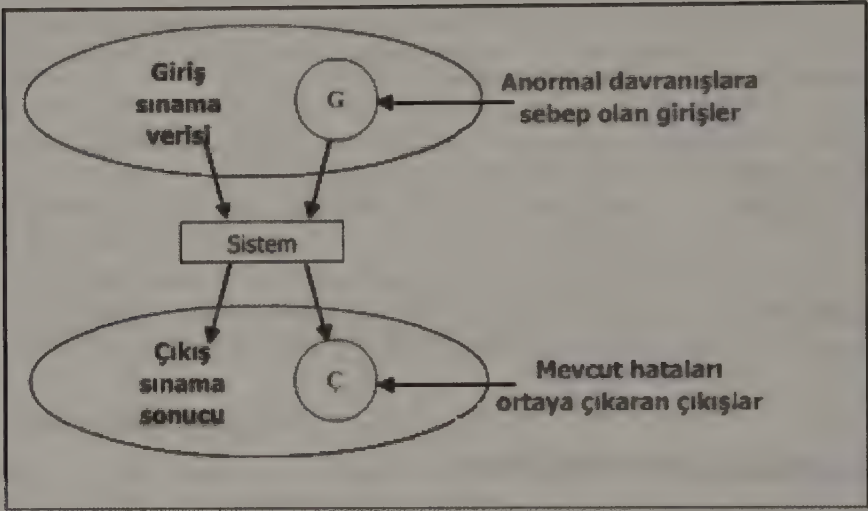
Kara kutu testi; yazılımın bütünlenmesi sırasında uygulanan ve yazılım arabirimi üzerinde yapılan bir sınamadır. Bu sınama ile, yazılım işlevlerinin yerine getirildiği, girdilerin kabul edildiği, çıktıların doğru olarak üretildiği ve dış bilişim (örneğin, veri kütükleri) bütünlüğünün sağlandığı gösterilmektedir. Kara kutu testinde yazılımın mantıksal iç yapısından çok, temel sistem modeli denenmiş olmaktadır. Bu nedenle, kara ve saydam kutu testleri birlikte uygulanarak, yazılım arabiriminin geçerliliği onaylanmakta ve yazılımın iç işlerliğinin doğruluğu kısmen güvence altına alınmaktadır.

Kara kutu testi yöntemleri, yazılımın işlevsel gereksinimlerinin denenmesine yöneliktir. Programın bütün işlevsel gereksinimlerini denemek üzere, bir dizi girdi du-

rumları oluşturulmaktadır. Bu test, saydam kutu testinin alternatifi olmayıp, tamamlayıcı niteliktedir. Böylece, değişik karakterdeki hatalar bulunabilmektedir. Kara kutu testi ile;

- Hatalı ve eksik olan işlevler,
- Arabirim hataları,
- Veri yapılarında ve veri tabanı erişimindeki hatalar,
- Performans hataları,
- Başlama ve bitirme hataları

bulunabilmektedir (Şekil-14.5).



Şekil-14.5. Kara kutu testi modeli

Saydam kutu testi sinama aşamasının başında ve program parçaları (modüller) için uygulandığı halde, kara kutu testi son basamakta ve daha geniş ölçüde, programın bütünü için kullanılmaktadır. Ayrıca; saydam kutu testi program kontrol yapısına yönelik olmasına karşın, kara kutu testinde yazılımın bilişim alanına ağırlık verilmiştir.

Kara kutu testi tekniklerinde de bir dizi test programları (*test cases*) oluşturulmaktadır.

Kara kutu testi ile sistemin dış spesifikasyonları ve gereksinimleri sınanmaktadır. Burada, sistem düzeyinde elde edilen sonuçların kanıtlanması esas alınmaktadır. Sistemin nasıl geliştirildiği ve programdaki bütün deyimlerin işlerliği üzerinde durulmamaktadır. Başlıca yöntemler; eşdeğerli bölümlleme, sınır değer analizi, neden-sonuç grafi çizimi, veri onaylama testi olarak sayılmaktadır.

Eşdeğerli bölümlendirme (*equivalence partitioning*) yönteminde; programın girdi alanı, test programları oluşturulabilecek veri sınıflarına bölünmektedir. Böylece her test programı belirli sınıftaki hataları ortaya çıkarmakta ve daha az sayıda test programı ile yetinilmektedir.

Sınır değer analizi, test verisinin; girdi alanı (ya da çıktı serisi) sınıflarının, veri yapılarının, altprogram parametrelerinin vb. sınırlarından veya uç değerlerinden seçilmesi tekniğidir. Genellikle en büyük-en küçük veya en az beklenen değerler ve parametreler seçilmektedir. Bu yöntem, dayanıklılık testi (stress testing) adı da verilmektedir.

Neden-sonuç graf çizimi, mantıksal durumları ve bu durumlarla ilgili işlemlerin özlü olarak gösterimini sağlamaktadır. Bu teknik;

- Modüle ait girdi koşullarının ve sonuçların (işlemlerin) liste haline getirilmesi ve her biri için tanıtıcılar atanması,
- Bir neden-sonuç grafının geliştirilmesi,
- Grafın bir karar tablosuna dönüştürülmesi,
- Karar tablosu kurallarının test programlarına dönüştürülmesi

basamakları halinde uygulanmaktadır.

Veri onaylama testi, etkileşimli sistemlerde (*interactive systems*) kullanıcı tarafından türetilen özel komutların doğruluğunun onaylanması için uygulanmaktadır. Kullanıcı tarafından türetilen özel komutlar, sistemin buna uygun yanıt vereceği birer söz dizimidir. Veri onaylama testi ile;

- Yanlış söz dizimli komutlar,
- Sırasız ya da zamansız komut girişleri,
- Çok sayıda niteleyici veri gerektirmesi,
- Komut girişi halinde sistemin kesilmesi

durum ve olayları denetlenmektedir. Ayrıca, komutların sınırlandırılması ve ayırımı bakımından da; eksik ya da yanlış işaret kullanma halleri ortaya çıkarılmaktadır. Veri onaylama testi, diğer kara kutu testlerinin bıraktığı boşlukları doldurmak amacı ile uygulanmakta ve insan-makine arabirimlerini denetlemekte kullanılmaktadır.

14.6. Özet

Kaliteli yazılım, doğruluk, güvenilirlik, verimlilik, güvenlik, kullanılabilirlik, hata bulma kolaylığı, esneklik, sınama kolaylığı, taşınabilirlik, tekrar kullanılabilirlik ve bağlanabilirlik özelliklerine sahip olmalıdır.

Yazılım kalitesini sağlamak için hata ve eksiklerin anında bulunması ve düzeltilmesi büyük önem taşımaktadır. Çünkü bir basamakta yapılan hata, sonraki basamakları da etkilemektedir. Bu nedenle bilgi sistemi geliştirme sürecinde kalite kontrolü gerçekleştirilmektedir. Ayrıca, yazılım ürününün alıcıya teslimi sırasında da “kabul muayenesi” yapılmaktadır.

Sınama bir programdaki hataları bulmak amacı ile yapılan işlemlerdir. Sınama, yazılımın fonksiyonel, performans, dayanıklılık ve yapısal açılarından yeterliliğini denetlemek için gerçekleştirilir.

14.7. Sorular

- 14.7.1) Yazılım kalitesini oluşturan faktörler nelerdir?
- 14.7.2) Yazılım kalitesini sağlamak için gerçekleştirilen aktiviteler nelerdir, kısaca açıklayınız.
- 14.7.3) Yazılım test sürecinde bulunan aşamaları yazılım geliştirme süreci ile ilişkili olarak açıklayınız.
- 14.7.4) Sistem testinin amacı nedir, hangi test tipleri ile gerçekleştirilir?
- 14.7.5) Sınır değer analizi hangi tip test yöntemidir, nasıl gerçekleştirilir?
- 14.7.6) Kara kutu testi ile hangi hatalar bulunabilir?
- 14.7.7) Kara kutu testi olarak kullanılan yöntemleri açıklayınız.
- 14.7.8) Veri onaylama testi ile hangi durum ve olaylar denetlenebilir?
- 14.7.9) Test edilebilir yazılımın özellikleri nelerdir?
- 14.7.10) Uzun bir metin okuyarak içindeki kelimeleri ayıran ve her bir kelimenin sayısını hesaplayan bir yazılımın modüllerini tek seviye olarak gösteriniz. Tüm modüllerinin birim testinin yapıldığını varsayarak, bütünleme test stratejisinin adımlarını açıklayınız.

BÖLÜM 15

VAR OLAN SİSTEMDEN YENİ SİSTEME GEÇİŞ

Oluşturulan yeni sistemin başarıya ulaşmasında hem sistemin hayata geçirilmesi (kuruluşu) hem de kullanıma geçiriliş yöntemi önemli rol oynamaktadırlar.

Ön tasarımda belirlenen sistem donanımlarının kurulmasıyla ağ yapısının oluşturulması, bir sistemdeki kayıtların yeni sisteme aktarımı, eğitim ve geçiş tipinin belirlenmesi mevcut sistemden yeni sisteme geçiş planı çerçevesinde yapılmalıdır.

15.1. Kullanıma Hazırlık İşlemleri

15.1.1. Yeni Sistemin Eğitimi

Sistemin başarıya ulaşmasında kullanıcı eğitiminin payı da göz ardı edilemez. Çünkü sistem ne kadar fonksiyonel ve ne kadar verimli çalışıyor olsa da onu doğru kullanan olmadığı takdirde hiçbir anlam ifade etmez. Öncelikle eğitim alacaklar gruplara ayrılır ve her grup için belirlenen amaca göre eğitim programı düzenlenir. Bu gruplar genelde sistemle ilgilenecekler, uç kullanıcılar ve üst yönetimdekiler olmak üzere üç gruba ayrılır.

Sistemle ilgilenecekler işletim sistemiyle ve program modüllerinin yapısıyla ilgili daha fazla bilgi verilmelidir. Bu grup kendi içinde işletim sisteminden sorumlu ve gerektiğinde yazılıma müdahale edecekler olarak ikiye ayrılabilir.

İkinci grupta yer uç kullanıcılara ise sistemde gerçekleştirebilecekleri işlemler, bu işlemler için hangi modülü kullanmaları gerektiği ve bu modüllere menüler üzerinden nasıl erişebileceklerine dair eğitim verilmelidir.

Son gruptaki üst düzey yöneticiler genelde sistemi doğrudan kullanmadıkları için onlar için hazırlana eğitim programında sistem aracılığıyla hangi bilgileri alabilecekleri sistemin kısıtlarıyla beraber anlatılmalıdır.

15.2. Yeni Sisteme Mevcut Verilerin Aktarılması

Yeni sistemin altyapısı oluşturulduktan sonra kullanıma alınmadan önce tamamlanması gereken diğer adım ise sistem üstünde çalışacağı veritabanına kayıtların aktarılmasıdır (*database migration*). Günümüzde sadece bir sistemden başka bir sisteme aktarımı sağlayan araçların hatta şirketlerin olması bu konunun önemi daha da vurgulamaktadır.

İlk yükleme (*initial loading*) adı da verilen bu işlem sırasındaki zorluklar ve zaman kaybı; yüklenecek verinin kaynağına ve formatına, kayıt sayısına, oluşturulacak in-

deks sayısına ve kullanılacak veritabanı yönetim sistemine göre değişmektedir. Örneğin, kaynak verinin bulunduğu araçların yeni sisteme aktarmaya uygunluğu ve o kuma olanakları, formatların ve kodlamanın farklılığı çok etkili olmaktadır.

Kayıt ve indeks sayılarının çok olması veritabanı aktarımında önemli olan yükleme süresini etkilemektedir. Özellikle ileride anlatılan bir sistemden bir sisteme doğru geçiş yönteminde veritabanı aktarımı süresi çok kritiktir.

Veritabanının ilk yükleme aşamasında, dönüştürme (*converting*) ve bütünleştirme (*integration*) işlemleri gerçekleştirilir. Dönüştürme, bir veri gösteriminin, kayıt içeriğini kaybetmeksizin, başka bir veri gösterimine çevrilmesidir. Bütünleştirme, komutların paylaşımı ve bileşiminin bir programdan diğerine akışının sağlanmasıdır.

Veritabanı ortamında en çok zaman alıcı işlem, eldeki veri kütüklerinin verisini veritabanına dönüştürmektir. Çoğu ortamda, dönüştürülecek veri çok sayıda kütüğe dağıtılmış olarak ve farklı formatlar halinde bulunmaktadır. Bu veri içerisinde de önemli ölçüde gereksiz ve yinelemeli bilgi olabilmektedir. Dağınık veriyi veritabanı merkezinde toplamak için, çok sayıda uygulama programı yazılmasını gerektirmektedir.

Dönüştürmenin “nereden-nereye” yapılacağının kararlaştırılması, uygulama kodlarının yazılması, test edilmesi ve gözden geçirilmesi, çok sayıdaki diğer programlarla ayarlanması zaman alıcı işlemlerden olduğu için zaman planının yapılmasını gerektirir. Nitekim dönüştürme ve bütünleştirme işlemi sırasında, önce verinin geçerliliğinin sağlanması (onaylanması) ve sonra yerleştirilmesi gerekmektedir. Veri dönüştürmede yapılacak hatalar, zaman içinde daha büyük sorunlara neden olmaktadır. Dönüştürme işlemi hemen bitirilememekte, aylarca sürmektedir. Bu arada, bir yandan kurumun işlevleri ve görevlilerin işleri de sürdürülecektir.

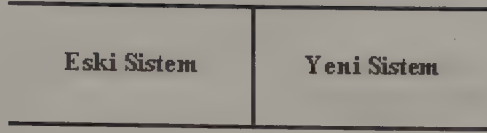
Genel olarak, bir dönüştürme ve bütünleştirme projesi, iki bölüm halinde yürütülmektedir. Önce veri kütükleri sonra uygulama programı dönüştürülmektedir. Dönüştürme sırasında günlük işlemler durdurulamayacağı için, bu işlemler ile ilgili veri bir süre eski veri kütüklerinde de tutulacaktır. Bir yandan da yeni veritabanı sistemi çalıştırılmaya başlanacaktır. Bu durum, bilgisayar sisteminin işletimi ve bakımındaki görevlilerin yükünü bir süre için iki katına çıkarmış olacaktır.

15.2. Yeni Sisteme Geçiş Yöntemleri

Kurum içindeki eğitim tamamlandıktan sonra izlenen aşamada ise var olan sistemden yeni sisteme nasıl geçileceğine karar verilir. Beş farklı stratejinin olduğu bu aşamada her yöntemin yararları ve zararları olduğunu unutmadan kurum ve sistemin yapısını göz önünde tutarak en uygununu seçmek sistem analistinin görevidir.

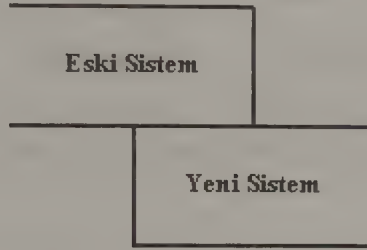
Var olan sistemden yeni sisteme geçiş yöntemleri:

Doğrudan Geçiş : Belirlenen bir tarihten itibaren eski sistem tamamen bırakılarak yeni sistem kullanılmaya başlanır. Oldukça riskli olan bu yöntemde ön hazırlıkların; eğitimin ve veritabanı aktarımının tamamlanmış olması gerekir. Özellikle eğitim ve veritabanı dönüşümünün çok iyi hazırlanmış olması gerekir (Şekil-15.1).



Şekil-15.1. Yeni sisteme doğrudan geçiş

Paralel Geçiş: Adından anlaşılacağı üzere eski sistemle yeni sistem belirlenen süre boyunca birlikte çalışır (Şekil-15.2). Ancak her iki sistemde de verilerin birbirini tutması sağlamak için kullanıcıların gerçekleştirdikleri işlemi hem yeni hem de eski sistemde yapmaları gerekmektedir ki bu da kullanıcının daha fazla vaktini alacaktır. Bu yöntemde eğitim geçiş süreci sırasında da verilebilirken veritabanı aktarımı yeni sistem kullanılmaya başlanmadan tamamlanmalıdır.



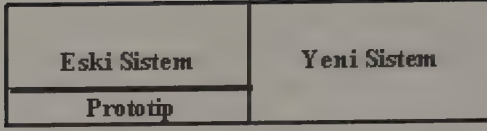
Şekil-15.2. Yeni sisteme paralel geçiş

Dereceli Geçiş: Yeni sistem belli işlevleriyle kullanılmaya başlanır. Zaman içinde yeni sistemin diğer işlevleri de aşamalı olarak aktarılacak şekilde sistem bütünleştirilir. Yeni sistem tüm fonksiyonlarıyla beraber kullanılmaya başlandığı anadan itibaren ise eski sistem kullanımdan kaldırılır (Şekil-12.3). Hem eğitim hem de veri aktarımı daha ilk işlev kullanıma geçmeden tamamlanması tavsiye edilir. Ancak birbirinden tamamıyla bağımsız olan işlevlerde veri aktarımı da dereceli olarak yapılabilir.



Şekil-15.3. Yeni sisteme dereceli geçiş

Prototip Geçiş: Dereceli geçişe göre daha organize bir yapısı vardır. Sistemin bir prototipi oluşturulup kullanıma geçirilir. Prototipin çalışmasından elde edilen sonuçlara göre sistem üzerinde gerekli düzenlemeler yapıldıktan sonra tamamen yeni sisteme geçilir (Şekil-15.4). Prototip yapısı üzerinde eğitim verilebilir. Veritabanı aktarımı prototip öncesinde tamamlanmalıdır. Böylece prototip sistemde veritabanı aktarımı da test edilmiş olur. Prototip yapıdan yeni sisteme geçişte gerçek veritabanı aktarımı gerçekleştirilir.



Şekil-15.4. Yeni sisteme prototip ile geçiş

Dağıtılmış Geçiş: Dağıtılmış geçişte kurumun her bölümünde aşamalı olarak tek tek geçiş söz konusudur. Sistemin belirli periyotlarla genel olarak değerlendirilmesi ve bakımının yapılması gerekir (Şekil-15.5). Veritabanı aktarımı bakımından dereceli geçiş yöntemiyle benzerlik göstermektedir. Ancak eğitim daha geniş süre içinde örneğin her bölüme ayrı ayrı verilebilir.



Şekil-15.5. Yeni sisteme dağıtık geçiş

15.3. Özet

Sistemin başarılı olmasında geçiş yapısının kurgusu ve eğitimi de en az sistemin fizibilite çalışmasından test edilmesine kadar olan tüm aşamalar kadar önemlidir. Sistemin kullanılacağı çalışma ortamına, etkileyeceği alana ve sistemin yapısına göre beş geçiş tipinden (doğrudan, paralel, dereceli, prototip ve dağıtılmış geçiş tipi) birine karar verilir.

Geçiş tipine ve yapılan kurguya göre eğitim planlaması yapılır. Eğitimin, yeni sistem tamamıyla kullanıma alınmadan bitmesi önerilmektedir. Ayrıca eğitim, sistemden etkilenen/etkileyen gruplara göre (örneğin üst düzey yönetici eğitimi, kullanıcı eğitimi vb. şeklinde) içeriği düzenlenerek ayrı ayrı verilmelidir.

Yeni sisteme geçilmeden önce yapılan hazırlıklardan biri de genelde uzun bir zaman dilimine ihtiyaç duyulan veritabanı aktarımıdır (*database migration*). Veritabanı aktarımının nereden nereye, nasıl ve ne zaman yapılacağı konularında karar verilir. Veritabanının ilk yükleme aşamasında dönüştürme (*converting*) ve bütünleştirme (*integration*) işlemleri gerçekleştirilir. Dönüştürme bir veri gösteriminin, kayıt içeriğini kaybetmeksizin, başka bir veri gösterimine çevrilmesidir. Bütünleştirme ise komutların bir programdan diğerine akışının sağlanmasıdır.

15.4. Sorular

- 15.4.1) Bilgi sisteminin kullanıma hazırlık sürecinde hangi eğitimler verilmelidir?
- 15.4.2) Var olan sistemden yeni sisteme geçiş yolları nelerdir?
- 15.4.3) Paralel geçiş hangi durumlarda kullanılmalıdır?
- 15.4.4) Sistem kullanıcısının eğitiminde hangi yöntemlerden ve araçlardan yararlanıldığını araştırınız.
- 15.4.5) Havaalanında uçakların iniş-kalkışlarını kontrol eden bir sistem yenisiyle değiştirilecektir. Bu geçişin kurgusunu nedenlerini açıklayarak planlayınız.

KISIM V

İŞLETMELERDE BİLGİ SİSTEMİ UYGULAMALARI

Son kısımda bilgi sistemlerinin günlük hayatta en fazla kullanım alanı olan işletmelerdeki önemi ve uygulama türleri ele alınmıştır.

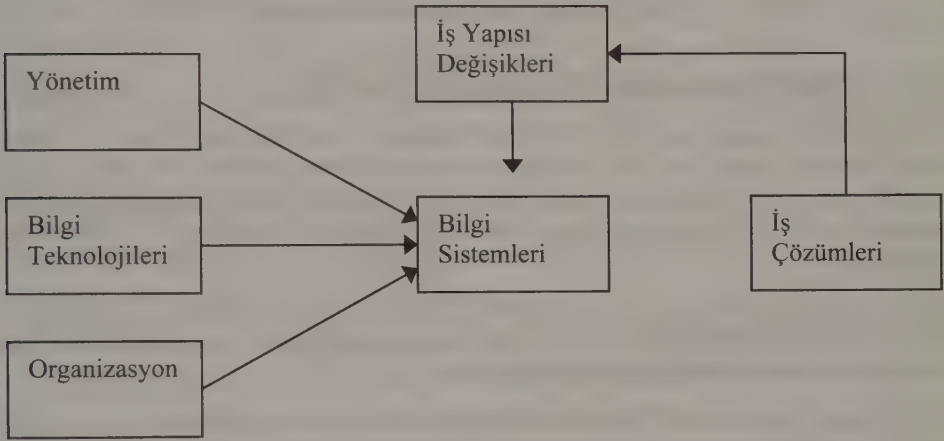
Bölüm 16: Organizasyonlar ve bilgi sistemleri arasındaki etkileşim, işletme bilgi sistemlerinin hazırlanma ve kurulum aşamaları anlatılmıştır.

Bölüm 17: Kurumsal kaynak planlaması, tedarik zinciri yönetimi ve müşteri ilişkileri yönetimi olmak üzere temel uygulama yazılımları genel özellikleriyle örneklenmiştir.

BÖLÜM 16

İŞLETMELERDE BİLGİ SİSTEMLERİNİN YERİ

Bilgi sistemleri, bir veya daha fazla işletmecilik iş akışlarında bilgi gereksinimlerini sağlamak amacıyla birarada çalışan elemanlardan oluşmaktadır. Bilgi sistemlerinin etkin olarak kullanımı organizasyon, yönetim ve bilgi teknolojilerinin anlaşılmasını gerektirir (Şekil-16.1). Sistem analizi ve tasarımı sürecinde kullanıcıların gereksinimlerini karşılamak üzere tüm bu elemanlar en iyi düzeyde birleştirilmektedir.



Şekil-16.1. Bilgi sistemleri ile organizasyonu yeniden düzenleme

16.1. Organizasyonlar ve Bilgi Sistemleri Arasındaki İlişki

Bilgi sistemleri ve organizasyonlar arasındaki ilişki karmaşıktır ve aşağıdaki sorular ile açıklanmaya çalışılır:

- Bilgi sistemleri, organizasyonlardaki düzey sayısını azaltarak yataylaştırmalı mı?
- Bilgi sistemleri, organizasyonlardaki orta düzey yönetici sayısını azaltacak mı?
- Organizasyonlar, iş süreçlerini yeniden oluşturmak için bilgi sistemlerini kullanabilirler mi?
- Web uygulamaları, organizasyonların yerine geçebilir mi?

Organizasyonların tüm işletme planlaması göz önünde bulundurularak, bilgi sistemlerinin planlaması geliştirilmelidir.

Bilgi sistemleri ve organizasyonlar arasında ortam faktörlerinden etkilenen iki yönlü ilişki vardır (Şekil-16.2).



Şekil-16.2. Organizasyonlar ve bilgi sistemleri arasındaki ilişki

16.1.1. Bilgi Sistemlerinin Organizasyonlara Etkisi

Bilgi sistemleri, kurumsal veri ve bilgi kullanımında verimliliği arttırmakta ve planlama, kontrol, karar alma gibi yönetim fonksiyonlarında yöneticilere yardımcı olmaktadır. Ayrıca bilgi sistemleri, yeni bilim ve teknolojinin, kararlara modeller yoluyla yaklaşma anlayışının yerleşmesinde yöneticinin eğitimine katkıda bulunmaktadır.

Bilgi sistemi tasarımı kurumun amaçlarının yeniden gözden geçirilmesi ve belirlenmesini gerektirir. Sistem, organizasyonun amaçlarına, sorunlarına, yapısına ve yöneticilerinin niteliklerine göre düzenlenmelidir.

Bilgi sistemi tasarımında, yönetimin aldığı kararların çözülmesi ve karar süreçlerinin ortaya konması, hem yeni sistemin başarılı tasarımını sağlar, hem de yöneticinin yönetim etkinliğini artırır.

Kurumlarda gerçekleştirilen sistem analizi ve tasarımı uygulamalarında, ön inceleme aşamasındaki görüşmelerde yöneticilerin kararları nasıl aldıkları belirsiz iken, sistem kuruluşundan sonraki görüşmelerde kendi karar alma süreçlerini daha iyi tanımladıkları gözlenmiştir.

16.1.2. Organizasyonlar Bilgi Sistemlerini Nasıl Etkiler

Organizasyonlar, yöneticiler ve çalışanlarının verdiği kararlar nedeniyle bilgi sistemlerini etkilerler. Yöneticiler sistemlerin nasıl tasarlanacağı hakkında karar verirler ve bilgi teknolojilerini kullanırlar. Yöneticiler bilgi sistemlerini kimin oluşturacağına ve işleteceğine karar verirler.

Organizasyonların bilgi sistemlerini etkilemesi konusunda dört önemli soru vardır:

- Organizasyonlar bilgi sistemlerini nasıl kullanırlar?
- Bilgi sistemlerinin organizasyondaki rolü nasıl değiştirilir?

- Bilgi sistemlerini kim işletir?
- Organizasyonlar bilgi sistemlerini nasıl kabul ederler?

16.2. Bilgi Sistemleri ile Desteklenen Kurumsal İşlevler

Kurumlar, satış, pazarlama, üretim, finans, muhasebe ve insan kaynakları gibi farklı özellik ve uzmanlık alanlarında yapılandırılmış bölümlerden oluşur (Tablo-16.1)

Tablo-16.1. Temel kurumsal işlemler

İşlev	Amaç
Satış ve Pazarlama	Kurum hizmet ve ürünlerinin satışı
Üretim	Ürün ve hizmet üretimi
Finans	Kurumun finans değerlerini yönetir
Muhasebe	Kurumun finansal kayıtlarının bakımını gerçekleştirir
İnsan Kaynakları	Kurumun emek gücünü geliştirme ve çalışan kayıtlarının bakımını yapma

Bilgi teknolojileri, işletmecilik süreçlerini;

- Bir süreçteki insan gücünü azaltmak veya yerini almak,
- Bir süreç hakkında ayrıntılı bilgi verme,
- Bir sürecin girdi ve çıktılarının durumlarını ayrıntılı olarak izleyebilmek,
- Bilgiyi hızlı ve kolaylıkla coğrafi olarak uzaktaki yerlere taşıyabilmek,
- Yapılandırılmamış süreçleri rutin işlemlere dönüştürmek

yoluyla geliştirmektedirler.

16.2.1. Bilgi Sistemlerinin Stratejik Etkisi

Her kurumun, temel değerleri, varsayımları ve üyelerinin kabul ettiği iş gerçekleştirme yollarından oluşan bir kurum kültürü vardır. Yöneticiler; işleri koordine etmek, uygun kararlar almak üzere insan ve finansal kaynakları doğru kullanarak kurumsal stratejileri belirlemektedirler.

Bilgi sistemlerinin organizasyonlarda;

- maliyetin düşmesi,
- ürün ve serviste farklılaşma,
- yaratıcılık,
- büyümeyi sağlama ve
- kalite ve etkinliği artırma

gibi stratejik görevleri vardır. Bilgi teknolojileri, bu görevleri gerçekleştirerek, zamansal, coğrafi, mali ve yapısal engelleri ortadan kaldırmaktadır.

Bilgi sistemlerinin organizasyonlardaki işlevsel ve stratejik görevleri yerine getirmesi amacıyla, geliştirilmeleri sürecinde;

- Bilgisayar bilimi,
- Yöneylem araştırması,
- Yönetim bilimi,
- Sosyoloji ve
- Politika bilimi

gibi birçok disiplinin desteği alınmaktadır. İletişim teknolojileri de kurumlarda pazardaki değişikliklere cevap verme yeteneği kazandıracak esnekliğin artmasına yol açarlar.

16.3. İşletmelerde Uygulama Yazılımlarının Elde edilmesi ve Kullanımı

Bu bölümde,

- İşletmelerde uygulama yazılımlarının sağlanması için en iyi yol nedir?
- Kurumlar yazılımı elde ettikten sonra hedeflerine en uygun kullanımı nasıl sağlarlar?

sorularına cevap aranacaktır.

16.3.1. Uygulama Yazılımlarının Elde Edilmesi

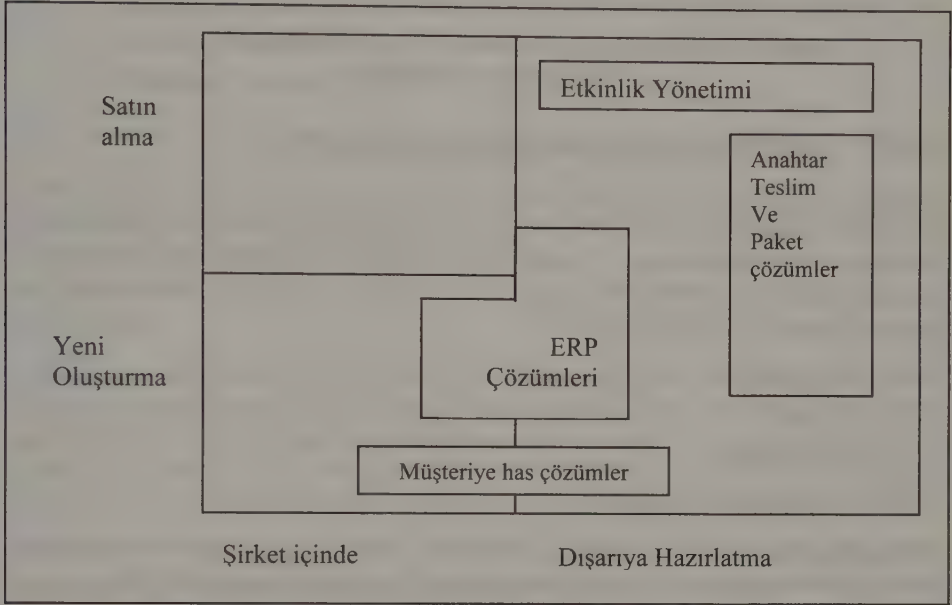
Bilgi sistemleri endüstrisi geliştikçe, yaygın kullanılan ve standart işlemleri gerçekleştiren paket yazılımlar, kullanıcı olan işletmeler dışında yazılım firmaları tarafından üretilmektedir. Uygulama oldukça standart ise, kurum gerekli çözümü sağlayacak yazılımı sağlayabilir. Böylece aynı amaçlı uygulama yazılımlarının farklı kurumlarda hazırlanması nedeniyle emek ve mali kaynak harcanmamış olmaktadır. Eğer hazır yazılımlar gereken işlevi karşılayamayacak ise müşteriye özel yazılım geliştirme yoluna gidilmektedir.

Uygulama yazılım paketinin satın alınması ile satın alınan sistem anında kullanılabilir hale gelmekte, yararları kısa sürede elde edilmekte ve maliyet, tamamlama süresinin aşılması gibi riskler azalmaktadır. Satıcılar, aynı problem üzerinde birçok firma ile çalıştıkları için, genellikle, organizasyon içerisindeki çalışanlardan daha iyi çözümler sağlayabilmektedir.

Şekil-16.3 yazılım sisteminin gerçekleştirilmesi için alternatifleri göstermektedir, tüm sistem satın alınabilir yada tüm sistem yeniden oluşturulabilir. Fakat yeni kullanılacak sistem satın alınsa da, varolan sistemler ile arabirimlerinin oluşturulması gerekir. Burada yatay eksen, bilgi sisteminin şirket içinde üretimi (*inhouse*) veya

dışarda bir kuruma hazırlatılmasını (*outsource*), düşey eksen ise, satın alma yada yeni oluşturmaya göstermektedir.

Etkinlik yönetimi (*Facilities management*), kurumun tüm bilgi sistemi ve bilgi teknolojileri hizmetlerinin bir dış satıcı firmaya ihale edilmesidir. Tüm bilgi sistemi işlemlerinin dış firmaya verilmesi uzun dönemli ve stratejik bir karardır ve tek bir proje için değil, tüm kurum işlemlerine uygulanır.



Şekil-16.3. Uygulama yazılımlarının elde edilme alternatifleri

16.3.2. Uygulama Yazılım Paketlerinin Kuruluşu ve Desteklenmesi

Analiz aşamasında sistemin kullanıcı gereksinimleri tanımlanır. Bu gereksinimlerin tümünü karşılayan yazılım paketleri seçilir. Tasarım aşamasında, yazılım paketinin kullanımı için kullanıcıların nasıl eğitileceği ve varolan sistemden nasıl bir değişim oluşacağı konusunda incelemeler yapılmaktadır.

Yazılım yaşam döngüsü adımlarında yazılım paketlerinin kullanımı çerçevesinde;

- Müşteriye özel hale getirme (*customization*),
- Bütünleştirme (*integration*),
- Güncelleme (*Upgrading packages*)

temel kavramları önem kazanmaktadır (Şekil-16.4).

Planlama	Analiz	Tasarım	Gerçekleştirme	Destek
	Müşteriye Özelleştirme			
	Bütünleştirme			
	Güncelleştirme			

Şekil-16.4. Yazılım geliştirme yaşam döngüsünde yazılım paketi kullanımı temel kavramları

16.3.2.1. Yazılım Paketlerini Müşteriye Özelleştirme

Yazılım paketleri, endüstrideki en yaygın uygulamaların tüm özellik ve seçeneklerinin dikkate alınarak, müşteriye özelleştirme işlemine gereksinim olmayacak şekilde tasarlanması ile hazırlanmıştır. Ancak yine de kullanıcılar işlevsel gereksinimler ya da kişisel-kurumsal tercihlerinin bilgi sistemlerinde sağlanmasını istemektedir. Bu amaçla müşteriye özel hale getirme,

- Konfigürasyon
- Değişiklik
- Zenginleştirme

şeklinde gerçekleştirilmektedir.

Konfigürasyon, yazılım modüllerini bir sistem halinde düzenleme veya yazılımdaki parametrelerin değerlerini yeniden tanımlama gibi işlemleri kapsamaktadır.

Değişiklik (Modifikasyon), işlemlerdeki değişikliği sağlamak üzere program kodunu değiştirmektir.

Zenginleştirme ise yazılıma yeni işlevler eklemek için yeni program kodu veya modülleri yazmaktır.

16.3.2.2. Yazılım Paketlerini Bütünleştirme

Uygulama yazılımı sadece bir bölümün ihtiyaçları doğrultusunda değil, diğer bölümlere ve tüm organizasyona hizmet vermek üzere kullanılırsa yararlı olmaktadır. Yazılım bütünlüğü ayrı ayrı üretilen yazılımlar veya alt sistemleri birleştirme ve bölümler arasındaki etkileşim problemlerini çözme işlemidir. Bilgi teknolojilerindeki son ilerlemeler ile işletmelerin içindeki farklı bölümler, dış müşteriler ve sağlayıcılar arasında yazılım bütünlüğü sağlanmaktadır.

İnsan kaynakları, finans, pazarlama, üretim gibi önemli birimlerde gerekli veriye standart formatta ve gerçek zamanlı olarak erişebilmek için bu birimlerdeki uygulamaları bütünleştirmek, organizasyonların performansını arttıracaktır.

Kurumsal uygulama bütünlüğü denilen yazılım paketlerini bütünleştirmeye;

- Teknik kaynakların eksikliği
- Organizasyonda eşfikir oluşturma zorluğu
- Teknoloji geliştirmedeki zorluklar

engel yaratmaktadır.

İşletmelerdeki bu yazılım bütünlüğü, kurumsal kaynak planlaması ile sağlanmaktadır.

16.3.2.3. Yazılım Paketlerini Güncelleştirme

Yazılım satıcıları, uygulama yazılımlarının kullanımını kolaylaştırmak ya da yeni işlevler eklemek amacıyla düzeltme ve yükseltme yapmaktadırlar.

Yazılım satıcısı firmalar, ürün güncelleştirme gereksinimlerini kullanıcı anketlerinin geri beslemelerinden almaktadırlar.

16.4. Özet

İşletmelerde kullanılan bilgi sistemleri ile işletmenin yapısı arasında karşılıklı bir etkileşim söz konusudur. Örneğin bilgi sisteminin altyapısının dayandığı teknolojilerden birinin gelişmesiyle mevcut iş kurallarının değişmesine veya yeni iş kurallarının alınmasına neden olurken işletmede oluşan yeni ihtiyaçlar kullanılan bilgi sisteminin kısmen veya tamamen değişmesine neden olabilmektedir.

İşletmenin tüm fonksiyonel alanlarında (satış ve pazarlama, üretim, finans, muhasebe, insan kaynakları vb.) farklı amaçlara hizmet eden çeşitli bilgi sistemi uygulamaları kullanılmaktadır. Ancak aynı işletmede bu uygulamaların birbiriyle entegrasyonu ve amaçlara uygunluğu önemlidir. Bunun için uygulamalarda müşteriye özel hale getirme (*customization*), bütünleştirme (*integration*) ve güncelleme (*Upgrading packages*) işlemleri gerçekleştirilir.

16.5. Sorular

- 16.5.1) Bilgi sistemleri ile işletmelerin ortak etkileşim alanlarını açıklayınız.
- 16.5.2) Bilgi sistemlerinin kuruluşu ve desteklenmesi açısından bakıldığında şirket içinde mi yoksa dışında mı yapılacağı kararı hangi kriterlere göre verilir?
- 16.5.3) Bir işletmenin pazarlama bölümü ve finans bölümünde kullanılan bilgi sistemlerin birbiriyle entegre olmaması durumunda yaşanabilecek sorunlar nelerdir?

[The main body of the page contains extremely faint, illegible text, likely bleed-through from the reverse side of the paper. It appears to be several paragraphs of text.]

BÖLÜM 17

İŞLETMELERDE BİLGİ SİSTEMİ UYGULAMALARI

Şirketlerin bilgi sistemleri genellikle çok karmaşıktır. Bu bölümde, uygulama yazılım paketleriyle çalışmada temel esasları incelemek amacıyla, kurumsal kaynak planlaması (*Enterprise Resource Planning*, ERP), tedarik zinciri yönetimi (*Supply Chain Management*, SCM) ve Müşteri ilişkileri yönetimi (*Customer Relationship Management*, CRM) uygulamaları üzerinde durulmaktadır.

17.1. Kurumsal Kaynak Planlaması

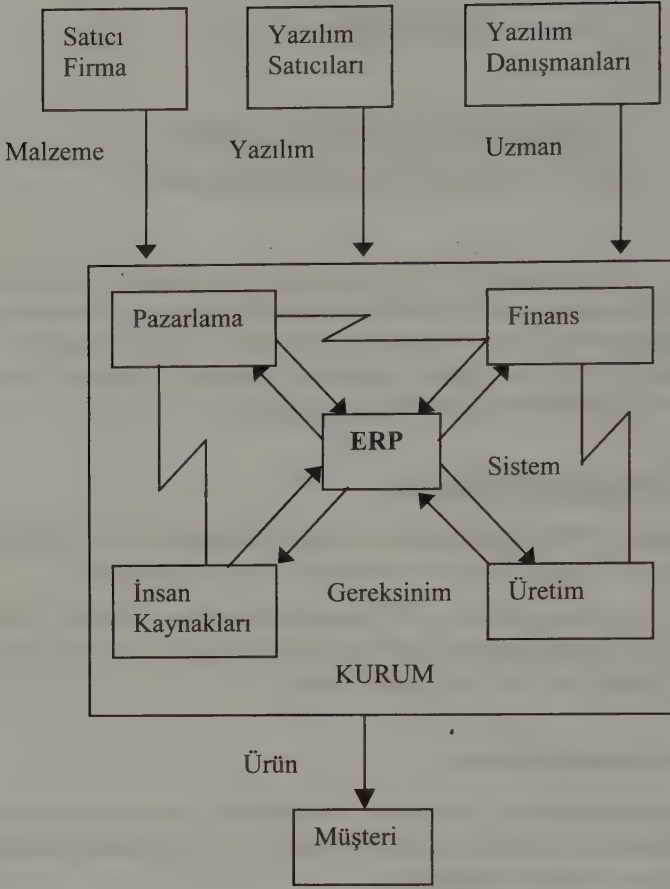
İşletmelerdeki yazılım bütünlüğü, kurumsal kaynak planlaması (*Enterprise Resource Planning*, ERP) yönetim bilgi sistemi uygulamaları ile sağlanmaktadır.

Bu bölümde, ERP ve işletme ortamı, ERP sistemlerinin bilgi sistemleri içindeki yeri, ERP özellikleri ve fonksiyonları, ERP sistemlerinin başarıyla uygulanması için gerekli olan kritik başarı faktörleri incelenmektedir.

17.1.1. ERP ve İşletme Ortamı

ERP sistemleri, çok işletmeli ve çok uluslu kurumlarda, bir birimin gereksinimini karşılamının ötesinde tüm organizasyonun çalışmasını düzenlemek, geliştirmek ve bütünleştirmek amacıyla tasarlanmaktadır.

ERP sistemleri, kurumların, satıcı firmalardan malzeme sağlamaktan, müşterilere ürün dağıtımına kadar tüm iş süreçlerini bütünsel bir yönetim bilgi sistemi desteğiyle düzenlemesini sağlayan modüler yapıya sahip yazılım paketleridir (Şekil-17.1).



Şekil-17.1. Kurumsal kaynak planlaması (ERP) sistemindeki anahtar elemanlar

ERP yazılım paketleri, tüm iş süreçlerinin ve fonksiyonlarının bütünleşmesini sağlayan, işletmenin özel gereksinimleri doğrultusunda uyarlanabilen, insan kaynakları, satış, finans ve üretim gibi modüllerden oluşmuş geniş kapsamlı yazılım paketleridir.

Birçok kurumda ERP projelerinin başlatılma nedenleri;

- Çekirdek işlemsel bilgi sistemlerinin işyüklerinin azaltılması,
- İşletme uygulamalarının küreselleşmesini destekleme,
- Yazılım paketi çözümleri ile bilgi sistemi geliştirme maliyetini azaltma,
- Genel veritabanı kullanımı ile veri bütünlüğünü sağlama,
- Zamanında raporlama ve ayrıntılı sorgulamayı sağlama ile karar desteği geliştirme,
- Stratejik kararların alınmasına yardımcı olmaktadır.

Dünyadaki birçok kuruluş, artık günümüz gereksinimlerine göre güncelleştirilmesi mümkün olmayan eski önemli sistemlerini (*legacy systems*), ERP sistemleri ile değiştirme çabasıdadır. İnternet uygulamaları yaygınlaştıkça, ERP, işlemsel bilgi sis-

temlerini bütünleştirmek, çalışan, müşteri ve ortaklara daha kolay erişmek için olanaklar sunmaktadır.

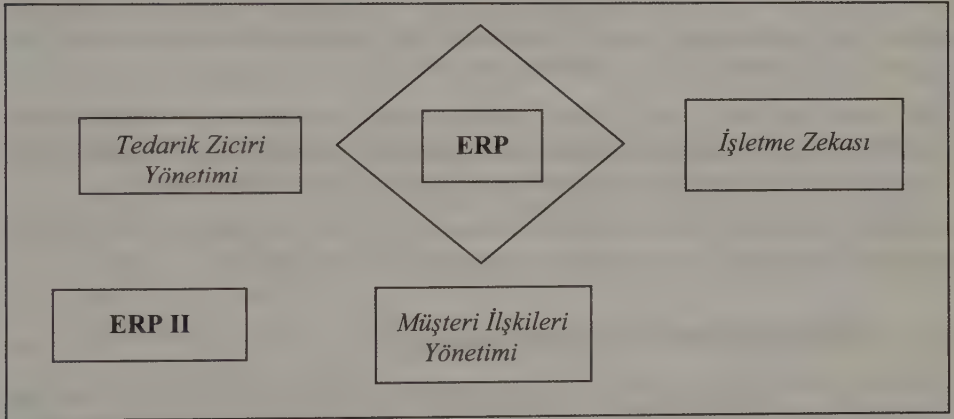
17.1.1.1. ERP Sistemleri Gelişme Süreci

1970'li yıllarda kurumların üretim birimlerinde, mevcut stoklara ve verilmiş siparişlere bakarak ısmarlanması gereken hammadde miktarını belirlemek ve tam zamanında (*just in time*, JIT) üretim yapmak üzere işlemsel bilgi sistemi (TPS) yapısında malzeme ihtiyaç planlaması (MRP) yazılımları kullanılmıştır.

Daha sonra Üretim Kaynakları Planlaması (*Manufacturing Resource Planning*, MRP II) yazılımı ile işletmelerin malzeme kaynakları yanında işgücü, makine ve para kaynaklarını da etkin olarak planlamak ve kontrol etmek amaçlanmıştır. MRP II sistemi, satın alma, üretim planlama ve kontrol, muhasebe, stok yönetimini kapsamıştır.

1990'lı yıllarda, MRP sistemleri tüm sektörlerde hizmet verecek ve tüm faaliyet birimlerini kapsayacak yönetim bilgi sistemleri olarak geliştirilerek ERP sistemleri hazırlanmıştır.

ERP II veya genişletilmiş ERP kapsamında ERP ile birlikte tedarik zinciri yönetimi ve müşteri ilişkileri yönetimi de sisteme eklenmiştir (Şekil-17.2).



Şekil-17.2. Genişletilmiş ERP sistemi yapısı

SAP, lojistik, muhasebe ve insan kaynakları temel uygulama alanlarında düzenlenen dünyada yaygın kullanılan bir ERP sistemidir.

17.1.2. ERP Sistemlerinin Bilgi Sistemleri İçindeki Yeri

İkinci bölümde incelediğimiz üzere, kurumlarda, işlemsel bilgi sistemleri (TPS), yönetim bilgi sistemleri (MIS), karar destek sistemleri (DSS) gibi bilgi sistemleri vardır. ERP sistemleri, kurumlardaki finans, üretim, insan kaynakları gibi birimlerdeki işlemsel bilgi sistemlerinin, bütünleşik (*integrated*), gerçek zamanlı (*real-time*),

küresel (*global*) ve genişletilmiş (*extended*) özellikte yönetim bilgi sistemlerine dönüştürülmüş şeklidir.

Küresel ve gerçek zamanlı özelliği ile ERP sistemleri, çok uluslu şirketlerin işletilmesi ve yönetilmesinde coğrafik sınırların ötesine ulaşabilmektedir. Genişletilmiş özelliği ile de, kurum, satıcı firmalar (tedarikçiler) ve müşteriler arasında ilişki kurabilmektedir.

Yönetim bilgi sistemleri, karar destek sistemleri, bilgi tabanlı sistemler gibi sistemlerden farklı olarak ERP sistemleri, olaylara iş süreçleri tarafından yaklaşmaktadır.

17.1.2.1. Kurumlarda ERP Sistemlerinin Sağlanması

Diğer işletme uygulama yazılımları gibi ERP paketlerinin elde edilmesi ve kuruluşu için de,

- Hazır bir ERP yazılım paketi almak,
- ERP sisteminin şirket içinde üretimi (*inhouse*),
- Dışarıda bir kuruma hazırlanması (*outsourcing*)

yollarına gidilmektedir. Kurumlar kendi iş süreç yapılarını ve gereksinimlerini analiz ederek, bu yollardan birini seçmektedirler.

ERP yazılımını hazır satın alan firmalar yönetimde standart işlemler gerçekleştirmektedirler. Uluslararası yazılım firmalarının ERP ürünlerini kullanmak üzere, kurumlar iş süreçlerini yeniden yapılandırmak zorunda kalabilirler.

ERP sisteminin şirket içinde üretimi, kurumun kendi nitelikli yazılım uzmanları ile uygulamaları geliştirmesidir. Yazılım uzmanları, ERP yazılımının geliştirilmesinde, iş süreçleri ve uygulamalarını anlamak üzere iş uzmanlarından yardım alırlar.

Kurumun özellikleri ve iş süreçlerini anlatacak bir grup oluşturarak, ERP yazılım paketinin dışarıda bir yazılım şirketine hazırlanması sağlanabilir.

17.1.3. ERP Özellikleri ve Fonksiyonları

ERP sistemi kurulmuş olan işletmelerde küresel ekonomide büyük önem taşıyan malzeme, makine, işgücü, metot, para ve pazar gibi kaynakları dengelenmektedir.

ERP sisteminde, üretim, pazarlama, satış ve finans birimlerinde ayrı ayrı sistemler, uygulamalar ve veri tabanı kullanımı yerine tek bir sistem, uygulama ve veri tabanı kullanılmaktadır.

ERP sistemi;

- Farklı üretim yapılarına uyum gösterebilme,
- Personel, bakım, kalite gibi çok sayıda uygulamayı kapsaması,
- Farklı birimler arasında yatay elektronik bilgi değişim hızının yüksekliği,

- Farklı ülkelerdeki tesisler veya bu ülkelerle olan ilişkiler nedeni ile çok dilli kullanımı,
- Uygulama, veri tabanı ve sunum olmak üzere üç katmandan oluşan bir istemci-sunucu mimarisi,
- Dağıtılmış ve ilişkisel veri tabanı,
- Standart sorgulama dilleri,
- İşletim sistemi ve donanımdan bağımsız olmaları,
- Kuşak programlama dili kullanımı,
- Tüm uygulama alanlarında birbiriyle tutarlı grafik ara yüzleri,
- Raporlama esneklikleri

gibi temel teknik özelliklere sahiptir.

- ERP sisteminin modüler yapısı ile gerçekleştirilen fonksiyonlar;
- Kaynak akışının yönetilmesi (Malzeme listesi, süreç planlama, veriyi merkezi işlemek)
- İşlerin dokümantasyonu (Atölye siparişlerinin teslimi, malzeme teslimi, montaj parçaların rota kartları)
- Veri kontrolleri (Malzeme listesi, süreç planlama, veriyi merkezi işlemek)
- Atölye düzeyinde kontrol ve yönetim
- Maliyetlendirme, bakım yönetimi, lojistik yönetimi,

olarak sıralanmaktadır.

17.1.4. ERP Sistemleri Uygulamalarında Kritik Başarı Faktörleri

ERP sistemlerinin başarıyla kuruluşu ve uygulanması için gerekli olan faktörler;

- doğru ERP yazılımının seçilmesi,
- üst yönetimin desteği,
- proje lideri başarı ve etkinliği,
- proje takımının yeterliliği,
- yazılıma göre iş süreçlerinde yeniden düzenleme yapılabilmesi,
- kullanıcı katkısının sağlanması,
- kullanıcıların yeni sistem için eğitimi
- ve kurallı proje yönetimidir.

Kurumun iş süreçleri ve yönetim yapısına uygun olan ERP yazılımının seçimi için, ön değerlendirmede belirlenen yazılımlar üzerinde tanıtım istenerek, ayrıntılı kıyaslama tabloları oluşturulur ve son karar verilir.

Organizasyonda başarılı bir ERP sistemi kuruluşu projesi için, gerekli kaynakların sağlanması, hızlı karar alınabilmesi ve tüm organizasyonda kabul edilmesi açısından üst yönetimin desteği çok önemlidir.

Proje lideri, teknolojik yenilikleri izleyerek, projenin standartlara uygun gerçekleştirilebilme ve projenin kullanıcılara sunumunu başarı ile sağlayabilmelidir.

Proje ekibinde çalışanlar, ERP yazılım sistemini geliştirecekler ya da ERP paket yazılımının danışmanları ve işletme içi çalışanlardan oluşmalıdır.

ERP sistemi kuruluş projesinin kurallara uygun olarak yönetilmesi ve izlenmesi gerekmektedir. ERP Proje yönetimi yapısında;

- Hedeflerin tanımlanması
- Proje çizelgeleme ve planlama
- Risk yönetimi
- Yönetim komitesi oluşturma
- Kullanıcı eğitimi
- Beklentilerin yönetimi
- Tedarikçi/müşteri ilişkisi
- Uygun yazılım paketinin seçimi
- İş süreçlerini yeniden yapılandırma ve minimum özelleştirme
- Kaynak planlama
- Değişim yönetimi
- Departmanlar arası iletişim ve iş birliği
- Tedarikçi desteğinin sürekliliği
- Aksaklıkları önleme
- İzleme, performansın değerlendirilmesi ve geri besleme

özelliklerinin bulunması, başarılı bir sistem kuruluşunun gereğidir.

Yönetim komitesi; farklı birimlerin üst düzey yöneticileri, proje yönetimi temsilcileri ve ERP ile ilgili günlük faaliyetleri yürütecek olan kullanıcılardan oluşmaktadır.

17.1.5. ERP Sistemlerinin Organizasyonların Başarısına Katkısı

Kurumsal bilgi yönetim sisteminin varlığı ve fonksiyonların bütünleşik çalışmasına olanak verilmesi organizasyona aşağıda belirtilen yararları sağlayabilmektedir.

- Gelirlerin artması,
- Personel ihtiyacının azalması,
- Satın alma giderlerinin azalması,
- Sipariş yönetiminin gelişmesi,
- Nakit akışının gelişmesi,
- Verimliliğin artması,
- Müşteri hizmetlerinin artması,
- Gereksiz veri işlemlerin azalması,

- Güvenilir bilginin kolay, doğru ve hızlı elde edilmesi,
- Talep tahminlerinin doğru yapılması,
- İletişim ve lojistik giderlerin azalması,
- Sistem bakım ve onarım giderlerinin azalması,
- Tüm işletme yazılımlarının bir sistemde toplanması,
- Proje yönetiminin gelişmesi,
- Anında veri ulaşımı sayesinde karar verme mekanizmasının güçlenmesi,
- Organizasyon içindeki bütün sistemlerin bütünleşik olarak çalışması.

17.2. Tedarik Zinciri Yönetimi (Supply Chain Management, SCM)

Bir şirketin ham madde hazırlanmasından, üretiminden bitmiş ürünlerin pazara sunulmasına kadar olan bütün aşamalarının tamamına tedarik zinciri adı verilmektedir. Bu zincir içinde ham madde tedarikçileri, yarı mamül işletmecileri, imalat işletmecileri, lojistik, perakendeciler ve hatta müşteriler yer almaktadır. Tedarik zinciri yönetimi sistemi ise tüm bu süreç boyunca tarafları izleyip, yöneterek müşteri ihtiyaçlarına en kısa zamanda daha etkin ve verimli şekilde cevap vermek üzere, maliyetleri azaltarak kaynaklardan en yüksek düzeyde yararlanılmasını amaçlamaktadır.

Ticarette, küreselleşme ile fiziksel sınırların ortadan kalkması, tedarik zinciri sistemlerinin tek bir merkezden yönetiminin yerini farklı bölgeler arasında iletişim içinde düzenlenen yönetim almıştır. Hatta üretim bile artık sadece bir yerde yapılmamaktadır. Örneğin cep telefonlarında “Kore’de yapıldı; Macaristan’da bitti.” (“*Made in Korea finished in Hungary*”) ifadesine rastlamak olasıdır. Bu, hem küreselleşmenin hem de iş alanlarında uzmanlaşmanın sonucudur.

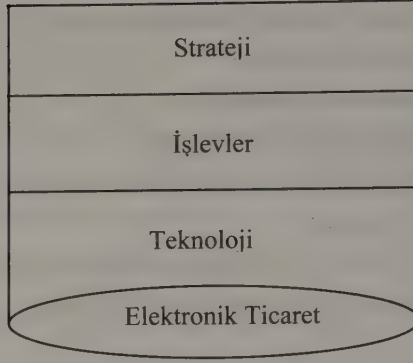
Bir tedarik zinciri sisteminin genel olarak;

- Ana firmanın çalıştığı diğer şirketlerle haberleşmeyi sağlayan,
- Ham madde, yarı mamul ve ürün bilgilerini kontrol edip yöneten,
- Üretimi tüm riskleriyle, zaman ve kaynak planlamasıyla projelendiren,
- Tüm üretim süreçleri boyunca tedarikçiler ile alıcılar arasındaki koordinasyonu sağlayan

özelliklere sahip olması gerekmektedir. Bu özellikleriyle tedarik zinciri sistemi şirketin ihtiyaçlarına göre tek başına kullanılabileceği gibi ERP paketlerinin içinde bir alt modül olarak da temin edilebilir.

17.3. Müşteri İlişkileri Yönetimi (Customer Relationship Management, CRM)

Müşteri ilişkileri yönetimi (*Customer Relationship Management, CRM*), müşteri odaklı bir stratejik anlayıştır ve yazılım teknolojileri bu stratejiye destek sağlamaktadır (Şekil-17.3).



Şekil-17.3. CRM stratejik yapısı

CRM, her müşterinin özelliğini belirleyerek ayrı ayrı davranılmasını sağlar ve tüm bölümlere yatay ve dikey olarak yerleşerek kuruma hakim olursa başarılı olur.

CRM, şirketin var oluş amacı olan müşterilerin nasıl bölüm ve alt bölümlere ayrılabilceği, müşteriye nasıl ulaşılabilceği ve onlarla nasıl kalıcı ilişkiler kurulabileceğini içerir.

17.3.1. Pazarlama Bilgi Sistemlerinden Müşteri İlişkileri Sistemlerine

Pazarlama sistemlerinde, anlayışlarına göre;

- Kitle Pazarlama,
- Hedef Odaklı Pazarlama,
- Müşteri Odaklı Pazarlama ve
- Birebir Pazarlama

olmak üzere, farklı özelliklerde olan dört ana aşama bulunmaktadır.

Tarihsel olarak incelendiğinde, pazarlama anlayışlarındaki gelişim de kitleselden müşteri odaklıya ve birebir pazarlamaya doğru bir yönelim izlemektedir. Birebir pazarlama, müşterinin kendisi için almak istediği en yalın hizmet şeklini belirlerken, organizasyon büyüdükçe daha karmaşık bir operasyona ve işgücüne ihtiyaç duymakta, bu nedenle de teknolojik olarak doğru araçların desteğini gerektirmektedir.

Kitlesel pazarlamada, daha çok demografik bazı gerçekler ve bilgiler kullanılarak kaba bir segmentasyon uygulanırken; birebir pazarlama, müşteriye ön planda tutan ve müşteriye özel hizmet sunmayı amaçlayan ve müşterinin ayrıntılı izlenmesini gerektiren bir *Müşteri İlişkileri Yönetimi* stratejisidir.

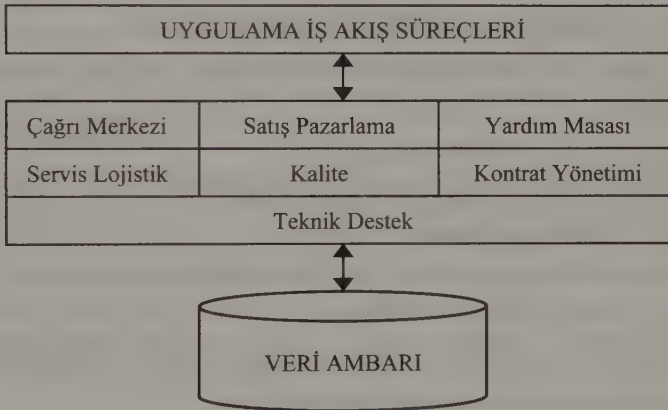
Bire-bir pazarlama stratejisinin temeli, müşterilerin nasıl farklılıklar gösterdiğini anlamayı ve bu farklılıkların her bir müşteriye göre firmanın nasıl davranması gerektiği konusunda bir strateji oluşturmaktır.

Gerçek anlamda bire-bir şirket olabilmek, birey müşterilerle ilişki içinde olmayı ve onlarla sürekli olarak etkileşim sonucunda tüm şirketin üretim ve sunum yapısını değiştirmeyi içerir.

17.3.2. CRM Sistemi Yapısı

Bir CRM yazılımının içerisinde genel olarak bulunabilecek modüller Şekil-17.4’de görülmektedir. Bu modül isimleri farklı firmalarda geliştirilmiş CRM paketlerine göre değişebilmektedir. Çağrı merkezi, satış, pazarlama, destek, saha satış, kontrat yönetimi, kalite, iş akışı olarak adlandırabileceğimiz bu modüller kampanya yönetimi, e-posta yönetimi, doküman yönetimi, mobil satış gibi pakete dahil olan ya da eklenebilen programlarla desteklenebilmektedir. CRM’in ana özellikleri:

- Bir veri ambarına dayanır.
- Müşteri davranışının farklı görünümüne odaklanmıştır.
- Farklı kanallarda müşteri etkinliklerinin izlemek için tutarlı metriklere sahiptir.
- Müşteriyi merkeze koyan yeni pazar dinamiklerine uyumludur.
- Müşterinin karlılığını ya da kar potansiyelini belirlemek üzere yapılandırılmıştır.
- Değişen ve artan koşullara göre ölçeklendirilebilir.

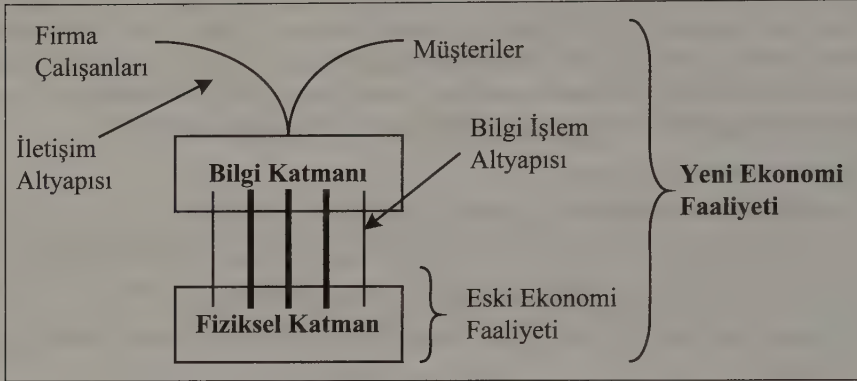


Şekil-17.4. Temel CRM modülleri

Günümüzde müşteri ilişkileri yönetiminin bu kadar öne çıkmasının temel sebebi, müşteri izleme yeteneklerinin diğer teknolojilere göre oldukça yüksek olması ve bu nedenle de Web’e dayalı daha yenilikçi müşteri ilişkileri yöntemlerinin (eCRM) ortaya çıkmasıdır.

17.3.2.1. Yeni Ekonomi

1980'lerin ortasından beri yaşanan gelişmeler özellikle de teknolojik gelişmeler sonucu "yeni ekonomi" kavramı oluşmuştur. Eski ekonomi aktiviteleri fiziksel unsurlardan oluşurken yeni ekonomide, Şekil-17.5'de de görüldüğü gibi aktivitelerin hepsinde bir fiziksel faaliyet katmanı, bir bilgi katmanı, bilgisayar ve yazılımlardan oluşan bir bilgi işlem altyapısı ve bir de iletişim altyapısı vardır.



Şekil-17.5. Yeni ekonomi faaliyetinin tanımı

17.3.3. Müşteri İlişkileri Yönetimi Sistemi Kuruluşu

Etkin CRM sisteminin gerçekleştirilebilmesi amacıyla üzerinde durulması gereken kapsam ve amaçlar;

- Mevcut karlı müşterilerin belirlenmesi
- Bu müşterilerle uzun vadeli, güçlü ilişkiler kurulması
- Müşteri sadakatinin sağlanması
- Müşterinin cüzdan payı (*share of wallet*) ve karlılığının artırılması
- Potansiyel karlı müşterilerin belirlenmesi
- Bu müşterilere doğru, etkili ve daha düşük maliyetli yöntemlerle ulaşılması

CRM dört pazarlama stratejisi üzerine kurulmaktadır:

- Terk etmiş olan müşterileri geri kazanma (*win-back*)
- Müşteriler nezdinde sadakat yaratma (*loyalty*)
- Yeni müşteri bulma (*prospecting*)
- Var olan ve potansiyel müşterilere çapraz satış yapma (*cross-selling*)

Bu adımlardan kazanma ve tutma aşamasında kampanya yönetimi, çağrı merkezi, web sitesi, satış otomasyonu ve satış sonrası destek gibi CRM bazlı teknik yazılım ve araçlar kullanılabilir; geriye kalan geliştirme, kurtulma analiz ve seçme aşamalarında ise özellikle bu modüllerin üzerine ağırlıklı olarak kullanılacak pazarlama

ve analitik ürünler katılmaktadır. Analitik ile veri ambarları, online analiz araçları ve diğer karar destek sistemleri kullanılmaktadır. Bu analizler sektör ve firma bazında önemli farklılıklar göstermektedir. Ayrıca, müşteri ile ilgili olarak CRM paketleri dışında, ERP paketleri gibi ek arka ofis uygulamalarının da detaylı entegrasyonuna gereksinim duyulabilmektedir

17.3.4. CRM Uygulamalarında Başarı Faktörleri

CRM uygulamalarının %70 ile %90'ının başarılı bulunmadıkları gözlenmektedir. Bu başarısızlık konusunda en fazla karşılaşılan sebepler aşağıda verilmektedir;

- Satış ekibini veri giriş memurlarına dönüştürmek
- Kendinizi, müşterinizin yerine koymamak
- Sadece teknolojiye odaklanmak
- Stratejiyi pakete göre belirlemek
- Entegrasyon problemleri
- Satış ekibinin müşteriye yeterince çabuk cevap verememesi
- Firma içindeki diğer MIS sistemleri ile entegrasyonun olmaması (stok, sipariş, finans)
- Mükerrer veri girişlerine neden olan entegrasyon yapıları
- Firma içi MIS sistemlerindeki kısıtlar

Başarılı CRM Projelerinin ortak noktası, sadece teknolojiye odaklanma yerine teknoloji, süreç ve insanı aynı zamanda inceleyerek dengeli olarak ele alma - CRM için kapsamı ve stratejiyi doğru çizme - organizasyonun bütününde bu bilginin paylaşımını sağlama olmalıdır.

Ayrıca, hızlı karar verebilme ve dolayısıyla analiz yapabilme yeteneğinin kazandırılması CRM projelerini başarılı kılmaktadır. Bu amaç doğrultusunda; kolektif ve operasyonel alanda elde edilen verilerin düzenli olarak analiz tablolarına işlenmesi ve verinin veri ambarları, OLAP araçları ve diğer karar destek sistemleri ile hızlı olarak ve işlemsel sistemlere yük olmadan bilgiye dönüştürülmesi sağlanmalıdır. Bu aşamada hangi verinin nerede ve nasıl birleştirileceğinin bilinmesi çok önemlidir. Bazı CRM programları, analitik çözümleri doğrudan üzerlerinde bulundurmaktadır.

17.3.5. Elektronik Müşteri İlişkileri Yönetimi (e-CRM)

Kişiselleştirmenin yanı sıra müşteri hareketlerinin izlenmesinin de diğer teknolojilere göre yüksek olması son yıllarda Web'e dayalı müşteri ilişkileri yönetiminin öne çıkmasına neden olmuştur. Bu yaklaşımın etkisiyle CRM'in Web'e dayalı biçimi olan e-CRM oluşmuştur. Ancak birbirlerine çok benzerlik gösterdikleri için günümüzde halen e-CRM tanımı yerine CRM de kullanılmaktadır. Aşağıda kısaca bu ortak özelliklere değinilmiştir.

e-CRM de CRM gibi dört pazarlama stratejisi üzerine kurulmuştur:

- Yeni müşteri bulma (*prospecting*)
- Müşteriler nezdinde sadakat yaratma (*loyalty*)
- Var olan ve potansiyel müşterilere çapraz satış yapma (*cross-selling*)
- Terk etmiş olan müşterileri geri kazanma (*win-back*)

Belirlenen bu stratejilerin başarıyla gerçekleştirilebilmesi içinse yapılması gereken, müşterinin alışveriş sürecini dikkatle yakından gözlemleyerek, müşteriyle sürekli bir etkileşim süreci oluşturmaktır. Ancak CRM ve e-CRM sadece birer yazılım değil aynı zamanda stratejilerdir. Teknoloji, sadece bu stratejinin hayata geçirilmesi-ne yarayan önemli bir araçtır. CRM'in ve e-CRM'in ana özelliklerini maddeler halinde belirtilirse;

- Bir veri ambarına dayanır.
- Müşteri davranışının farklı görünümüne odaklanmıştır.
- Farklı kanallarda müşteri etkinliklerinin izlemek için tutarlı metriklere sahiptir.
- Müşteriyi merkeze koyan yeni pazar dinamiklerine uyumludur.
- Müşterinin karlılığını ya da kar potansiyelini belirlemek üzere yapılandırılmıştır.
- Değişen ve artan koşullara göre ölçeklendirilebilir.

17.4. Özet

Rekabetin hızlı artışı ve küreselleşmesinin etkisiyle 1990'lı yıllardan itibaren işletmelerde bilgi sistemi uygulamaları etki alanları genişleyerek daha da önem kazanmıştır. Satıcı firmalardan malzeme sağlamaktan, müşterilere ürün dağıtımına kadar tüm iş süreçlerini bütünlük bir yönetim bilgi sistemi desteğiyle düzenleyen kurumsal kaynak planlaması (ERP) uygulaması kullanmadan başarıya ulaşması oldukça zordur. Müşteri memnuniyetini arttırmak veya yeni müşteriler kazanmak isteyen bir işletmenin müşteri ilişkileri yönetimi (CRM) uygulaması zorunlu hale gelmiştir. Tedarik zinciri yönetimi uygulaması da, tedarikçiler ve bayilerle daha güçlü bir iletişim ve kontrol mekanizması sağlayacaktır. E-ticaretin artmasıyla beraber CRM'in Web'e dayalı çeşidi olan e-CRM kullanımı da artmıştır.

Ancak bu uygulama türlerinin hepsi bir işletmede kurulurken daha önceki bölümlerde anlatıldığı gibi fizibilite kapsamında saylayacağı yarar ve maliyet analizi yapılmalıdır.

17.5. Sorular

- 17.5.1)** ERP'yi oluşturan alt sistemleri araştırınız.
- 17.5.2)** ERP ile ERP II arasındaki farklar nedir?
- 17.5.3)** Bu bölümde anlatılan tüm bilgi sistemi uygulamalarına birer ticari paket örneği veriniz.
- 17.5.4)** CRM'in olmadığı büyük ölçekli bir işletmede birebir pazarlama nasıl yapılabilir tartışınız.
- a)** CRM'in hangi pazarlama stratejileri üzerine odaklanmıştır?
- b)** CRM bu stratejileri nasıl gerçekleştirir açıklayınız?
- 17.5.5.)** İlet A.Ş. isimli kargo şirketinde bir tedarik zinciri uygulamasını nasıl gerçekleştirirsiniz?

KAYNAKLAR

- ARİFOĞLU, A., DOĞRU, A., *Yazılım Mühendisliği*, Sas Bilişim Yayınları, 2001
- BROWN, D., *An Introduction to Object-Oriented Analysis*, John Wiley & Sons, Inc., 1997
- DÜNDAR, D., KALIPSIZ, O., *Bilgi İşlem Sistemi*, Graphics Basım A.Ş., 1990
- EREN, E., *İşletmelerde Stratejik Planlama ve Yönetim Cilt 1*, İ.Ü. Yayın No: 3448, 1989
- ERKUT, H., *Sistem Analizi*, Kıyı Yayınları, 1989
- ERTÜRK, M., *İşletme Biliminin Temel İlkeleri*, (3.Baskı), Beta Yayınları, 1998
- ESEN, H.Ö., *İşletme Yönetiminde Sistem Yaklaşımı*, Alfa Basım Yayın Dağıtım, 1998
- FRAWLEY, A., *e-CRM*, Yeni Teknoloji, 2002
- HOFFER, J.A., GEORGE, J.F., VALAICH, J.S., *Modern Systems Analysis And Design*, The Benjamin/Cummings Publishing Company, Inc., 1996
- KALIPSIZ, O., *Bilgisayar Yazılım Mühendisliği*, İ.Ü. Basımevi, 1992
- KALIPSIZ, O., Tıpta Bilgisayar Kullanımı ve Bir Bilişim Sistemi Geliştirme Uygulaması, İ.Ü. Doktora Tezi, 1989
- KALIPSIZ, A., *Bilim ve Araştırma*, İ.Ü. Yayın No: 3492, 1988
- KARLIDERE, T., KALIPSIZ, O., *Yazılım Mühendisliğinde Çevik Yaklaşımların Yeri*, 1.Ulusal Yazılım Mühendisliği Sempozyumu, 2003
- KAYA, İ., *Pazarlama Bilgi Sistemleri*, İ.Ü.Yayın No:3226, 1988
- KENDALL, K.E., KENDALL, J.E., *System Analysis and Design*, Prentice Hall, 2004
- KIRCOVA, İ., *İşletmelerarası Elektronik Ticaret*, İstanbul Ticaret Odası Yayınları No:2001-32, 2001
- KIRIM, A., *Strateji ve Bire-Bir Pazarlama CRM*, (2.baskı), Sistem Yayıncılık, 2001
- KROEBER, D.W., WATSON, H.J., *Computer Based Information Systems*, Maxwell MacMillan, 1990
- LARMAN, C., *Applying UML and Patterns*, (3.Edition), Prentice Hall, 2005
- LEESON, M., *System Analysis And Design*, (2.Edition), Science Research Associates, Inc., 1985
- LOUDON, K.C., LOUDON, J.P., *Management Information Systems : managing the digital firm*, (8.Edition), Prentice Hall, 2004
- LUCAS, H.C., *The Analysis, Design, And Implementation Of Information Systems*, (3.Edition), McGraw-Hill, 1985
- MACIASZEK L.A., *Requirements Analysis And System Design*, Addison Wesley, 2001
- MACIASZEK L.A., LIONG, B.L., *Practical Software Engineering*, Pearson Education, 2005

- MARAKAS, G.M., *System Analysis and Design: An Active Approach*, Prentice Hall, 2001
- O'BRIEN, J.A., *Management Information Systems: Managing Information Technology in the Internetworked Enterprise*, (5.Edition), McGraw-Hill, 2002
- OLSON, D.L., *Introduction to Information Systems Project Management*, McGraw-Hill, 2001
- PRESSMAN, R.S., *Software Engineering-A Practitioner's Approach*, (2.Edition), McGraw-Hill, 1987
- PRESSMAN, R.S., *Software Engineering-A Practitioner's Approach*, (4.Edition), McGraw-Hill, 1997
- ROYCE, W., *Software Project Management-A Unified Framework*, Prentice Hall, 1998
- RUMBAUGH, J., ve ark., *Object Oriented Modeling And Design*, Prentice Hall, 1991
- SATZINGER, J.W., JACKSON, R.B., BURD, S.D., *System Analysis and Design in a Changing World*, Course Technology-Thomson Learning, 2002
- SENN, J.A., *Analysis & Design of Information Systems*, (2.Edition), McGraw-Hill, 1989
- SOMMERVILLE, I., *Software Engineering*, (6.Edition), Addison-Wesley, 2001
- TOKOL, T., *Pazarlama Araştırması*, (10.baskı), Vipaş A.Ş., 2000
- TULGA, S.Ş., *Eski Ekonomiye Yeni Kurallar*, Hepbizz Yeni Ekonomi Platformu Dergisi Sayı:1, 2001

İnternet Kaynakları

- <http://www.doc.mmu.ac.uk/online/SAD/T02/sdlc.htm>
- http://www.qafqaz.edu.az/akademikpersonel/fatihyegul/LectureNotes_SystemAnalysis_StudentCopy.pdf

PAPATYA YAYINCILIK EĞİTİM

Dil/Türk Dili Kitaplarımız

- **Uygulamalı Türk Dili** ⇒ Yusuf ÇOTUKSÖKEN
- **Biçimbilim: Temel Kavramlar** ⇒ Nadir Engin UZUN
- **Dilin İşlevleri ve İletişim** ⇒ Veysel KILIÇ
- **Türkiye Türkçesinde Kök-Ek Bileşmeleri** ⇒ Ömer DEMİRCAN
- **Türkiye Türkçesinde Biçimbirimler** ⇒ Oya ADALI
- **Türkiye Türkçesinin Sözdizimi** ⇒ Neşe ATABAY, Sevgi ÖZEL ve Ayfer ÇAM
- **Sözcük Türleri** ⇒ Neşe ATABAY, Dr. İbrahim KUTLUK ve Sevgi ÖZEL
- **Türk Dilinde Çatı** ⇒ Ömer DEMİRCAN
- **Türkçe Üzerine I: Denemeler ve Eleştiriler** ⇒ Yusuf ÇOTUKSÖKEN
- **Uygulamalı Türk Dili - Tek Cilt** ⇒ Yusuf ÇOTUKSÖKEN
- **Uygulamalı Türk Dili-I** ⇒ Yusuf ÇOTUKSÖKEN
- **Uygulamalı Türk Dili-II** ⇒ Yusuf ÇOTUKSÖKEN

Tıp/Sağlık Kitaplarımız

- **Hafta Hafta Gebelik** ⇒ Op.Dr. Kağan KOCATEPE
- **9 Ay 10 Gün (Gebelik Süreci ve Bebeğin Gelişimi)** ⇒ Op.Dr. Kağan KOCATEPE
- **Çocuk Sağlığı Kılavuzu** ⇒ Dr. Doğu ERKER
- **Çocuğumuz ve Sağlıklı Beslenmesi** ⇒ Dr. İ. TUNÇDOĞAN ve Dr. C.A. TUNÇDOĞAN
- **28 Gün/Kadın Olmak (Kadın Sağlığı ve Cinselliği)** ⇒ Op.Dr. Kağan KOCATEPE
- **Gazlı Sindirim Bozukluğu - Kabızlık - Doğal Tedavisi** ⇒ Dr. Nidai Sulhi ATMACA
- **Menopoz: Yeni Bir Yaşam** ⇒ Op.Dr. Kağan KOCATEPE (hazırlanıyor)

Sosyal/İşletme/Hukuk Kitaplarımız

- **Satış Teknikleri Eğitim** ⇒ Erdoğan TAŞKIN
- **İşletme ve Yönetim (Fonksiyonları Açısından)** ⇒ Mehmet ÖZTÜRK
- **Satış Yönetimi Eğitimi** ⇒ Erdoğan TAŞKIN
- **İşletme Yönetiminde Eğitim ve Geliştirme** ⇒ Erdoğan Taşkın
- **Müşteri İlişkileri Eğitimi** ⇒ Erdoğan Taşkın
- **Öğrenen Organizasyon ve Rekabet Üstünlüğü** ⇒ Salim ÇAM
- **Türk İzcilik Tarihi** ⇒ Gökhan UZGÖREN
- **Politik Pazarlama** ⇒ Ahmet TAN
- **Politika'da Niye Kaybediyorlar? Nasıl Kazanırlar?** ⇒ Ahmet TAN
- **İnsan Kaynakları: Kişi ve Kurumlara Öneriler** ⇒ Meltem YAMAN
- **İnternet ve Hukuk** ⇒ Ali Osman ÖZDİLEK
- **Sosyal Güvenlik Terimleri, Ansiklopedik Sözlük** ⇒ Ali SEYYAR

Bilgisayar/Elektronik Kitaplarımız

- **8051 Mikrodenetleyici Uygulamaları (C ile)** ⇒ A. Turan ÖZCERİT, Murat ÇAKIROĞKLU ve Cüneyt BAYILMIŞ
- **JAVA ve Yazılım Tasarımı** ⇒ Altuğ B. ALTINTAŞ
- **Yazılım Mühendisliği** ⇒ Erhan SARIHAN
- **Programlama Sanatı Algoritmalar (C ile)** ⇒ Rifat ÇÖLKESEN
- **C++ Nesneye Yönelik Programlama** ⇒ Erhan SARIHAN
- **MATLAB ve Programlama** ⇒ Aslan İNAN
- **Yapay Sinir Ağları** ⇒ Ercan ÖZTEMELE
- **İnternet Teknolojileri ve İnternet Uygulamaları** ⇒ Türker CAMBAZOĞLU
- **Veri Yapıları ve Algoritmalar Temelleri** ⇒ Sefer KURNAZ
- **Veri Yapıları ve Algoritmalar** ⇒ Rifat ÇÖLKESEN
- **Bilgisayar Ağları** ⇒ B. Demir ÖNER
- **Bilgisayar Haberleşmesi ve Ağ Teknolojileri** ⇒ R. ÇÖLKESEN ve B. ÖRENCİK
- **Veri Haberleşmesi Temelleri** ⇒ Yasin KAPLAN
- **NETWORK/Veri Haberleşmesi Uygulamaları** ⇒ Yasin KAPLAN
- **Veri Haberleşmesi Kavramları** ⇒ Yasin KAPLAN
- **NETWORK TCP/IP ve UNIX El Kitabı** ⇒ Rifat ÇÖLKESEN
- **C Programlama Dili** ⇒ Rifat ÇÖLKESEN
- **Uygulamalı C Programlama Dili** ⇒ Bora TUNÇER
- **GTK+/GNOME Programlama** ⇒ M. Ali VARDAR
- **LINUX Altında Program Geliştirme** ⇒ M. Ali VARDAR
- **Sistem Analizi ve Tasarımı** ⇒ Oya KALIPSIZ ve ark.
- **Lojik Devre Tasarımı** ⇒ Taner ARSAN ve Rifat ÇÖLKESEN

Temel Bilimler/Teknik Kitaplarımız

- **Matematik Analizi ve Uygulamaları** ⇒ Elimhan MAHMUDOV
- **Diferansiyel Denklemler Teorisi** ⇒ E.HASANOV, G.UZGÖREN, A.BÜYÜKAKSOY
- **Lineer CEBİR** ⇒ Veli ŞAHMUROV ve Gökhan UZGÖREN
- **Lineer CEBİR Uygulamaları** ⇒ Veli ŞAHMUROV ve Gökhan UZGÖREN
- **Atmosfer Fiziği** ⇒ Zafer ASLAN, Sema TOPÇU, M. Celal BARLA ve Güven ÖZDEMİR
- **Ekolojik Sistemlerin Analizi, Yönetimi ve Modellenmesi** ⇒ Fatih EVRENDİLEK
- **Enerji Sistemleri (Cilt I, III)** ⇒ Nariman ŞERİFOĞLU
- **Enerji Sistemleri (Cilt II)** ⇒ N. ŞERİFOĞLU ve Oğuz SOYSAL
- **Elkt. Dalga Teorisi Çözümlü Problemleri** ⇒ G. UZGÖREN, A. BÜYÜKAKSOY
- **Elek. Alan Teorisi Çözümlü Problemleri** ⇒ G. UZGÖREN, A. BÜYÜKAKSOY
- **ISI Transferi: Teorik ve Uygulamalı** ⇒ Tuncay YILMAZ

DİZİN

- açık uçlu sorular 62
 aday sınıf 116
 aktör 112
 alt sınıf 107
 anket yönetimi 62
 arabirim 124
 tasarımı 96
 araştırma metotları 60
 raporu 65
 ardışıl diyagram 126
 artırılmalı model (incremental) 44
 aşırı programlama (Extreme Programming - XP) 45
 yükleme (overloading) 108
 aykırı programlama (EP) 43, 45
 ayrıntılı tasarım 88
 az bağımlılık (low coupling) 122
 bağlantı (association) 105
 belirlenmesi 117
 başlangıç nesnesi 104
 bilgi sistemi tipleri 25
 sistemleri 25
 tabanlı iş sistemleri (BTİS) 25,30
 bilginin uzman kalıbı 122
 uzmanı (expert) 122
 birleştirme (composition) 106
 bütünleme testi 150
 CAD 30
 CASE 34, 36
 CPM 52, 173, 179-184
 sistemi yapısı 181
 çıktı tasarımı 91
 çoğullama sayısı (multiplicity) 106
 çok şekillilik (polymorphism) 107, 123
 değişimlerden korunma 125
 denetçi (controller) 123
 dereceli geçiş 159
 doğrudan geçiş 158
 dolaylılık 124
 Drill Down 31-32
 durak noktaları 52, 53, 56
 durum diyagramı 121
 geçiş diyagramı 67, 68
 e-CRM 183-184
 ekonomik fizibilite 49
 ERP özellikleri 173, 176, 178
 ERP ve İşletme Ortamı 173
 eşdeğerli bölünme 155
 etkileşim döngüler 129
 diyagramları 125
 karşılıklı dışlamalı mesaj 128
 kendine mesaj 127
 koşullu mesaj 127
 evrimsel süreç 41, 44
 fizibilite analizi 41, 49
 matrisi 54
 tipleri 49
 Gantt 51
 geçersiz kılma (overriding) 107
 genel sistem teorisi 19
 tasarım 82
 genelleştirme (generalization) 107
 genişletme (extend) 114
 gereksinim analizi 57
 geri ödeme yönetimi 51
 girdi tasarımı 92
 görünürlük 104, 135
 görüşme 60
 hedef arama (goal seeking) 29
 HIPO 89
 IRR 80
 içermek (include) 114
 işbirliği diyagramı 126
 işlem 105
 sözlük girişi 78
 işlemsel bilgi sistemleri (İBS) 25, 26
 iyi uyum (high cohesion) 123
 kalıtım (inheritance) 107
 kapalı uçlu sorular 62, 63
 kara kutu testi 153
 karar destek sistemleri (DSS) 25, 28
 tabloları 74
 kavramsal sınıf 116
 kaynak ağacı 78
 program 142
 seviyeleme 52, 56
 kilometre taşı (milestone) 52, 53, 56
 klasik süreç (waterfall model) 41

konfigürasyon 170
 kullanım diyagramı 113
 dokümantasyonu 114
 gerçeklenmesi 131
 modellenmesi 111
 senaryoları 113
 kümeleme (aggregation) 106
 mesaj 104
 metot 105
 mimari tasarım 87
 model oluşturma 42
 müşteri ilişkileri yönetimi 173, 179-184
 nesne 103
 NPV 50
 ofis otomasyon sistemleri (OOS) 28
 OLAP 31-32, 183
 oluşum (composition) 106
 onaylama testi 150
 ön inceleme 49
 özelleştirme (specialization) 114
 özellik (attribute) 104
 belirlenmesi 116
 PERT 52
 private (özel) 104
 proje yöneticisi 33
 protected (korunmalı) 104
 prototip geçiş 159
 model 43
 public (açık) 104
 RAD 41, 43
 model 43
 risk yönetimi 37
 sarmalama (encapsulation) 125
 saydam kutu testi 152
 senaryo 113
 sınıf 103
 diyagramı 117, 135
 modellenmesi 115
 sınıflar arası ilişkiler (relations) 105
 sınır değer analizi 155
 simülasyon 30
 sistem 17
 analisti 33, 34
 analizi yöntemleri 58
 arabirim 18
 bileşenler 17
 değişkeni 18
 mimarisi 87

tasarımı 81
 tasarımı 81
 testi 148, 151
 sistemin özellikleri 17
 sorumluluklar atanması 131
 sosyal fizibilite 53
 soyut sınıf (abstract class) 107, 108
 sözleşmeler (contracts) 115
 spiral model 44, 45
 tasarım kalıpları 121
 sınıf diyagramı 135
 taslak veri akış diyagramı 71
 tedarik zinciri yönetimi (SCM) 173, 179
 teknik fizibilite 49
 temel yollar testi 153
 toplam kalite kontrolü 23
 tümleştirilmiş modelleme dili 101
 UML 101
 Unified Process 101
 uygulama yazılımı 168, 170
 ünite testi 148, 149
 üst yönetim bilgi sistemleri (EIS) 25, 29
 varlık ilişki diyagramı (ER) 67, 73
 veri akış diyagramı 67, 68, 69, 70
 akış sözlük girişi 76
 deposu sözlük girişi 77
 elemanı sözlük girişi 78
 sözlüğü 67, 76, 76, 77
 tabanı yönetimi sistemleri 36
 tasarımı 95
 yapısı sözlük girişi 77
 yapay sınıf (pure fabrication) 124
 yapısal kodlama 143
 yaratıcı (creator) 122
 kalıbı 122
 sınıf 124
 yasal fizibilite 53
 yazılım kalite faktörleri 145
 kalite planı 147
 yazılımın sınanması 148
 yeni ekonomi 182
 yönetim bilgi sistemi (YBS) 25, 27
 fizibilitesi 53
 zaman fizibilitesi 51
 zenginleştirme 170

SİSTEM ANALİZİ VE TASARIMI

NESNEYE YÖNELİK MODELLEME

Bu kitap "Sistem Analizi ve Tasarımı" derslerinde öğrencilerin yararlanması ve bilgi sistemleri geliştirme projelerinde çalışanlara yol gösterici bir kaynak olması amacıyla hazırlanmıştır.

Sistem Analizi ve Tasarımı kitabında sistem ve bilgi sistemleri ile ilgili temel tanımlar verildikten sonra bilgi sistemi geliştirme süreci, nesneye yönelik yaklaşımla sistem analizi ve tasarımı, sistem gerçekleştirme ve işletmelerde bilgi sistemi uygulamalarına yer verilmiştir.

Kitap, bilgi sistemi modellemede geleneksel ve nesneye yönelik yeni yaklaşımları içermek üzere tasarlanmıştır.

Bölümler:

Sistem, Bilgi Sistemleri, Bilgi Sistemi Tarafları

Sistem Geliştirme Süreci ve Modelleri

Ön İnceleme ve Fizibilite Çalışması

Sistem Analizi (Gereksinim Analizi)

Sistem Analizi Modeli

Sistem Tasarımı

Nesneye Yönelik Temel Kavramlar

Nesneye Yönelik Sistem Analizi

Nesneye Yönelik Sistem Tasarımı

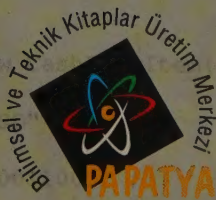
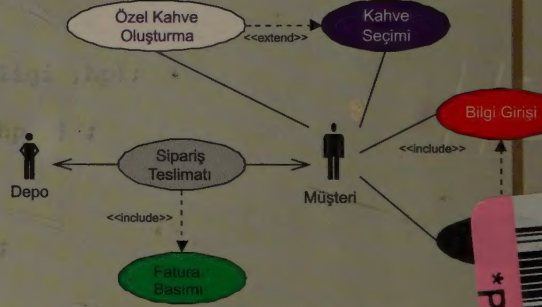
Kodlama

Yazılımda Kalite ve Test Teknikleri

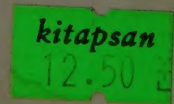
Var Olan Sistemden Yeni Sisteme Geçiş

İşletmelerde Bilgi Sistemlerinin Yeri

İşletmelerde Bilgi Sistemi Uygulamaları



<http://www.papatya.gen.tr>
<http://www.papatya.info>



P7-BET-853

